THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that **Ms. GAJADEVI.G (20524UCSC072)** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO:01 Creation of a DDL operations using Books Table

DATE: 05.08.2026

Aim

To create a Books table, insert book records, display all records, and retrieve books published from the year 2010 onwards.

Software Used

Oracle /MySQL/sqlite3

Description

DDL (Data Definition Language), it is used to create and modify the structure of database objects in database

DDL COMMANDS:

- CREATE
- ALTER
- DROP
- TRUNCATE

CREATE: Create command is used to create a new table and also use constraints.

Syntax

```
CREATE TABLE book_table
   ( column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE books (
        id INTEGER PRIMARY KEY,
        title TEXT,
        author TEXT,
        published_year INTEGER
       );
```

Table Created

ALTER: Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP

ADD: Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE book_table ADD column_name datatype;

Query:

```
SQL> ALTER TABLE books ADD genre VARCHAR(50);
```

Table Altered

MODIFY: Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax:

ALTER TABLE book_table MODIFY column_name datatype;

Query:

SQL>ALTER TABLE books MODIFY name VARCHAR(200);

ALTER TABLE books ALTER COLUMN name VARCHAR(200);

DROP: Drop command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE book_table DROP COLUMN column_name;

Query:

SQL> ALTER TABLE books DROP COLUMN genre;

RENAME: Rename command is used to re-change the table name.

SQL> RENAME TABLE books TO novels;

Table renamed.

(OR)

ALTER TABLE books RENAME TO novels;

Column Rename

ALTER TABLE books RENAME COLUMN title TO book_title;

TRUNCATE: Truncate command is used to delete a value in the table.

Syntax: TRUNCATE TABLE tablename;

Query:

TRUNCATE TABLE books;

DROP: Deletes the table and all its data permanently.

Syntax:

DROP TABLE tablename;

Query:

DROP TABLE books;

Explanation

- Create a table named **movies**.
- Insert movie details into the table.
- Display all movie records.
- Retrieve movies released in or after the year **2000** and sort them by release year.

Create Table

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author TEXT,
    published_year INTEGER
);
```

Sample Data

INSERT INTO books VALUES (1, 'The Alchemist', 'Paulo Coelho', 1988);

INSERT INTO books VALUES (2, 'The Da Vinci Code', 'Dan Brown', 2003);

INSERT INTO books VALUES (3, 'The Kite Runner', 'Khaled Hosseini', 2003);

INSERT INTO books VALUES (4, 'The Hunger Games', 'Suzanne Collins', 2008);

INSERT INTO books VALUES (5, 'The Fault in Our Stars', 'John Green', 2012);

INSERT INTO books VALUES (6, 'Where the Crawdads Sing', 'Delia Owens', 2018);

SQL Query

Query 1: Display all books records

SQL> SELECT * FROM books;

Resultant Query

id	name	author	published_year
1	The Alchemist	Paulo Coelho	1998
2	The Da Vinci Code	Dan Brown	2003
3	The Kite Runner	Khaled Hosseini	2003
4	The Hunger Games		2008
5	The Fault in Our Stars	John Green	2012
6	Where the Crawdads Sing	Delia Owens	2018

Query 2: Display movies released from the year 2000 onwards

```
SQL> SELECT *
FROM books
WHERE published_year >= 2010
ORDER BY published_year;
```

Resultant Query

id	name	author	release_year
5	The Fault in Our Stars	John Green	2012
6	Where the Crawdads Sing	Delia Owens	2018

RESULT:

Thus, the Movies table was created successfully, records were inserted, all movie details were displayed, and the movies released from the year 2000 onwards were retrieved successfully.

EX. NO:02 Creation of a DML operations using Students Table

DATE: 06.08.2026

Aim

To create a Students table, insert student records, update existing records, delete records, and display the results using DML commands.

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language), it is used to manipulate the data present in the database.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE

INSERT: Insert command is used to add new records/rows into a table.

Syntax

INSERT INTO table_name VALUES (value1, value2, value3, ...);

Query:

SQL> INSERT INTO students VALUES (1, 'Arun', 'CSE', 85);

1 row created.

UPDATE: Update command is used to modify existing records in a table.

Syntax

UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;

Query:

SQL> UPDATE students SET mark = 90 WHERE id = 1;

1 row updated.

DELETE: Delete command is used to remove existing records from a table.

Syntax

DELETE FROM table_name WHERE condition;

Query:

SQL> DELETE FROM students WHERE id = 6;

1 row deleted.

Explanation

- Create a table named students.
- Insert student details into the table using INSERT.
- Update the mark of a particular student using UPDATE.
- Delete a student record using DELETE.
- Display the final table contents.

Create Table

```
CREATE TABLE students (
        id INTEGER PRIMARY KEY,
        name TEXT,
        department TEXT,
        mark INTEGER
       );
```

Sample Data (INSERT)

```
INSERT INTO students VALUES (1, 'Arun', 'CSE', 85);
INSERT INTO students VALUES (2, 'Bala', 'ECE', 78);
INSERT INTO students VALUES (3, 'Charan', 'IT', 92);
INSERT INTO students VALUES (4, 'Divya', 'CSE', 88);
INSERT INTO students VALUES (5, 'Elango', 'MECH', 65);
INSERT INTO students VALUES (6, 'Farida', 'CSE', 74);
```

Query 1: Display all student records after insertion

```
SQL> SELECT * FROM students;
```

Resultant Query

id	name	department	mark
1	Arun	CSE	85
2	Bala	ECE	78
3	Charan	IT	92
4	Divya	CSE	88
5	Elango	MECH	65
6	Farida	CSE	74

Query 2: Update the mark of student with id 1 to 90

SQL> UPDATE students SET mark = 90 WHERE id = 1;

SQL> SELECT * FROM students;

Resultant Query

id	name	department	mark
1	Arun	CSE	90
2	Bala	ECE	78
3	Charan	IT	92
4	Divya	CSE	88
5	Elango	MECH	65
6	Farida	CSE	74

Query 3: Delete the student record with id 6

SQL> DELETE FROM students WHERE id = 6;

SQL> SELECT * FROM students;

Resultant Query

id	name	department	mark
1	Arun	CSE	90
2	Bala	ECE	78
3	Charan	IT	92
4	Divya	CSE	88
5	Elango	MECH	65

RESULT:

Thus, the Students table was created successfully, records were inserted using INSERT, an existing record was modified using UPDATE, a record was removed using DELETE, and the final table contents were displayed successfully.

EX. NO:03 Creation of a DQL operations using Sales Table

DATE: 08.08.2026

Aim

To create a Sales table, insert sales records, and retrieve data using DQL commands such as SELECT, WHERE, ORDER BY, GROUP BY, and HAVING.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to retrieve/query data from the database.

DQL COMMANDS:

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

SELECT: Select command is used to retrieve records from a table.

Syntax

SELECT column1, column2, ... FROM table_name;

Query:

SQL> SELECT * FROM sales;

WHERE: Where clause is used to filter records based on a specified condition.

Syntax

SELECT column1, column2, ... FROM table_name WHERE condition;

Query:

SQL> SELECT * FROM sales WHERE units_sold > 50;

ORDER BY: Order by clause is used to sort the result set in ascending or descending order.

Syntax

SELECT column1, column2, ... FROM table_name ORDER BY column1 [ASC|DESC];

Query:

SQL> SELECT * FROM sales ORDER BY revenue DESC;

GROUP BY: Group by clause is used to group rows that have the same values in specified columns, often used with aggregate functions.

Syntax

SELECT column1, AGG_FUNC(column2) FROM table_name GROUP BY column1;

Query:

SQL> SELECT region, SUM(revenue) FROM sales GROUP BY region;\

HAVING: Having clause is used to filter groups based on a condition, similar to WHERE but applied after GROUP BY.

Syntax

SELECT column1, AGG_FUNC(column2) FROM table_name GROUP BY column1 HAVING condition;

Query:

SQL> SELECT region, SUM(revenue) FROM sales GROUP BY region HAVING SUM(revenue) > 100000;

Explanation

- Create a table named sales.
- Insert sales details into the table.
- Retrieve sales with units sold greater than 50 using WHERE.
- Sort sales by revenue in descending order using ORDER BY.
- Group sales by region and calculate total revenue using GROUP BY.
- Filter regions whose total revenue exceeds 100000 using HAVING.

Create Table

```
CREATE TABLE sales (
        id INTEGER PRIMARY KEY,
        product TEXT,
        region TEXT,
        units_sold INTEGER,
        revenue INTEGER
       );
```

Sample Data

INSERT INTO sales VALUES (1, 'Laptop', 'North', 40, 200000);

INSERT INTO sales VALUES (2, 'Mobile', 'South', 60, 120000);

INSERT INTO sales VALUES (3, 'Tablet', 'North', 70, 105000);

INSERT INTO sales VALUES (4, 'Laptop', 'East', 30, 150000);

INSERT INTO sales VALUES (5, 'Mobile', 'South', 90, 180000);

INSERT INTO sales VALUES (6, 'Tablet', 'East', 20, 30000);

Query 1: Display all sales records

SQL> SELECT * FROM sales;

Resultant Query

id	product	region	units_sold	revenue
1	Laptop	North	40	200000
2	Mobile	South	60	120000
3	Tablet	North	70	105000
4	Laptop	East	30	150000
5	Mobile	South	90	180000
6	Tablet	East	20	30000

Query 2: Display sales with units sold greater than 50 using WHERE

SQL> SELECT * FROM sales WHERE units_sold > 50;

Resultant Query

id	product	region	units_sold	revenue
2	Mobile	South	60	120000
3	Tablet	North	70	105000
5	Mobile	South	90	180000

Query 3: Display sales sorted by revenue in descending order using ORDER BY

SQL> SELECT * FROM sales ORDER BY revenue DESC;

Resultant Query

id	product	region	units_sold	revenue
1	Laptop	North	40	200000
5	Mobile	South	90	180000
4	Laptop	East	30	150000
2	Mobile	South	60	120000
3	Tablet	North	70	105000
6	Tablet	East	20	30000

Query 4: Group sales by region and calculate total revenue using GROUP BY

SQL> SELECT region, SUM(revenue) AS total_revenue FROM sales GROUP BY region;

Resultant Query

region	total_revenue
North	305000
South	300000
East	180000

Query 5: Display regions whose total revenue exceeds 100000 using HAVING

SQL> SELECT region, SUM(revenue) AS total_revenue FROM sales GROUP BY region HAVING SUM(revenue) > 100000;

Resultant Query

region	total_revenue
North	305000
South	300000
East	180000

RESULT:

Thus, the Sales table was created successfully, records were inserted, and data was retrieved successfully using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING clauses.

EX. NO:04 Creation of a TCL operations using Accounts Table

DATE: 10.08.2026

Aim

To perform Transaction Control Language (TCL) operations on an accounts table to manage database transactions using COMMIT, SAVEPOINT, and ROLLBACK commands.

Software Used

Oracle / MySQL / sqlite3

Description

TCL (Transaction Control Language),it deals with transaction within the database

TCL COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT: COMMIT command is used to permanently save all the changes made during the current transaction into the database.

Syntax

COMMIT;

Query

SQL> UPDATE accounts SET balance = balance + 2000 WHERE account_id = 101;

SQL> COMMIT;

ROLLBACK: ROLLBACK command is used to undo the changes made during the current transaction after the last COMMIT.

Syntax

ROLLBACK;

Query

SQL> UPDATE accounts SET balance = balance - 3000 WHERE account_id = 102;

SQL> ROLLBACK;

SAVEPOINT: SAVEPOINT command is used to create a temporary point within a transaction. It allows the transaction to be rolled back to a specific point without cancelling the entire transaction.

Syntax

SAVEPOINT savepoint_name;

Query:

SQL> SAVEPOINT sp1;

Explanation

- Create a table named accounts.
- Insert account details into the table.
- Use COMMIT to permanently save the changes.
- Use ROLLBACK to undo uncommitted changes.
- Use SAVEPOINT to create an intermediate point in a transaction.
- Use ROLLBACK TO SAVEPOINT to undo changes made after a particular savepoint.
- Use SELECT to verify the changes in the table.

Create Table

```
CREATE TABLE accounts (
   account_id INTEGER PRIMARY KEY,
   account_holder TEXT,
   balance INTEGER
);
```

Sample Data

```
INSERT INTO accounts VALUES (101, 'Arun', 10000);

INSERT INTO accounts VALUES (102, 'Priya', 15000);

INSERT INTO accounts VALUES (103, 'Karthik', 20000);

INSERT INTO accounts VALUES (104, 'Divya', 12000);

INSERT INTO accounts VALUES (105, 'Rahul', 18000);
```

Query 1: Display all accounts records

```
SQL> SELECT * FROM accounts;
```

Resultant Query

account_id	account_holder	balance
101	Arun	10000
102	Priya	15000
103	Karthik	20000
104	Divya	12000
105	Rahul	18000

Query 2: Perform COMMIT operation

```
SQL> UPDATE accounts
    SET balance = balance + 2000
    WHERE account_id = 101;
```

SQL> COMMIT;

Resultant Query

account_id	account_holder	balance
101	Arun	12000

Query 3: Perform ROLLBACK operation

```
SQL>  UPDATE accounts
    SET balance = balance - 3000
    WHERE account_id = 102;

SQL> ROLLBACK;
```

Resultant Query

account_id	account_holder	balance
102	Priya	15000

Query 4: Perform SAVEPOINT operation

```
SQL> UPDATE accounts SET balance = balance + 1000 WHERE account_id = 103;

SQL> SAVEPOINT sp1;
```

Resultant Query

account_id	account_holder	balance
101	Arun	12000
102	Priya	15000
103	Karthik	21000
104	Divya	12000
105	Rahul	18000

RESULT:

Thus, the accounts table was created successfully, records were inserted, and TCL operations such as COMMIT, ROLLBACK and SAVEPOINT were successfully performed to manage database transactions.

EX. NO:05 Creation of a JOINS operations using Accounts Table

DATE: 12.08.2026

Aim

To perform different types of SQL Joins on customers and orders tables using INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and SELF JOIN.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Join is used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN

INNER JOIN: Inner Join is used to return the records that have matching values in both tables.

Syntax

```
SELECT column1, column2, ...
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Query

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id
    FROM customers
    INNER JOIN orders
    ON customers.customer_id = orders.customer_id;
```

LEFT JOIN: Left Join is used to return all records from the left table and the matching records from the right table.

Syntax

```
SELECT column1, column2, ...
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Query

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id
    FROM customers
    LEFT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

RIGHT JOIN: Right Join is used to return all records from the right table and the matching records from the left table.

Syntax

```
SELECT column1, column2, ...
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Query

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id
    FROM customers
    RIGHT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

FULL JOIN: Full Join is used to return all records from both tables, including matching and non-matching records.

Syntax

```
SELECT column1, column2, ...
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

Query

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id
    FROM customers
    FULL JOIN orders
    ON customers.customer_id = orders.customer_id;
```

SELF JOIN: Self Join is used to join a table with itself. It is useful when records in the same table are related to each other.

Syntax

```
SELECT a.column1, b.column2
FROM table_name a
JOIN table_name b
ON a.column = b.column;
```

Query

```
SQL> SELECT c1.customer_name AS customer,
    c2.customer_name AS referred_by
    FROM customers c1
    JOIN customers c2
    ON c1.referred_by = c2.customer_id;
```

Explanation

- Create two tables named customers and orders.
- Insert customer details into the customers table.
- Insert order details into the orders table.
- Use INNER JOIN to display matching records from both tables.
- Use LEFT JOIN to display all records from the left table and matching records from the right table.
- Use RIGHT JOIN to display all records from the right table and matching records from the left table.
- Use FULL JOIN to display all records from both tables.
- Use SELF JOIN to join the customers table with itself.

CREATE TABLE

Customers Table

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    city TEXT,
    referred_by INTEGER
);
```

Orders Table

```
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    product TEXT,
    amount INTEGER
);
```

Sample Data

Customers

```
INSERT INTO customers VALUES (301, 'Meena', 'Chennai', NULL);

INSERT INTO customers VALUES (302, 'Ravi', 'Madurai', 301);

INSERT INTO customers VALUES (303, 'Kavya', 'Trichy', 301);

INSERT INTO customers VALUES (304, 'Suresh', 'Salem', 302);

INSERT INTO customers VALUES (305, 'Nisha', 'Pondicherry', 302);
```

Orders

INSERT INTO orders VALUES (501, 301, 'Laptop', 55000);

INSERT INTO orders VALUES (502, 302, 'Mobile', 20000);

INSERT INTO orders VALUES (503, 303, 'Tablet', 30000);

INSERT INTO orders VALUES (504, 304, 'Headphone', 5000);

INSERT INTO orders VALUES (505, 306, 'Keyboard', 2500);

Query 1: Perform INNER JOIN

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id, orders.product, orders.amount
    FROM customers
    INNER JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Resultant Query

customer_id	customer_name	order_id	product	amount
301	Meena	501	Laptop	55000
302	Ravi	502	Mobile	20000
303	Kavya	503	Tablet	30000
304	Suresh	504	Headphone	5000

Query 2: Perform LEFT JOIN

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id, orders.product, orders.amount
    FROM customers
    LEFT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Resultant Query

customer_id	customer_name	order_id	product	amount
301	Meena	501	Laptop	55000
302	Ravi	502	Mobile	20000
303	Kavya	503	Tablet	30000
304	Suresh	504	Headphone	5000
305	Nisha	NULL	NULL	NULL

Query 3: Perform RIGHT JOIN

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id, orders.product, orders.amount
    FROM customers
    RIGHT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Resultant Query

customer_id	customer_name	order_id	product	amount
301	Meena	501	Laptop	55000
302	Ravi	502	Mobile	20000
303	Kavya	503	Tablet	30000
304	Suresh	504	Headphone	5000
NULL	NULL	505	Keyboard	2500

Query 4: Perform FULL JOIN

```
SQL> SELECT customers.customer_id, customers.customer_name,
    orders.order_id, orders.product, orders.amount
    FROM customers
    FULL JOIN orders
```

ON customers.customer_id = orders.customer_id;

Resultant Query

customer_id	customer_name	order_id	product	amount
301	Meena	501	Laptop	55000
302	Ravi	502	Mobile	20000
303	Kavya	503	Tablet	30000
304	Suresh	504	Headphone	5000
305	Nisha	NULL	NULL	NULL
NULL	NULL	505	Keyboard	2500

Query 5: Perform SELF JOIN

SQL> SELECT c1.customer_name AS customer,
c2.customer_name AS referred_by
FROM customers c1
JOIN customers c2
ON c1.referred_by = c2.customer_id;

Resultant Query

customer	referred_by
Ravi	Meena
Kavya	Meena
Suresh	Ravi
Nisha	Ravi

RESULT

Thus, the customers and orders tables were created successfully, records were inserted, and different types of SQL Joins such as INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and SELF JOIN were successfully performed.

EX. NO:06 Implementation of SQL Functions using Campus Events Table

DATE: 14.08.2026

Aim

To perform different SQL functions on a `campus_events` table using Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Functions are used to perform operations on data and return a result.

SQL FUNCTIONS:

- AGGREGATE FUNCTIONS
- STRING FUNCTIONS
- DATE FUNCTIONS

AGGREGATE FUNCTION: Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common Aggregate Functions

- `COUNT()` – Counts the number of records.
- `SUM()` – Calculates the total value.
- `AVG()` – Calculates the average value.
- `MAX()` – Returns the maximum value.
- `MIN()` – Returns the minimum value.

Syntax

SQL>SELECT AGGREGATE_FUNCTION(column_name)FROM table_name;

Query

SQL> SELECT COUNT(*) FROM camp;

STRING FUNCTIONS: String functions are used to perform operations on text or character values.

Common String Functions

- `UPPER()` – Converts text into uppercase.
- `LOWER()` – Converts text into lowercase.
- `LENGTH()` – Returns the length of a string.
- `CONCAT()` – Combines two strings.

Syntax

SELECT STRING_FUNCTION(column_name)FROM table_name;

Query

SQL> SELECT UPPER(event_name)FROM camp:

DATE FUNCTIONS: Date functions are used to perform operations on date values.

Common Date Functions

- `CURRENT_DATE` – Returns the current date.
- `YEAR()` – Returns the year from a date.

- `MONTH()` – Returns the month from a date.
- `DAY()` – Returns the day from a date.

Syntax

SELECT DATE_FUNCTION(date_column)FROM table_name;

Query

SQL> SELECT CURRENT_DATE;

CREATE TABLE

CREATE TABLE camp (event_id INTEGER PRIMARY KEY, event_name TEXT,

coordinator TEXT,participants INTEGER,budget INTEGER, event_date DATE);

Sample Data

INSERT INTO camp VALUES(101, 'Tech Fest', 'Anitha', 150, 25000, '2026-01-15');

INSERT INTO camp VALUES(102, 'Cultural Day', 'Bharath', 200, 30000, '2026-02-20’);

INSERT INTO camp VALUES(103, 'Sports Meet', 'Chitra', 180, 20000, '2026-03-10');

INSERT INTO camp VALUES(104, 'Coding Contest', 'Deepak', 120, 15000, '2026-04-05');

INSERT INTO camp VALUES(105, 'Science Expo', 'Elango', 100, 18000, '2026-05-25');

Query 1: Display all campus event records

SQL> SELECT * FROM camp;

Resultant Query

event_id	event_name	coordinator	participants	budget	event_date
101	Tech Fest	Anitha	150	25000	2026-01-15
102	Cultural Day	Bharath	200	30000	2026-02-20
103	Sports Meet	Chitra	180	20000	2026-03-10
104	Coding Contest	Deepak	120	15000	2026-04-05
105	Science Expo	Elango	100	180000	2026-05-25

Query 2: Count the number of events using COUNT()

SQL> SELECT COUNT(*) AS total_events FROM camp;

Resultant Query

total_events
5

Query 3: Calculate total participants using SUM()

SQL> SELECT SUM(participants) AS total_participants FROM camp;

Resultant Query

total_participants
750

Query 4: Calculate average participants using AVG()

SQL> SELECT AVG(participants) AS average_participants FROM camp;

Resultant Query

average_participants
150

Query 5: Find maximum budget using MAX()

SQL> SELECT MAX(budget) AS maximum_budget FROM camp;

Resultant Query

maximum_budget
30000

Query 6: Find minimum budget using MIN()

SQL> SELECT MIN(budget) AS minimum_budget FROM camp;

Resultant Query

minimum_budget
15000

Query 7: Convert event names into uppercase using UPPER()

SQL> SELECT event_name,UPPER(event_name) AS uppercase_name FROM camp;

Resultant Query

event_name	uppercase_name
Tech Fest	TECH FEST
Cultural Day	CULTURAL DAY
Sports Meet	SPORTS MEET
Coding Contest	CODING CONTEST
Science Expo	SCIENCE EXPO

Query 8: Convert coordinator names into lowercase using LOWER()

SQL> SELECT coordinator,LOWER(coordinator) AS lowercase_name FROM camp;

Resultant Query

coordinator	lowercase_name
Anitha	anitha
Bharath	bharath
Chitra	chitra
Deepak	deepak
Elango	elango

Query 9: Find the length of event names using LENGTH()

SQL> SELECT event_name, LENGTH(event_name) AS name_length FROM camp;

Resultant Query

event_name	name_length
Tech Fest	9
Cultural Day	12
Sports Meet	11
Coding Contest	14
Science Expo	12

Query 10: Display the current date using CURRENT_DATE

SQL> SELECT CURRENT_DATE AS current_date;

Resultant Query

current_date
Current system date

Query 11: Extract the year from the event date

SQL> SELECT event_name,YEAR(event_date) AS event_year FROM camp;

Resultant Query

event_name	event_year
Tech Fest	2026
Cultural Day	2026
Sports Meet	2026
Coding Contest	2026
Science Expo	2026

Query 12: Extract the month from the event date

SQL> SELECT event_name,MONTH(event_date) AS event_month FROM camp;

Resultant Query

event_name	event_month
Tech Fest	1
Cultural Day	2
Sports Meet	3
Coding Contest	4
Science Expo	5

RESULT

Thus, the ‘camp’ table was created successfully, records were inserted, and different SQL Functions such as Aggregate Functions, String Functions, and Date Functions were successfully performed.

EX. NO:07 Implementation of Subqueries and Nested Queries Using Library Table

DATE: 07.09.2026

Aim

To perform different Subqueries and Nested Queries on a library table using Single-Row Subqueries, Multi-Row Subqueries, and Correlated Subqueries.

Software Used

Oracle / MySQL / SQLite3

Description

A **Subquery** or **Nested Query** is a query placed inside another SQL query (such as SELECT, INSERT, UPDATE, or DELETE). The inner query executes first, and its result is used by the outer query.

TYPES OF SUBQUERIES:

- SINGLE-ROW SUBQUERIES ·
- MULTI-ROW SUBQUERIES ·
- CORRELATED SUBQUERIES

SINGLE-ROW SUBQUERY: Returns a single row/value to the outer query. Uses operators like =, >, <, >=, <=, <>.

Syntax:

SELECT column_name FROM table_name WHERE column_name OPERATOR (SELECT column_name FROM table_name WHERE condition);

Query:

SQL> SELECT * FROM library WHERE price > (SELECT AVG(price) FROM library);

MULTI-ROW SUBQUERY: Returns multiple rows to the outer query. Uses operators like IN, ANY, ALL.

Syntax:

SELECT column_name FROM table_name WHERE column_name IN (SELECT column_name FROM table_name WHERE condition);

Query:

SQL> SELECT * FROM library WHERE category IN (SELECT category FROM library WHERE price > 600);

CORRELATED SUBQUERY: Evaluates once for each row processed by the outer query using values from the outer query.

Syntax:

SELECT c1.column_name FROM table_name c1 WHERE c1.column_name > (SELECT AVG(c2.column_name) FROM table_name c2 WHERE c2.category = c1.category);

Query:

SQL> SELECT l1.book_name, l1.price FROM library l1 WHERE l1.price > (SELECT AVG(l2.price) FROM library l2 WHERE l2.category = l1.category);

Explanation ·

- Create a table named library. ·
- Insert book details into the table. ·
- Display all library records. ·
- Retrieve books using Single-Row Subqueries, Multi-Row Subqueries, and Correlated Subqueries based on price and category.

CREATE TABLE

CREATE TABLE library (book_id INTEGER PRIMARY KEY, book_name TEXT, category TEXT, price INTEGER, pub_date DATE);

Sample Data

INSERT INTO library VALUES(401, 'Database Systems', 'Tech', 650, '2021-03-10');

INSERT INTO library VALUES(402, 'Python Programming', 'Tech', 450, '2022-06-15');

INSERT INTO library VALUES(403, 'World History', 'History', 550, '2019-11-20');

INSERT INTO library VALUES(404, 'Financial Accounting', 'Finance', 700, '2023-01-05');

INSERT INTO library VALUES(405, 'Data Structures', 'Tech', 500, '2020-08-12');

SQL QUERY

Query 1: Display all library records

SQL> SELECT * FROM library;

Resultant Query

book_id	book_name	category	price	pub_date
401	Database Systems	Tech	650	2021-03-10
402	Python Programming	Tech	450	2022-06-15
403	World History	History	550	2019-11-20
404	Financial Accounting	Finance	700	2023-01-05
405	Data Structures	Tech	500	2020-08-12

Query 2: Find books priced higher than the average price (Single-Row Subquery)

SQL> SELECT book_name, price FROM library WHERE price > (SELECT AVG(price) FROM library);

Resultant Query

book_name	Price
Database Systems	650
Financial Accounting	700

Query 3: Display details of the most expensive book(Single-Row Subquery)

SQL> SELECT * FROM library WHERE price = (SELECT MAX(price) FROM library);

Resultant Query

book_id	book_name	category	Price
404	Financial Accounting	Finance	700

Query 4: **Find books priced equal to the cheapest book in the Tech category (Single-Row Subquery)**

SQL> SELECT book_name, category, price FROM library WHERE price = (SELECT MIN(price) FROM library WHERE category = 'Tech');

Resultant Query

book_name	category	price
Python Programming	Tech	450

Query 5: Find books belonging to categories that have books priced above 600 (Multi-Row Subquery using IN)

SQL> SELECT book_name, category FROM library WHERE category IN (SELECT category FROM library WHERE price > 600);

Resultant Query

book_name	category
Database Systems	Tech
Python Programming	Tech
Financial Accounting	Finance
Data Structures	Tech

Query 6: Find books whose price is greater than ANY book in the History category (Multi-Row Subquery using IN)

SQL> SELECT book_name, price FROM library WHERE price > ANY (SELECT price FROM library WHERE category = 'History');

Resultant Query

book_name	price
Database Systems	650
Financial Accounting	700

Query 7: Display books priced lower than the average price of all books (Nested Query)

SQL> SELECT book_name, price FROM library WHERE price < (SELECT AVG(price) FROM library);

Resultant Query

book_name	price
Python Programming	450
World History	550

Data Structures	500

Query 8: Find books whose price is greater than ALL books in the Tech category (Multi-Row Subquery using ALL)

SQL> SELECT book_name, price FROM library WHERE price > ALL (SELECT price FROM library WHERE category = 'Tech');

Resultant Query

book_name	price
Financial Accounting	700

Query 9: Find books priced higher than the average price of their respective category (Correlated Subquery)

SQL> SELECT l1.book_name, l1.category, l1.price FROM library l1 WHERE l1.price > (SELECT AVG(l2.price) FROM library l2 WHERE l2.category = l1.category);

Resultant Query

book_name	category	price
Database systems	Tech	650

Query 10: Check if any book belongs to the Finance category using EXISTS

SQL> SELECT book_id, book_name FROM library l1 WHERE EXISTS (SELECT 1 FROM library l2 WHERE l2.category = 'Finance' AND l1.book_id = l2.book_id);

Resultant Query

book_id	book_name
404	Financial Accounting

RESULT

Thus, the ‘Library’ table was created successfully, records were inserted, and different Subqueries and Nested Queries (Single-Row, Multi-Row, and Correlated Subqueries) were successfully performed.

EX. NO:08 Implementation of Views, Indexes, and Sequences using EMPLOYEE Table

DATE: 10.09.2026

Aim

To create an `employee` table and perform operations related to Database Objects such as Views, Indexes, and Sequences.

Software Used

Oracle / MySQL / SQLite3

Description

Database objects are built-in structures used to hold, manage, and optimize access to data in a relational database.

DATABASE OBJECTS:

- VIEW
- INDEX
- SEQUENCE

VIEW: A virtual table based on the result-set of an SQL statement. It simplifies complex queries and enhances data security.

Syntax:

CREATE VIEW view_name AS SELECT column1, column2 FROM table_name WHERE condition;

Query:

SQL> CREATE VIEW high_salary_emp AS SELECT emp_name, department, salary FROM employee WHERE salary > 50000;

INDEX: A database object created on table columns to speed up retrieval operations.

Syntax:

CREATE INDEX index_name ON table_name (column_name);

Query:

SQL> CREATE INDEX idx_dept ON employee (department);

SEQUENCE: An object used to generate a sequence of numeric values automatically (commonly used for primary keys).

Syntax:

CREATE SEQUENCE sequence_name START WITH initial_value INCREMENT BY step_value;

Query:

SQL> CREATE SEQUENCE emp_seq START WITH 501 INCREMENT BY 1;

Explanation

- Create a table named employee.
- Insert records using an auto-generated sequence or manually.
- Create a View to filter employees earning above a threshold.
- Create an Index on department column to optimize search performance.

CREATE TABLE

```
CREATE TABLE employee (
    emp_id INTEGER PRIMARY KEY,
    emp_name TEXT,
```

```
    department TEXT,
    salary INTEGER,
    hire_date DATE
);
```

Sample Data

INSERT INTO employee VALUES (501, 'Arun', 'IT', 65000, '2021-03-10');

INSERT INTO employee VALUES (502, 'Bhavana', 'HR', 45000, '2022-06-15');

INSERT INTO employee VALUES (503, 'Charan', 'IT', 75000, '2019-11-20');

INSERT INTO employee VALUES (504, 'Divya', 'Finance', 48000, '2023-01-05');

INSERT INTO employee VALUES (505, 'Eshwar', 'HR', 52000, '2020-08-12');

SQL Query

Query 1: Display all employee records

SQL> SELECT * FROM employee;

Resultant Query

emp_id	emp_name	department	salary	hire_date
501	Arun	IT	65000	2021-03-10
502	Bhavana	HR	45000	2022-06-15
503	Charan	IT	75000	2019-11-20
504	Divya	Finance	48000	2023-01-05
505	Eshwar	HR	52000	2020-08-12

Query 2: **Create a View for employees earning above 50,000**

SQL> CREATE VIEW high_salary_emp AS SELECT emp_name, department, salary FROM employee WHERE salary > 50000;

View created.

Query 3: Display records from the created View

SQL> SELECT * FROM high_salary_emp;

Resultant Query

emp_name	department	salary
Arun	IT	65000
Charan	IT	75000
Eshwar	HR	52000

Query 4: Create an Index on the department column

SQL> CREATE INDEX idx_dept ON employee (department);

Index created.

Query 5: Create a Sequence for employee IDs

SQL> CREATE SEQUENCE emp_seq START WITH 506 INCREMENT BY 1;

Sequence created.

RESULT:

Thus, the Employee table was created successfully, records were inserted, and database objects such as Views, Indexes, and Sequences were created and performed successfully.

EX. NO:09 Implementation of Stored Procedures and Functions using Inventory Table

DATE: 14.09.2026

Aim

To create an inventory table and perform operations using PL/SQL Stored Procedures and Functions.

Software Used

Oracle / MySQL / SQLite3

Description

Stored Procedures and Functions are subprograms composed of a set of SQL statements that are compiled and stored in the database for reuse and modular execution.

PL/SQL OBJECTS:

- STORED PROCEDURE
- STORED FUNCTION

STORED PROCEDURE: A subprogram designed to perform a specific action or business logic. It can accept parameters (IN, OUT, IN OUT) and may or may not return values directly.

Syntax:

```
SQL> CREATE OR REPLACE PROCEDURE procedure_name (param_name IN datatype) AS
BEGIN
  -- Executable statements
END;
/
```

STORED FUNCTION: A subprogram created to perform calculations and compute values. A function MUST return a value using the RETURN clause.

Syntax:

```
SQL> CREATE OR REPLACE FUNCTION function_name (param_name IN datatype)
RETURN return_datatype IS
  result_val return_datatype;
BEGIN
  -- Executable statements
  RETURN result_val;
END;
/
```

Explanation

- Create a table named inventory.
- Insert product inventory details into the table.
- Create a Stored Procedure to update item stock quantity based on item ID.
- Create a Stored Function to calculate the total stock valuation (quantity × price).

Create Table

```
SQL> CREATE TABLE inventory
   ( item_id INTEGER PRIMARY
   KEY,
   item_name TEXT,
   category TEXT,
   quantity INTEGER,
   unit_price INTEGER
);
```

Sample Data

INSERT INTO inventory VALUES (601, 'Keyboard', 'Electronics', 50, 1200);
INSERT INTO inventory VALUES (602, 'Mouse', 'Electronics', 80, 600);
INSERT INTO inventory VALUES (603, 'Monitor', 'Electronics', 25, 15000);
INSERT INTO inventory VALUES (604, 'Desk Chair', 'Furniture', 15, 8000);
INSERT INTO inventory VALUES (605, 'USB Cable', 'Accessories', 120, 250);

SQL Query

Query 1: Display all inventory records

SQL> SELECT * FROM inventory;

Resultant Query

item_id	item_name	category	quantity	unit_price
601	Keyboard	Electronics	50	1200
602	Mouse	Electronics	80	600
603	Monitor	Electronics	25	15000
604	Desk Chair	Furniture	15	8000
605	USB Cable	Accessories	120	250

Query 2: Create a Stored Procedure to update item stock quantity

```
SQL> CREATE OR REPLACE PROCEDURE update_stock (
      p_item_id IN INTEGER,
      p_new_qty IN INTEGER
    ) AS
    BEGIN
      UPDATE inventory
      SET quantity = p_new_qty
      WHERE item_id = p_item_id;
    END;
    /
```

Resultant Query

Procedure created.

Query 3: Execute the Stored Procedure to update stock for Keyboard (item_id 601)

```
SQL> EXEC update_stock(601, 75);
```

Resultant Query

PL/SQL procedure successfully completed.

Query 4: Display updated inventory record

```
SQL> SELECT * FROM inventory WHERE item_id = 601;
```

Resultant Query

item_id	item_name	category	quantity	unit_price
601	Keyboard	Electronics	75	1200

Query 5: Create a Stored Function to calculate the total inventory valuation of an item

```
SQL> CREATE OR REPLACE FUNCTION calculate_total_value
      ( p_item_id IN INTEGER
    ) RETURN INTEGER IS
      v_total INTEGER;
    BEGIN
      SELECT (quantity * unit_price) INTO v_total
      FROM inventory
      WHERE item_id = p_item_id;
      RETURN v_total;
    END;
    /
```

Resultant Query

Function created.

Query 6: Call the Stored Function to calculate total stock value for Monitor (item_id 603)

```
SQL> SELECT item_name, calculate_total_value(603) AS total_stock_value FROM inventory
WHERE item_id = 603;
```

Resultant Query

item_name	total_stock_value
Monitor	375000

RESULT:

Thus, the Inventory table was created successfully, records were inserted, and Stored Procedures and Functions were created and executed successfully.

EX. NO:10 Implementation of Triggers using account Table

DATE: 22.09.2026

Aim

To create an account table, an account_audit log table, and perform operations using Database Triggers (BEFORE and AFTER triggers).

Software Used

Oracle / MySQL / SQLite3

Description

A Database Trigger is a special type of stored procedure that automatically executes (fires) when a specified event (INSERT, UPDATE, or DELETE) occurs on a table or view.

TYPES OF TRIGGERS:

- BEFORE TRIGGER (Executes before the triggering statement/action)
- AFTER TRIGGER (Executes after the triggering statement/action)
- ROW-LEVEL TRIGGER (Fires once for each affected row)
- STATEMENT-LEVEL TRIGGER (Fires once for the SQL statement as a whole)

BEFORE TRIGGER SYNTAX:

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT OR UPDATE OR DELETE ON table_name
FOR EACH ROW
BEGIN
  -- Executable statements
END;
/
```

AFTER TRIGGER SYNTAX:

```
CREATE OR REPLACE TRIGGER trigger_name
AFTER INSERT OR UPDATE OR DELETE ON table_name
FOR EACH ROW
BEGIN
  -- Executable statements
END;
/
```

Explanation

- Create a primary table named account and an audit logging table named account_audit.
- Insert initial bank account details into the table.
- Create a BEFORE UPDATE trigger to validate that account balances cannot drop below zero.
- Create an AFTER UPDATE trigger to log all balance modifications into the audit table automatically.

CREATE TABLES

```
CREATE TABLE account (
   acc_no INTEGER PRIMARY KEY,
   holder_name TEXT,
   acc_type TEXT,
   balance INTEGER
);

CREATE TABLE account_audit (
   audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
   acc_no INTEGER,
   old_balance INTEGER,
```

```
    new_balance INTEGER,
    change_date DATE
);
```

Sample Data

```
INSERT INTO account VALUES (701, 'Anand', 'Savings', 25000);
INSERT INTO account VALUES (702, 'Bhavna', 'Current', 50000);
INSERT INTO account VALUES (703, 'Cyril', 'Savings', 15000);
INSERT INTO account VALUES (704, 'Deepika', 'Savings', 30000);
INSERT INTO account VALUES (705, 'Elango', 'Current', 42000);
```

SQL Query

Query 1: Display all account records

```
SQL> SELECT * FROM account;
```

Resultant Query

acc_no	holder_name	acc_type	balance
701	Anand	Savings	25000
702	Bhavna	Current	50000
703	Cyril	Savings	15000
704	Deepika	Savings	30000
705	Elango	Current	42000

Query 2: Create a BEFORE UPDATE Trigger to restrict negative balance updates

```
SQL> CREATE OR REPLACE TRIGGER check_min_balance
    BEFORE UPDATE OF balance ON account
    FOR EACH ROW
```

```
BEGIN
  IF :NEW.balance < 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Balance cannot be negative!');
  END IF;
END;
/
```

Resultant Query

Trigger created.

Query 3: Create an AFTER UPDATE Trigger to log balance changes into the account_audit table

```
SQL> CREATE OR REPLACE TRIGGER log_balance_update
  AFTER UPDATE OF balance ON account
  FOR EACH ROW
  BEGIN
    INSERT INTO account_audit (acc_no, old_balance, new_balance, change_date)
    VALUES (:OLD.acc_no, :OLD.balance, :NEW.balance, CURRENT_DATE);
  END;
  /
```

Resultant Query

Trigger created.

Query 4: Perform an UPDATE statement to trigger the log_balance_update trigger

```
SQL> UPDATE account SET balance = balance + 5000 WHERE acc_no = 701;
```

Resultant Query

1 row updated.

Query 5: Display updated record from account table

SQL> SELECT * FROM account WHERE acc_no = 701;

Resultant Query

acc_no	holder_name	acc_type	balance
701	Anand	Savings	30000

Query 6: Display audit log generated automatically by the AFTER trigger

SQL> SELECT * FROM account_audit;

Resultant Query

audit_id	acc_no	old_balance	new_balance	change_date
1	701	25000	30000	2026-09-28

RESULT:

Thus, the Account and Account Audit tables were created successfully, records were inserted, and Database Triggers (BEFORE and AFTER) were successfully created, executed, and verified.

EX. NO:11 Implementation of Cursors using orders Table

DATE: 26.09.2026

Aim

To create an orders table and write a PL/SQL block to retrieve and process records sequentially row-by-row using Cursors (Implicit and Explicit Cursors).

Software Used

Oracle / MySQL / SQLite3

Description

A Cursor is a pointer to a context area (a private SQL area allocated by the database) that holds the rows returned by a SQL query. It allows row-by-row processing of multi-row query result sets.

TYPES OF CURSORS:

- IMPLICIT CURSOR (Automatically created by Oracle for DML statements and single-row SELECT queries)
- EXPLICIT CURSOR (User-defined cursor created for processing multi-row SELECT queries)

EXPLICIT CURSOR STEPS:

1. DECLARE (Define cursor name and SQL query)
2. OPEN (Allocate memory and execute query)
3. FETCH (Retrieve row data into PL/SQL variables)
4. CLOSE (Release allocated memory)

CURSOR ATTRIBUTES:

- %FOUND – Returns TRUE if fetch was successful.
- %NOTFOUND – Returns TRUE if no row was fetched.
- %ROWCOUNT – Returns total number of rows fetched so far.
- %ISOPEN – Returns TRUE if cursor is open.

Syntax:

```
DECLARE
  CURSOR cursor_name IS
    SELECT column1, column2 FROM table_name;
  v_col1 table_name.column1%TYPE;
  v_col2 table_name.column2%TYPE;
BEGIN
  OPEN cursor_name;
  LOOP
    FETCH cursor_name INTO v_col1, v_col2;
    EXIT WHEN cursor_name%NOTFOUND;
    -- Statements
  END LOOP;
  CLOSE cursor_name;
END;
/
```

Explanation

- Create a table named orders.
- Insert sample order transactions.
- Use an Implicit Cursor (%ROWCOUNT) to check affected rows after an update operation.
- Use an Explicit Cursor to fetch each order row by row and calculate total bill with a 10% tax discount/addition.

Create Table

```
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    item_name TEXT,
    bill_amount INTEGER,
    order_date DATE
);
);
```

Sample Data

SQL

```
INSERT INTO orders VALUES (801, 'Arun', 'Laptop', 55000, '2026-01-10');
INSERT INTO orders VALUES (802, 'Bhavana', 'Tablet', 25000, '2026-02-15');
INSERT INTO orders VALUES (803, 'Charan', 'Smartwatch', 8000, '2026-03-20');
INSERT INTO orders VALUES (804, 'Divya', 'Headphones', 4000, '2026-04-12');
INSERT INTO orders VALUES (805, 'Elango', 'Monitor', 18000, '2026-05-18');
```

SQL Query

Query 1: Display all order records

```
SQL> SELECT * FROM orders;
```

Resultant Query

order_id	customer_name	item_name	bill_amount	order_date
801	Arun	Laptop	55000	2026-01-10
802	Bhavana	Tablet	25000	2026-02-15
803	Charan	Smartwatch	8000	2026-03-20
		Headphones	4000	2026-04-12
805	Elango	Monitor	18000	2026-05-18

Query 2: Demonstrate Implicit Cursor using SQL%ROWCOUNT after UPDATE operation

```
SQL> DECLARE
       v_rows INTEGER;
     BEGIN
       UPDATE orders SET bill_amount = bill_amount + 500 WHERE bill_amount < 10000;
       IF SQL%FOUND THEN
         v_rows := SQL%ROWCOUNT;
         DBMS_OUTPUT.PUT_LINE(v_rows || ' order records updated.');
       END IF;
     END;
     /
```

Resultant Query

```
2 order records updated.
PL/SQL procedure successfully completed.
/SQL procedure successfully completed.
```

Query 3: Demonstrate Explicit Cursor to fetch each order and process total bill sequentially

```
SQL
SQL> DECLARE
       CURSOR order_cursor IS
         SELECT order_id, customer_name, bill_amount FROM orders;
       v_id orders.order_id%TYPE;
       v_name orders.customer_name%TYPE;
       v_amount orders.bill_amount%TYPE;
       v_final_amount NUMBER;
     BEGIN
       OPEN order_cursor;
       LOOP
```

```
        FETCH order_cursor INTO v_id, v_name, v_amount;
        EXIT WHEN order_cursor%NOTFOUND;
        v_final_amount := v_amount * 1.10; -- Adding 10% tax
        DBMS_OUTPUT.PUT_LINE('Order ID: ' || v_id || ' | Customer: ' || v_name || ' | Total with
Tax: ' || v_final_amount);
    END LOOP;
    CLOSE order_cursor;
END;
/
```

Resultant Query

Order ID: 801 | Customer: Arun | Total with Tax: 60500
Order ID: 802 | Customer: Bhavana | Total with Tax: 27500
Order ID: 803 | Customer: Charan | Total with Tax: 9350
Order ID: 804 | Customer: Divya | Total with Tax: 4950
Order ID: 805 | Customer: Elango | Total with Tax: 19800

PL/SQL procedure successfully completed.

RESULT

Thus, the Orders table was created successfully, records were inserted, and PL/SQL blocks using Implicit and Explicit Cursors were executed and verified successfully.

EX. NO:12 Implementation of PL/SQL Exception Handling using bank_account Table
DATE: 26.09.2026

Aim

To create a bank_account table and write PL/SQL blocks to demonstrate Exception Handling using predefined system exceptions, user-defined exceptions, and OTHERS.

Software Used

Oracle / MySQL / SQLite3

Description

An Exception is an error or abnormal condition that occurs during the execution of a PL/SQL program. Exception Handling ensures that the program catches errors gracefully without terminating unexpectedly.

TYPES OF EXCEPTIONS:

- PREDEFINED SYSTEM EXCEPTIONS (Automatically raised by Oracle for common errors like NO_DATA_FOUND, ZERO_DIVIDE, TOO_MANY_ROWS)
- USER-DEFINED EXCEPTIONS (Explicitly defined by the programmer using EXCEPTION declaration and raised using the RAISE statement)

PREDEFINED EXCEPTION SYNTAX:

```
BEGIN
  -- Executable statements
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    -- Error handling code
  WHEN OTHERS THEN
    -- Default error handling
END;
```

/

USER-DEFINED EXCEPTION SYNTAX:

SQL

```
DECLARE
  exception_name EXCEPTION;
BEGIN
  IF condition THEN
    RAISE exception_name;
  END IF;
EXCEPTION
  WHEN exception_name THEN
    -- Error handling code
END;
/
```

Explanation

- Create a table named bank_account.
- Insert initial customer account records.
- Handle a predefined exception (NO_DATA_FOUND) when querying a non-existent account number.
- Handle a user-defined exception (insufficient_funds) when attempting to withdraw an amount greater than the current balance.

Create Table

```
CREATE TABLE bank_account
   ( acc_no INTEGER PRIMARY
   KEY,
   holder_name TEXT,
   balance INTEGER
);
```

Sample Data

INSERT INTO bank_account VALUES (901, 'Anand', 15000);
INSERT INTO bank_account VALUES (902, 'Bhavna', 8000);
INSERT INTO bank_account VALUES (903, 'Cyril', 25000);
INSERT INTO bank_account VALUES (904, 'Deepika', 3000);
INSERT INTO bank_account VALUES (905, 'Elango', 42000);

SQL Query

Query 1: Display all bank account records

SQL> SELECT * FROM bank_account;

Resultant Query

acc_no	holder_name	balance
901	Anand	15000
902	Bhavna	8000
903	Cyril	25000
904	Deepika	3000
905	Elango	42000

Query 2: Demonstrate Predefined System Exception (NO_DATA_FOUND)

```
DECLARE
    v_name bank_account.holder_name%TYPE;
    v_balance bank_account.balance%TYPE;
  BEGIN
    SELECT holder_name, balance INTO v_name, v_balance
    FROM bank_account
```

```
    WHERE acc_no = 999;

    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ' | Balance: ' || v_balance);
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Error: Account number 999 does not exist.');
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');
  END;
  /
```

Resultant Query

```
Error: Account number 999 does not exist.
PL/SQL procedure successfully completed.
```

Query 3: Demonstrate User-Defined Exception (insufficient_funds)

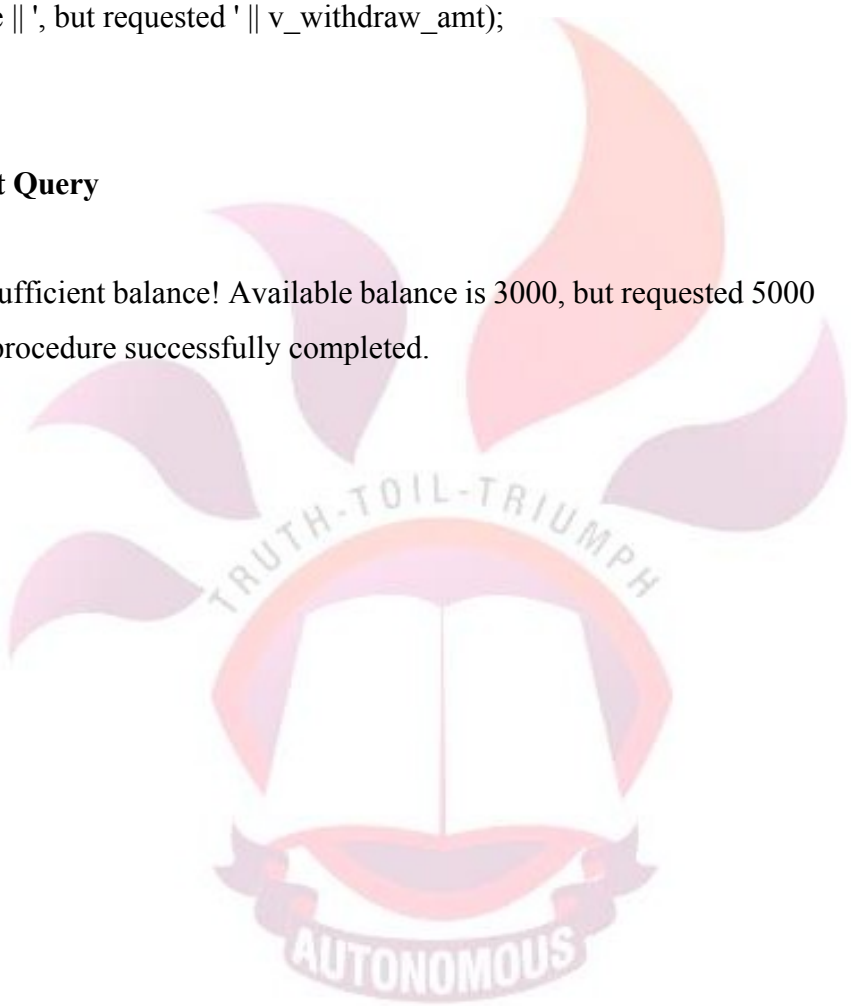
```
DECLARE
    v_acc_no INTEGER := 904;
    v_withdraw_amt INTEGER := 5000;
    v_balance bank_account.balance%TYPE;
    insufficient_funds EXCEPTION;
  BEGIN
    SELECT balance INTO v_balance FROM bank_account WHERE acc_no = v_acc_no;

    IF v_withdraw_amt > v_balance THEN
      RAISE insufficient_funds;
    ELSE
      UPDATE bank_account SET balance = balance - v_withdraw_amt WHERE acc_no =
v_acc_no;
```

```
        DBMS_OUTPUT.PUT_LINE('Withdrawal successful. New Balance: ' || (v_balance -
v_withdraw_amt));
      END IF;
    EXCEPTION
      WHEN insufficient_funds THEN
        DBMS_OUTPUT.PUT_LINE('Error: Insufficient balance! Available balance is ' ||
v_balance || ', but requested ' || v_withdraw_amt);
    END;
    /
```

Resultant Query

```
Error: Insufficient balance! Available balance is 3000, but requested 5000
PL/SQL procedure successfully completed.
```



RESULT:

Thus, the Bank Account table was created successfully, records were inserted, and PL/SQL blocks using Predefined and User-Defined Exception Handling were executed and verified successfully.

EX. NO:13 Implementation of Database Normalization (1NF, 2NF, 3NF, BCNF) using student_courses Table

DATE: 28.09.2026

Aim

To demonstrate the process of Database Normalization (1NF, 2NF, 3NF, and BCNF) on an unnormalized table by decomposing it step-by-step into well-structured relational tables to eliminate data redundancy and insertion, update, and deletion anomalies.

Software Used

Oracle / MySQL / SQLite3

Description

Normalization is a systematic approach to organizing data in a database. It reduces data redundancy and prevents data modification anomalies (Insertion, Update, Deletion) by decomposing large tables into smaller, well-structured tables.

NORMALIZATION FORMS:

- **FIRST NORMAL FORM (1NF):** Eliminates repeating groups and ensures every column contains atomic (indivisible) values.
- **SECOND NORMAL FORM (2NF):** Must be in 1NF and must eliminate partial dependencies (every non-key column must depend on the entire composite primary key).
- **THIRD NORMAL FORM (3NF):** Must be in 2NF and must eliminate transitive dependencies (non-key columns must not depend on other non-key columns).
- **BOYCE-CODD NORMAL FORM (BCNF):** An extension of 3NF where for every functional dependency $X \rightarrow Y$, X must be a super key.

Explanation

- Start with an unnormalized table containing multi-valued attributes (courses, professors).
- **1NF Conversion:** Split multi-valued rows so each cell contains a single atomic value.
- **2NF Conversion:** Remove partial dependency on composite key {student_id, course_id} by separating course details.
- **3NF / BCNF Conversion:** Remove transitive/determinant dependencies by creating distinct entities for Students, Courses, Instructors, and Enrollments.

Unnormalized Table (UNF)

In an unnormalized state, a single record contains comma-separated values (repeating groups).

student_id	student_name	**courses**	**professors**	**dept_name**
1001	Anand	DBMS, OS	Dr. Raman, Dr. Priya	Computer Science
1002	Bhavna	Networks, DBMS	Dr. Kumar, Dr. Raman	Information Tech

Step 1: First Normal Form (1NF)

Ensure all attributes contain atomic values (single values per cell).

Create Table (1NF)

```
SQL> CREATE TABLE student_courses_1nf
   ( student_id INTEGER,
   student_name TEXT,
   dept_id INTEGER,
   dept_name TEXT,
```

```
    course_id TEXT,
    course_name TEXT,
    professor_id INTEGER,
    professor_name TEXT,
    PRIMARY KEY (student_id, course_id)
);
```

Sample Data (1NF)

```
INSERT INTO student_courses_1nf VALUES (1001, 'Anand', 10, 'Computer Science', 'CS101', 'DBMS', 501, 'Dr. Raman');
INSERT INTO student_courses_1nf VALUES (1001, 'Anand', 10, 'Computer Science', 'CS102', 'OS', 502, 'Dr. Priya');
INSERT INTO student_courses_1nf VALUES (1002, 'Bhavna', 20, 'Information Tech', 'IT101', 'Networks', 503, 'Dr. Kumar');
INSERT INTO student_courses_1nf VALUES (1002, 'Bhavna', 20, 'Information Tech', 'IT101', 'DBMS', 501, 'Dr. Raman');
```

Resultant Query (1NF)

student_id	student_name	dept_id	dept_name	course_id	course_name	professor_id	professor_name
1001	Anand	10	Computer Science	CS101	DBMS	501	Dr. Raman
1001	Anand	10	Computer Science	CS102	OS	502	Dr. Priya

Step 2: Second Normal Form (2NF)

Eliminate Partial Dependencies. Non-key attributes like student_name depend only on student_id, and course_name depends only on course_id.

Create Tables (2NF)

Table 1: Student Details

```
CREATE TABLE students_2nf
   ( student_id INTEGER PRIMARY
   KEY, student_name TEXT,
   dept_id INTEGER,
   dept_name TEXT
);
```

Table 2: Course Details

```
CREATE TABLE courses_2nf
   ( course_id TEXT PRIMARY KEY,
   course_name TEXT,
   professor_id INTEGER,
   professor_name TEXT
);
```

Table 3: Student-Course Enrollment (Junction Table)

```
CREATE TABLE enrollments_2nf
   ( student_id INTEGER,
   course_id TEXT,
   PRIMARY KEY (student_id, course_id),
   FOREIGN KEY (student_id) REFERENCES students_2nf(student_id),
   FOREIGN KEY (course_id) REFERENCES courses_2nf(course_id)
);
```

Resultant Query (2NF - students_2nf)

student_id	student_name	dept_id	dept_name
1001	Anand	10	Computer Science
1002	Bhavna	20	Information Tech

Resultant Query (2NF - courses_2nf)

course_id	course_name	professor_id	professor_name
CS101	DBMS	501	Dr. Raman
CS102	OS	502	Dr. Priya
IT101	Networks	503	Dr. Kumar

Resultant Query (2NF - enrollments_2nf)

student_id	course_id
1001	CS101
1001	CS102
1002	IT101
1002	CS101

Step 3: Third Normal Form (3NF) & BCNF

Eliminate Transitive Dependencies. dept_name depends on dept_id (not directly on student_id), and professor_name depends on professor_id.

Create Normalized Tables (3NF / BCNF)

1. Department Table

```
CREATE TABLE department
   (  dept_id INTEGER PRIMARY
   KEY,
   dept_name TEXT
);
```

2. Student Table

```
CREATE TABLE student (
   student_id INTEGER PRIMARY KEY,
   student_name TEXT,
   dept_id INTEGER,
   FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

3. Professor Table

```
CREATE TABLE professor (
   professor_id INTEGER PRIMARY KEY,
   professor_name TEXT
);
```

4. Course Table

```
CREATE TABLE course (
   course_id TEXT PRIMARY KEY,
   course_name TEXT,
   professor_id INTEGER,
```

```
    FOREIGN KEY (professor_id) REFERENCES professor(professor_id)
);
```

5. Enrollment Table

```
CREATE TABLE enrollment
    ( student_id INTEGER,
    course_id TEXT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);
```

Sample Insert Data

```
INSERT INTO department VALUES (10, 'Computer Science');
INSERT INTO department VALUES (20, 'Information Tech');

INSERT INTO student VALUES (1001, 'Anand', 10);
INSERT INTO student VALUES (1002, 'Bhavna', 20);

INSERT INTO professor VALUES (501, 'Dr. Raman');
INSERT INTO professor VALUES (502, 'Dr. Priya');
INSERT INTO professor VALUES (503, 'Dr. Kumar');

INSERT INTO course VALUES ('CS101', 'DBMS', 501);
INSERT INTO course VALUES ('CS102', 'OS', 502);
INSERT INTO course VALUES ('IT101', 'Networks', 503);

INSERT INTO enrollment VALUES (1001, 'CS101');
INSERT INTO enrollment VALUES (1001, 'CS102');
INSERT INTO enrollment VALUES (1002, 'IT101');
INSERT INTO enrollment VALUES (1002, 'CS101');
```

SQL Query: Verify normalized tables via JOIN query

```
SQL> SELECT s.student_id, s.student_name, d.dept_name, c.course_name, p.professor_name
    FROM enrollment e
    JOIN student s ON e.student_id = s.student_id
    JOIN department d ON s.dept_id = d.dept_id
    JOIN course c ON e.course_id = c.course_id
    JOIN professor p ON c.professor_id = p.professor_id;
```

Resultant Query

student_id	student_name	dept_name	course_name	professor_name
1001	Anand	Computer Science	DBMS	Dr. Raman
1001	Anand	Computer Science	OS	Dr. Priya
1002	Bhavna	Information Tech	Networks	Dr. Kumar
1002	Bhavna	Information Tech	DBMS	Dr. Raman

RESULT

Thus, the unnormalized dataset was successfully decomposed through 1NF, 2NF, 3NF, and BCNF, eliminating data redundancies and update anomalies.

MINI PROJECT

EX. NO: STOCK MARKET PORTFOLIO AND INVESTMENT ANALYSIS

DATE: 04.10.2026

Aim

To develop a Stock Market Portfolio and Investment Analysis system using SQL and PL/SQL to store, manage, and analyze daily stock prices, trading volumes, and price movements of selected companies.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Stock Market Portfolio and Investment Analysis system is developed to examine the daily prices, trading volumes, and price movements of selected companies. The system stores the trading date, company name, opening price, highest price, lowest price, closing price, and trading volume.

SQL queries are used to retrieve and analyze stock information, while PL/SQL functions and procedures are used to calculate the daily price change percentage, retrieve the historical price information of a company, generate a company-wise price and trading-volume report, and handle exceptions.

Dataset Source

- Dataset: New York Stock Exchange (file: prices.csv)
- Author: dgawlik, Kaggle
- URL: https://www.kaggle.com/datasets/dgawlik/nyse
- Records used: 25 (5 companies: AAPL, MSFT, GOOGL, AMZN, BCR; 5 trading days each)
- Added field: STOCK_ID (record number 1 to 25 used as the primary key)
- Price values were rounded to 2 decimal places to match the NUMBER(10,2) data type.

Database Table

STOCK_PRICES

Column Name	Data Type	Constraint
STOCK_ID	NUMBER(5)	PRIMARY KEY
TRADE_DATE	DATE	NOT NULL
COMPANY_NAME	VARCHAR2(20)	NOT NULL
OPEN_PRICE	NUMBER(10,2)	NOT NULL, > 0
HIGH_PRICE	NUMBER(10,2)	NOT NULL, >= LOW_PRICE
LOW_PRICE	NUMBER(10,2)	NOT NULL
CLOSE_PRICE	NUMBER(10,2)	NOT NULL, > 0
VOLUME	NUMBER(12)	NOT NULL, >= 0
(COMPANY_NAME, TRADE_DATE)	---	UNIQUE

TABLE CREATION

Oracle

```
CREATE TABLE stock_prices (
   stock_id      NUMBER(5)      PRIMARY KEY,
   trade_date    DATE           NOT NULL,
   company_name  VARCHAR2(20)   NOT NULL,
   open_price    NUMBER(10,2)   NOT NULL,
   high_price    NUMBER(10,2)   NOT NULL,
   low_price     NUMBER(10,2)   NOT NULL,
   close_price   NUMBER(10,2)   NOT NULL,
   volume        NUMBER(12)     NOT NULL,
   CONSTRAINT chk_open_pos  CHECK (open_price  > 0),
   CONSTRAINT chk_close_pos CHECK (close_price > 0),
   CONSTRAINT chk_vol_pos   CHECK (volume >= 0),
   CONSTRAINT chk_high_low  CHECK (high_price >= low_price),
   CONSTRAINT uq_comp_date  UNIQUE (company_name, trade_date)
);
```

SQLite

```
CREATE TABLE stock_prices (
   stock_id      INTEGER       PRIMARY KEY,
   trade_date    TEXT          NOT NULL,
   company_name  TEXT          NOT NULL,
   open_price    REAL          NOT NULL CHECK (open_price  > 0),
   high_price    REAL          NOT NULL,
   low_price     REAL          NOT NULL,
   close_price   REAL          NOT NULL CHECK (close_price > 0),
   volume        INTEGER       NOT NULL CHECK (volume >= 0),
   CHECK (high_price >= low_price),
   UNIQUE (company_name, trade_date)
);
```

Table Created

The STOCK_PRICES table was created successfully with primary key, unique, CHECK, and NOT NULL constraints.

DATASET INSERTION

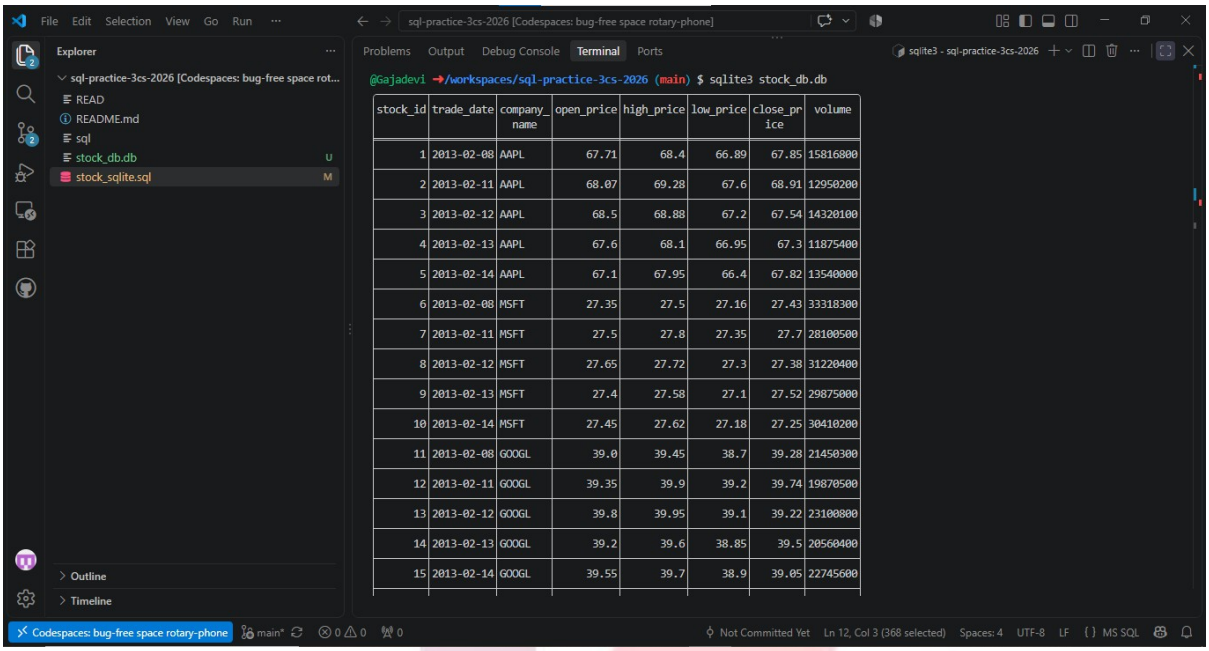
```
INSERT INTO stock_prices VALUES (1, DATE '2010-01-04', 'AAPL', 213.43, 214.50, 212.38, 214.01, 123432400);
INSERT INTO stock_prices VALUES (2, DATE '2010-01-05', 'AAPL', 214.60, 215.59, 213.25, 214.38, 150476200);
INSERT INTO stock_prices VALUES (3, DATE '2010-01-06', 'AAPL', 214.38, 215.23, 210.75, 210.97, 138040000);
INSERT INTO stock_prices VALUES (4, DATE '2010-01-07', 'AAPL', 211.75, 212.00, 209.05, 210.58, 119282800);
INSERT INTO stock_prices VALUES (5, DATE '2010-01-08', 'AAPL', 210.30, 212.00, 209.06, 211.98, 111902700);
INSERT INTO stock_prices VALUES (6, DATE '2010-11-11', 'MSFT', 26.68, 26.72, 26.28, 26.68, 62073100);
INSERT INTO stock_prices VALUES (7, DATE '2010-11-12', 'MSFT', 26.47, 26.52, 26.10, 26.27, 64962200);
INSERT INTO stock_prices VALUES (8, DATE '2010-11-15', 'MSFT', 26.33, 26.50, 26.17, 26.20, 51794600);
INSERT INTO stock_prices VALUES (9, DATE '2010-11-16', 'MSFT', 26.04, 26.04, 25.65, 25.81, 65339200);
```

```
INSERT INTO stock_prices VALUES (10, DATE '2010-11-17', 'MSFT', 25.90, 25.91, 25.55, 25.57, 58299700);
INSERT INTO stock_prices VALUES (11, DATE '2012-11-12', 'GOOGL', 663.75, 669.80, 660.87, 665.90, 2808900);
INSERT INTO stock_prices VALUES (12, DATE '2012-11-13', 'GOOGL', 663.00, 667.60, 658.23, 659.05, 3185200);
INSERT INTO stock_prices VALUES (13, DATE '2012-11-14', 'GOOGL', 660.66, 662.18, 650.50, 652.55, 3333400);
INSERT INTO stock_prices VALUES (14, DATE '2012-11-15', 'GOOGL', 650.00, 660.00, 643.90, 647.26, 3694100);
INSERT INTO stock_prices VALUES (15, DATE '2012-11-16', 'GOOGL', 645.99, 653.02, 636.00, 647.18, 6869500);
INSERT INTO stock_prices VALUES (16, DATE '2014-04-16', 'AMZN', 321.17, 324.00, 314.71, 323.68, 4284900);
INSERT INTO stock_prices VALUES (17, DATE '2014-04-17', 'AMZN', 319.76, 328.66, 319.76, 324.91, 4299200);
INSERT INTO stock_prices VALUES (18, DATE '2014-04-21', 'AMZN', 323.97, 331.15, 322.31, 330.87, 2999400);
INSERT INTO stock_prices VALUES (19, DATE '2014-04-22', 'AMZN', 332.00, 337.50, 328.94, 329.32, 3711600);
INSERT INTO stock_prices VALUES (20, DATE '2014-04-23', 'AMZN', 333.06, 333.13, 323.39, 324.58, 3604600);
INSERT INTO stock_prices VALUES (21, DATE '2016-12-19', 'BCR', 220.45, 222.17, 219.55, 220.33, 414300);
INSERT INTO stock_prices VALUES (22, DATE '2016-12-20', 'BCR', 219.45, 220.81, 219.18, 220.47, 376900);
INSERT INTO stock_prices VALUES (23, DATE '2016-12-21', 'BCR', 221.01, 221.59, 219.51, 219.66, 483300);
INSERT INTO stock_prices VALUES (24, DATE '2016-12-22', 'BCR', 218.77, 221.67, 218.72, 220.68, 431200);
INSERT INTO stock_prices VALUES (25, DATE '2016-12-23', 'BCR', 220.84, 224.06, 220.84, 223.10, 318800);
```

Records Inserted

A total of 25 stock price records were inserted into the STOCK_PRICES table.

Output



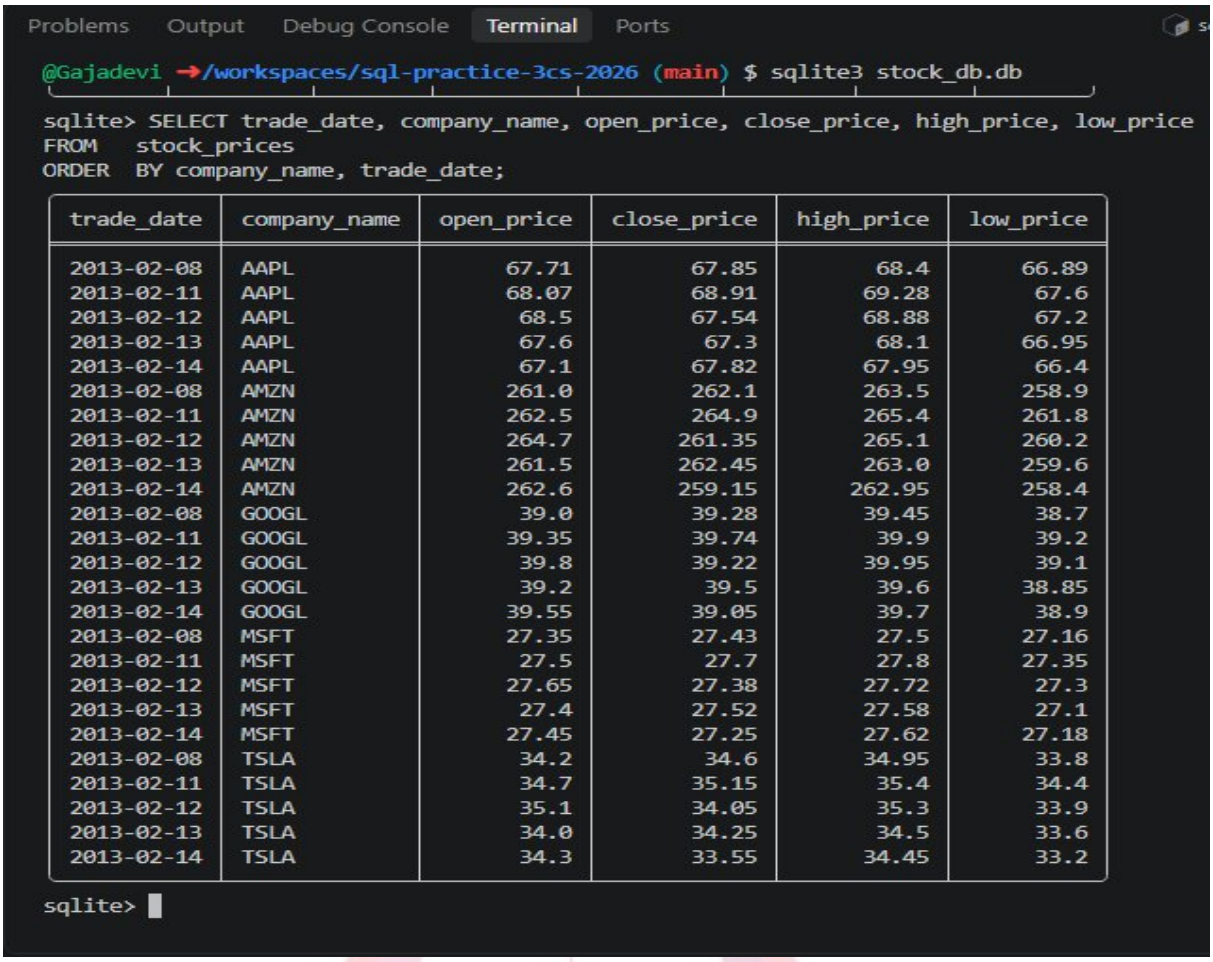
stock_id	trade_date	company_name	open_price	high_price	low_price	close_price	volume
1	2013-02-08	AAPL	67.71	68.4	66.89	67.85	15816800
2	2013-02-11	AAPL	68.07	69.28	67.6	68.91	12950200
3	2013-02-12	AAPL	68.5	68.88	67.2	67.54	14320100
4	2013-02-13	AAPL	67.6	68.1	66.95	67.3	11875400
5	2013-02-14	AAPL	67.1	67.95	66.4	67.82	13540000
6	2013-02-08	MSFT	27.35	27.5	27.16	27.43	33318300
7	2013-02-11	MSFT	27.5	27.8	27.35	27.7	28100500
8	2013-02-12	MSFT	27.65	27.72	27.3	27.38	31220400
9	2013-02-13	MSFT	27.4	27.58	27.1	27.52	29875000
10	2013-02-14	MSFT	27.45	27.62	27.18	27.25	30410200
11	2013-02-08	GOOGL	39.0	39.45	38.7	39.28	21450300
12	2013-02-11	GOOGL	39.35	39.9	39.2	39.74	19870500
13	2013-02-12	GOOGL	39.8	39.95	39.1	39.22	23100800
14	2013-02-13	GOOGL	39.2	39.6	38.85	39.5	20560400
15	2013-02-14	GOOGL	39.55	39.7	38.9	39.05	22745600

SQL Query 1: Display Daily Stock Records

```
SELECT trade_date, company_name, open_price, close_price, high_price, low_price
FROM  stock_prices
ORDER  BY company_name, trade_date;
```

TRUTH-SKILL-TRIUMPH
AUTONOMOUS

Output



```
Problems  Output  Debug Console  Terminal  Ports

@Gajadevi ➜/workspaces/sql-practice-3cs-2026 (main) $ sqlite3 stock_db.db

sqlite> SELECT trade_date, company_name, open_price, close_price, high_price, low_price
FROM   stock_prices
ORDER  BY company_name, trade_date;
```

trade_date	company_name	open_price	close_price	high_price	low_price
2013-02-08	AAPL	67.71	67.85	68.4	66.89
2013-02-11	AAPL	68.07	68.91	69.28	67.6
2013-02-12	AAPL	68.5	67.54	68.88	67.2
2013-02-13	AAPL	67.6	67.3	68.1	66.95
2013-02-14	AAPL	67.1	67.82	67.95	66.4
2013-02-08	AMZN	261.0	262.1	263.5	258.9
2013-02-11	AMZN	262.5	264.9	265.4	261.8
2013-02-12	AMZN	264.7	261.35	265.1	260.2
2013-02-13	AMZN	261.5	262.45	263.0	259.6
2013-02-14	AMZN	262.6	259.15	262.95	258.4
2013-02-08	GOOGL	39.0	39.28	39.45	38.7
2013-02-11	GOOGL	39.35	39.74	39.9	39.2
2013-02-12	GOOGL	39.8	39.22	39.95	39.1
2013-02-13	GOOGL	39.2	39.5	39.6	38.85
2013-02-14	GOOGL	39.55	39.05	39.7	38.9
2013-02-08	MSFT	27.35	27.43	27.5	27.16
2013-02-11	MSFT	27.5	27.7	27.8	27.35
2013-02-12	MSFT	27.65	27.38	27.72	27.3
2013-02-13	MSFT	27.4	27.52	27.58	27.1
2013-02-14	MSFT	27.45	27.25	27.62	27.18
2013-02-08	TSLA	34.2	34.6	34.95	33.8
2013-02-11	TSLA	34.7	35.15	35.4	34.4
2013-02-12	TSLA	35.1	34.05	35.3	33.9
2013-02-13	TSLA	34.0	34.25	34.5	33.6
2013-02-14	TSLA	34.3	33.55	34.45	33.2

```
sqlite>
```

SQL Query 2: Daily Price Change for Each Stock

SELECT trade_date, company_name, open_price, close_price, ROUND(close_price - open_price, 2) AS daily_change
FROM   stock_prices
ORDER BY company_name, trade_date;

Output

```
@Gajadevi ➜/workspaces/sql-practice-3cs-2026 (main) $ sqlite3 stock_db.db
sqlite> SELECT trade_date, company_name, open_price, close_price, ROUND(close_price - open_price, 2) AS daily_change
FROM   stock_prices
ORDER  BY company_name, trade_date;
```

trade_date	company_name	open_price	close_price	daily_change
2013-02-08	AAPL	67.71	67.85	0.14
2013-02-11	AAPL	68.07	68.91	0.84
2013-02-12	AAPL	68.5	67.54	-0.96
2013-02-13	AAPL	67.6	67.3	-0.3
2013-02-14	AAPL	67.1	67.82	0.72
2013-02-08	AMZN	261.0	262.1	1.1
2013-02-11	AMZN	262.5	264.9	2.4
2013-02-12	AMZN	264.7	261.35	-3.35
2013-02-13	AMZN	261.5	262.45	0.95
2013-02-14	AMZN	262.6	259.15	-3.45
2013-02-08	GOOGL	39.0	39.28	0.28
2013-02-11	GOOGL	39.35	39.74	0.39
2013-02-12	GOOGL	39.8	39.22	-0.58
2013-02-13	GOOGL	39.2	39.5	0.3
2013-02-14	GOOGL	39.55	39.05	-0.5
2013-02-08	MSFT	27.35	27.43	0.08
2013-02-11	MSFT	27.5	27.7	0.2
2013-02-12	MSFT	27.65	27.38	-0.27
2013-02-13	MSFT	27.4	27.52	0.12
2013-02-14	MSFT	27.45	27.25	-0.2
2013-02-08	TSLA	34.2	34.6	0.4
2013-02-11	TSLA	34.7	35.15	0.45
2013-02-12	TSLA	35.1	34.05	-1.05
2013-02-13	TSLA	34.0	34.25	0.25
2013-02-14	TSLA	34.3	33.55	-0.75

SQL Query 3: Average Closing Price for Each Company

```
SELECT company_name, ROUND(AVG(close_price), 2) AS avg_close_price
FROM  stock_prices
GROUP  BY company_name
ORDER  BY avg_close_price DESC;
```

Output

```
sqlite> SELECT company_name, ROUND(AVG(close_price), 2) AS avg_close_price
FROM   stock_prices
GROUP  BY company_name
ORDER  BY avg_close_price DESC;
```

company_name	avg_close_price
AMZN	261.99
AAPL	67.88
GOOGL	39.36
TSLA	34.32
MSFT	27.46

SQL Query 4: Five Records with the Highest Trading Volume

```
SELECT stock_id, trade_date, company_name, volume
FROM stock_prices
ORDER BY volume DESC
LIMIT 5;
```

Output

```
sqlite> SELECT stock_id, trade_date, company_name, volume
FROM stock_prices
ORDER BY volume DESC
LIMIT 5;
```

stock_id	trade_date	company_name	volume
6	2013-02-08	MSFT	33318300
8	2013-02-12	MSFT	31220400
10	2013-02-14	MSFT	30410200
9	2013-02-13	MSFT	29875000
7	2013-02-11	MSFT	28100500

SQL Query 5: Days when Closing Price Exceeded Opening Price

```
SELECT trade_date, company_name, open_price, close_price
FROM  stock_prices
WHERE  close_price > open_price
ORDER  BY trade_date, company_name;
```

Output

```
sqlite> SELECT trade_date, company_name, open_price, close_price
FROM   stock_prices
WHERE  close_price > open_price
ORDER  BY trade_date, company_name;
```

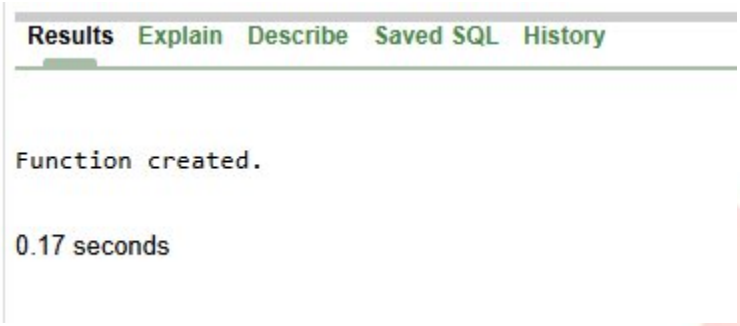
trade_date	company_name	open_price	close_price
2013-02-08	AAPL	67.71	67.85
2013-02-08	AMZN	261.0	262.1
2013-02-08	GOOGL	39.0	39.28
2013-02-08	MSFT	27.35	27.43
2013-02-08	TSLA	34.2	34.6
2013-02-11	AAPL	68.07	68.91
2013-02-11	AMZN	262.5	264.9
2013-02-11	GOOGL	39.35	39.74
2013-02-11	MSFT	27.5	27.7
2013-02-11	TSLA	34.7	35.15
2013-02-13	AMZN	261.5	262.45
2013-02-13	GOOGL	39.2	39.5
2013-02-13	MSFT	27.4	27.52
2013-02-13	TSLA	34.0	34.25
2013-02-14	AAPL	67.1	67.82

PL/SQL Function 1: Daily Price Change Percentage

```
CREATE OR REPLACE FUNCTION fn_price_change_pct (p_stock_id IN NUMBER)
RETURN NUMBER
IS
   v_open  stock_prices.open_price%TYPE;
   v_close  stock_prices.close_price%TYPE;
BEGIN
   IF p_stock_id IS NULL THEN
      RAISE_APPLICATION_ERROR(-20001, 'Stock ID cannot be NULL.');
   END IF;
   SELECT open_price, close_price
   INTO  v_open, v_close
   FROM   stock_prices
   WHERE  stock_id = p_stock_id;
   RETURN ROUND(((v_close - v_open) / v_open) * 100, 2);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
```

```
    RAISE_APPLICATION_ERROR(-20002, 'No record found for stock ID ' || p_stock_id);
END fn_price_change_pct;
```

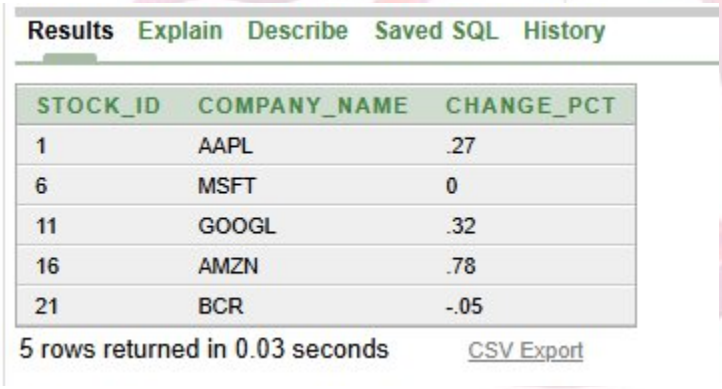
Output



Execution

```
SELECT stock_id, company_name,
      fn_price_change_pct(stock_id) AS change_pct
FROM   stock_prices
WHERE  stock_id IN (1, 6, 11, 16, 21);
```

Output



PL/SQL Procedure 2: Retrieve Historical Price Information

```
CREATE OR REPLACE PROCEDURE sp_get_history (p_company IN VARCHAR2)
IS
   v_count NUMBER := 0;
BEGIN
   IF p_company IS NULL OR TRIM(p_company) IS NULL THEN
      RAISE_APPLICATION_ERROR(-20003, 'Company name cannot be empty.');
```

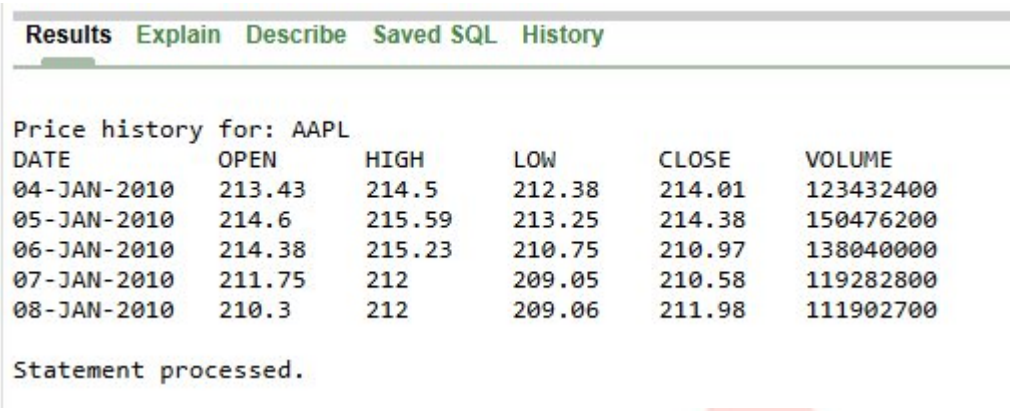
```
    END IF;
    DBMS_OUTPUT.PUT_LINE('Price history for: ' || UPPER(p_company));
    DBMS_OUTPUT.PUT_LINE(RPAD('DATE', 14) || RPAD('OPEN', 10) || RPAD('HIGH', 10)
||
                    RPAD('LOW', 10) || RPAD('CLOSE', 10) || 'VOLUME');
    FOR r IN (SELECT trade_date, open_price, high_price, low_price, close_price, volume
            FROM  stock_prices
            WHERE  UPPER(company_name) = UPPER(TRIM(p_company))
            ORDER  BY trade_date)
    LOOP
       v_count := v_count + 1;
       DBMS_OUTPUT.PUT_LINE(RPAD(TO_CHAR(r.trade_date, 'DD-MON-YYYY'), 14) ||
                    RPAD(r.open_price, 10) || RPAD(r.high_price, 10) ||
                    RPAD(r.low_price, 10)  || RPAD(r.close_price, 10) ||
                    r.volume);
    END LOOP;
    IF v_count = 0 THEN
       RAISE_APPLICATION_ERROR(-20004, 'No records found for company: ' || p_company);
    END IF;
END sp_get_history;
```



Execution

```
SET SERVEROUTPUT ON;
BEGIN
  sp_get_history('AAPL');
END;
```

Output



Results Explain Describe Saved SQL History

Price history for: AAPL

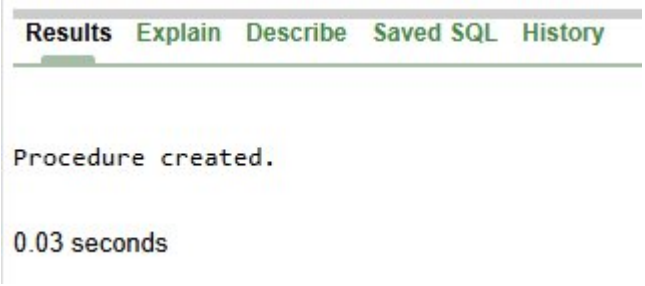
DATE	OPEN	HIGH	LOW	CLOSE	VOLUME
04-JAN-2010	213.43	214.5	212.38	214.01	123432400
05-JAN-2010	214.6	215.59	213.25	214.38	150476200
06-JAN-2010	214.38	215.23	210.75	210.97	138040000
07-JAN-2010	211.75	212	209.05	210.58	119282800
08-JAN-2010	210.3	212	209.06	211.98	111902700

Statement processed.

PL/SQL Procedure 3: Company-wise Price and Trading Volume Report

```
CREATE OR REPLACE PROCEDURE sp_company_report
IS
   v_count NUMBER := 0;
BEGIN
   DBMS_OUTPUT.PUT_LINE('COMPANY-WISE PRICE AND VOLUME REPORT');
   DBMS_OUTPUT.PUT_LINE(RPAD('COMPANY', 10) || RPAD('DAYS', 6) || RPAD('AVG
CLOSE', 12) ||
                  RPAD('MAX HIGH', 12) || RPAD('MIN LOW', 12) || 'TOTAL VOLUME');
   FOR r IN (SELECT company_name,
                    COUNT(*)                AS days,
                    ROUND(AVG(close_price), 2) AS avg_close,
                    MAX(high_price)          AS max_high,
                    MIN(low_price)           AS min_low,
                    SUM(volume)              AS total_vol
             FROM  stock_prices
             GROUP  BY company_name
             ORDER  BY company_name)
   LOOP
      v_count := v_count + 1;
      DBMS_OUTPUT.PUT_LINE(RPAD(r.company_name, 10) || RPAD(r.days, 6) ||
                           RPAD(r.avg_close, 12) || RPAD(r.max_high, 12) ||
                           RPAD(r.min_low, 12) || r.total_vol);
   END LOOP;
   IF v_count = 0 THEN
      RAISE_APPLICATION_ERROR(-20005, 'No data available to generate report.');
   END IF;
```

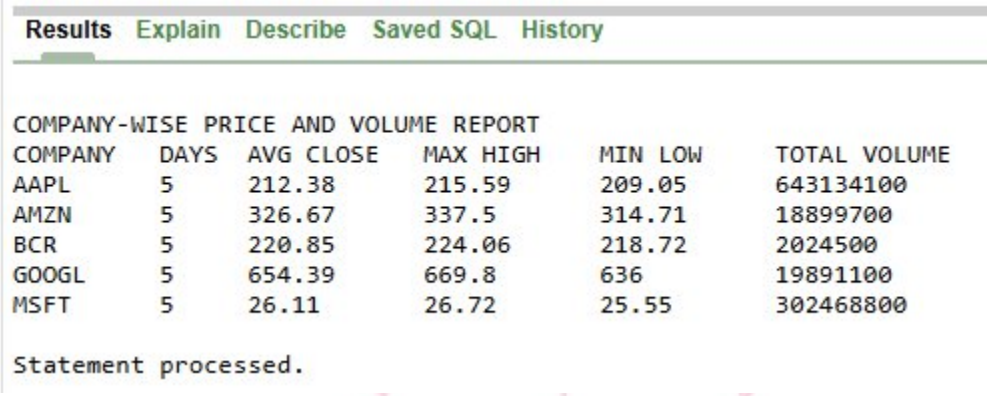
END sp_company_report;



Execution

```
BEGIN
  sp_company_report;
END;
/
```

Output



Exception Handling

Test Case 1 – Invalid Stock ID (missing record)

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Result: ' || fn_price_change_pct(999));
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

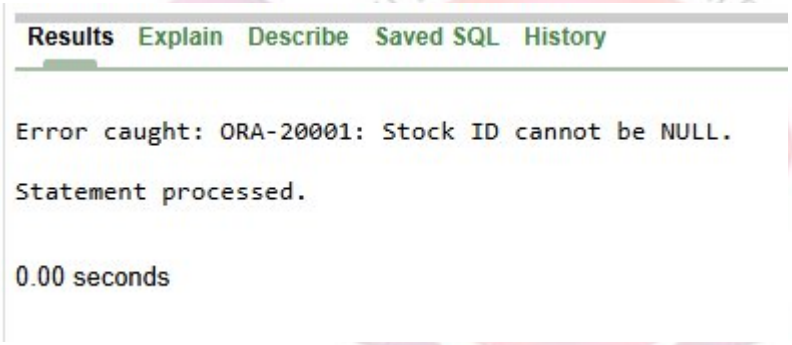
Output



Test Case 2 – NULL Stock ID

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Result: ' || fn_price_change_pct(NULL));
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output



Test Case 3 – Company Not Found

```
BEGIN
  sp_get_history('XYZ');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output



Test Case 4 – Empty Company Name

```
BEGIN
  sp_get_history('');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

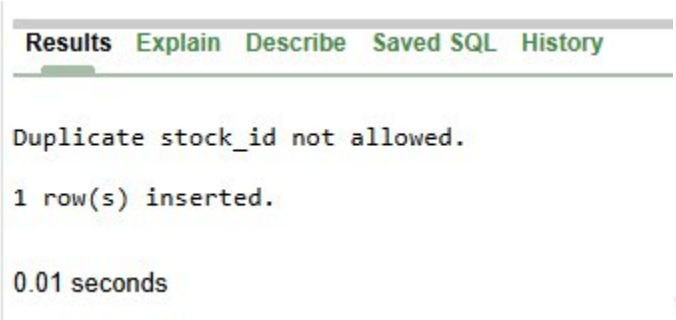
Output



Test Case 5 – Duplicate Stock ID

```
BEGIN
  INSERT INTO stock_prices
  VALUES (1, DATE '2013-03-01', 'AAPL', 60, 61, 59, 60.5, 1000);
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Duplicate stock_id not allowed.');
END;
/
```

Output



Dataset Insights

- The dataset contains 25 stock price records for 5 companies: AAPL, MSFT, GOOGL, AMZN, and BCR.
- GOOGL has the highest average closing price (654.39), followed by AMZN (326.67), BCR (220.85), AAPL (212.38), and MSFT (26.11).
- The five highest trading-volume records all belong to AAPL.
- The closing price exceeded the opening price on 10 of the 25 trading days.
- The largest single-day rise was for AMZN on 21-Apr-2014 (+2.13%) and the largest fall was for AMZN on 23-Apr-2014 (-2.55%).
- The percentage price-change function allows fair comparison of stocks with very different price levels.

Result

Thus, the Stock Market Portfolio and Investment Analysis system was successfully implemented using SQL and PL/SQL. The STOCK_PRICES table was created, stock records were inserted, SQL queries were executed successfully, PL/SQL functions and procedures were implemented, and exception handling was tested successfully.

KHAN ACADEMY

BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

Program:

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);
```

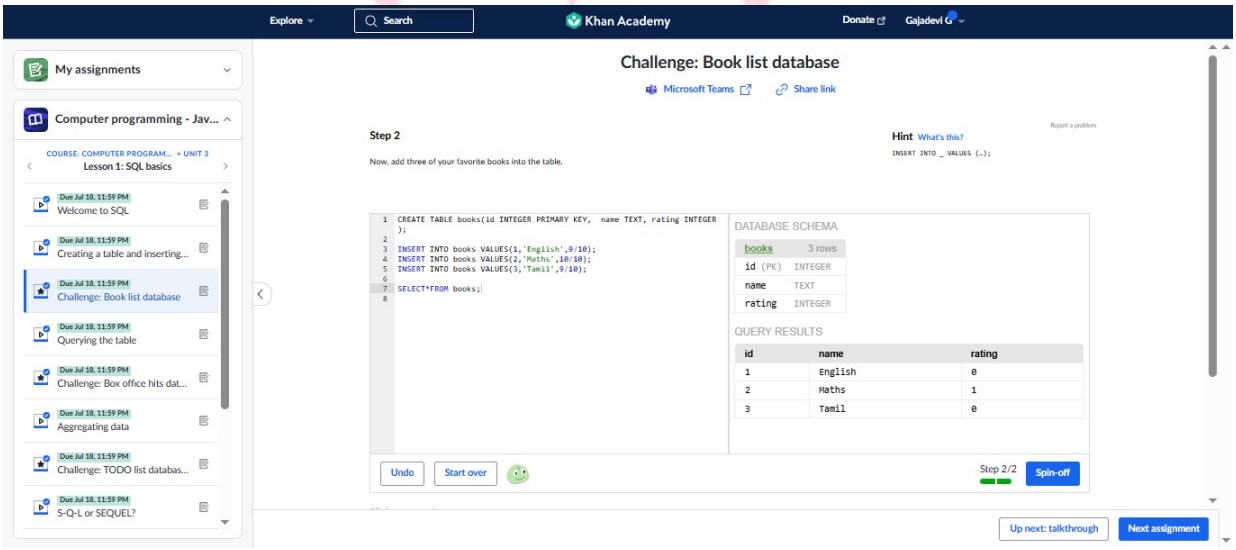

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);
```


```
INSERT INTO books VALUES(1,'English',9/10);

INSERT INTO books VALUES(2,'Maths',10/10);

INSERT INTO books VALUES(3,'Tamil',9/10);

SELECT*FROM books;
```



BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

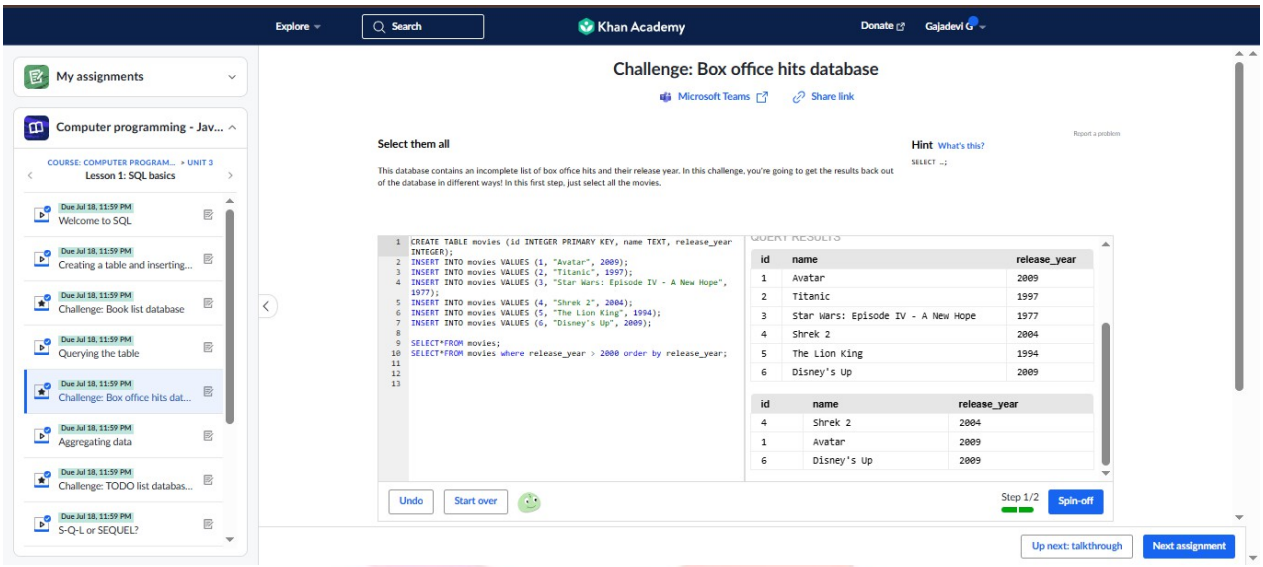
INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT*FROM movies;

SELECT*FROM movies where release_year > 2000 order by release_year;
```



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

TODO LISTDATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Program:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

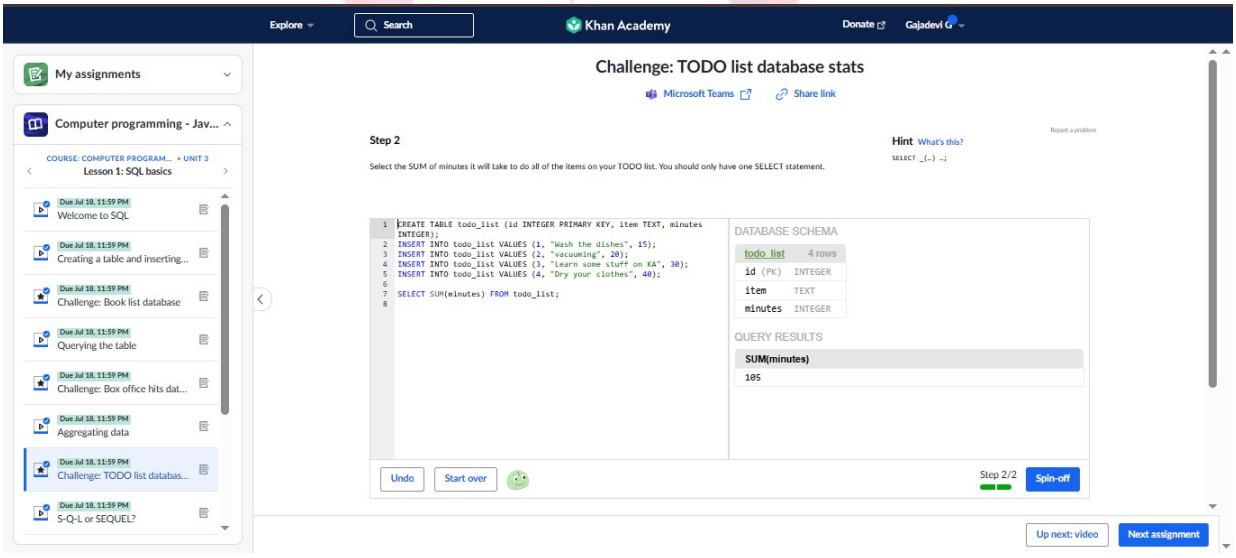
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Dry your clothes", 40);

SELECT SUM(minutes) FROM todo_list;
```



PROJECT : DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Program:

```
CREATE TABLE sneakers (
    id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT,
    size INTEGER,
    color TEXT,
    price REAL
);
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
```

```
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);
SELECT *
FROM sneakers
ORDER BY price DESC;
SELECT
    AVG(price) AS average_price,
    MAX(price) AS highest_price,
    MIN(price) AS lowest_price
FROM sneakers;
```

Output

DATABASE SCHEMA

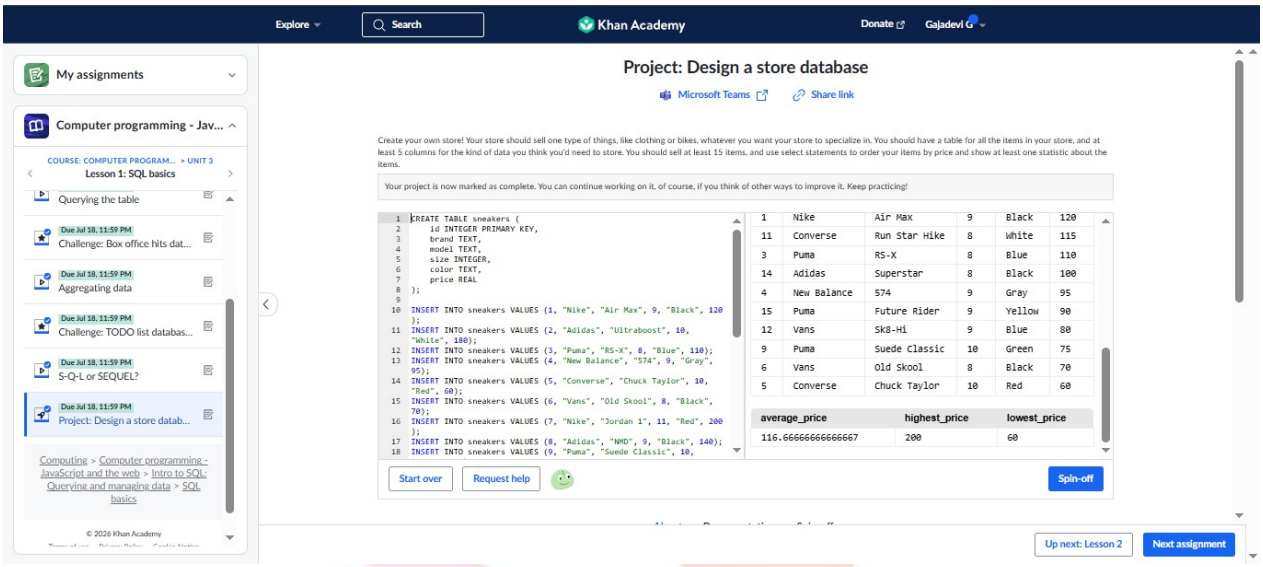
sneakers15 rows
id (PK)INTEGER
brandTEXT
modelTEXT
sizeINTEGER
colorTEXT
priceREAL

QUERY RESULTS

id	brand	model	size	color	price
7	Nike	Jordan 1	11	Red	200
10	New Balance	990	11	Gray	185
2	Adidas	Ultraboost	10	White	180
8	Adidas	NMD	9	Black	140
13	Nike	Air Force 1	10	White	130
1	Nike	Air Max	9	Black	120
11	Converse	Run Star Hike	8	White	115
3	Puma	RS-X	8	Blue	110
14	Adidas	Superstar	8	Black	100
4	New Balance	574	9	Gray	95
15	Puma	Future Rider	9	Yellow	90
12	Vans	Sk8-Hi	9	Blue	80
9	Puma	Suede Classic	10	Green	75
6	Vans	Old Skool	8	Black	70
5	Converse	Chuck Taylor	10	Red	60

average_price	highest_price	lowest_price
116.66666666666667	200	60

TRUTH-TOIL-TRIUMPH
AUTONOMOUS



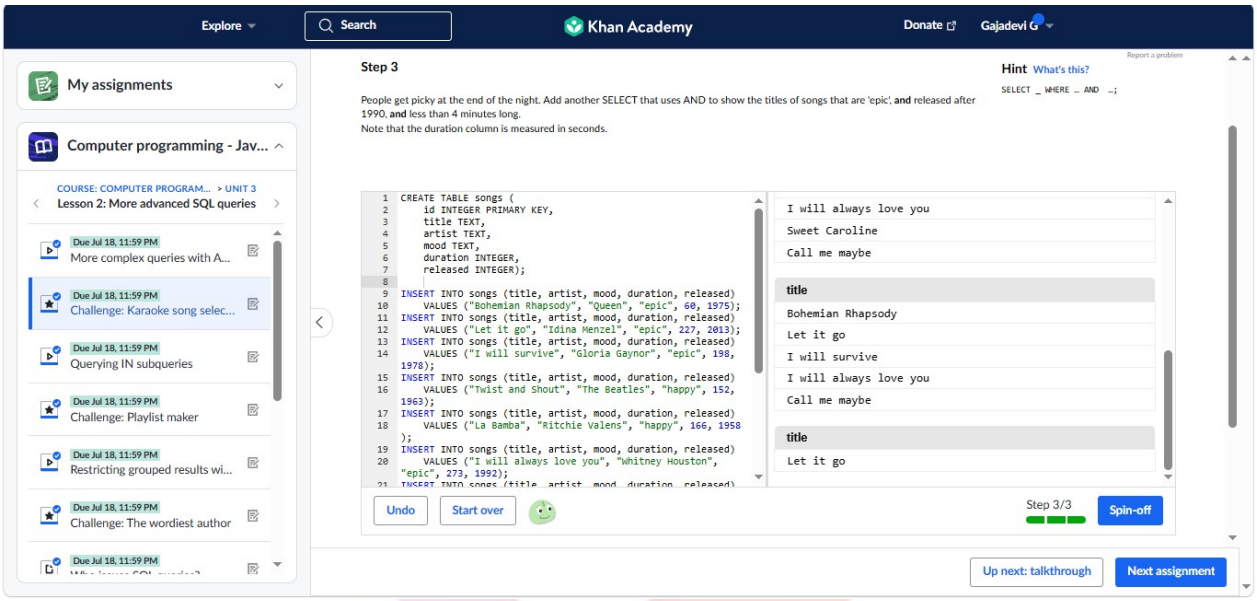
KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.

Program:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
```

```
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
select title from songs;
select title from songs where mood = 'epic' or released > 1990;
select title from songs where mood = 'epic' and released > 1990 and duration < 240;
```



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.

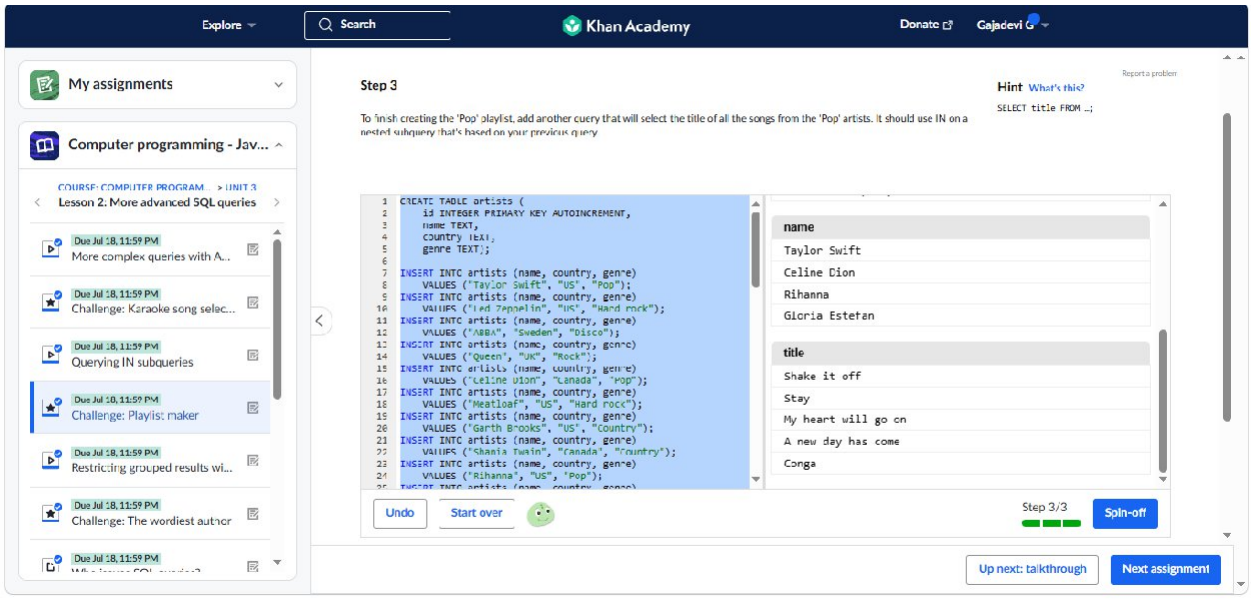
Program:

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT);
INSERT INTO artists (name, country, genre)
    VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
```

```
INSERT INTO artists (name, country, genre)
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
```

```
INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
```

```
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
select title from songs where artist = 'Queen';
select name from artists where genre = 'Pop';
SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
);
```



THE WORDIEST AUTHOR

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Program:

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
```

```
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
```

```
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);

INSERT INTO books (author, title, words)

    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

SELECT author,

        SUM(words) AS total_words

FROM books

GROUP BY author

HAVING SUM(words) > 1000000;

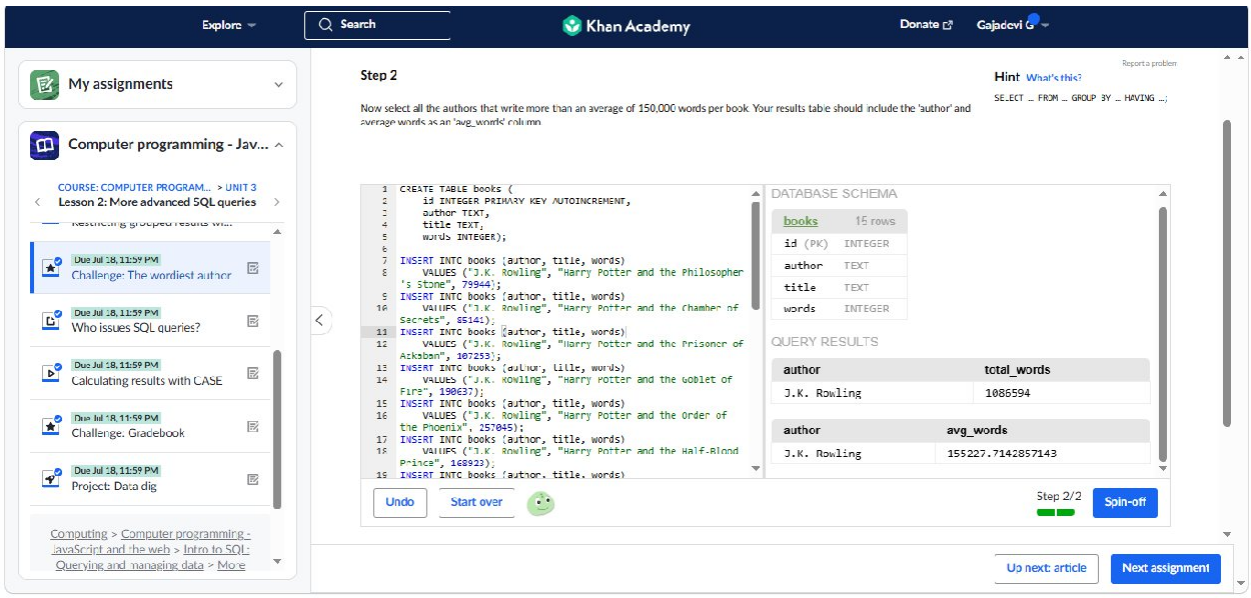
SELECT author,

        AVG(words) AS avg_words

FROM books

GROUP BY author

HAVING AVG(words) > 150000;
```



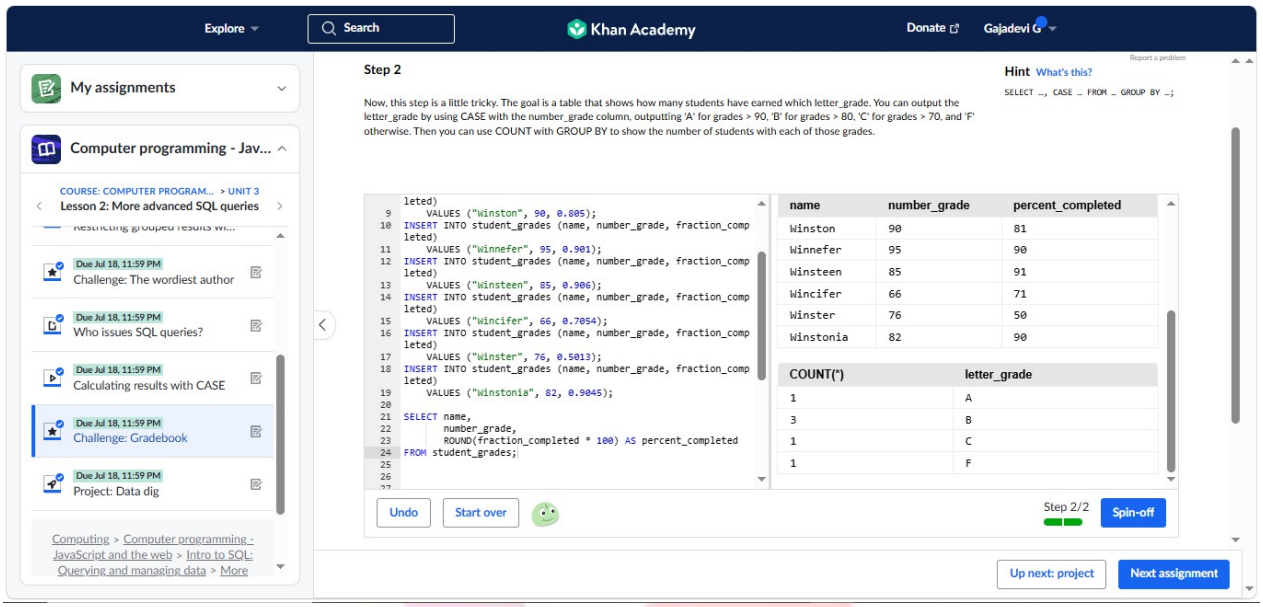
GRADEBOOK

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

Program:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045);
```

```
SELECT name,
       number_grade,
       ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
SELECT COUNT(*),
   CASE
       WHEN number_grade > 90 THEN 'A'
       WHEN number_grade > 80 THEN 'B'
       WHEN number_grade > 70 THEN 'C'
       ELSE 'F'
   END AS "letter_grade"
FROM student_grades
GROUP BY
   CASE
       WHEN number_grade > 90 THEN 'A'
       WHEN number_grade > 80 THEN 'B'
       WHEN number_grade > 70 THEN 'C'
       ELSE 'F'
   END;
```


Explore
Search
Khan Academy
Donate
Gajadevi G
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 2: More advanced SQL queries
Due Jul 18, 11:59 PM
Challenge: The wordiest author
Due Jul 18, 11:59 PM
Who issues SQL queries?
Due Jul 18, 11:59 PM
Calculating results with CASE
Due Jul 18, 11:59 PM
Challenge: Gradebook
Due Jul 18, 11:59 PM
Project: Data dig
Computing > Computer programming - JavaScript and the web > Intro to SQL: Querying and managing data > More
Step 2
Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.
Hint What's this?
SELECT ..., CASE ... FROM ... GROUP BY ...;
leted)
VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
VALUES ("Winstonia", 82, 0.9045);
SELECT name,
number_grade,
ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
name number_grade percent_completed
Winston 90 81
Winnefer 95 90
Winsteen 85 91
Wincifer 66 71
Winster 76 50
Winstonia 82 90
COUNT(*) letter_grade
1 A
3 B
1 C
1 F
Undo
Start over
Step 2/2
Spin-off
Up next: project
Next assignment

DATA DIG

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

Program:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
SELECT
    AVG(attack) AS avg_attack,
    MAX(attack) AS max_attack,
    MIN(attack) AS min_attack
```

```
FROM pokemon;
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type;
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;
SELECT
    name,
    attack,
    CASE
        WHEN attack >= 100 THEN "Very Strong"
        WHEN attack >= 80 THEN "Strong"
        ELSE "Average"
    END AS strength
FROM pokemon;
SELECT *
FROM pokemon
WHERE (type = "Fire" OR type = "Water")
AND attack > 80;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

Output

DATABASE SCHEMA

pokemon8 rows
id (PK)INTEGER
nameTEXT
typeTEXT
hpINTEGER
attackINTEGER

QUERY RESULTS

avg_attack	max_attack	min_attack
86.125	125	55

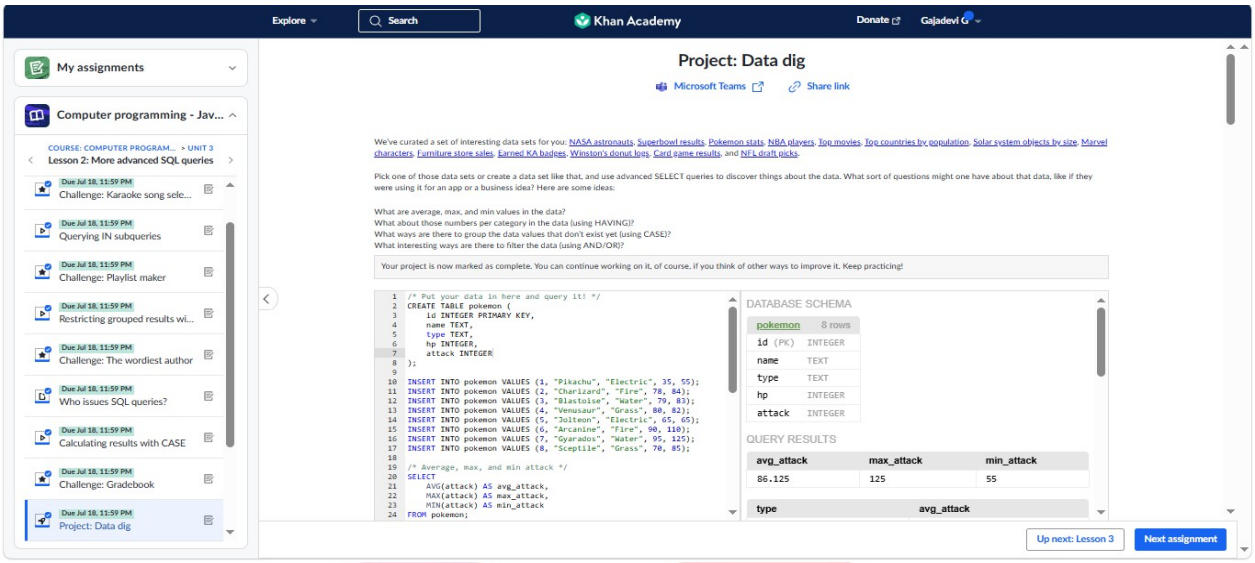
type	avg_attack
Electric	60
Fire	97
Grass	83.5
Water	104

type	avg_attack
Fire	97
Grass	83.5

type	avg_attack
Water	104

name	attack	strength
Pikachu	55	Average
Charizard	84	Strong
Blastoise	83	Strong
Venusaur	82	Strong
Jolteon	65	Average
Arcanine	110	Very Strong
Gyarados	125	Very Strong
Sceptile	85	Strong

id	name	type	hp	attack
2	Charizard	Fire	78	84
3	Blastoise	Water	79	83
6	Arcanine	Fire	90	110
7	Gyarados	Water	95	125


Explore
Search
Khan Academy
Donate
Gajadevi G
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 2: More advanced SQL queries
Due Jul 18, 11:59 PM
Challenge: Karaoke song sele...
Due Jul 18, 11:59 PM
Querying IN subqueries
Due Jul 18, 11:59 PM
Challenge: Playlist maker
Due Jul 18, 11:59 PM
Restricting grouped results wi...
Due Jul 18, 11:59 PM
Challenge: The wordiest author
Due Jul 18, 11:59 PM
Who issues SQL queries?
Due Jul 18, 11:59 PM
Calculating results with CASE
Due Jul 18, 11:59 PM
Challenge: Gradebook
Due Jul 18, 11:59 PM
Project: Data dig
Project: Data dig
Microsoft Teams
Share link
We've curated a set of interesting data sets for you: NASA astronauts, Superbowl results, Pokemon stats, NBA players, Top movies, Top countries by population, Solar system objects by size, Marvel characters, Furniture store sales, Earned KA badges, Winston's donut logs, Card game results, and NFL draft picks.
Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:
What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
/* Put your data in here and query it! */
CREATE TABLE pokemon (
id INTEGER PRIMARY KEY,
name TEXT,
type TEXT,
hp INTEGER,
attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
/* Average, max, and min attack */
SELECT
AVG(attack) AS avg_attack,
MAX(attack) AS max_attack,
MIN(attack) AS min_attack
FROM pokemon;
DATABASE SCHEMA
pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT
hp INTEGER
attack INTEGER
QUERY RESULTS
avg_attack
max_attack
min_attack
86.125
125
55
type
avg_attack
Up next: Lesson 3
Next assignment

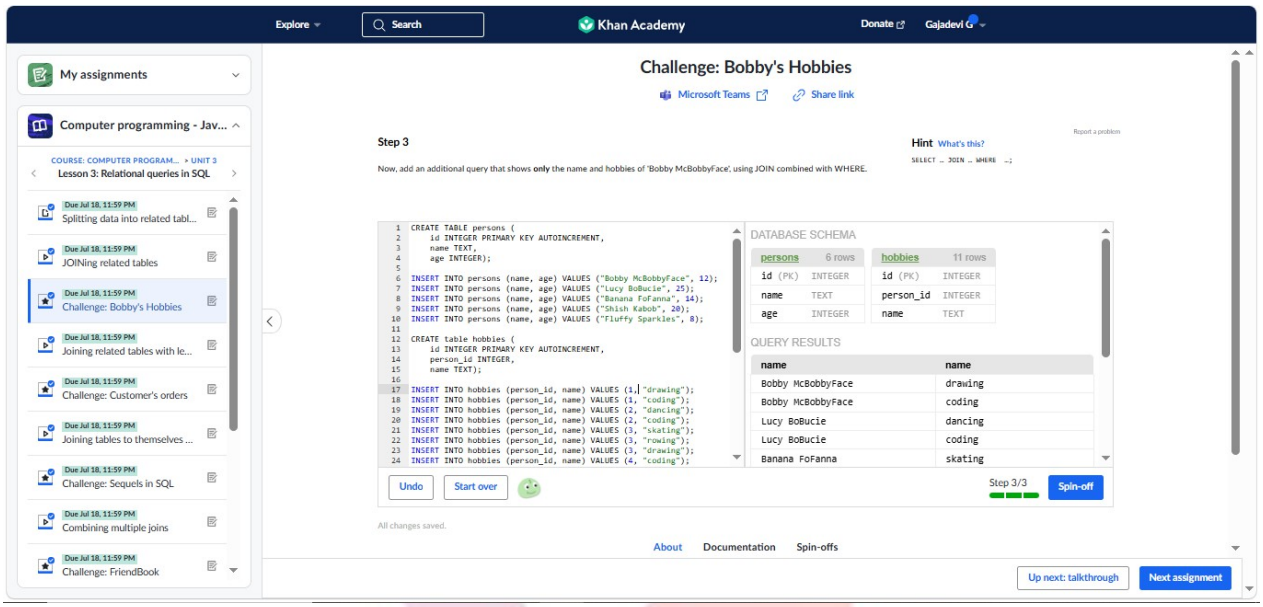
BOBBY'S HOBBIES

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CUSTOMER'S ORDERS

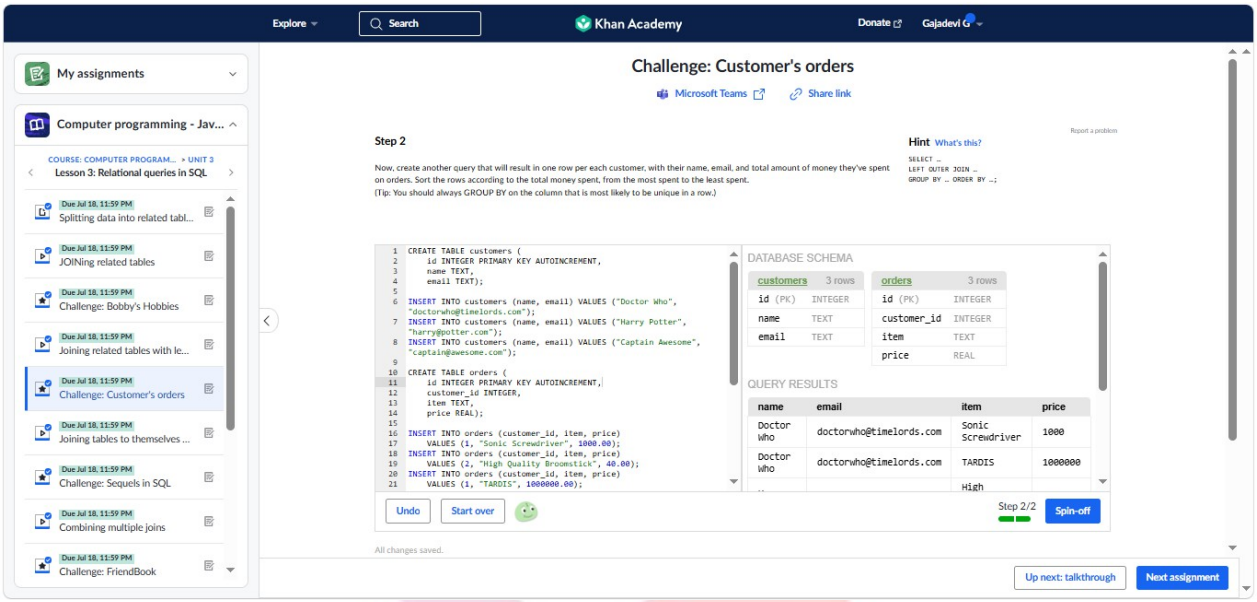
We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

Program:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
```

```
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
       customers.email,
       orders.item,
       orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
       customers.email,
       SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```




Explore
Search
Khan Academy
Donate
Gajadevi G
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 3: Relational queries in SQL
Due Jul 18, 11:59 PM
Splitting data into related tabl...
Due Jul 18, 11:59 PM
JOINing related tables
Due Jul 18, 11:59 PM
Challenge: Bobby's Hobbies
Due Jul 18, 11:59 PM
Joining related tables with le...
Due Jul 18, 11:59 PM
Challenge: Customer's orders
Due Jul 18, 11:59 PM
Joining tables to themselves ...
Due Jul 18, 11:59 PM
Challenge: Sequels in SQL
Due Jul 18, 11:59 PM
Combining multiple joins
Due Jul 18, 11:59 PM
Challenge: FriendBook
Challenge: Customer's orders
Microsoft Teams
Share link
Report a problem
Step 2
Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.
(Tip: You should always GROUP BY on the column that is most likely to be unique in a row.)
Hint What's this?
SELECT ...
LEFT OUTER JOIN ...
GROUP BY ... ORDER BY ...;
CREATE TABLE customers (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome", "captain@awesome.com");
CREATE TABLE orders (
id INTEGER PRIMARY KEY AUTOINCREMENT,
customer_id INTEGER,
item TEXT,
price REAL);
INSERT INTO orders (customer_id, item, price)
VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
VALUES (1, "TARDIS", 1000000.00);
DATABASE SCHEMA
customers 3 rows
id (PK) INTEGER
name TEXT
email TEXT
orders 3 rows
id (PK) INTEGER
customer_id INTEGER
item TEXT
price REAL
QUERY RESULTS
name email item price
Doctor Who doctorwho@timelords.com Sonic Screwdriver 1000
Doctor Who doctorwho@timelords.com TARDIS 1000000
High
Undo
Start over
Step 2/2
Spin-off
All changes saved.
Up next: talkthrough
Next assignment

SEQUELS IN SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    released INTEGER,
    sequel_id INTEGER);
INSERT INTO movies
    VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
    VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
    VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
    VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
```

```
    VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);

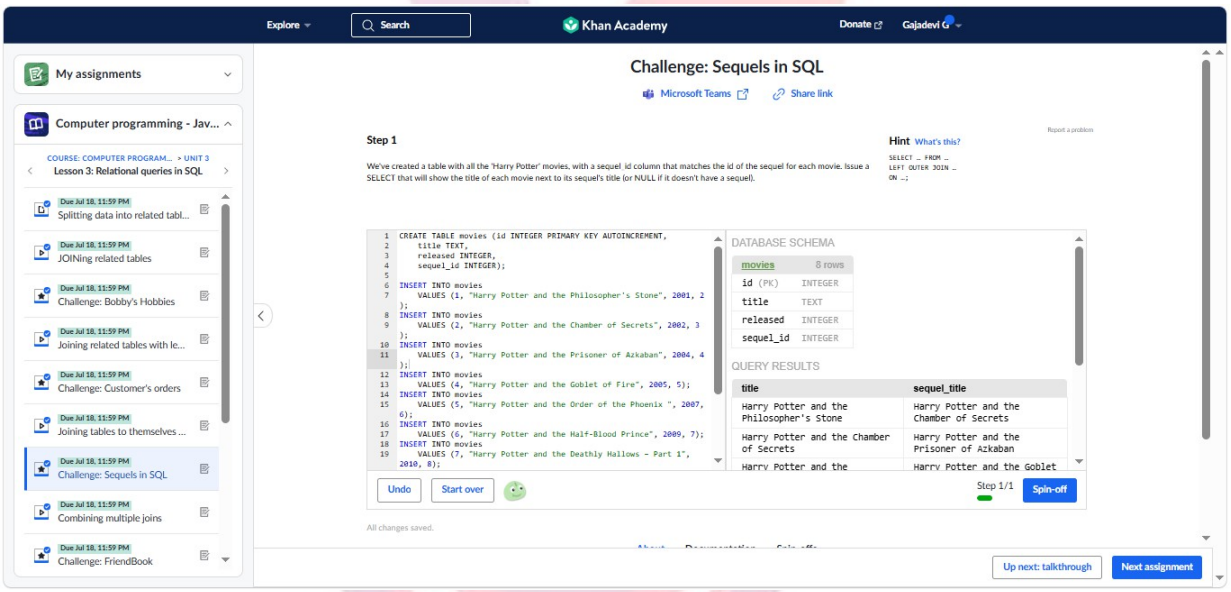
SELECT m1.title,

       m2.title AS sequel_title

FROM movies AS m1

LEFT OUTER JOIN movies AS m2

ON m1.sequel_id = m2.id;
```



FRIENDBOOK

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

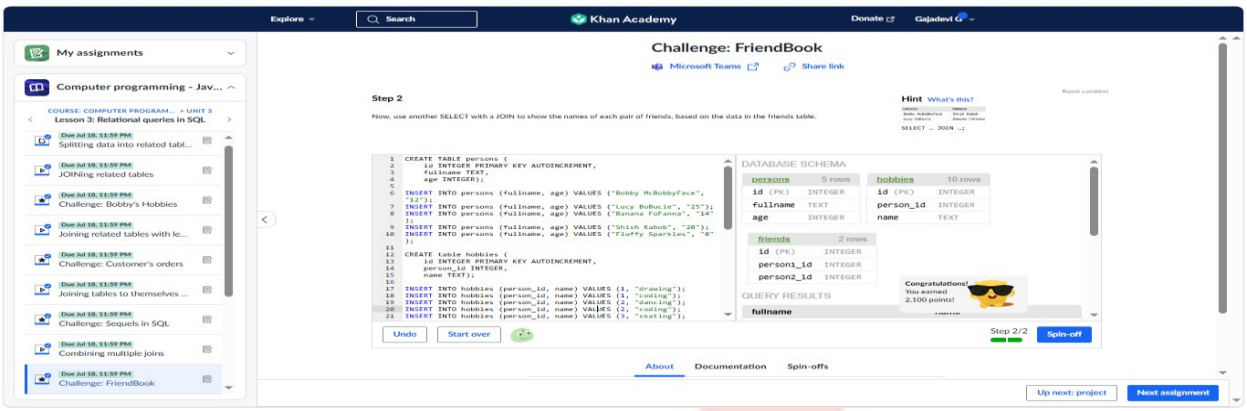
Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
```

```
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
CREATE table friends (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person1_id INTEGER,
   person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
   VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
   VALUES (2, 3);
SELECT persons.fullname,
      hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT p1.fullname AS person1,
      p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
```

JOIN persons AS p2

ON friends.person2_id = p2.id;



FAMOUS PEOPLE

Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Program:

```
CREATE TABLE authors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
```

```
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends
   ( id INTEGER PRIMARY
   KEY,
   author1_id INTEGER,
   author2_id INTEGER
);

INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
SELECT authors.name AS author,
      books.title AS book
FROM authors
JOIN books
ON authors.id = books.author_id;
SELECT name,
      country
FROM authors;
SELECT a1.name AS author1,
      a2.name AS author2
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

Output:

DATABASE SCHEMA

authors4 rows
id (PK)INTEGER
nameTEXT
countryTEXT

books6 rows
id (PK)INTEGER
titleTEXT
author_idINTEGER

author_friends2 rows
id (PK)INTEGER
author1_idINTEGER
author2_idINTEGER

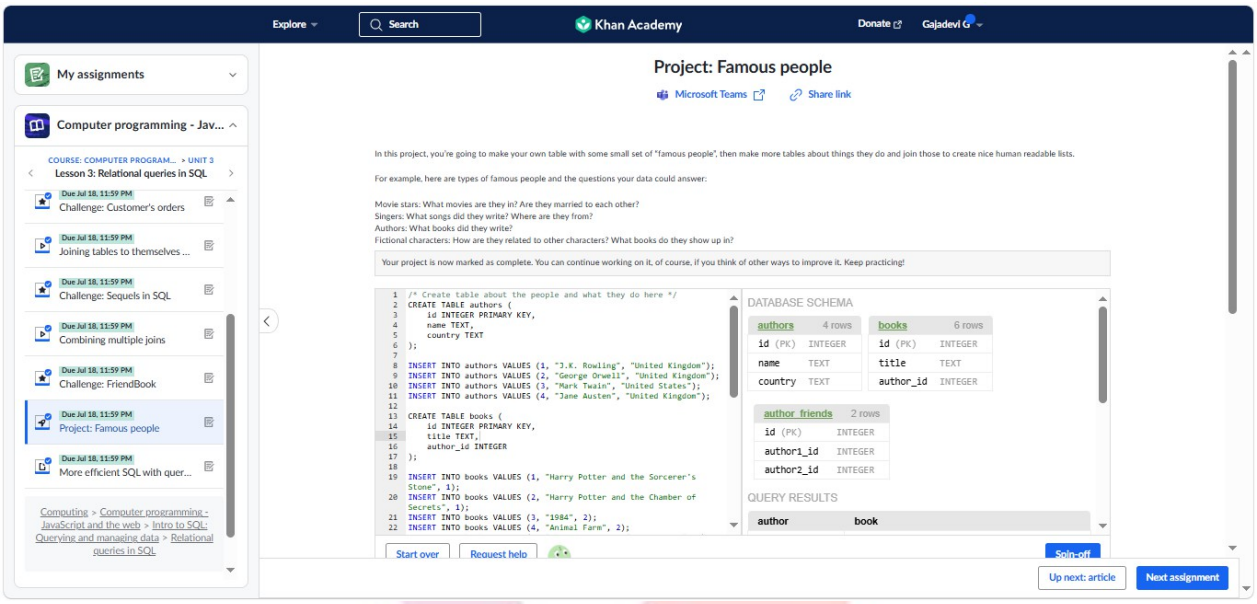
QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
J.K. Rowling	Harry Potter and the Chamber of Secrets
George Orwell	1984
George Orwell	Animal Farm

author	book
Mark Twain	The Adventures of Tom Sawyer
Jane Austen	Pride and Prejudice

name	country
J.K. Rowling	United Kingdom
George Orwell	United Kingdom
Mark Twain	United States
Jane Austen	United Kingdom

author1	author2
J.K. Rowling	George Orwell
Mark Twain	Jane Austen


Explore
Search
Khan Academy
Donate
Gajadevi G
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 3: Relational queries in SQL
Due Jul 18, 11:59 PM
Challenge: Customer's orders
Due Jul 18, 11:59 PM
Joining tables to themselves ...
Due Jul 18, 11:59 PM
Challenge: Sequels in SQL
Due Jul 18, 11:59 PM
Combining multiple joins
Due Jul 18, 11:59 PM
Challenge: FriendBook
Due Jul 18, 11:59 PM
Project: Famous people
Due Jul 18, 11:59 PM
More efficient SQL with quer...
Computing > Computer programming - JavaScript and the web > Intro to SQL: Querying and managing data > Relational queries in SQL
Project: Famous people
Microsoft Teams
Share link
In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.
For example, here are types of famous people and the questions your data could answer:
Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
/* Create table about the people and what they do here */
CREATE TABLE authors (
id INTEGER PRIMARY KEY,
name TEXT,
country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
id INTEGER PRIMARY KEY,
title TEXT,
author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
DATABASE SCHEMA
authors 4 rows
id (PK) INTEGER
name TEXT
country TEXT
books 6 rows
id (PK) INTEGER
title TEXT
author_id INTEGER
author_friends 2 rows
id (PK) INTEGER
author1_id INTEGER
author2_id INTEGER
QUERY RESULTS
author
book
Start over
Request help
Spin-off
Up next: article
Next assignment

DYNAMIC DOCUMENTS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

Program:

```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    content TEXT,
    author TEXT);
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
```

SET author = "Jackie Draper"

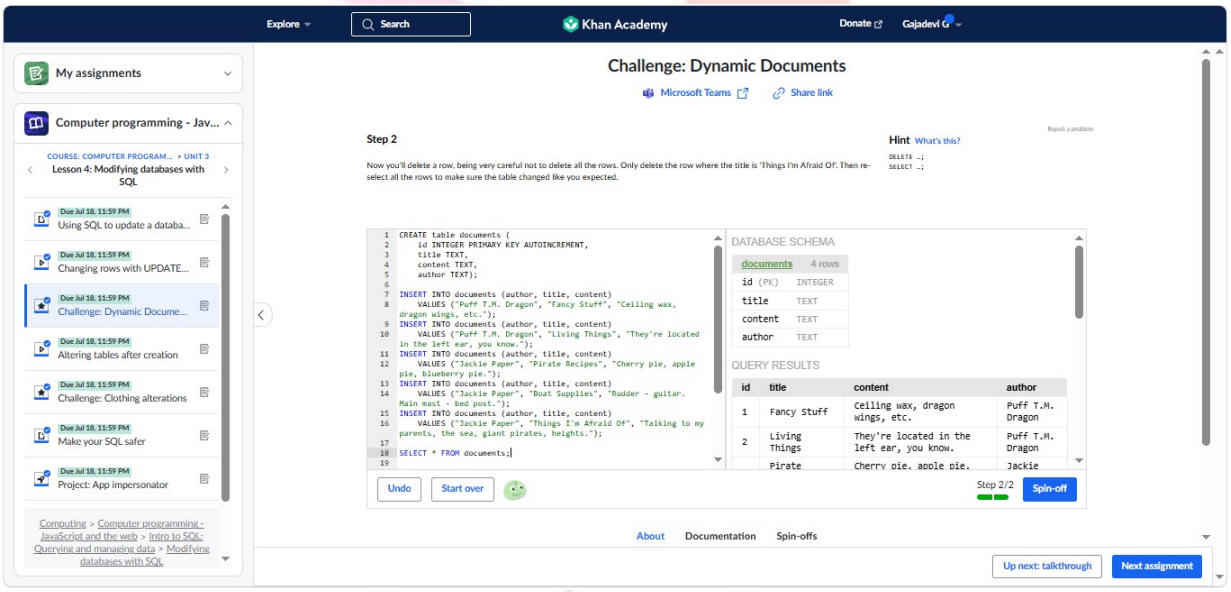
WHERE author = "Jackie Paper";

SELECT * FROM documents;

DELETE FROM documents

WHERE title = "Things I'm Afraid Of";

SELECT * FROM documents;



CLOTHING ALTERATIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Program:

```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);
INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
```

```
WHERE id = 1;

UPDATE clothes

SET price = 20

WHERE id = 2;

UPDATE clothes

SET price = 30

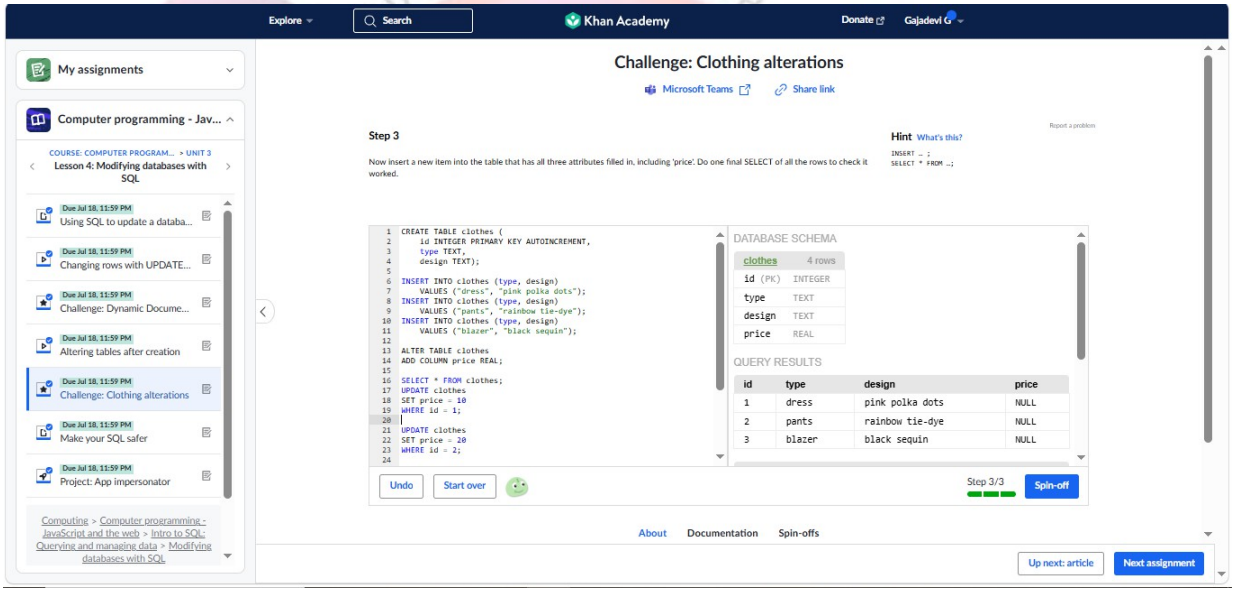
WHERE id = 3;

SELECT * FROM clothes;

INSERT INTO clothes (type, design, price)

VALUES ("shirt", "blue stripes", 15);

SELECT * FROM clothes;
```



PROJECT: APP IMPERSONATO

CREATE a table to store the data.
INSERT a few example rows in the table.
Use an UPDATE to emulate what happens when you edit data in the app.
Use a DELETE to emulate what happens when you delete data in the app.

Program:

```
CREATE TABLE posts (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT,
   content TEXT,
   likes INTEGER
);
INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);
INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);
SELECT * FROM posts;
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
SELECT * FROM posts;
DELETE FROM posts
WHERE id = 2;
SELECT * FROM posts;
```

Output:

DATABASE SCHEMA

posts2 rows
id (PK)INTEGER
usernameTEXT

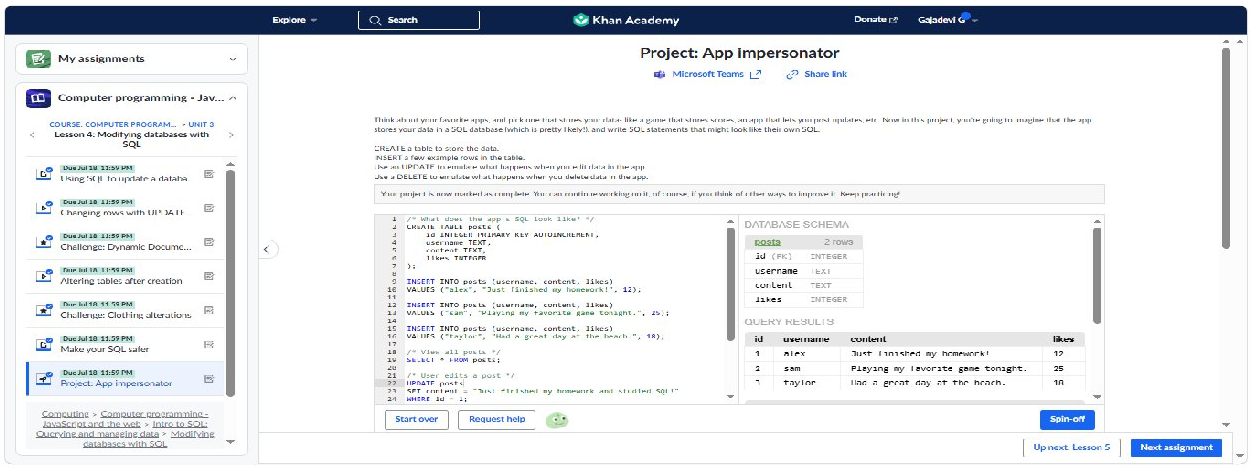
posts2 rows
contentTEXT
likesINTEGER

QUERY RESULTS

id	username	content	likes
1	alex	Just finished my homework!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
3	taylor	Had a great day at the beach.	18



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

HACKER RANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the **CITY** table with populations larger than `100000`.

The **CountryCode** for America is `USA`.

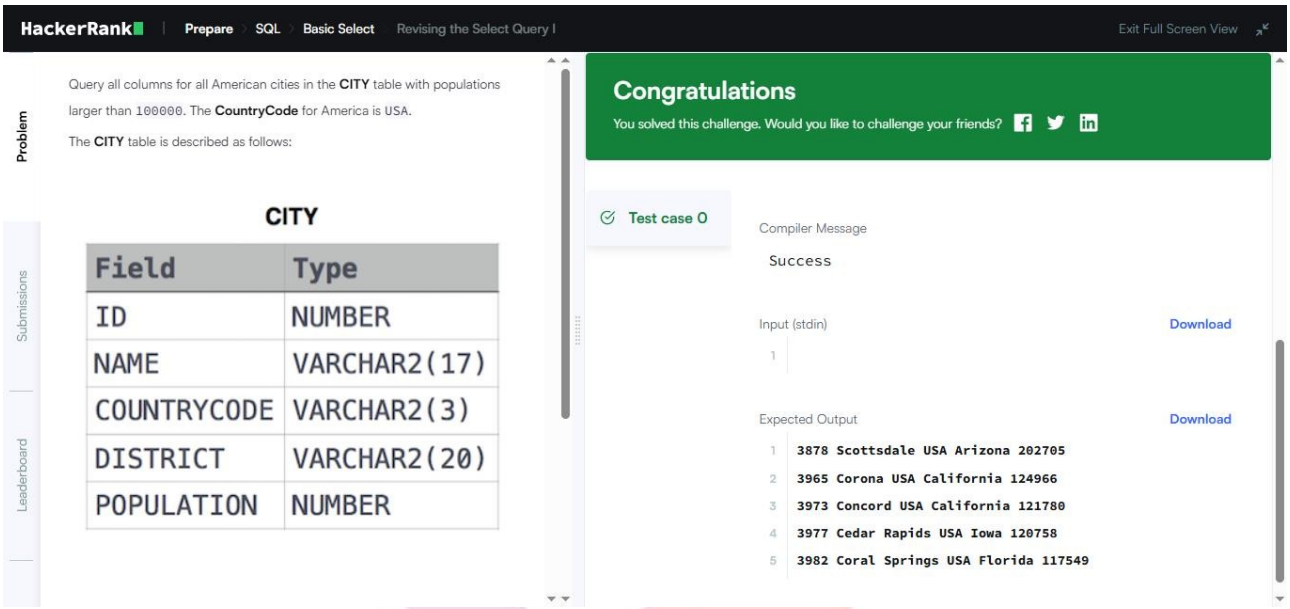
The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select * from city where countrycode = 'usa' and population > 100000;
```

AUTONOMOUS



REVISING THE SELECT QUERY II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than `120000`.

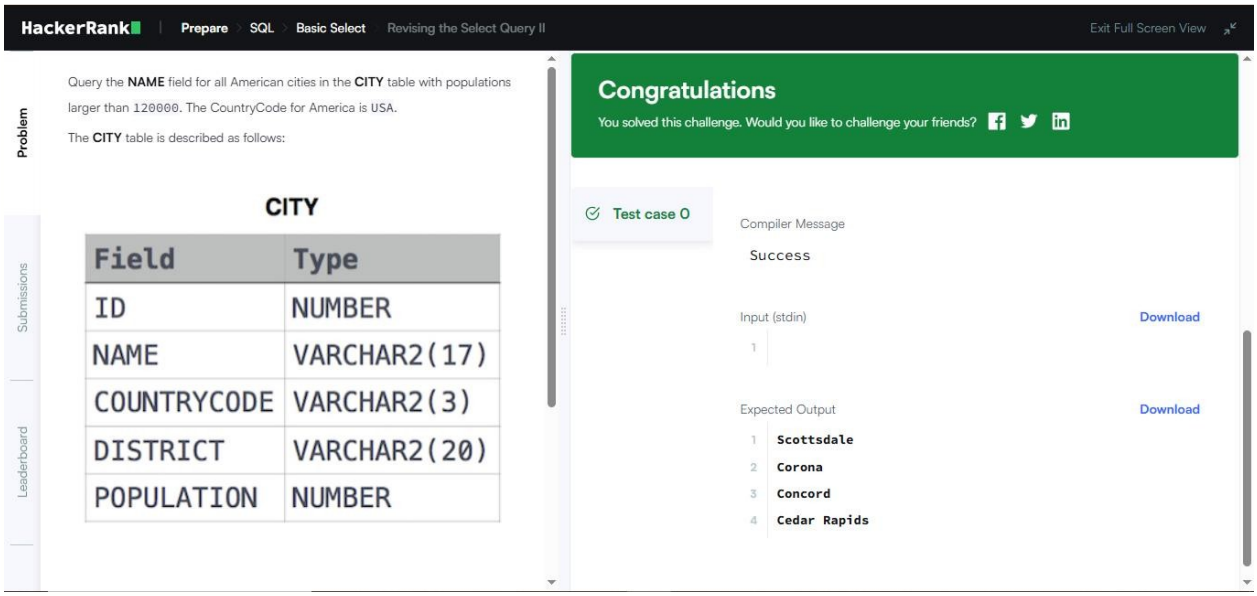
The *CountryCode* for America is `USA`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select name from city where countrycode = 'usa' and population > 120000;
```



SELECT ALL

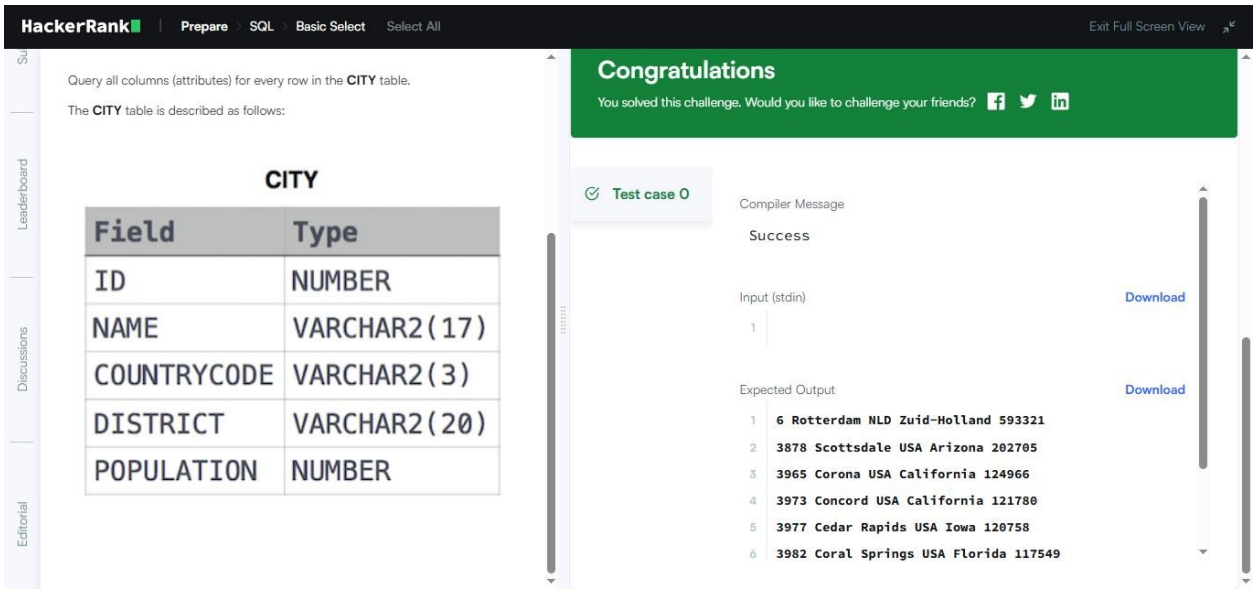
Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city;



SELECT BY ID

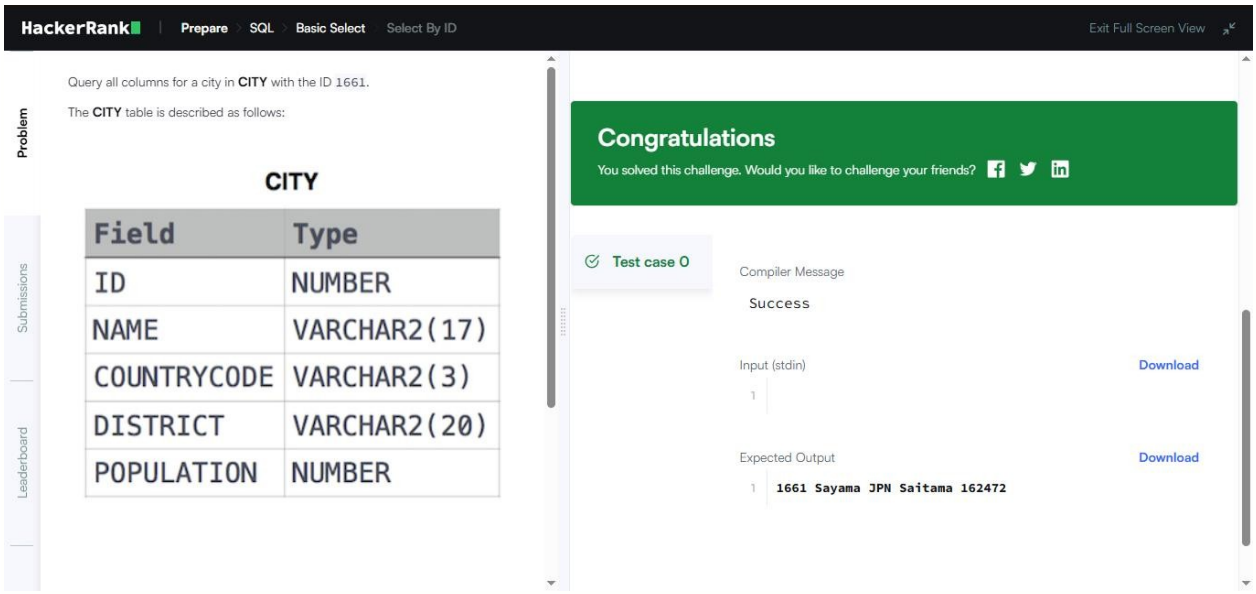
Query all columns for a city in **CITY** with the *ID* `1661`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where id = 1661;



JAPANESE CITIES' ATTRIBUTES

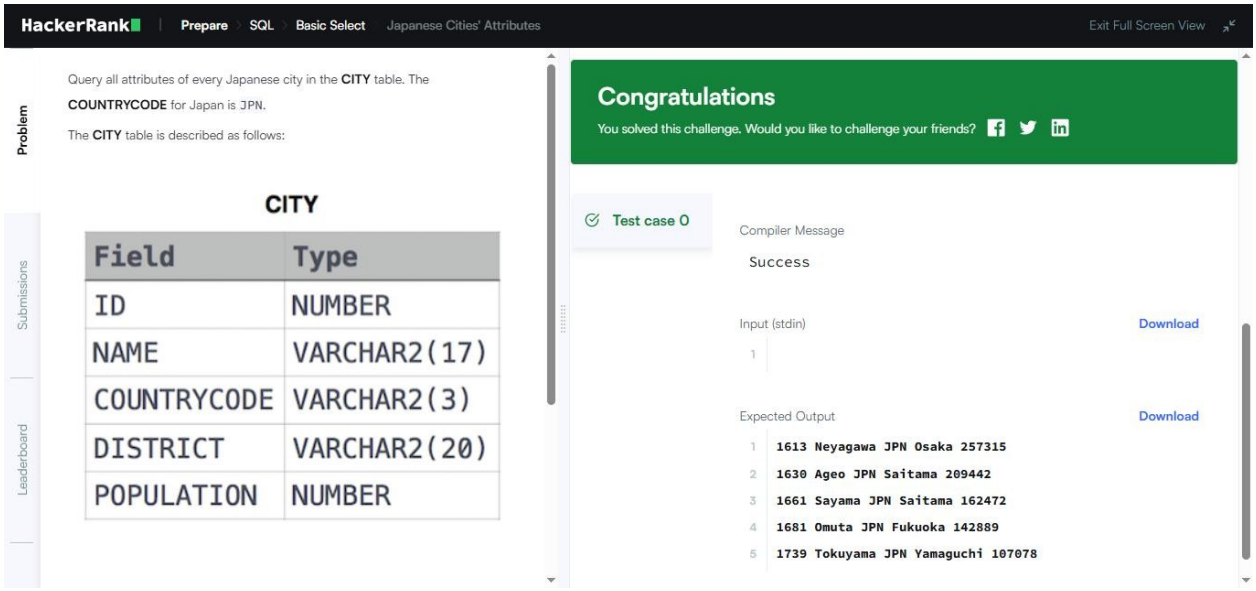
Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where countrycode = 'jpn';



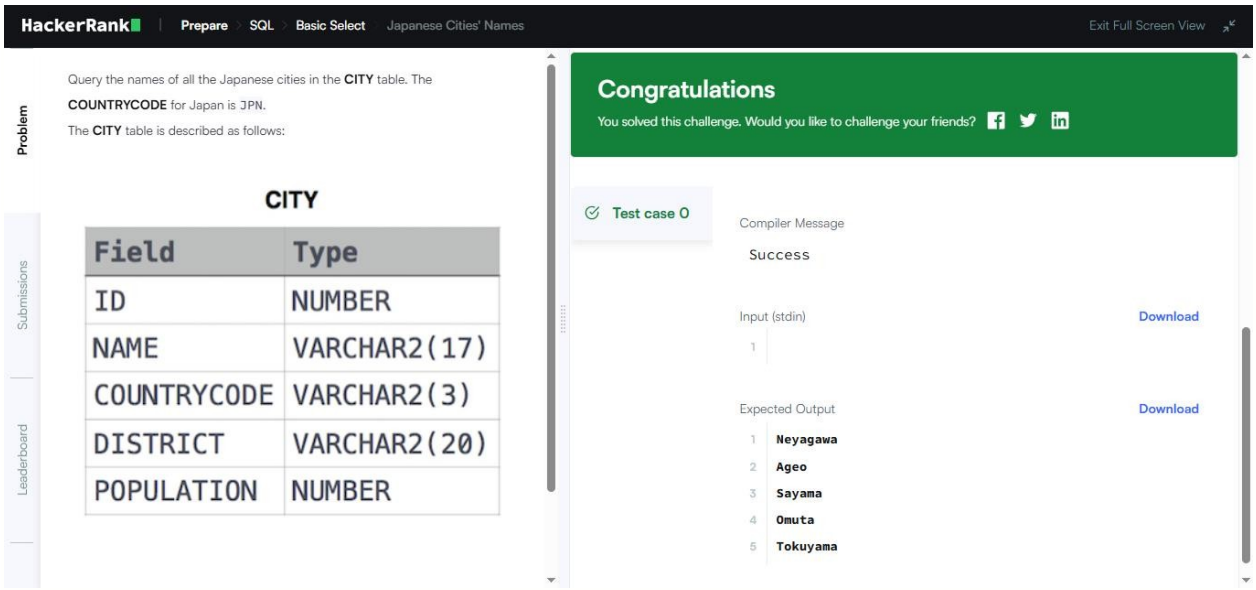
JAPANESE CITIES' NAMES

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.
The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select name from city where countrycode = 'jpn';



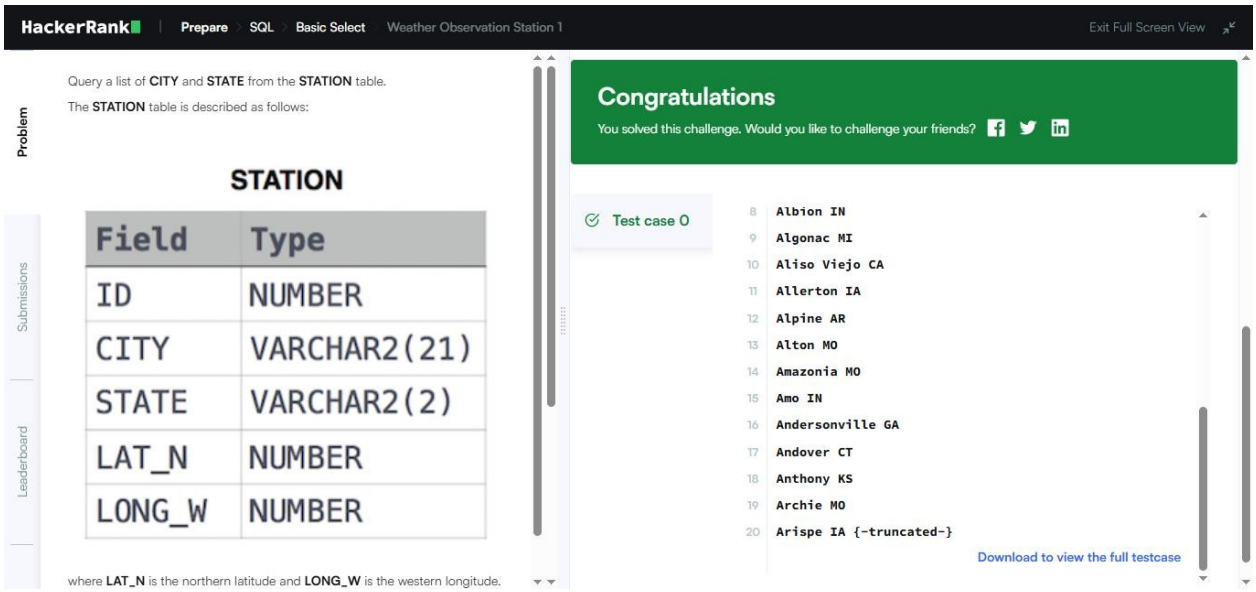
WEATHER OBSERVATION STATION 1

Query a list of **CITY** and **STATE** from the **STATION** table.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, state from station;



WEATHER OBSERVATION STATION 3

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.

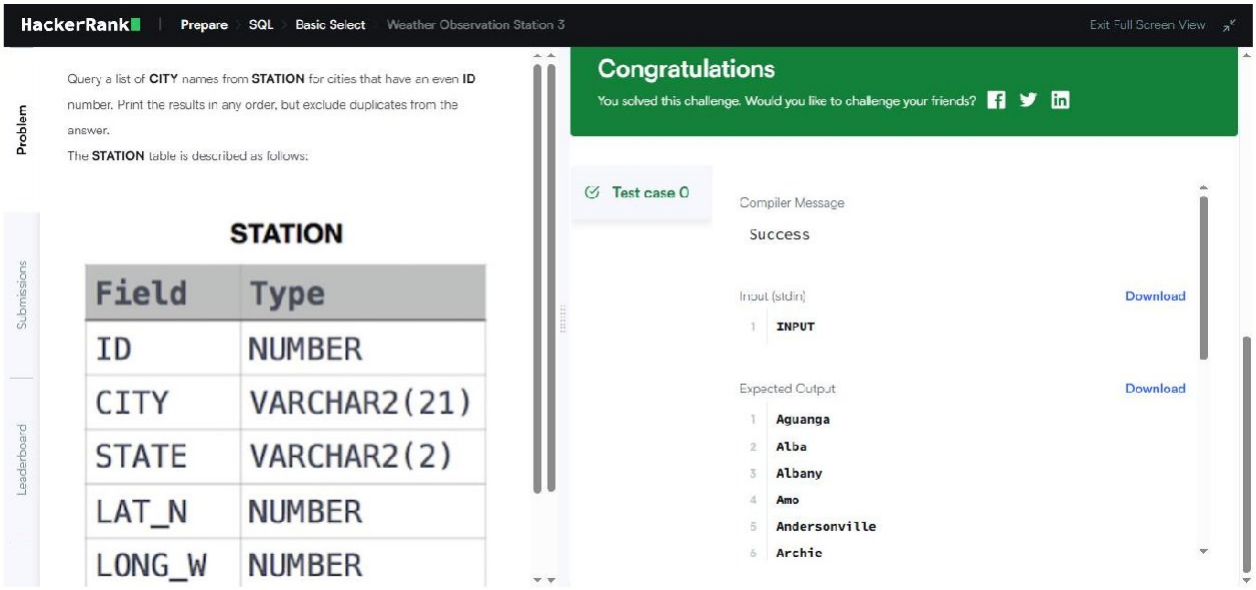
The **STATION** table is described as follows:

AUTONOMOUS

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where id % 2 = 0;



WEATHER OBSERVATION STATION 4

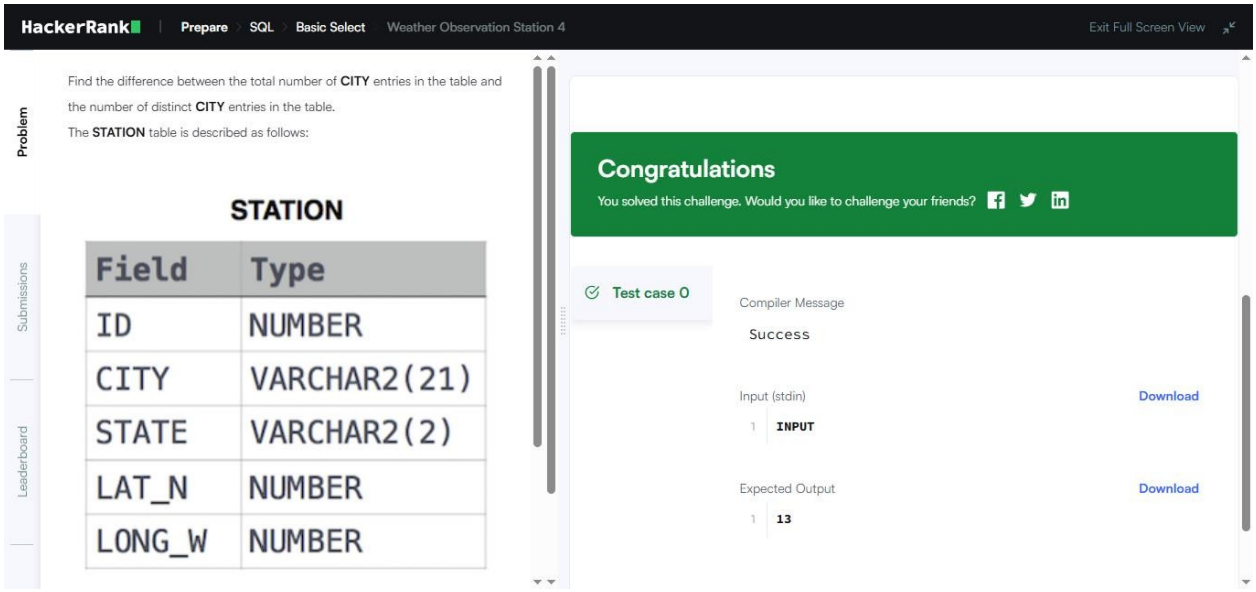
Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select count(city) - count(distinct city)
from station;
```



WEATHER OBSERVATION STATION 5

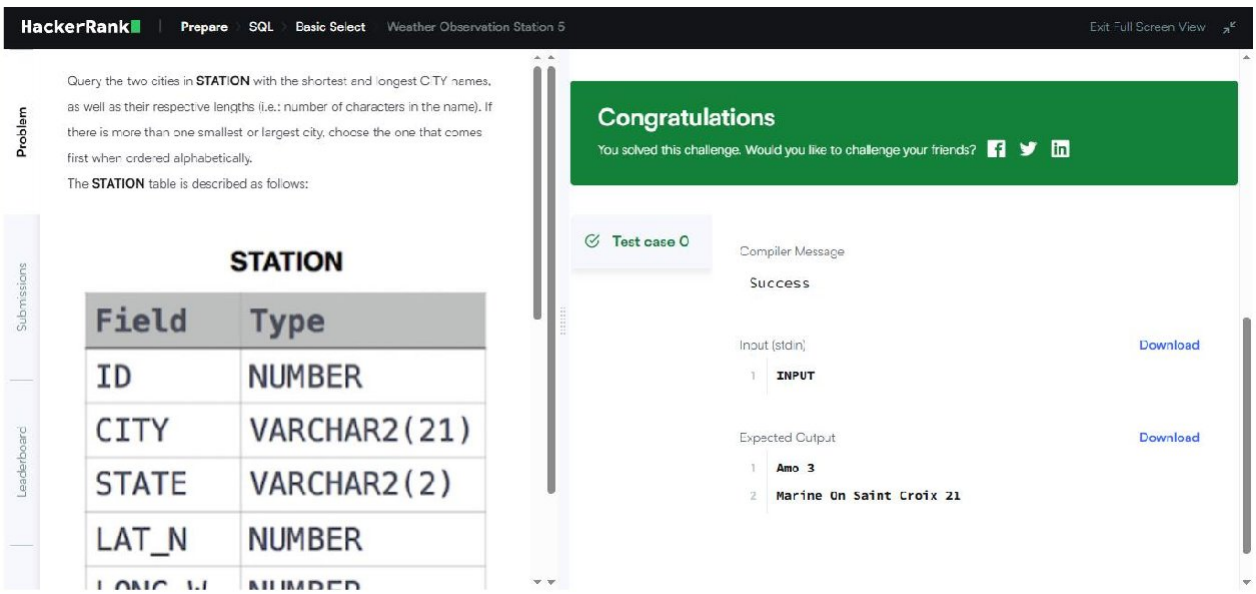
Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, length(city)from station order by length(city) asc ,city asc limit 1;
select city, length(city)from stationorder by length(city) desc, city asc limit 1;



WEATHER OBSERVATION STATION 6

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result *cannot* contain duplicates.

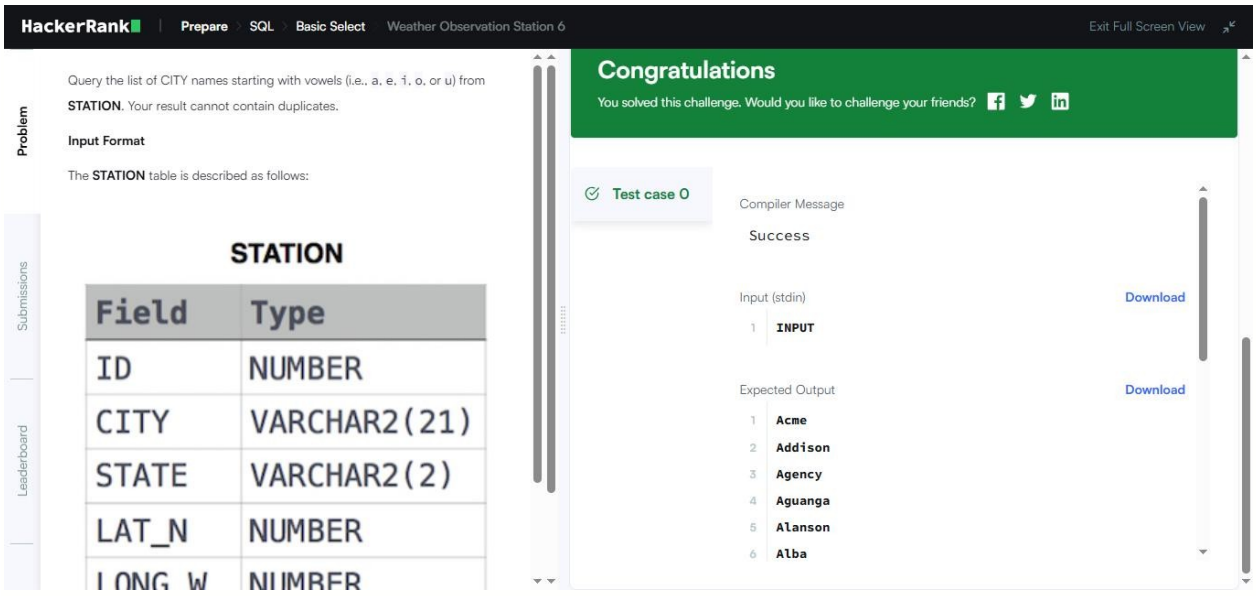
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiouaeiou]';



WEATHER OBSERVATION STATION 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

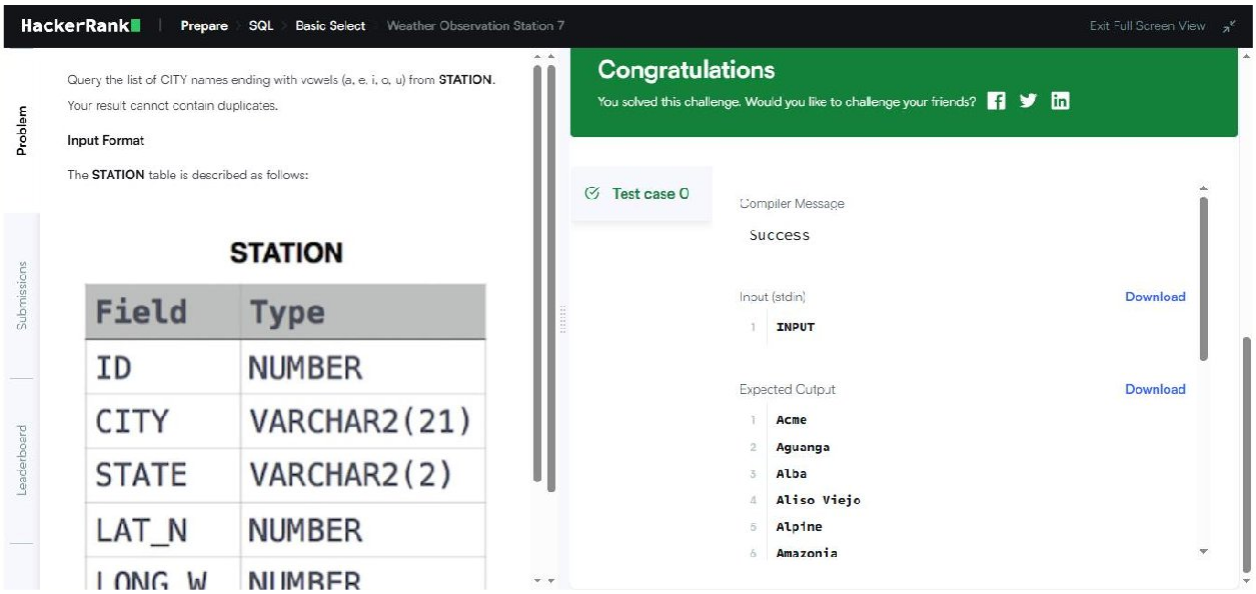
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '[aeiouaeiou]$';



WEATHER OBSERVATION STATION 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

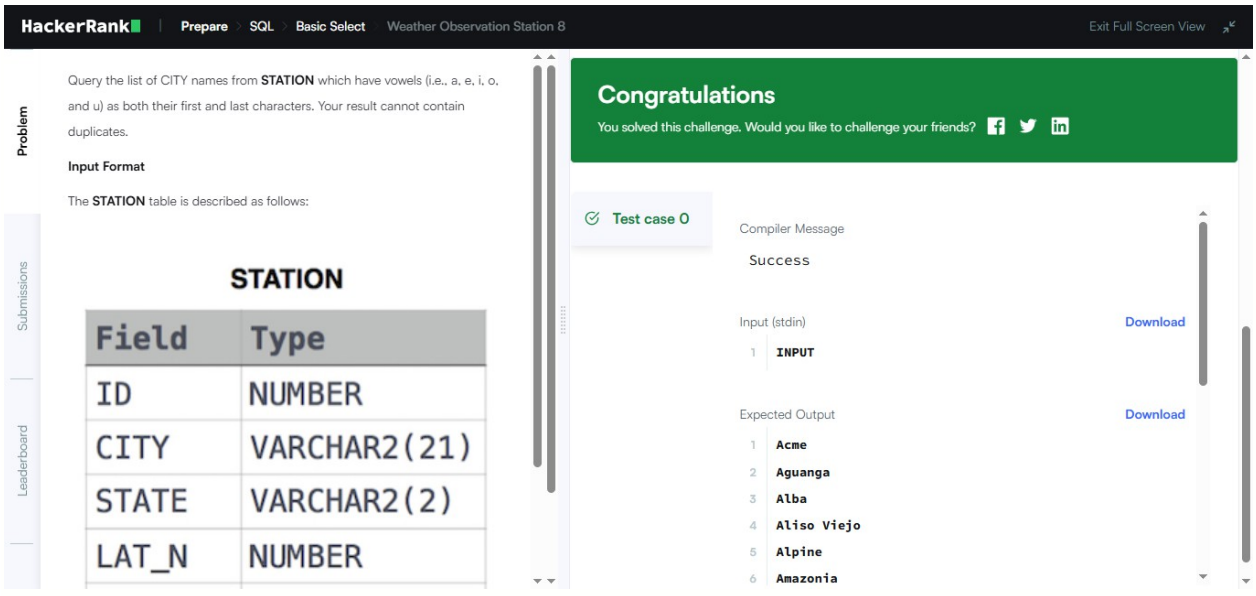
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiou].*[aeiou]$';



WEATHER OBSERVATION STATION 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

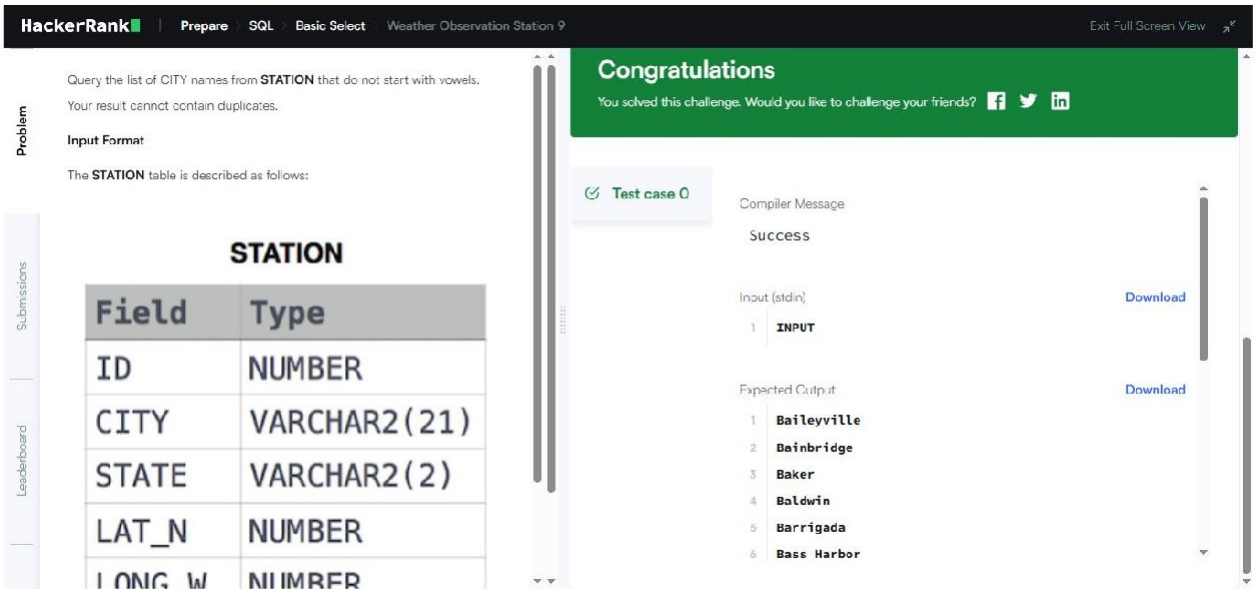
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiou].*[aeiou]$';



WEATHER OBSERVATION STATION 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

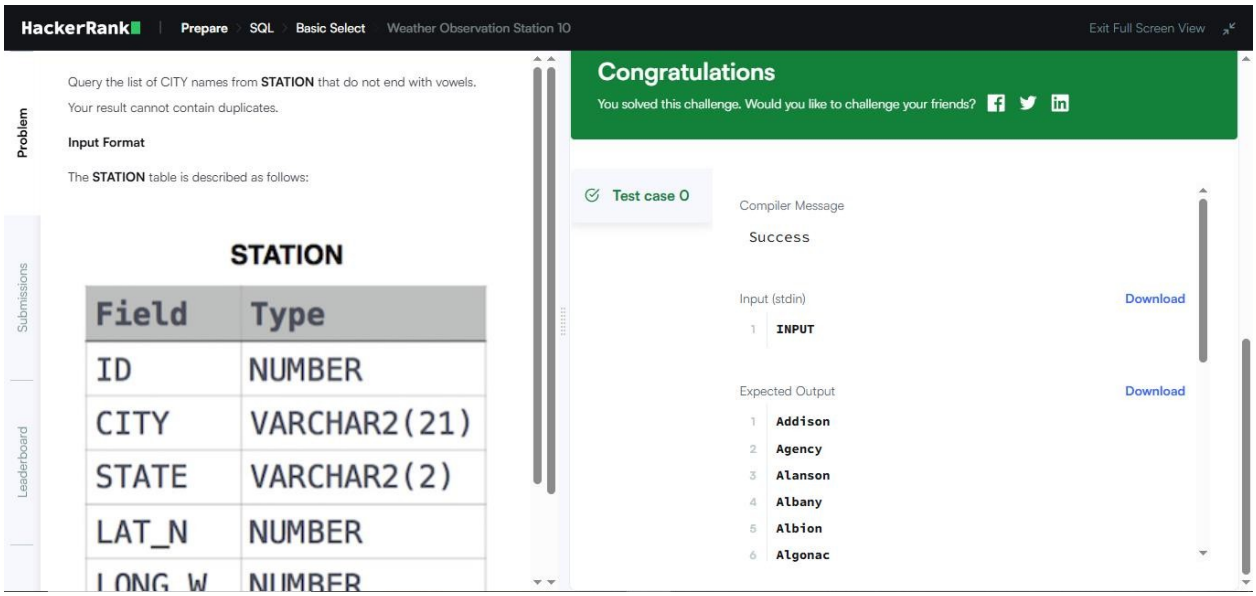
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '[^aeiouaeiou]$';



WEATHER OBSERVATION STATION 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels.

Your result cannot contain duplicates.

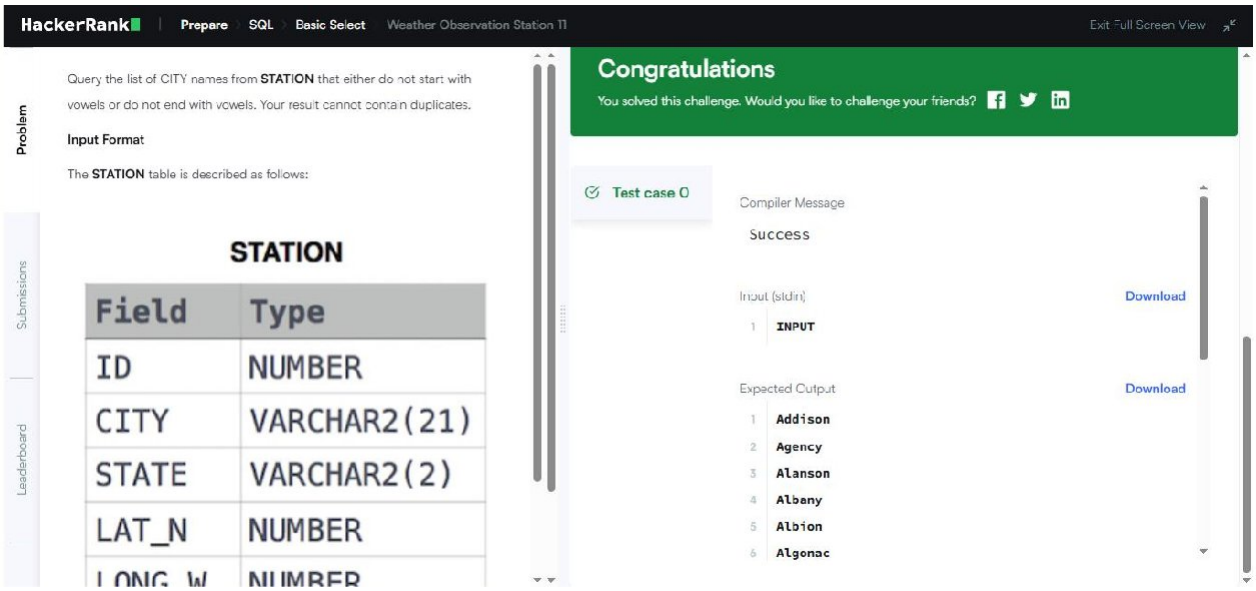
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where left(city, 1) not in ('a','e','i','o','u','a','e','i','o','u') or right(city, 1) not in ('a','e','i','o','u','a','e','i','o','u');



WEATHER OBSERVATION STATION 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

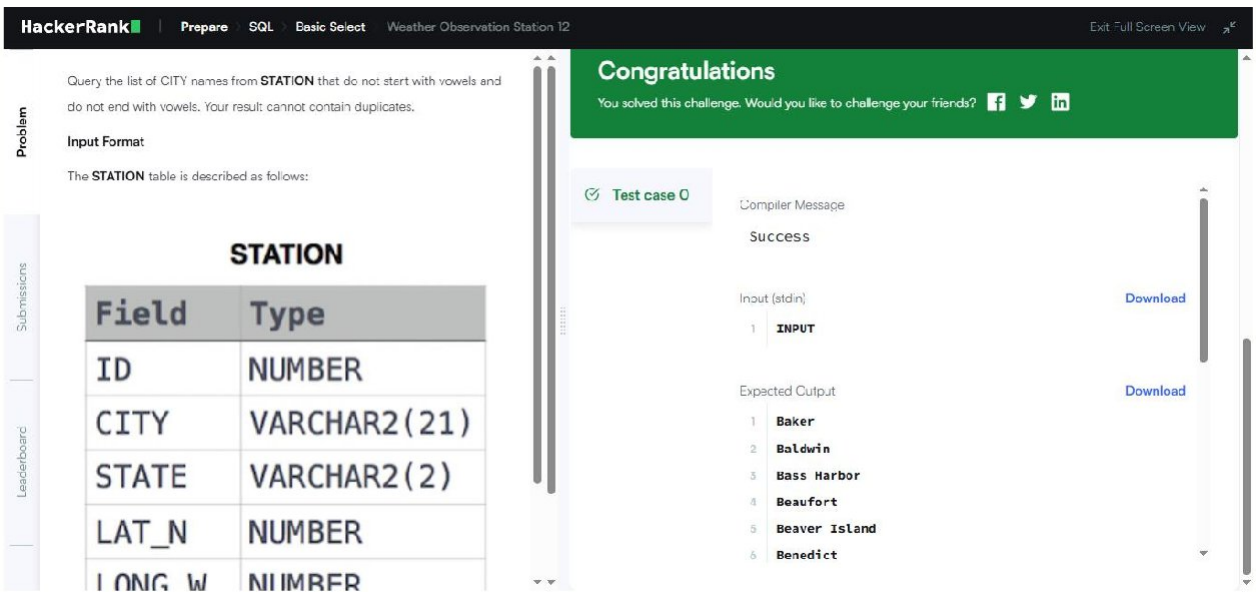
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city not regexp '^[aeiouaeiou]' and city not regexp '[aeiouaeiou]$';
```



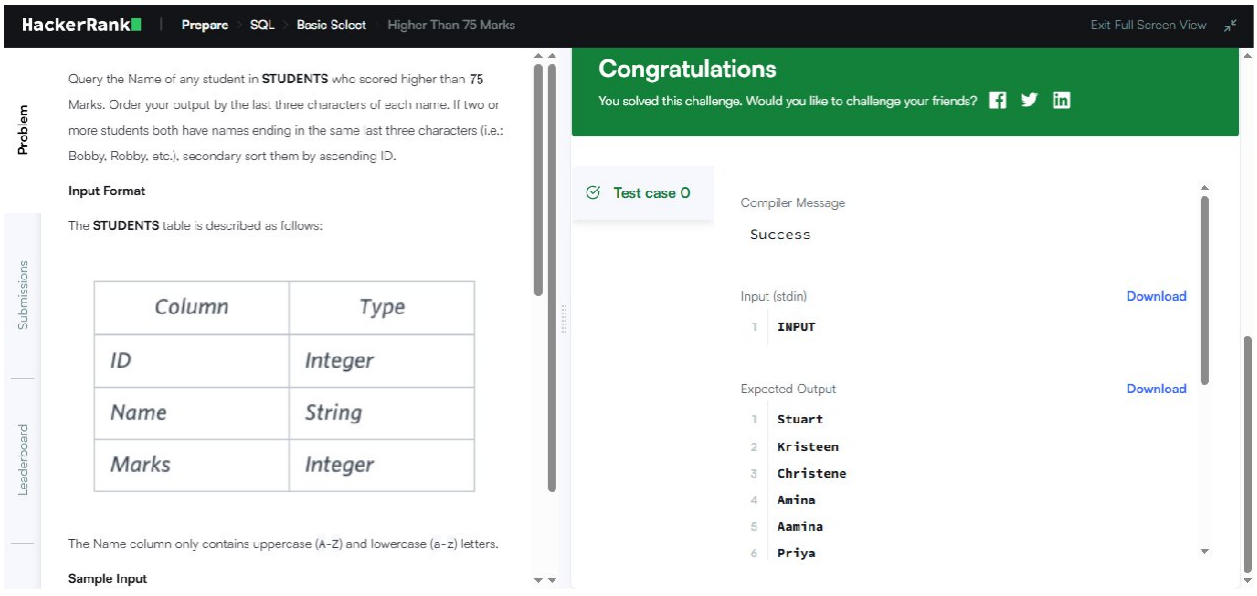
HIGHER THAN 75 MARKS

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

Column	Type
ID	Integer
Name	String
Marks	Integer

select name from students where marks > 75 order by right(name, 3), id asc;



EMPLOYEE NAMES

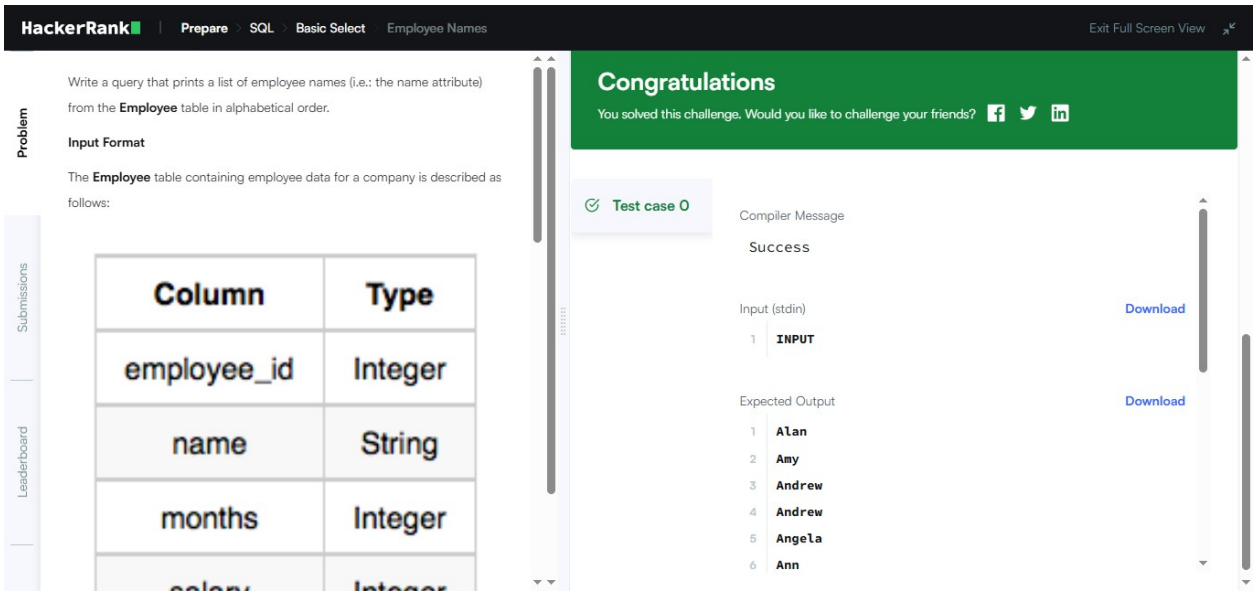
Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee order by name;



EMPLOYEE SALARIES

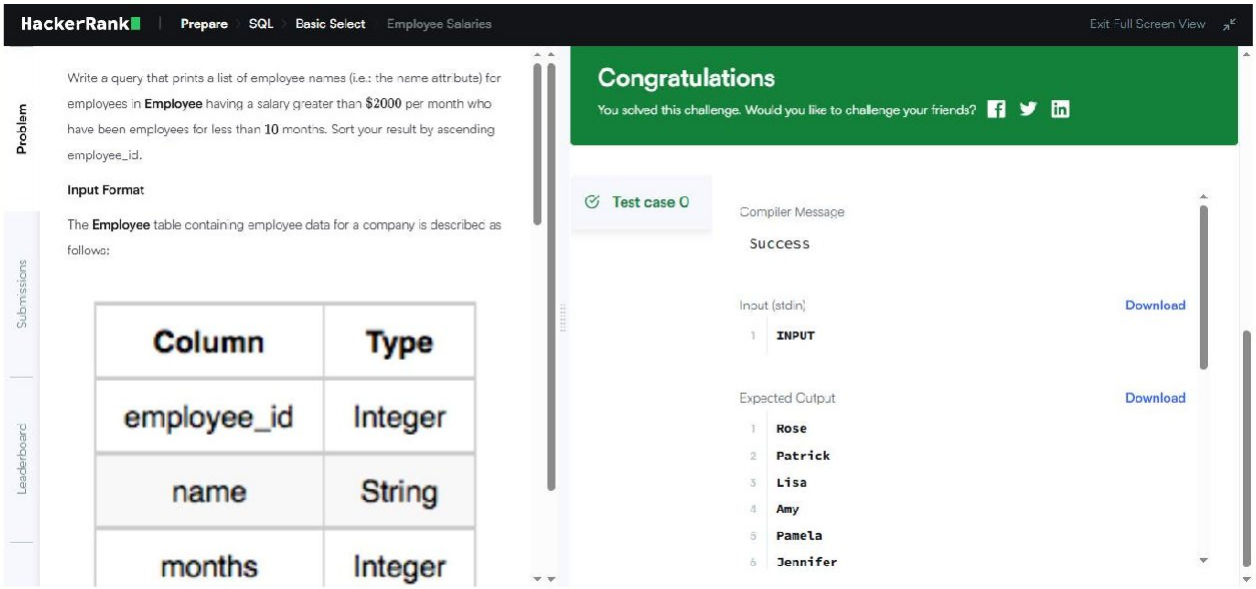
Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee where salary > 2000 and months < 10 order by employee_id asc;



TYPE OF TRIANGLE

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

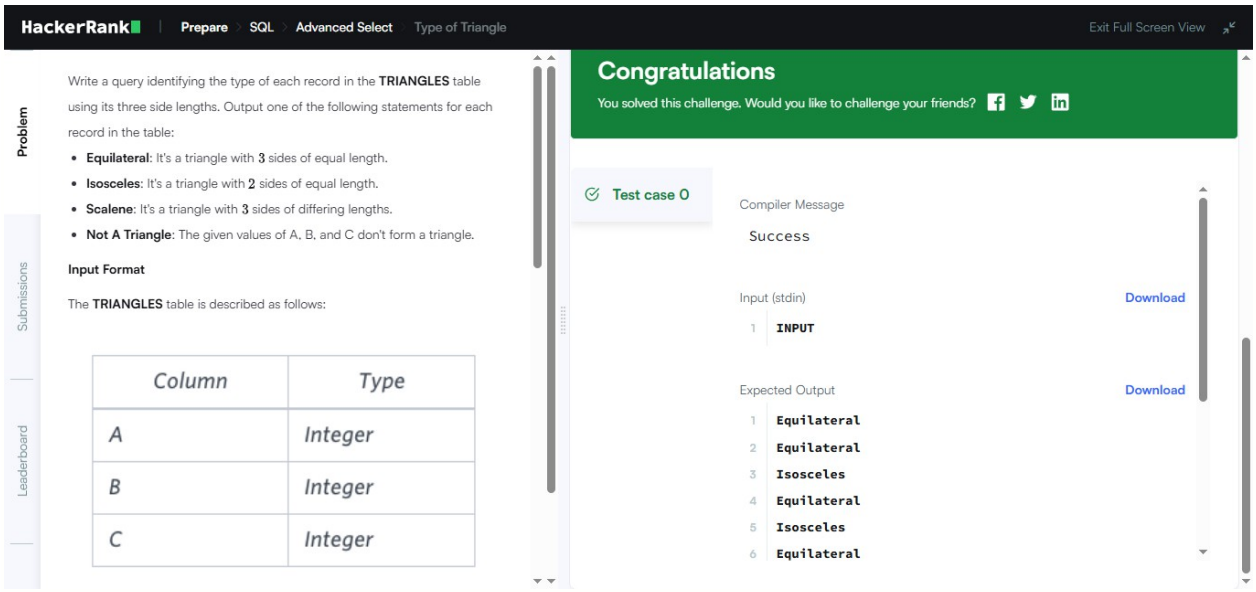
- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

```
select
   case
      when a + b <= c or a + c <= b or b + c <= a then 'not a triangle'
      when a = b and b = c then 'equilateral'
      when a = b or b = c or a = c then 'isosceles'
      else 'scalene'
   end
from triangles;
```



REVISING AGGREGATIONS - THE COUNT FUNCTION

Query a *count* of the number of cities in **CITY** having a *Population* larger than .

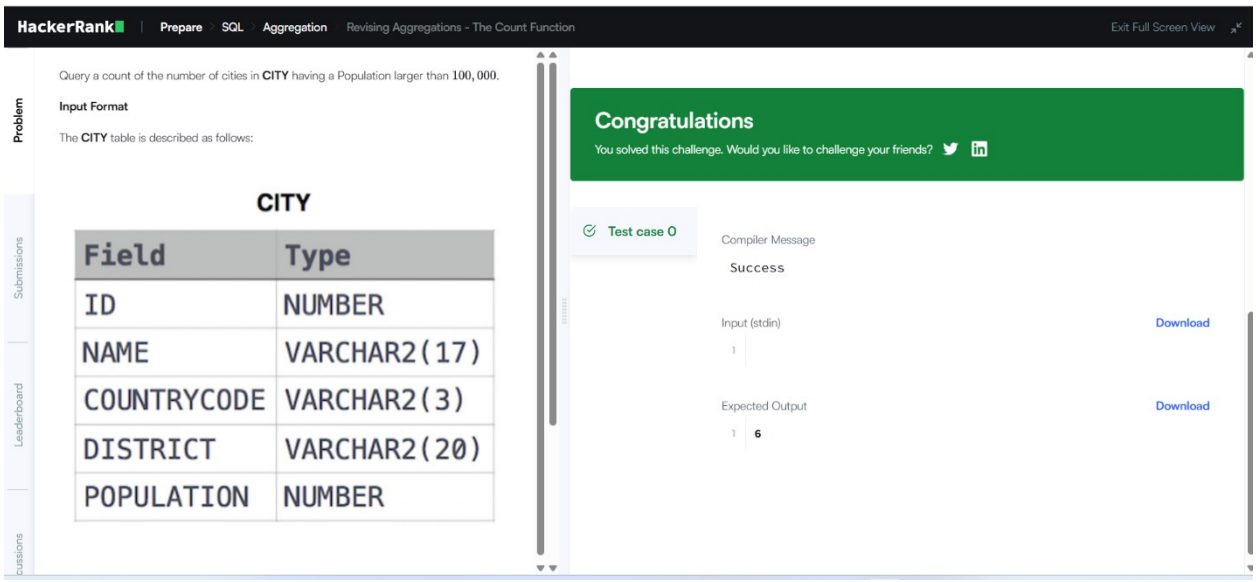
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;



REVISING AGGREGATIONS - THE SUM FUNCTION

Query the total population of all cities in **CITY** where *District* is **California**.

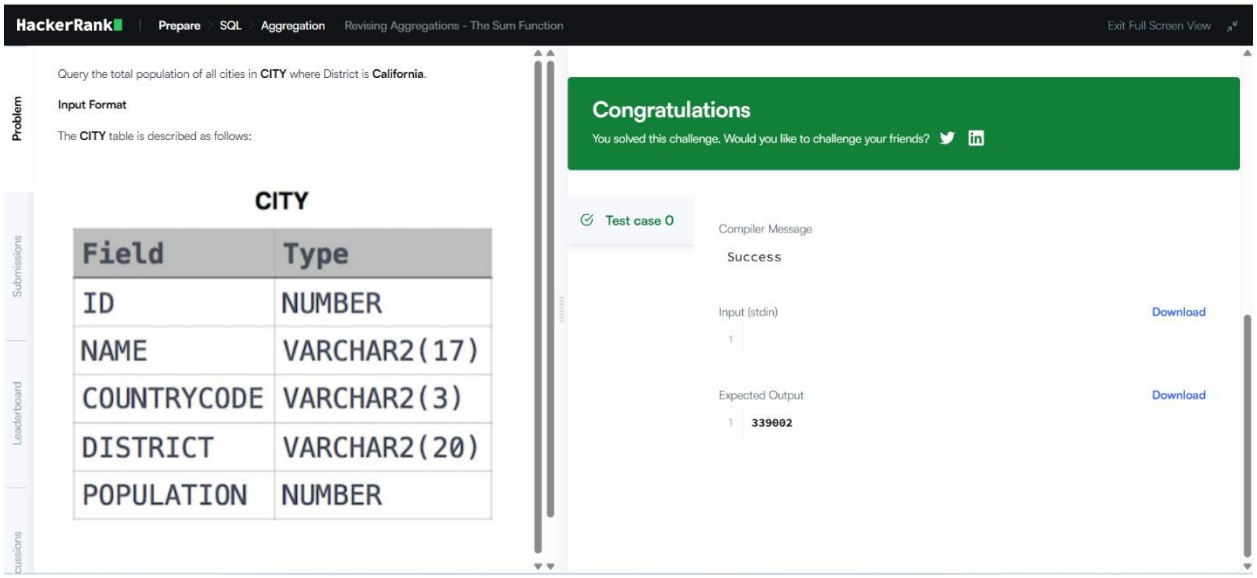
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

SELECT SUM(POPULATION) FROM CITY WHERE DISTRICT = 'California';



REVISING AGGREGATIONS – AVERAGES

Query the average population of all cities in **CITY** where *District* is **California**.

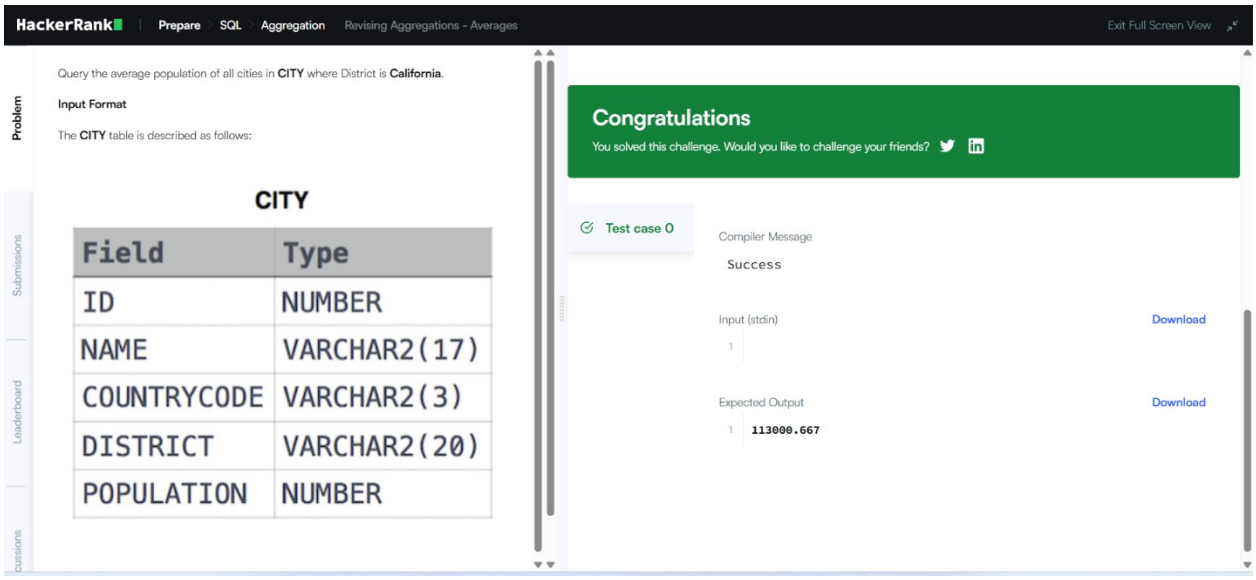
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```



AVERAGE POPULATION

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

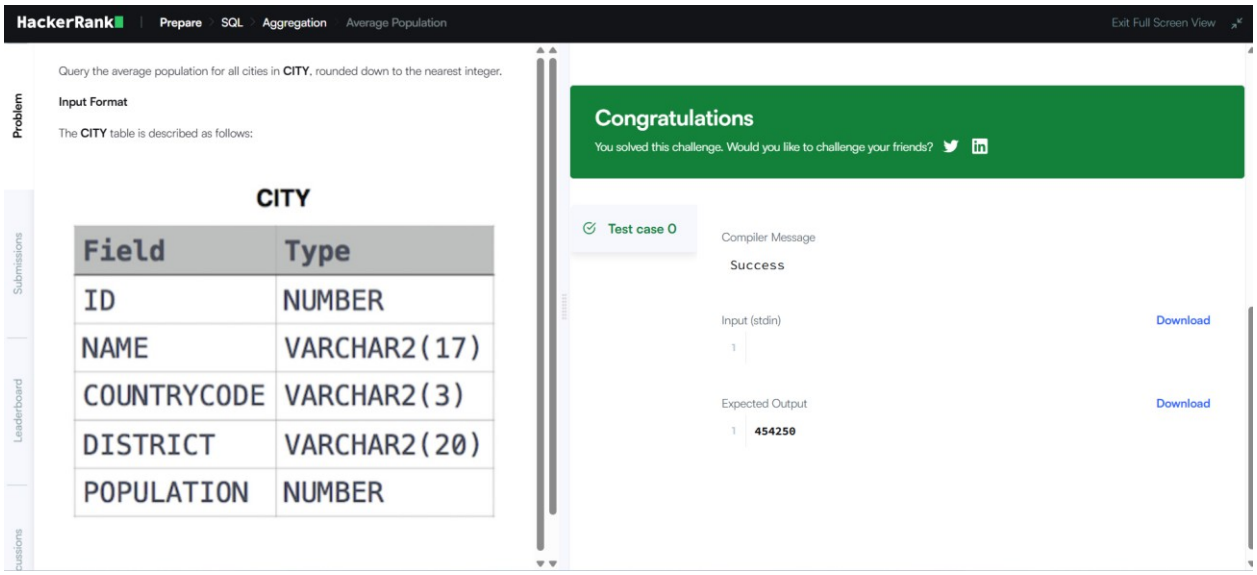
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT  FLOOR (AVG(POPULATION)) FROM CITY;
```



JAPAN POPULATION

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

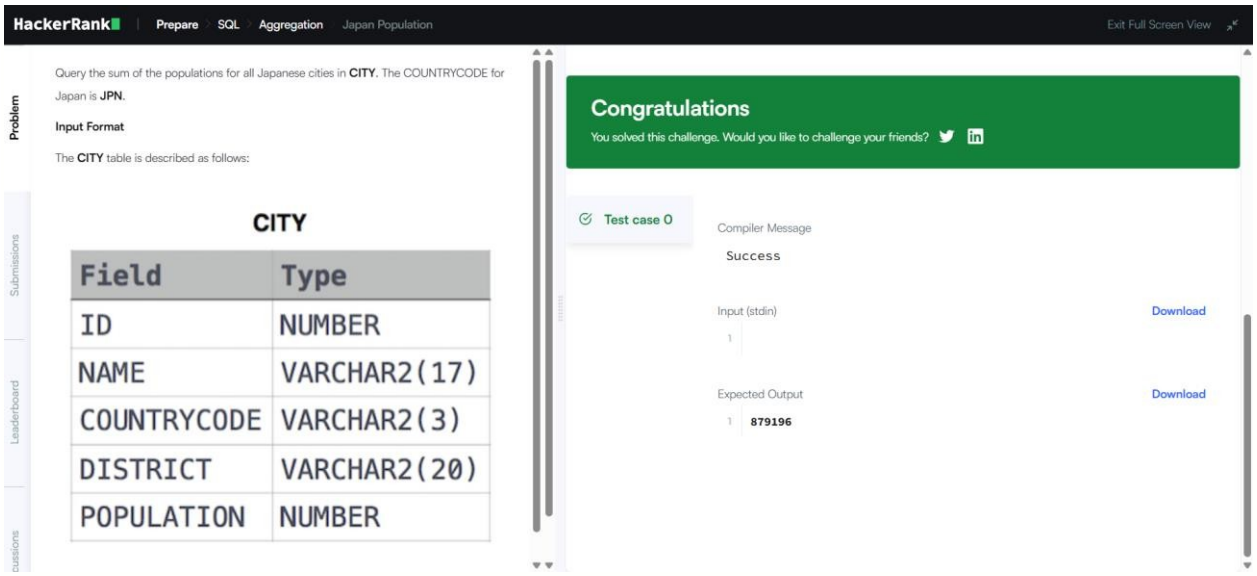
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```



POPULATION DENSITY DIFFERENCE

Query the difference between the maximum and minimum populations in **CITY**.

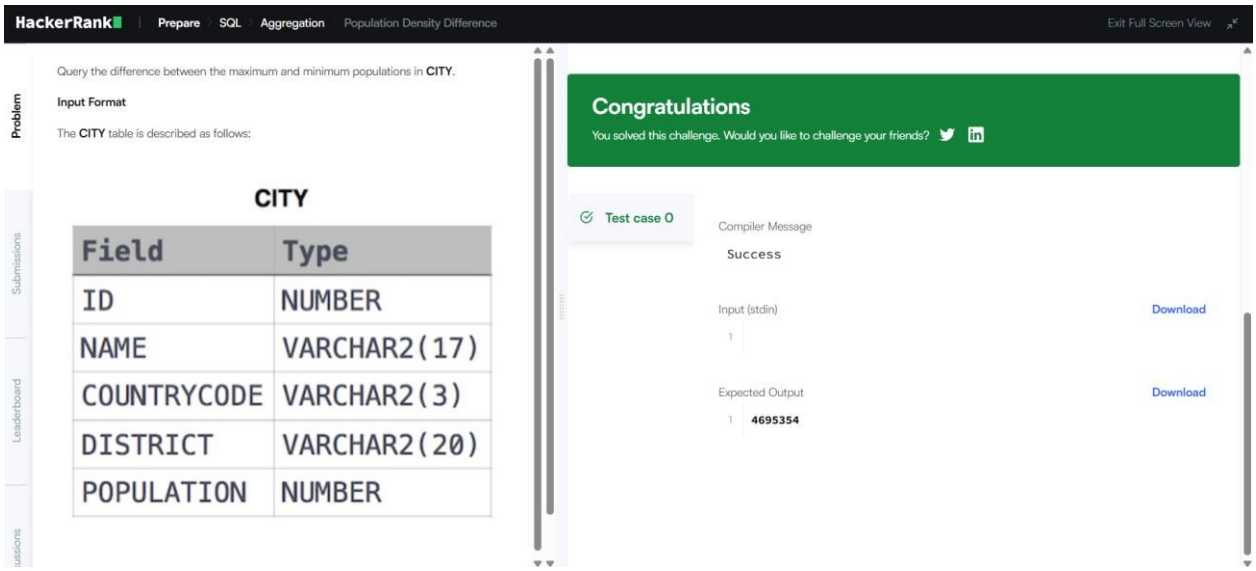
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```



THE BLUNDER

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

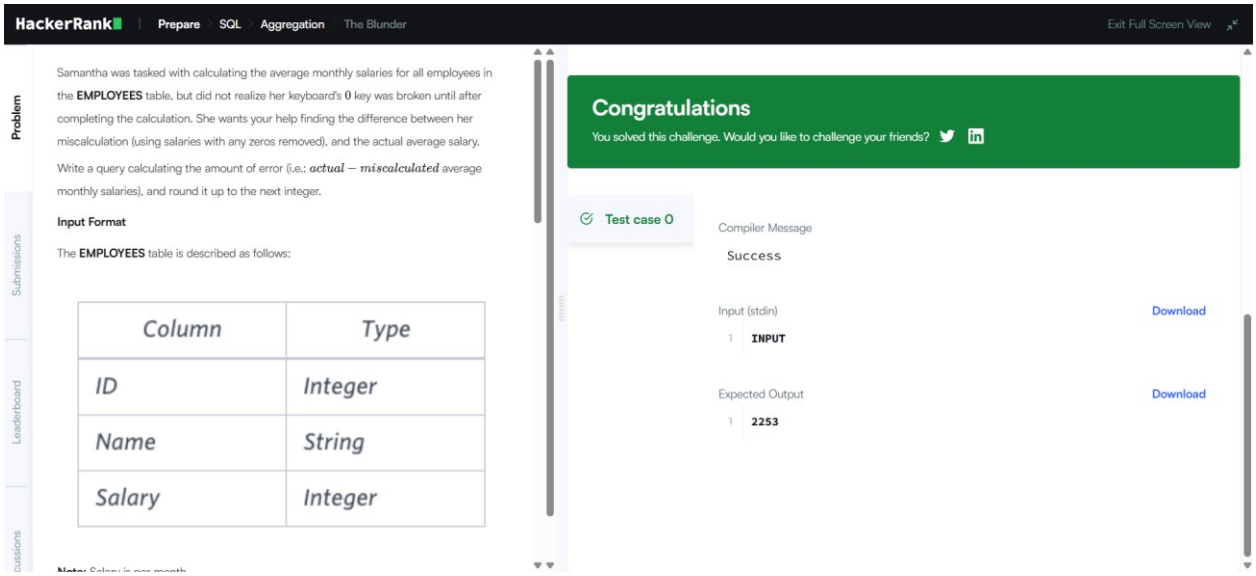
Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	*Type*
ID	*Integer*
Name	*String*
Salary	*Integer*

SELECT CEIL(AVG(Salary) - AVG(CAST(REPLACE(Salary, '0', '') AS UNSIGNED)))FROM EMPLOYEES;



TOP EARNERS

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

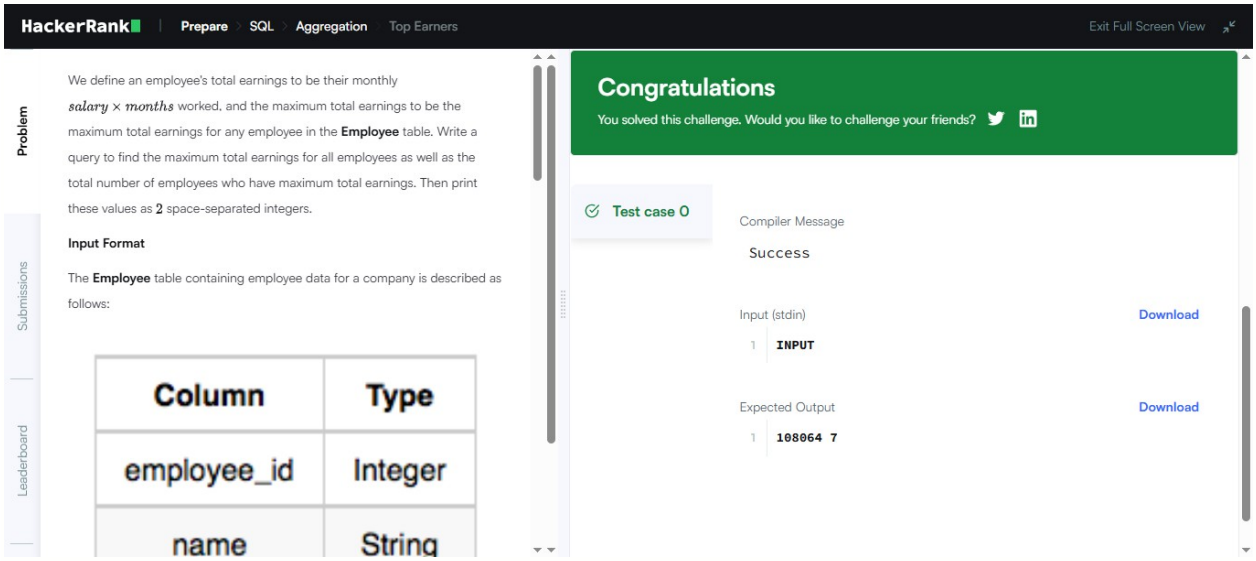
Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

```
SELECT
    (salary * months) AS earnings,
    COUNT(*) AS total_count
FROM EMPLOYEE
GROUP BY earnings
ORDER BY earnings DESC
LIMIT 1;
```

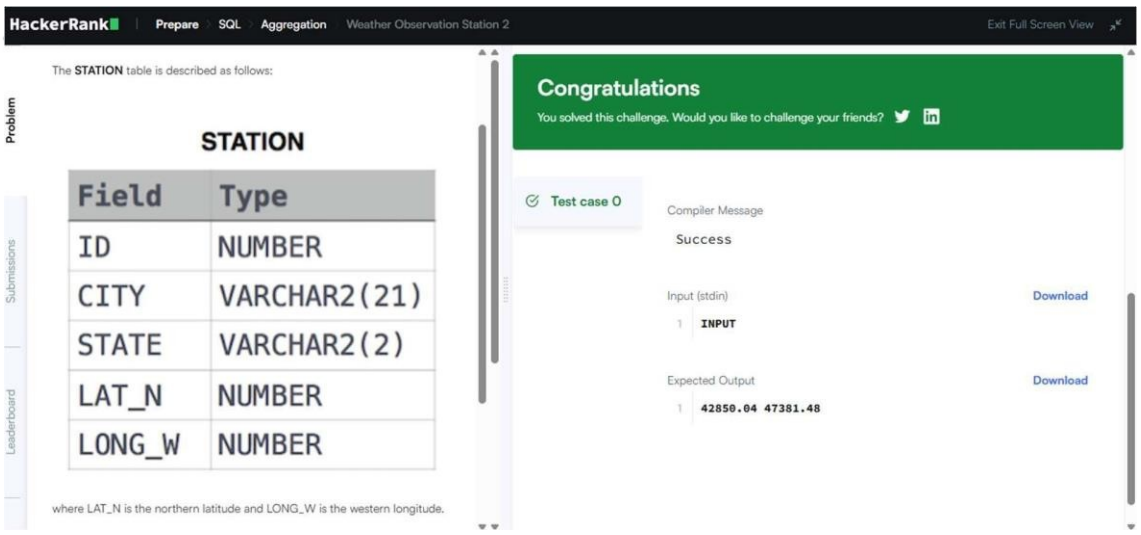


WEATHER OBSERVATION STATION 2

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER



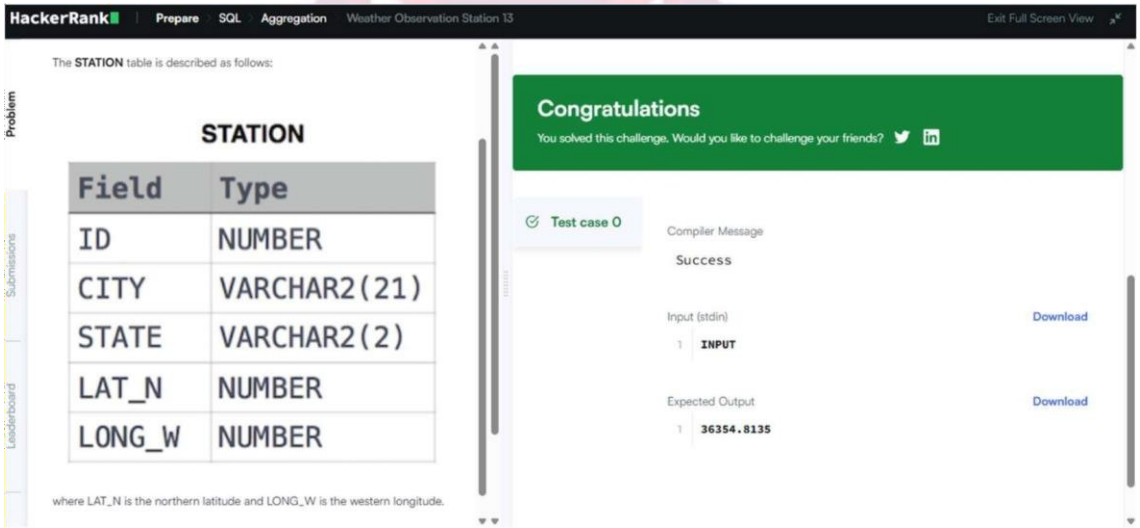
WEATHER OBSERVATION STATION 13

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



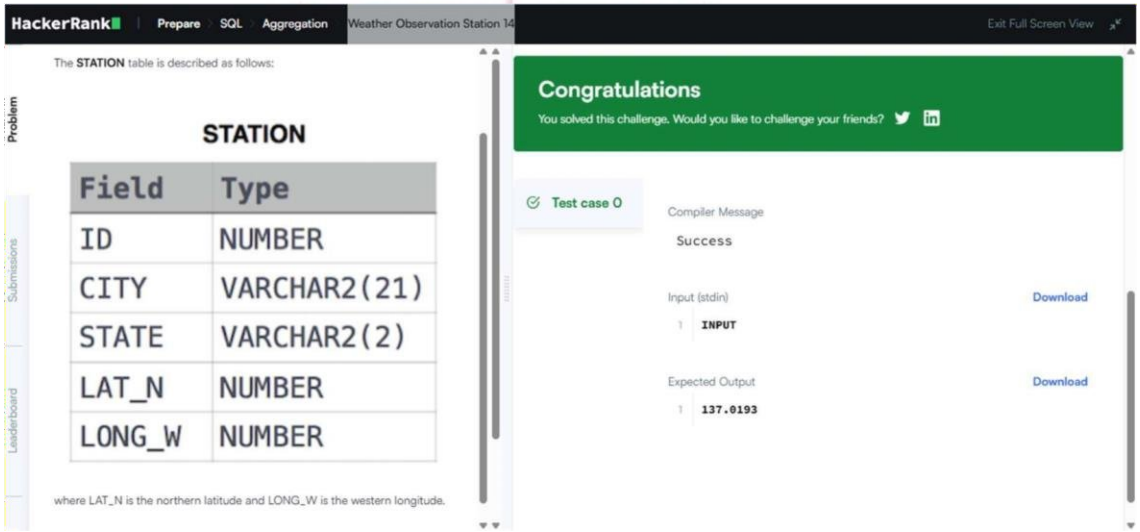
WEATHER OBSERVATION STATION 14

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



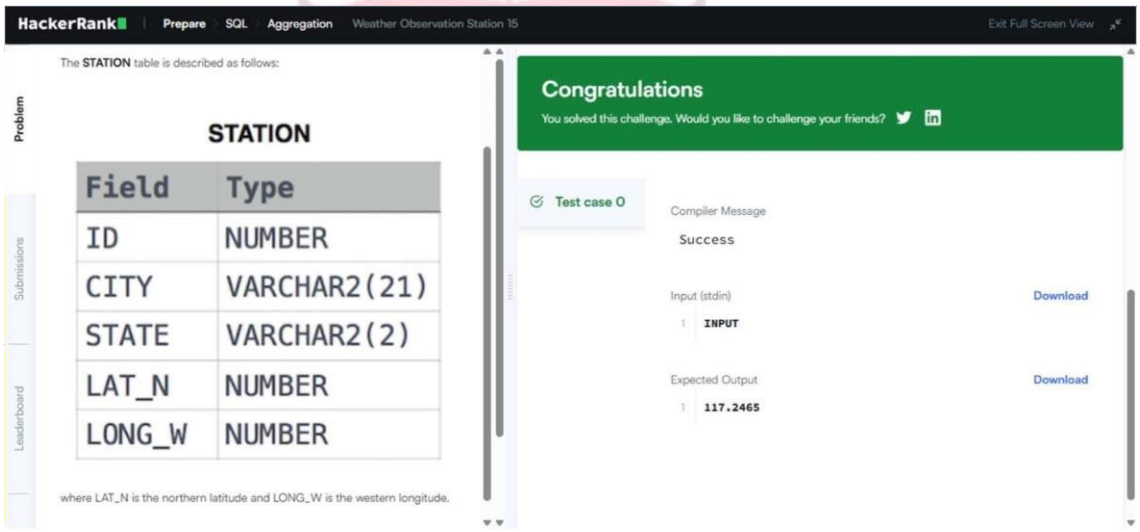
WEATHER OBSERVATION STATION 15

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



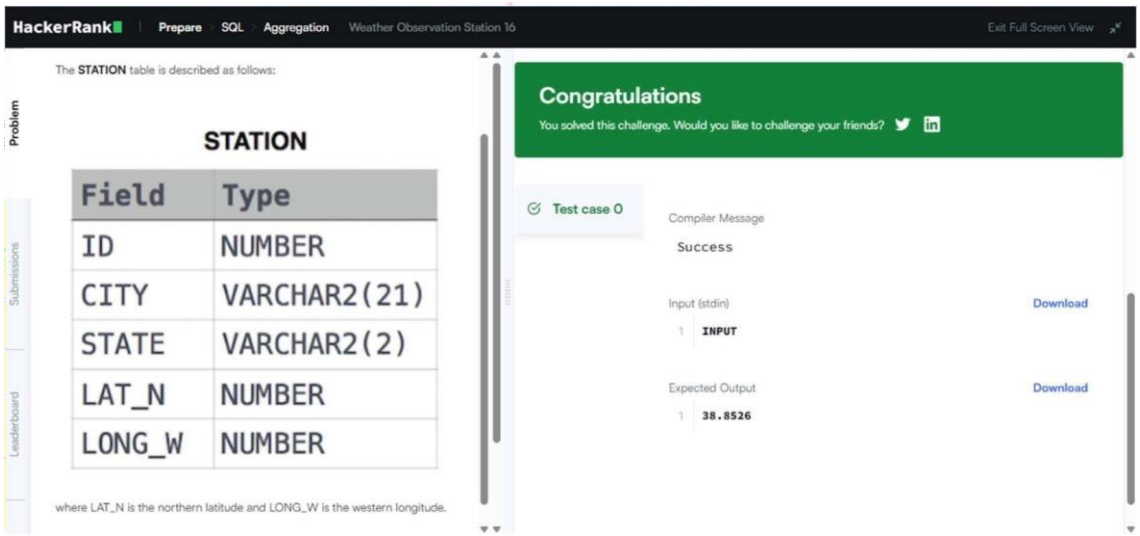
WEATHER OBSERVATION STATION 16

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



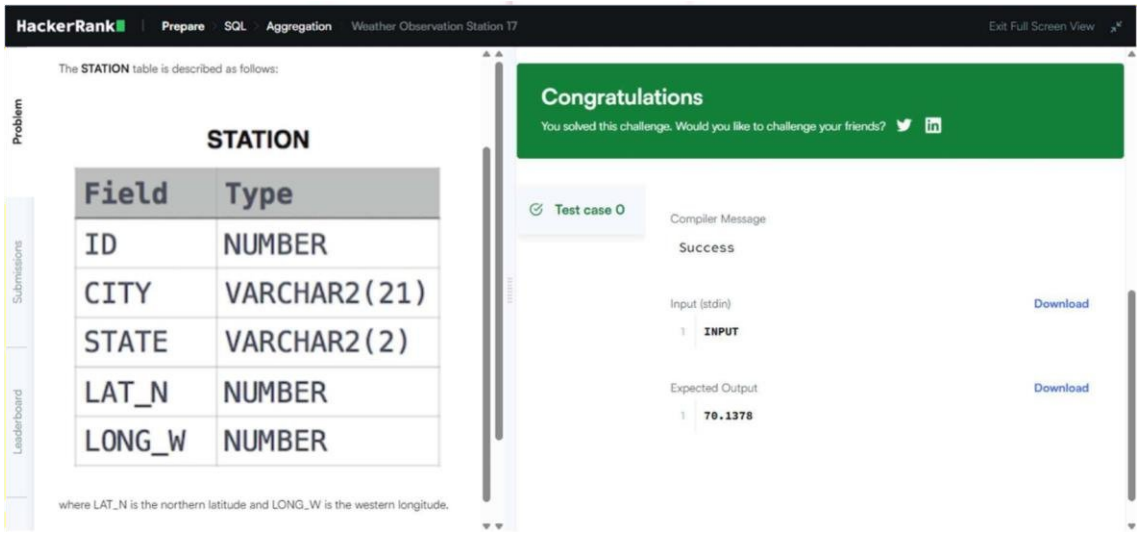
WEATHER OBSERVATION STATION 17

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



POPULATION CENSUS

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

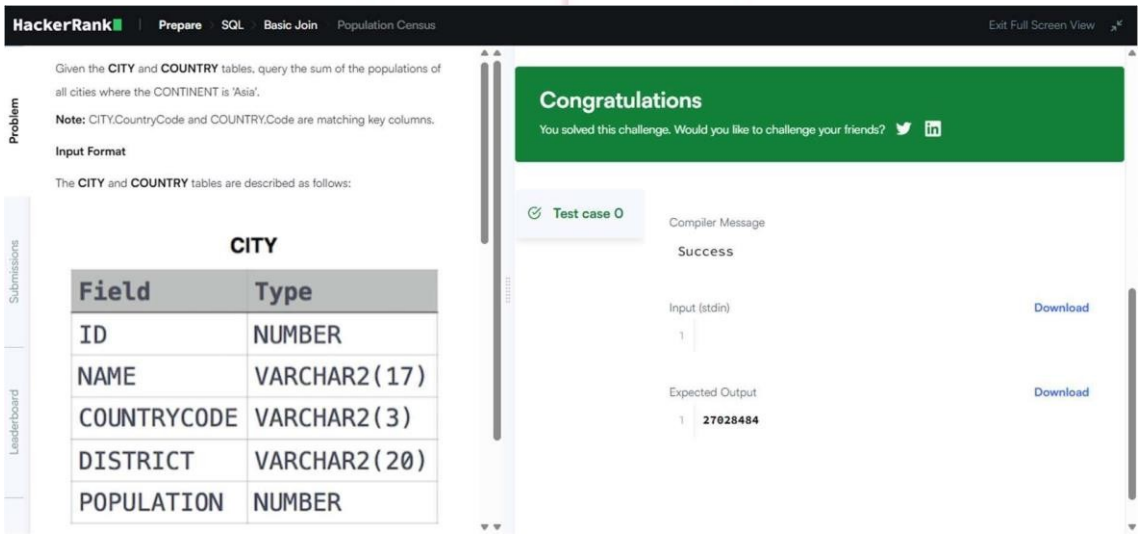
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AFRICAN CITIES

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the CONTINENT is 'Africa'.

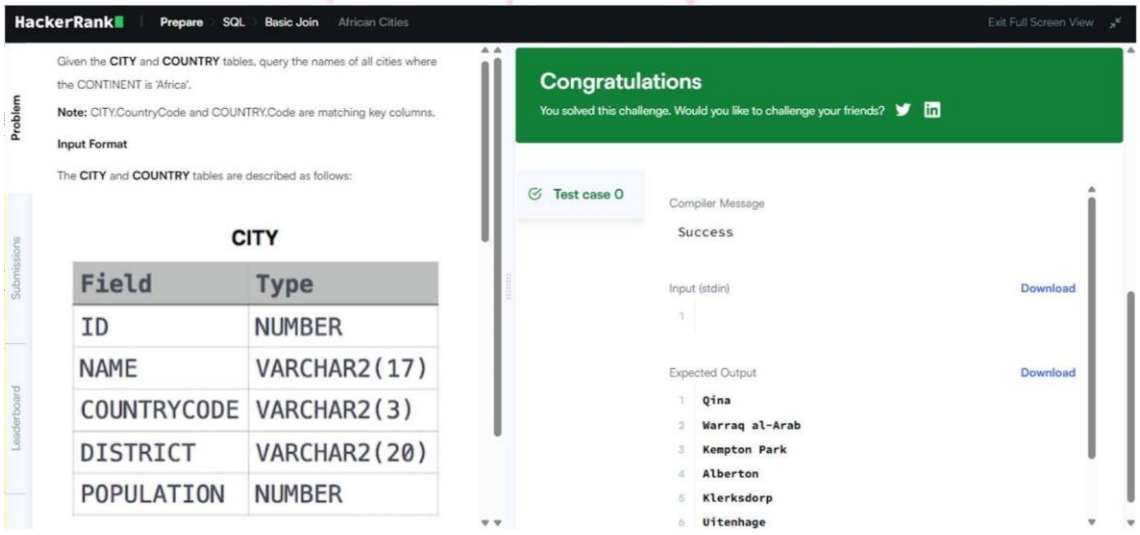
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AVERAGE POPULATION OF EACH CONTINENT

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

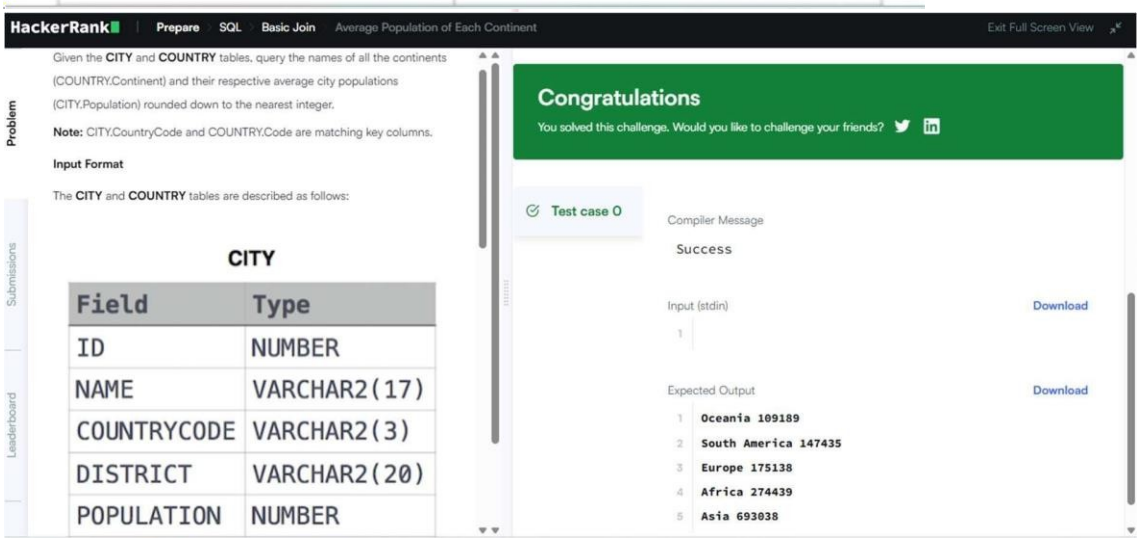
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

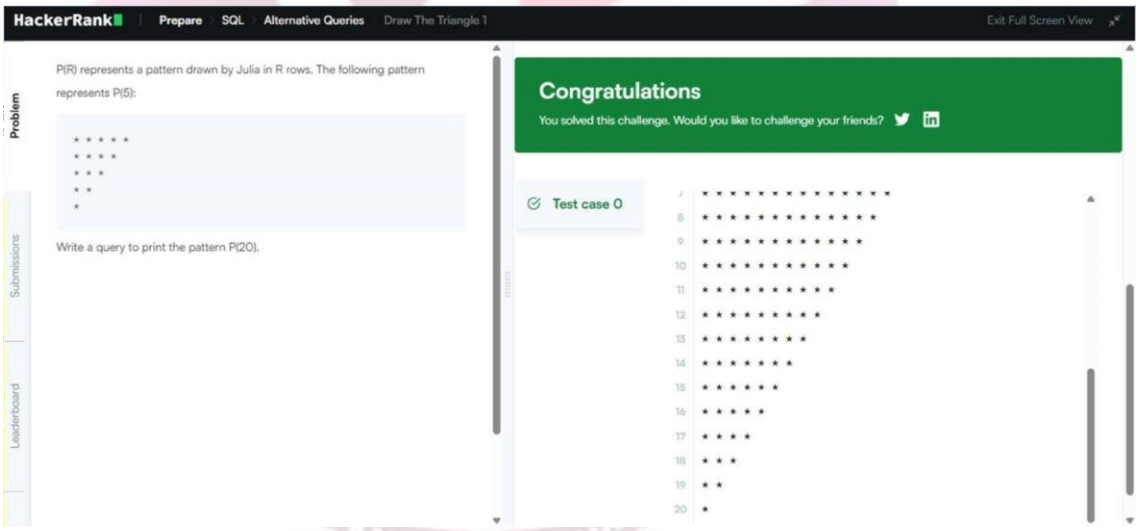


DRAW THE TRIANGLE 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern P(20).



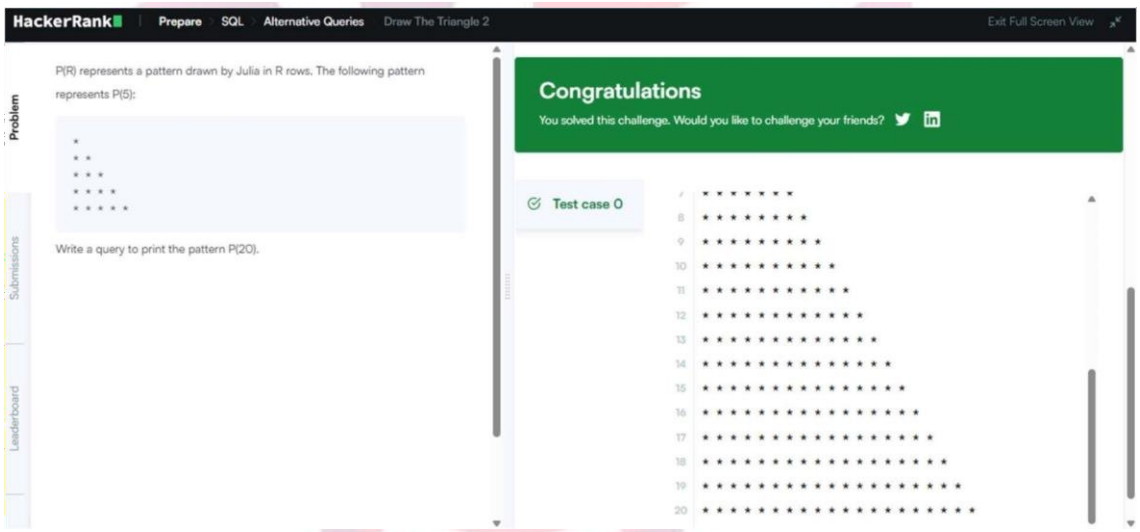
AUTONOMOUS

DRAW THE TRIANGLE 2

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern P(20).



AUTONOMOUS

THE PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).

1.Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in *ascending order*, and output them in the following format:where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Input Format

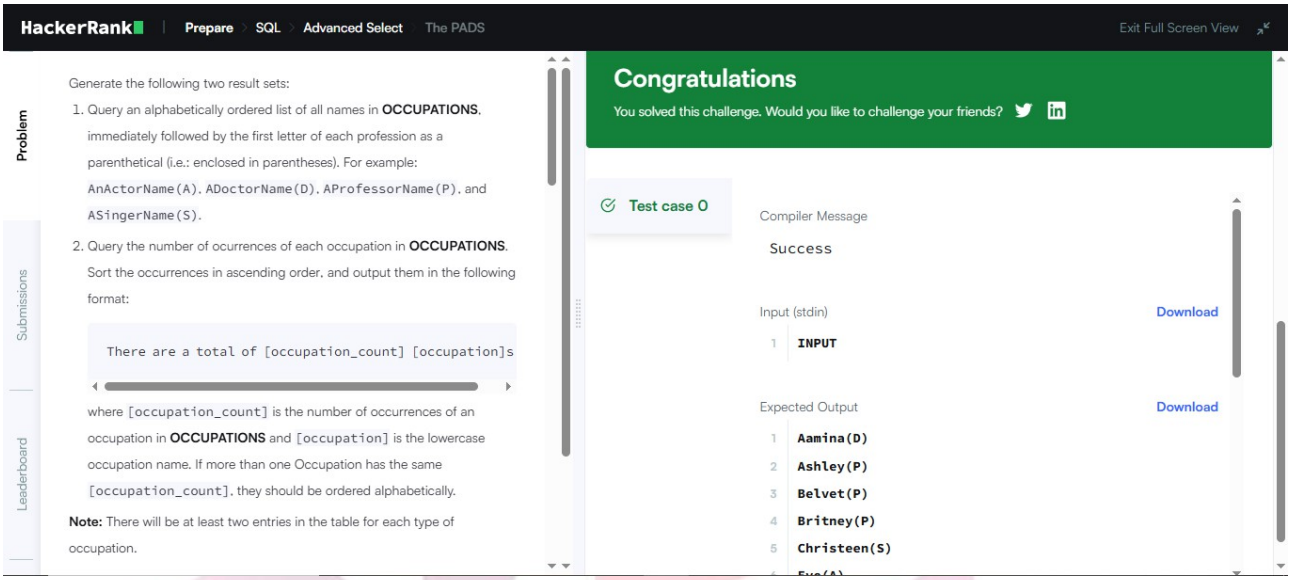
The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
```

GROUP BY Occupation

ORDER BY COUNT(Occupation), Occupation;



OCCUPATIONS

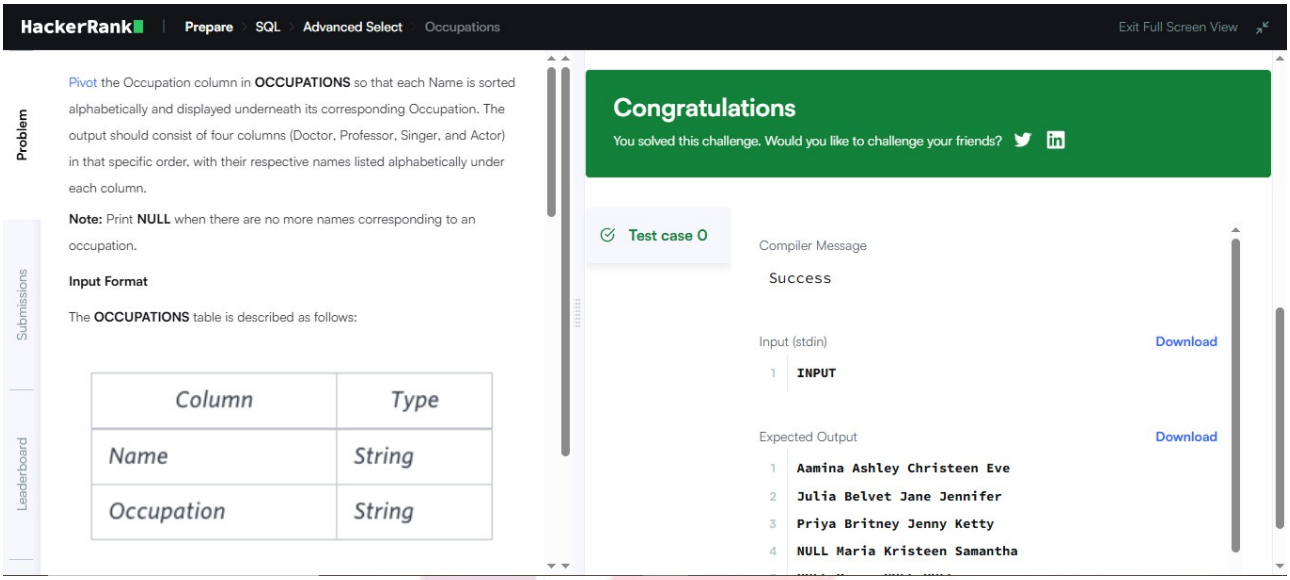
The *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SET @r1=0, @r2=0, @r3=0, @r4=0;
SELECT MIN(Doctor), MIN(Professor), MIN(Singer), MIN(Actor)
FROM (
   SELECT
      CASE
         WHEN Occupation = 'Doctor' THEN (@r1 := @r1 + 1)
         WHEN Occupation = 'Professor' THEN (@r2 := @r2 + 1)
         WHEN Occupation = 'Singer' THEN (@r3 := @r3 + 1)
         WHEN Occupation = 'Actor' THEN (@r4 := @r4 + 1)
      END AS RowNum,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor,
      CASE WHEN Occupation = 'Professor' THEN Name END AS Professor,
      CASE WHEN Occupation = 'Singer' THEN Name END AS Singer,
      CASE WHEN Occupation = 'Actor' THEN Name END AS Actor
   FROM OCCUPATIONS
   ORDER BY Name
) AS Temp
GROUP BY RowNum;
```



BINARY TREE NODES

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*

Column	Type
N	Integer
P	Integer

.Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

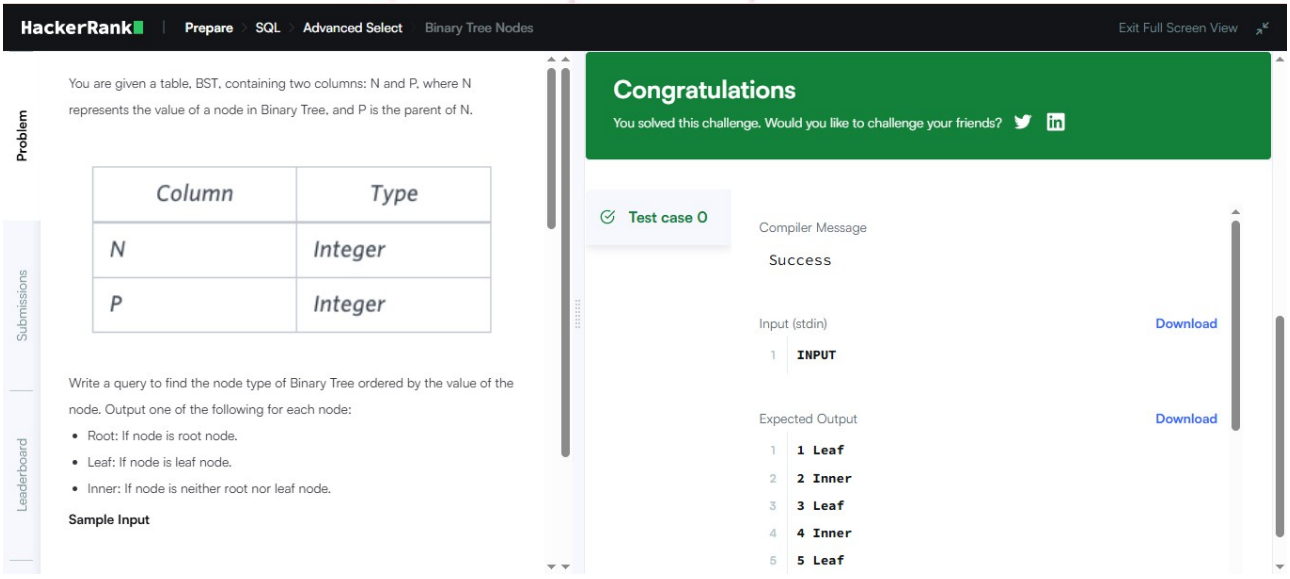
- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

SELECT N, CASE WHEN P IS NULL THEN 'Root' WHEN N IN (SELECT P FROM BST)

THEN 'Inner' ELSE 'Leaf' END FROM BST ORDER BY N;



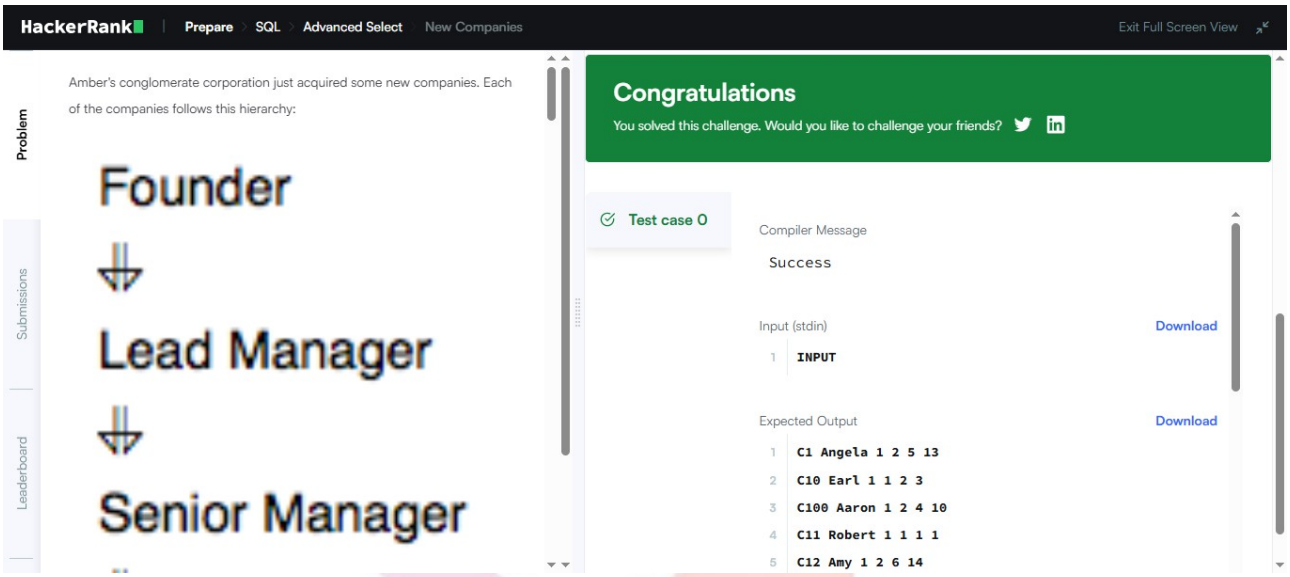
NEW COMPANIES

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:

Founder
⇓
Lead Manager
⇓
Senior Manager
⇓
Manager
⇓
Employee

Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

SELECT c.company_code, c.founder, COUNT(DISTINCT e.lead_manager_code), COUNT(DISTINCT e.senior_manager_code), COUNT(DISTINCT e.manager_code), COUNT(DISTINCT e.employee_code) FROM Company c LEFT JOIN Employee e ON c.company_code = e.company_code GROUP BY c.company_code, c.founder ORDER BY c.company_code;



WEATHER OBSERVATION STATION 18

Consider p1(a,b) and p2(c,d)to be two points on a *2D* plane.

- A happens to equal the minimum value in *Northern Latitude* (*LAT_N* in **STATION**).
- B happens to equal the minimum value in *Western Longitude* (*LONG_W* in **STATION**).
- C happens to equal the maximum value in *Northern Latitude* (*LAT_N* in **STATION**).
- D happens to equal the maximum value in *Western Longitude* (*LONG_W* in **STATION**).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

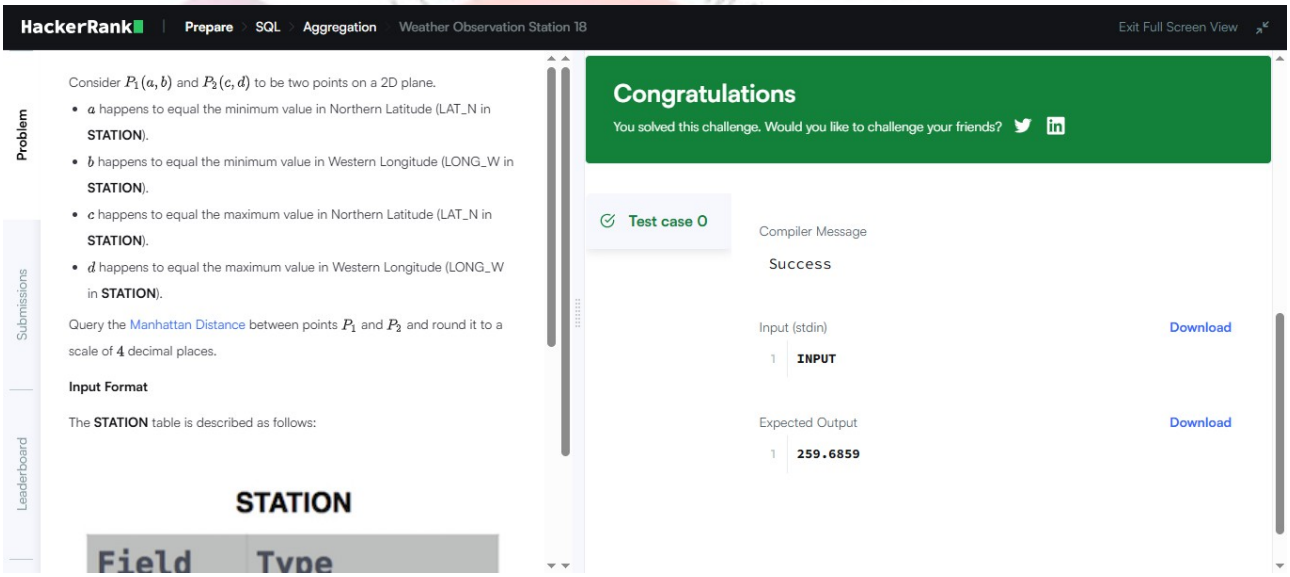
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(ABS(MIN(LAT_N) - MAX(LAT_N)) + ABS(MIN(LONG_W) -
MAX(LONG_W)), 4) FROM STATION;
```



WEATHER OBSERVATION STATION 19

Consider ranto be two points on a 2D plane where Are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in STATION.

Query the between point and *ormat your answer* to display decimal digits.

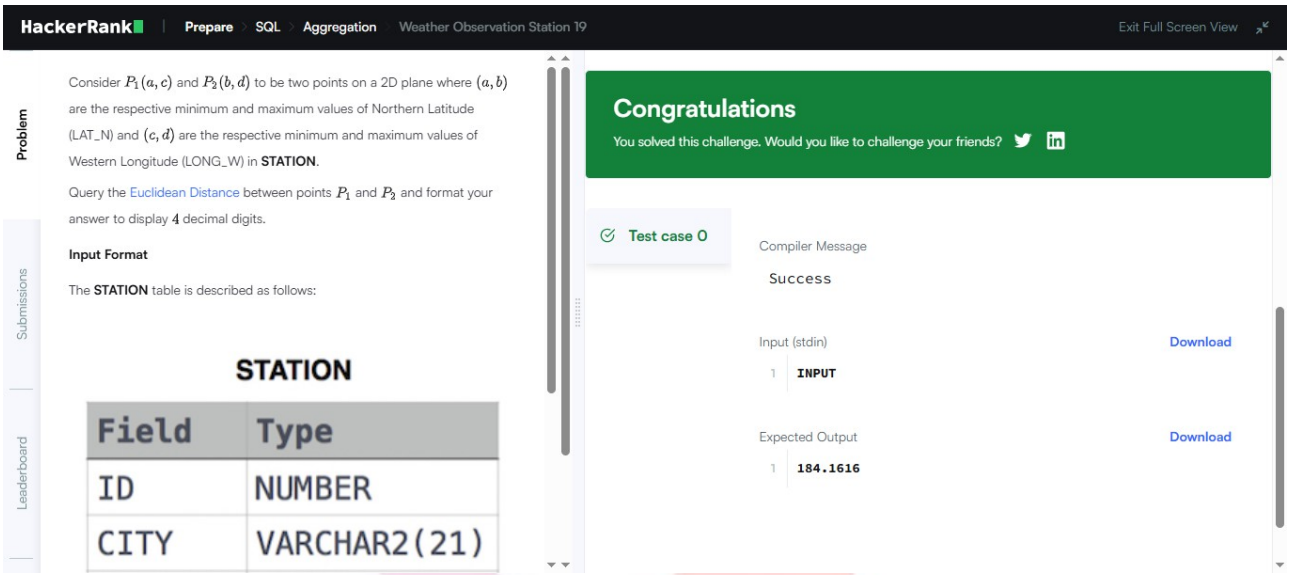
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

SELECT ROUND(SQRT(POW(MIN(LAT_N) - MAX(LAT_N), 2) + POW(MIN(LONG_W) - MAX(LONG_W), 2)), 4) FROM STATION;



WEATHER OBSERVATION STATION 20

A median is defined as a number separating the higher half of a data set from the lower half.

Query the *median* of the *Northern Latitudes* from STATION and round your answer to decimal places.

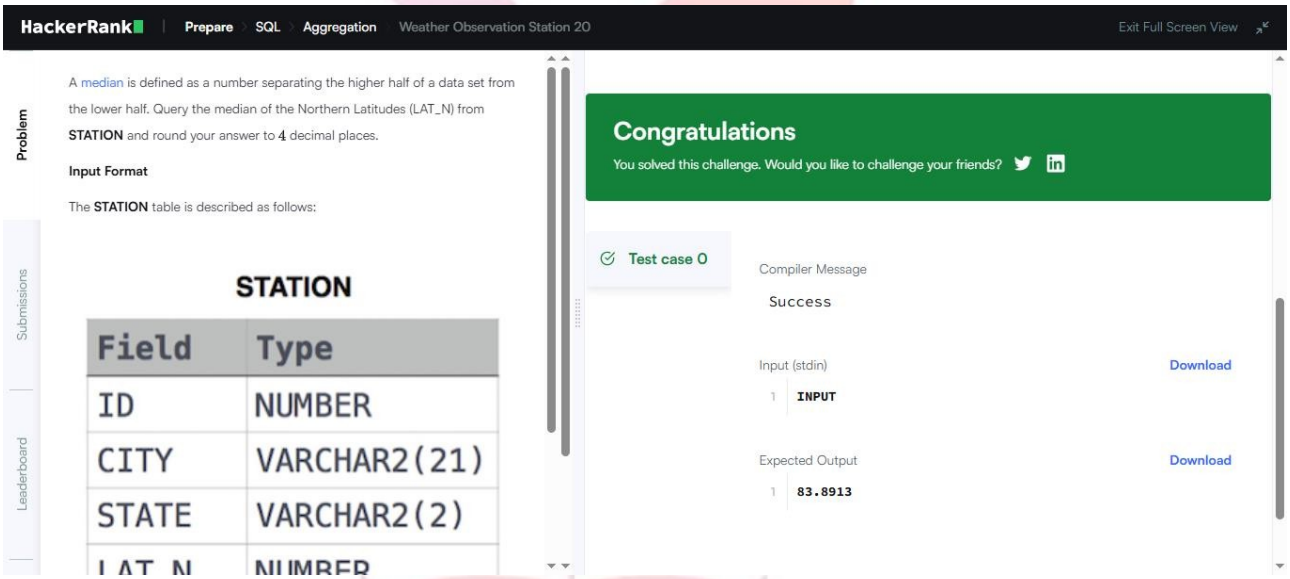
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS RowNum,
        COUNT(*) OVER () AS TotalRows
    FROM STATION
) AS RankedStation
WHERE RowNum IN (FLOOR((TotalRows + 1) / 2), CEIL((TotalRows + 1) / 2));
```



THE REPORT

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

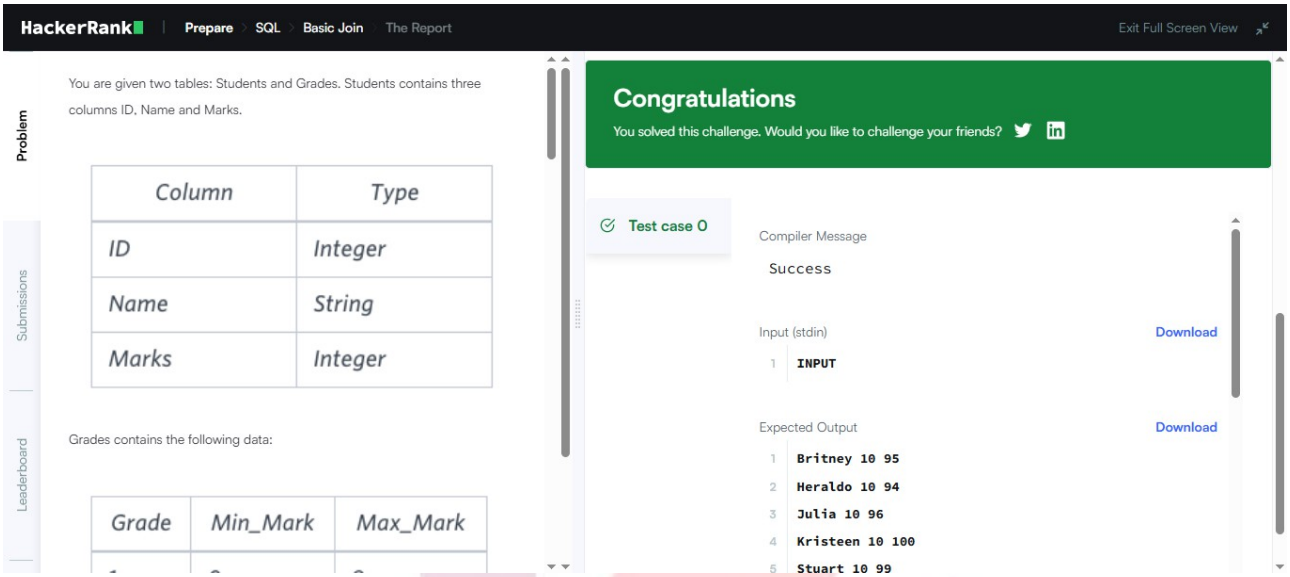
Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descenading order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their

name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

```
SELECT
    CASE WHEN g.Grade >= 8 THEN s.Name ELSE NULL END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS



TOP COMPETITORS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

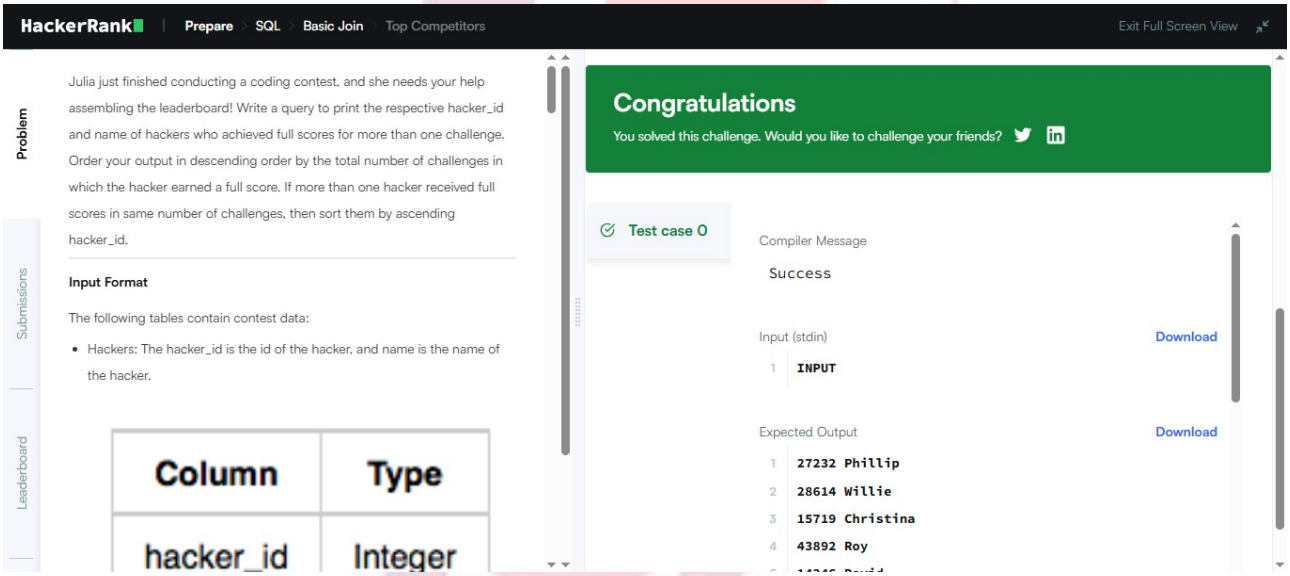
Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name
FROM Submissions s
JOIN Challenges c ON s.challenge_id = c.challenge_id
JOIN Difficulty d ON c.difficulty_level = d.difficulty_level
JOIN Hackers h ON s.hacker_id = h.hacker_id
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```



OLLIVANDER'S INVENTORY

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of

the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

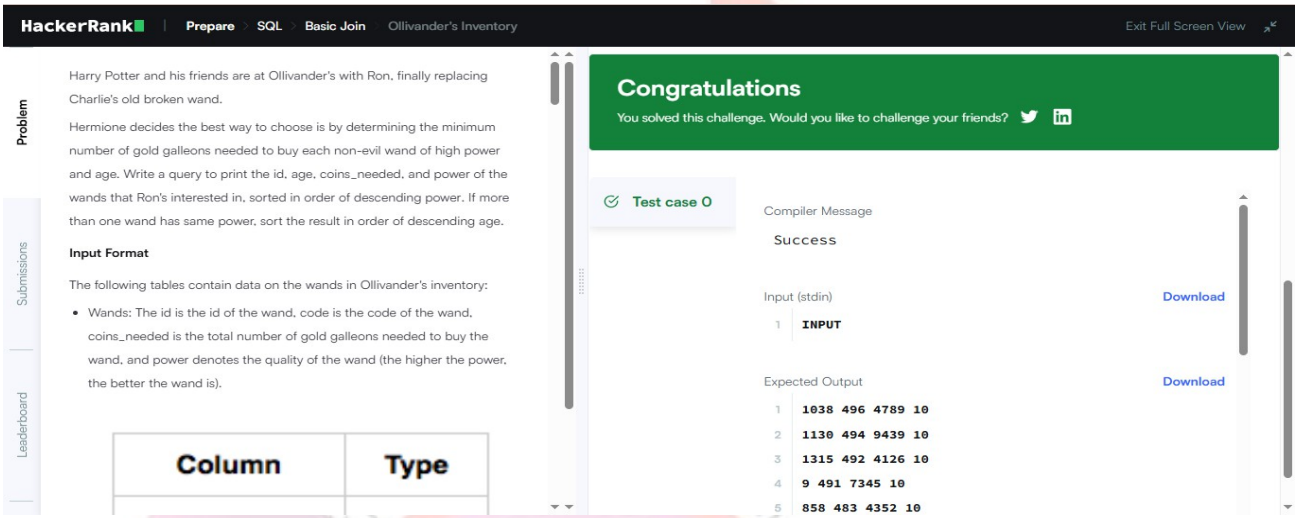
The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

```
SELECT w.id, p.age, w.coins_needed, w.power
FROM Wands AS w
JOIN Wands_Property AS p
ON w.code = p.code
WHERE p.is_evil = 0
  AND w.coins_needed =
     ( SELECT
     MIN(x.coins_needed) FROM
     Wands AS x
     JOIN Wands_Property AS y
```

```
    ON x.code = y.code
WHERE x.power = w.power
AND y.age = p.age
)
ORDER BY w.power DESC, p.age DESC;
```



CHALLENGES

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

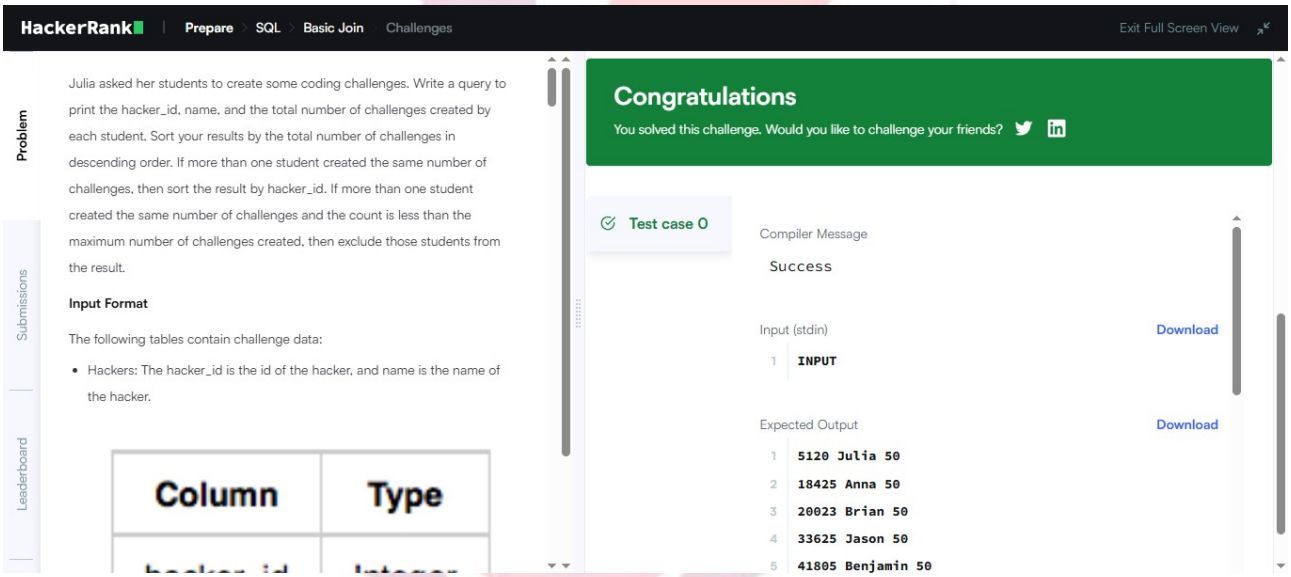
- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
WITH ChallengeCounts AS
    ( SELECT
        h.hacker_id,
        h.name,
        COUNT(c.challenge_id) AS total_challenges
    FROM Hackers h
    JOIN Challenges c ON h.hacker_id = c.hacker_id
    GROUP BY h.hacker_id, h.name
)

SELECT hacker_id, name, total_challenges
FROM ChallengeCounts
WHERE total_challenges = (
    /* Condition 1: Keep it if it matches the maximum challenges created by any hacker */
    SELECT MAX(total_challenges) FROM ChallengeCounts
)
```

```
OR total_challenges IN (
    /* Condition 2: Keep it if the challenge count is completely unique */
    SELECT total_challenges
    FROM ChallengeCounts
    GROUP BY total_challenges
    HAVING COUNT(total_challenges) = 1
)
ORDER BY total_challenges DESC, hacker_id ASC;
```



CONTEST LEADERBOARD

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker

achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

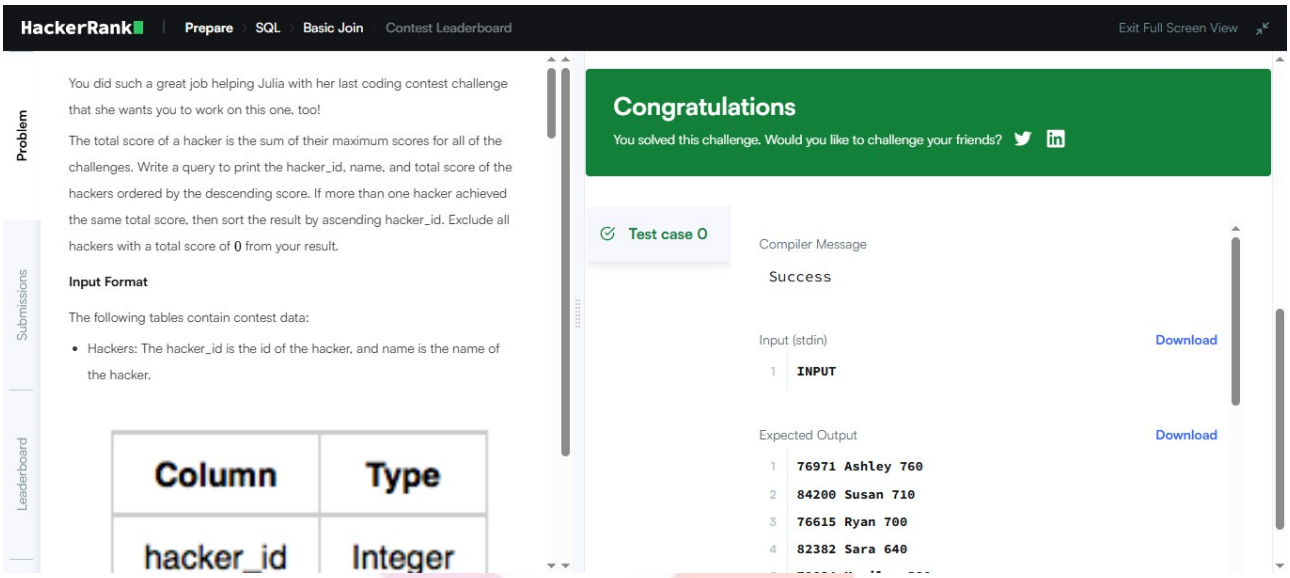
Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name, SUM(t.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) t ON h.hacker_id = t.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```



SQL PROJECT PLANNING

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

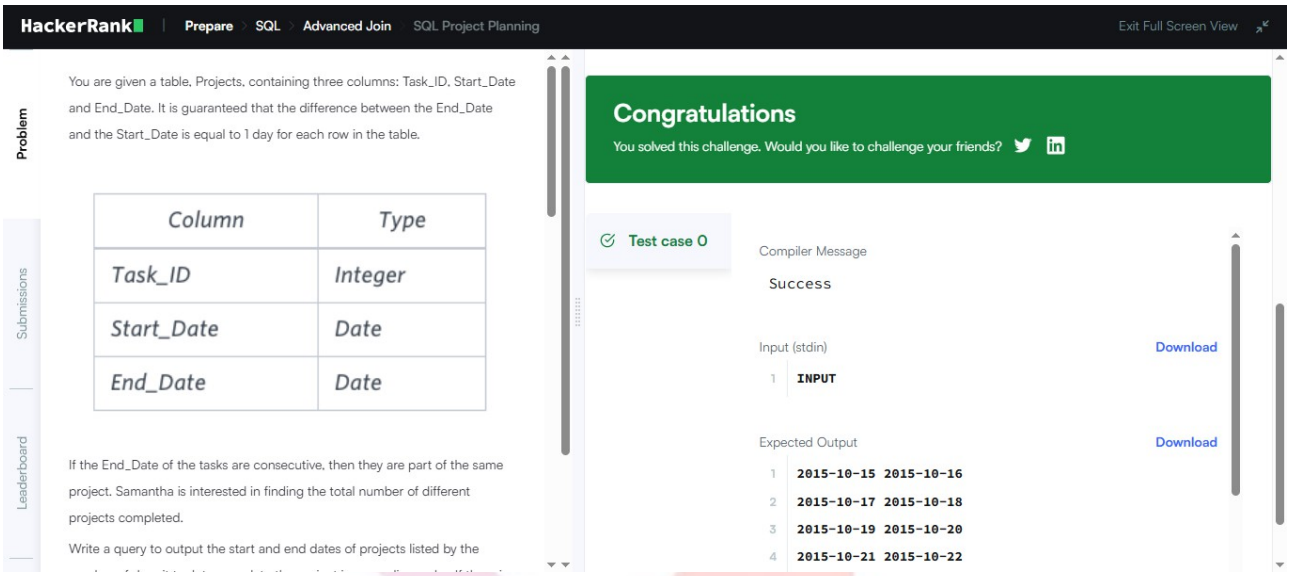
If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

```
SELECT A.Start_Date, MIN(B.End_Date)
FROM
    (SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)) AS A
    INNER JOIN
    (SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)) AS B
WHERE A.Start_Date < B.End_Date
GROUP BY A.Start_Date
ORDER BY MIN(B.End_Date) - A.Start_Date ASC, A.Start_Date;
```



PLACEMENTS

You are given three tables: *Students*, *Friends* and *Packages. Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

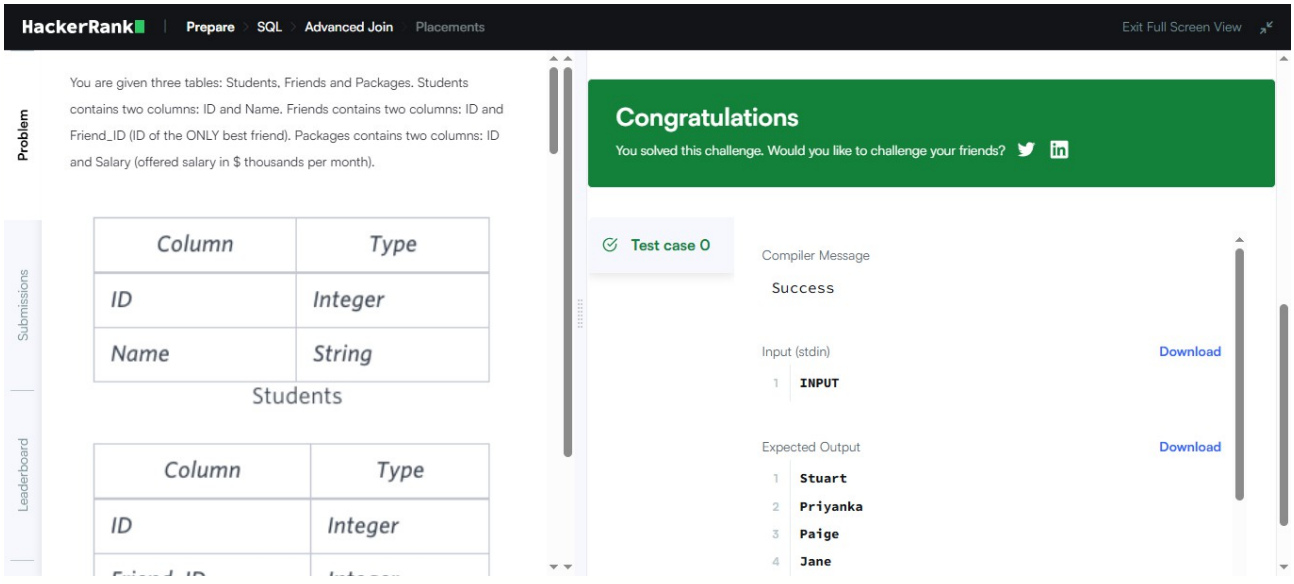
Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

```
SELECT s.Name
FROM Students s
JOIN Friends f ON s.ID = f.ID
JOIN Packages p1 ON s.ID = p1.ID
JOIN Packages p2 ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```



SYMMETRIC PAIRS

You are given a table, *Functions*, containing two columns: *X* and *Y*.

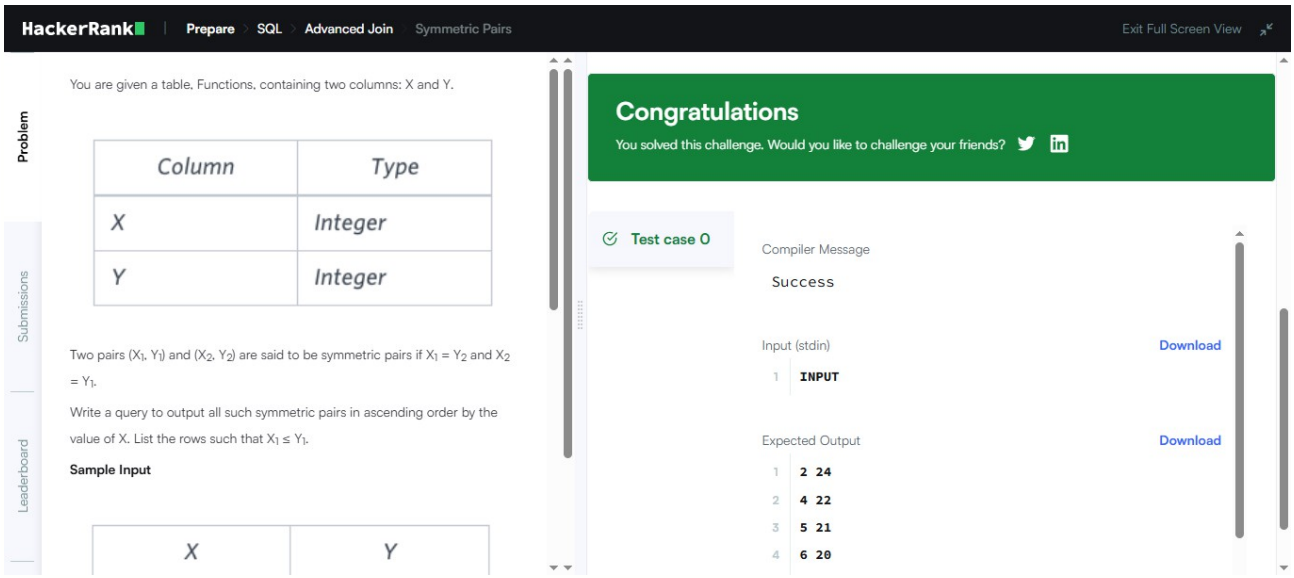
Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
    ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```

AUTONOMOUS



PRINT PRIME NUMBERS

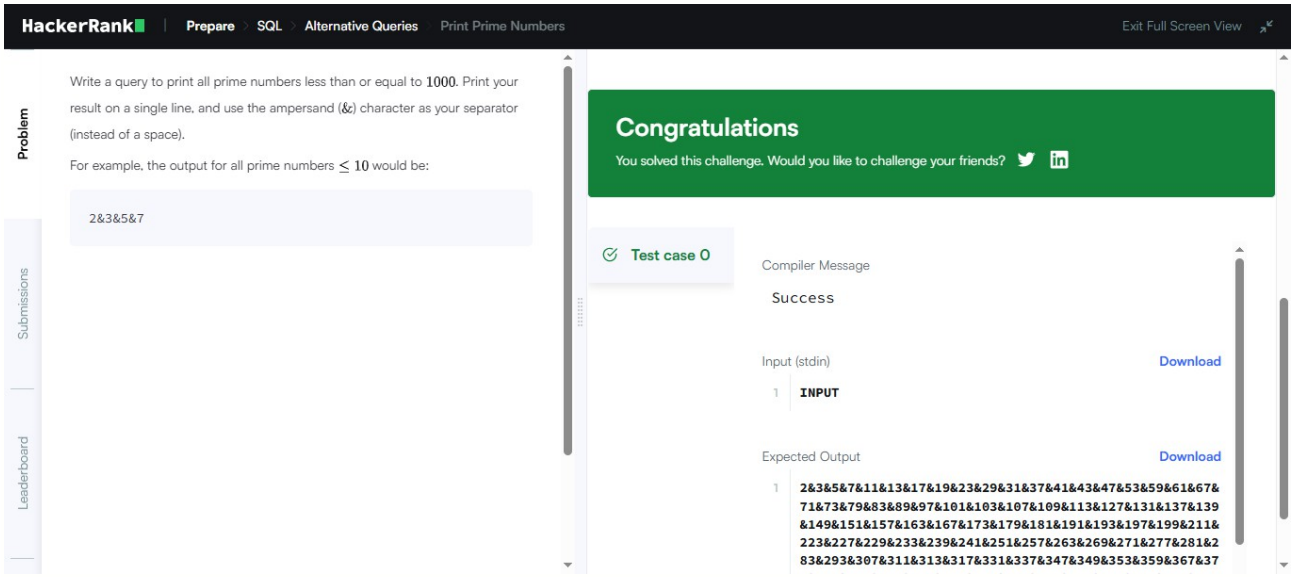
Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

```
2&3&5&7
```

```
WITH RECURSIVE Numbers AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM Numbers WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
```

```
FROM Numbers n1
WHERE NOT EXISTS (
    SELECT 1
    FROM Numbers n2
    WHERE n2.n < n1.n AND n1.n % n2.n = 0
);
```



INTERVIEWS

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table:

challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

SELECT

c.contest_id,

c.hacker_id,

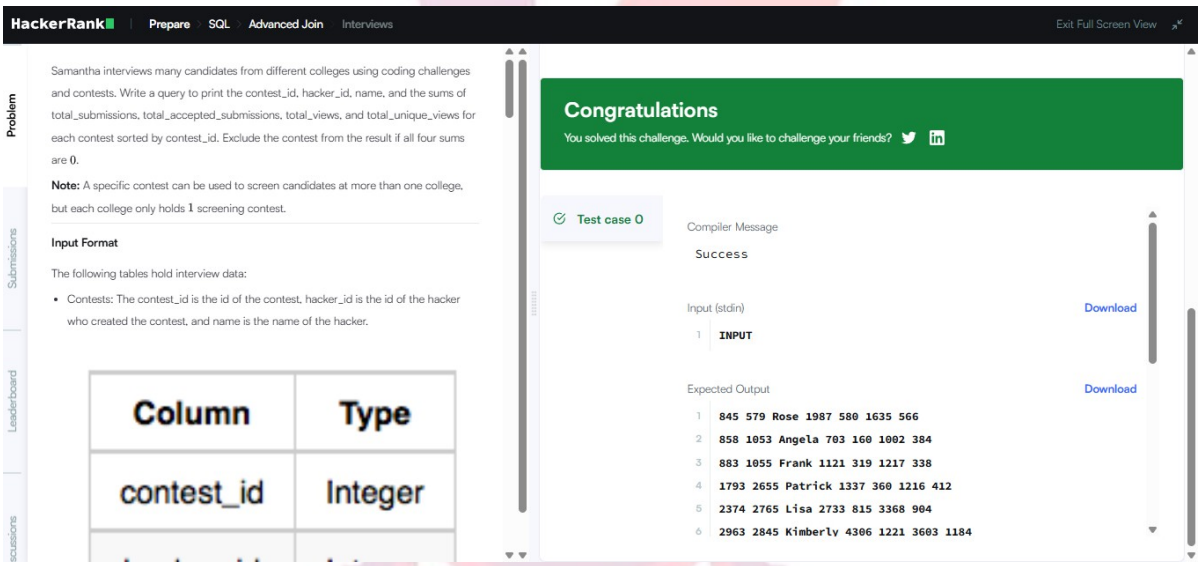
```
    c.name,
    SUM(COALESCE(s.total_submissions, 0)) AS total_submissions,
    SUM(COALESCE(s.total_accepted_submissions, 0)) AS total_accepted_submissions,

    SUM(COALESCE(v.total_views, 0)) AS total_views,
    SUM(COALESCE(v.total_unique_views, 0)) AS total_unique_views
FROM Contests c
JOIN Colleges col
    ON c.contest_id = col.contest_id
JOIN Challenges ch
    ON col.college_id = ch.college_id
LEFT JOIN (
    SELECT

        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s

ON ch.challenge_id = s.challenge_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
GROUP BY
    c.contest_id,
```

```
    c.hacker_id,
    c.name
HAVING
    SUM(COALESCE(s.total_submissions, 0)) +
    SUM(COALESCE(s.total_accepted_submissions, 0)) +
    SUM(COALESCE(v.total_views, 0)) +
    SUM(COALESCE(v.total_unique_views, 0)) > 0
ORDER BY
    c.contest_id;
```



15 DAYS OF LEARNING SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of

submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

Hackers Table:

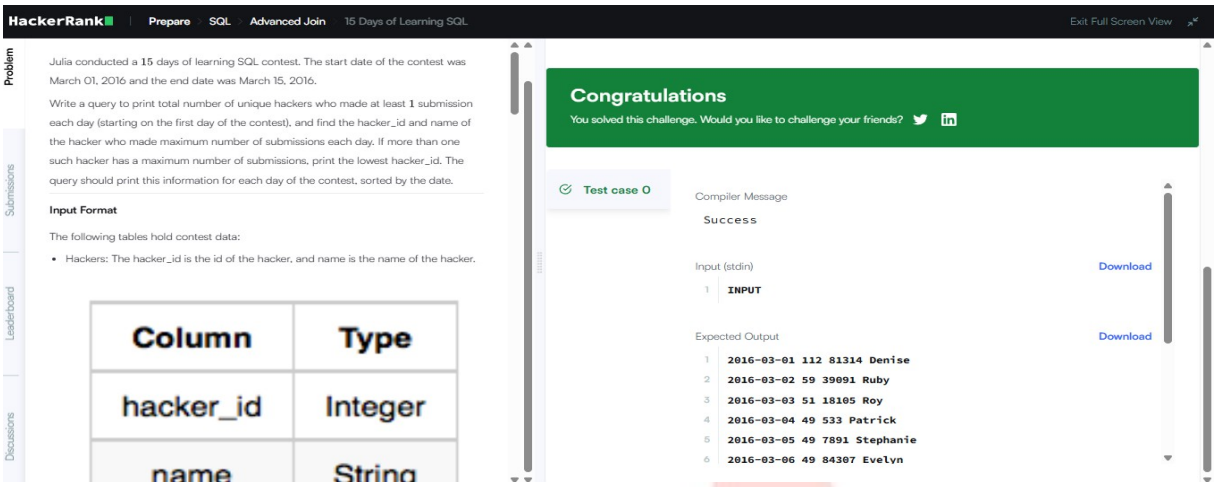
hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael



Submissions **Table:**

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

```
SELECT
    s.submission_date,
    (
        SELECT COUNT(DISTINCT hacker_id)
        FROM Submissions s2
        WHERE s2.submission_date <= s.submission_date
        AND (
            SELECT COUNT(DISTINCT s3.submission_date)
            FROM Submissions s3
            WHERE s3.hacker_id = s2.hacker_id
            AND s3.submission_date <= s.submission_date
        ) = DATEDIFF(s.submission_date, '2016-03-01') + 1
    ) AS total_hackers,
  s.hacker_id,
    h.name
FROM Submissions s
JOIN Hackers h
ON s.hacker_id = h.hacker_id
WHERE s.hacker_id = (
    SELECT s1.hacker_id
    FROM Submissions s1
    WHERE s1.submission_date = s.submission_date
    GROUP BY s1.hacker_id
    ORDER BY COUNT(*) DESC, s1.hacker_id
    LIMIT 1
)
GROUP BY s.submission_date, s.hacker_id, h.name
ORDER BY s.submission_date;
```


HackerRank
Prepare
SQL
Advanced Join
15 Days of Learning SQL
Exit Full Screen View
Problem
Submissions
Leaderboard
Discussions
Julia conducted a 15 days of learning SQL contest. The start date of the contest was March 01, 2016 and the end date was March 15, 2016.
Write a query to print total number of unique hackers who made at least 1 submission each day (starting on the first day of the contest), and find the hacker_id and name of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest hacker_id. The query should print this information for each day of the contest, sorted by the date.
Input Format
The following tables hold contest data:
Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.
Column
Type
hacker_id
Integer
name
String
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 2016-03-01 112 81314 Denise
2 2016-03-02 59 39091 Ruby
3 2016-03-03 51 18105 Roy
4 2016-03-04 49 533 Patrick
5 2016-03-05 49 7891 Stephanie
6 2016-03-06 49 84307 Evelyn

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CODECHEF

BASIC COMMANDS

1. ALL PRODUCTS

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

select*from products;

OUTPUT

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✔ Excellent work!

2. HIGH PRICE OF PRODUCTS

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

```
SELECT product_name, category FROM Products WHERE price > 100.00;
```

OUTPUT

Your Output

```
┌──────────────┬─────────────┐
│ product_name │  category   │
├──────────────┼─────────────┤
│ Laptop       │ Electronics │
│ Desk Chair   │ Furniture   │
│ Smartphone   │ Electronics │
│ Standing Desk │ Furniture  │
│ Tablet       │ Electronics │
└──────────────┴─────────────┘
```

✔ Excellent work!

3. AVERAGE SALARY

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

OUTPUT

Your Output

```
avg_salary
62600.155
```

Hooray, you did it!

4. LOCATE PEOPLE

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

INPUT

```
SELECT department_name, location FROM departments WHERE location LIKE 'S%';
```

OUTPUT

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

✔ Awesome, you nailed it!

5. DISTINCT COMPANIES

Task

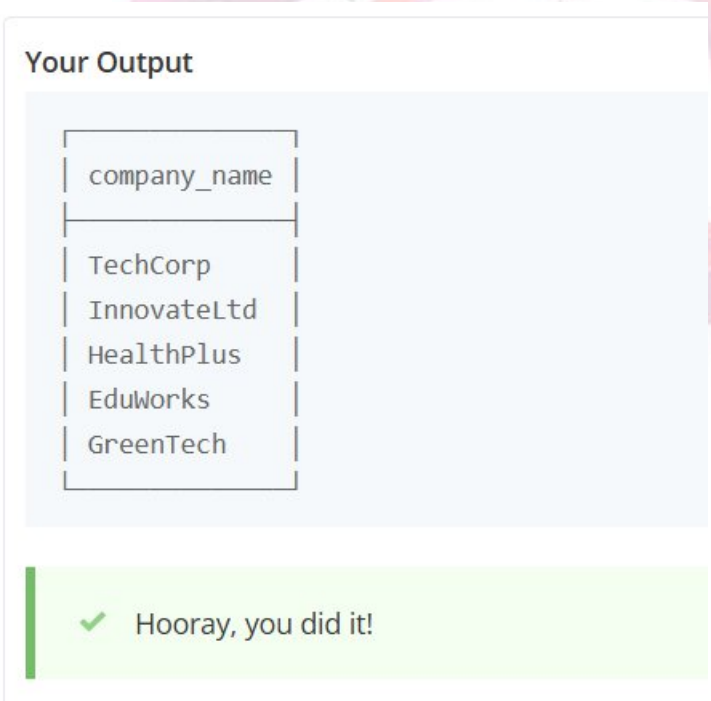
Write a query to select all the distinct companies (company_name) in the Works table.
Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT



6. FICTION COLLECTION SIZE

Task

Write a query to find the total count of books whose genre is **Fiction**.
Note: Output column name should be **fiction_count**.

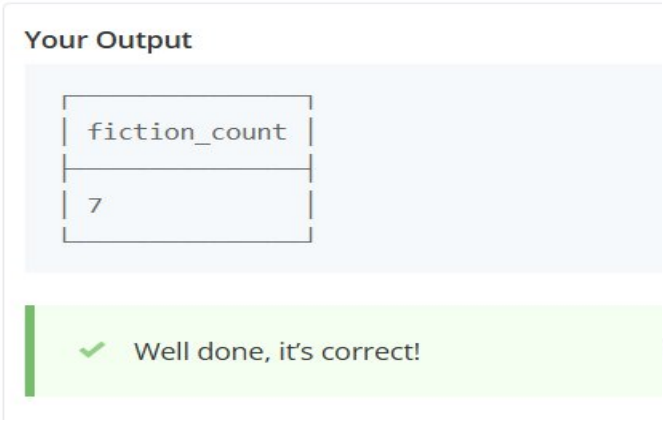
Table name: **Books**

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

INPUT

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```

OUTPUT



7. LIST OF MOVIES WITH RATINGS

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

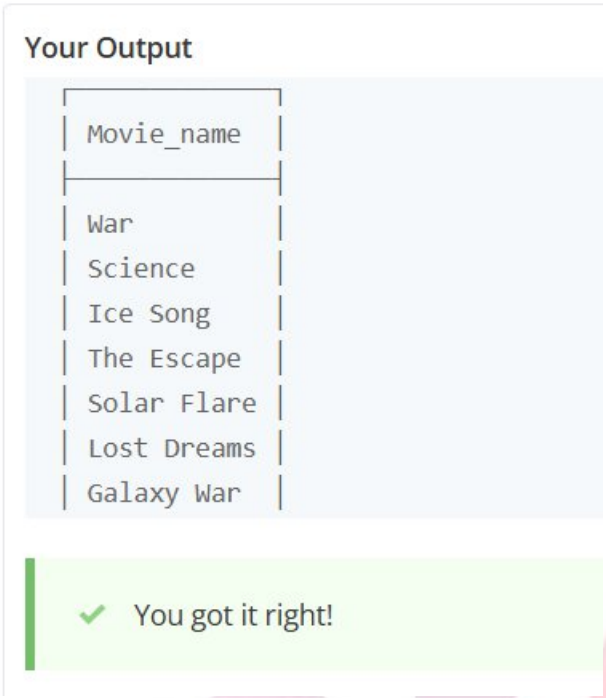
Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

INPUT

SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;

OUTPUT



8. HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have **NULL** rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

INPUT

select book_id, title, author, published_year from library where rating is null;

OUTPUT

Your Output

book_id	title	author	published_year
2	To Kill a Mockingbird	Harper Lee	1960
4	The Catcher in the Rye	J.D. Salinger	1951

✔ Excellent work!

9. SALARY OF EMPLOYEES

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;

OUTPUT

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✔ You got it right!

10. DEPARTMENT OF EACH EMPLOYEE

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

select department, count(*) as total_employees from employees group by department;

OUTPUT

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

✔ Perfect answer!

11. ARTICLE VIEWS

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

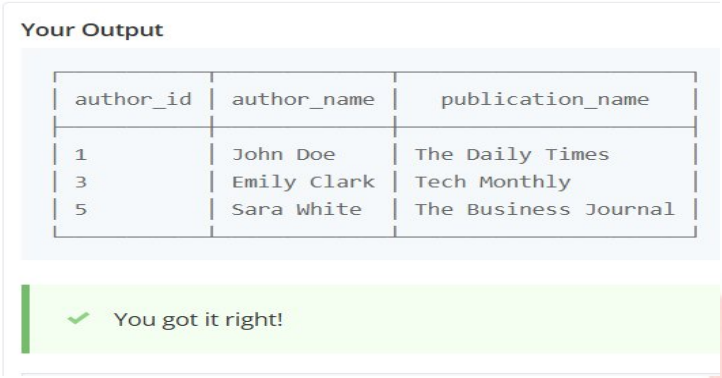
Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

INPUT

select distinct author_id, author_name, publication_name from views where view_count = 0 order by author_id asc;

OUTPUT



12. PLAYER PERFORMANCE INSIGHTS

Task

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

INPUT

select distinct player_name, score from players join matches where winner=player_name

order by score desc limit 3;

OUTPUT

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✔ Awesome, you nailed it!

13. PLAYER DETAILS

Task

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

INPUT

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT

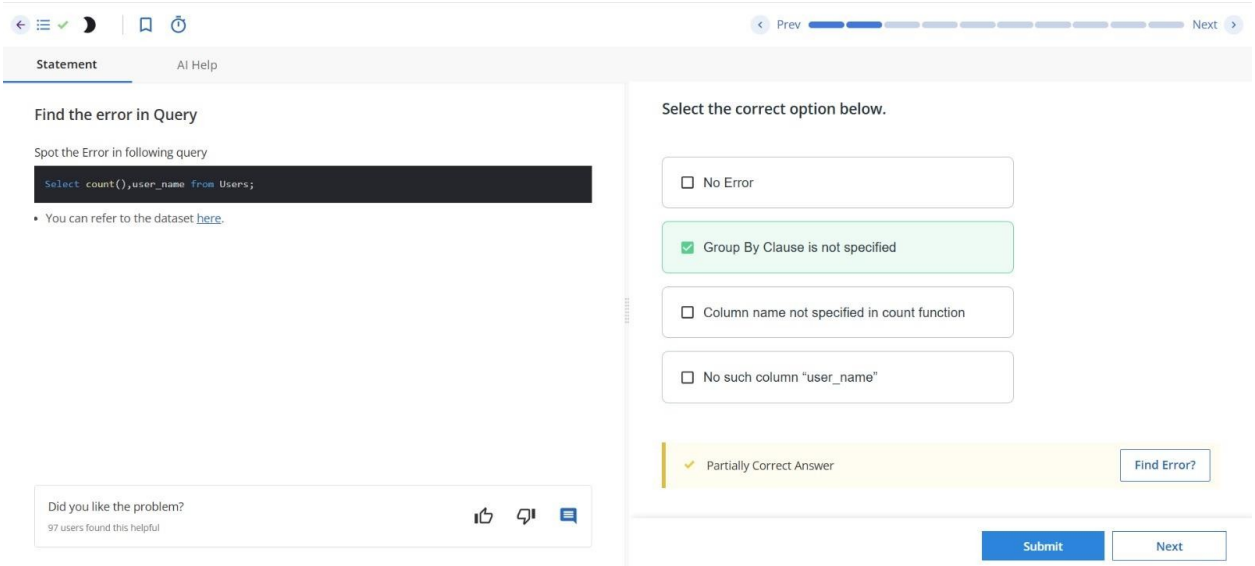
Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

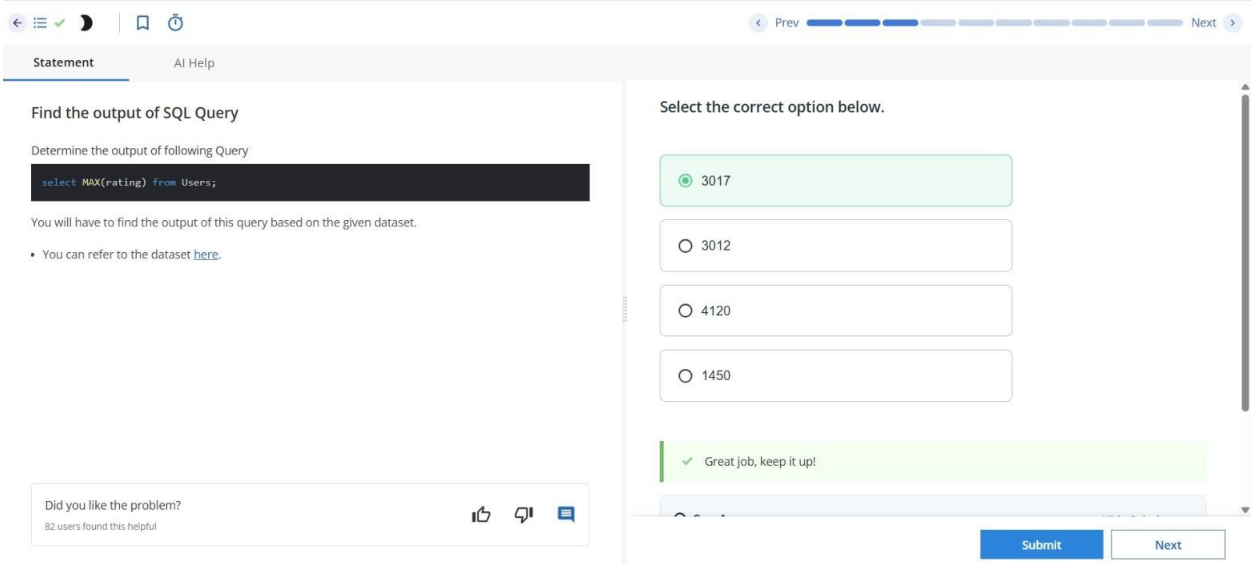
✔ Awesome, you nailed it!

CASE STUDY – CODECHEF DATABASE

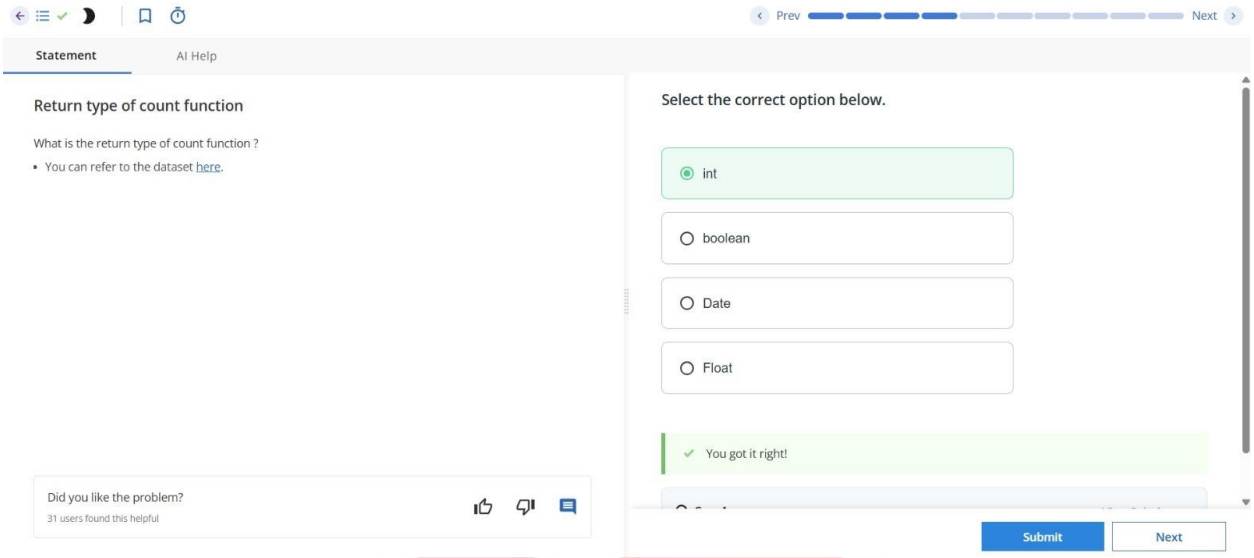
1. FIND THE ERROR IN QUERY



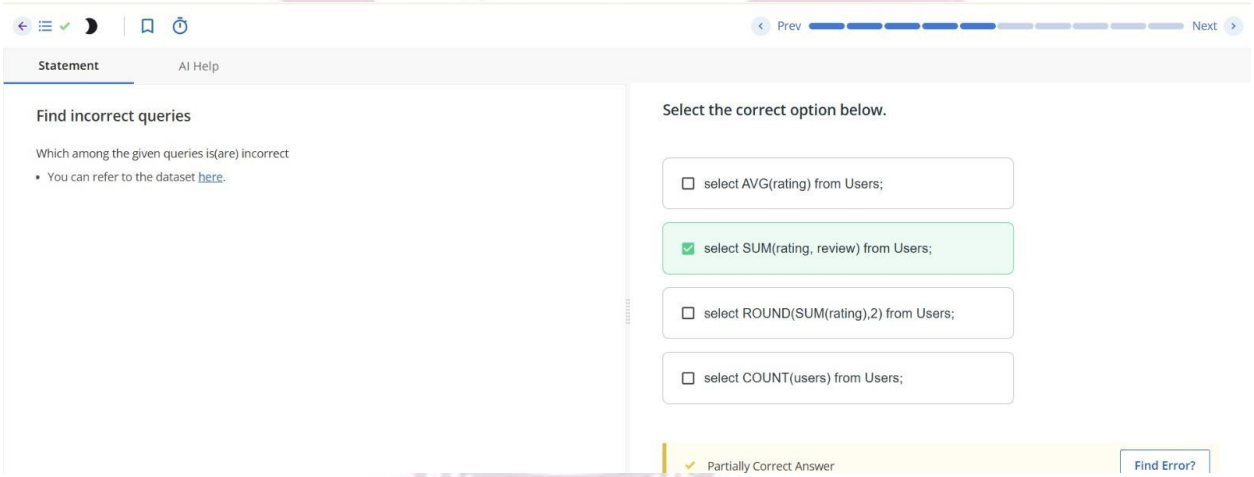
2. FIND THE OUTPUT OF SQL QUERY



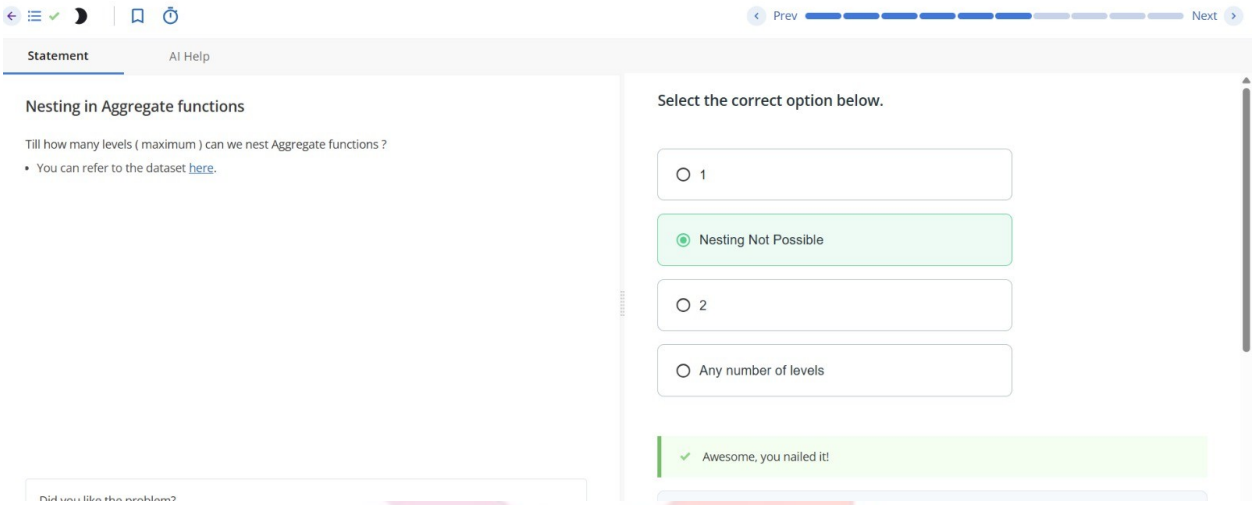
3. RETURN TYPE OF COUNT FUNCTION



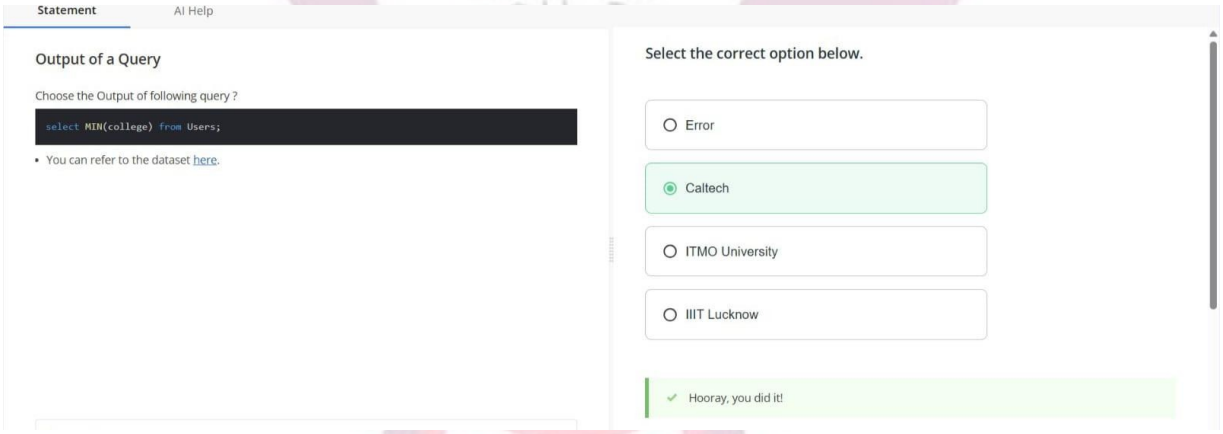
4. FIND INCORRECT QUERIES



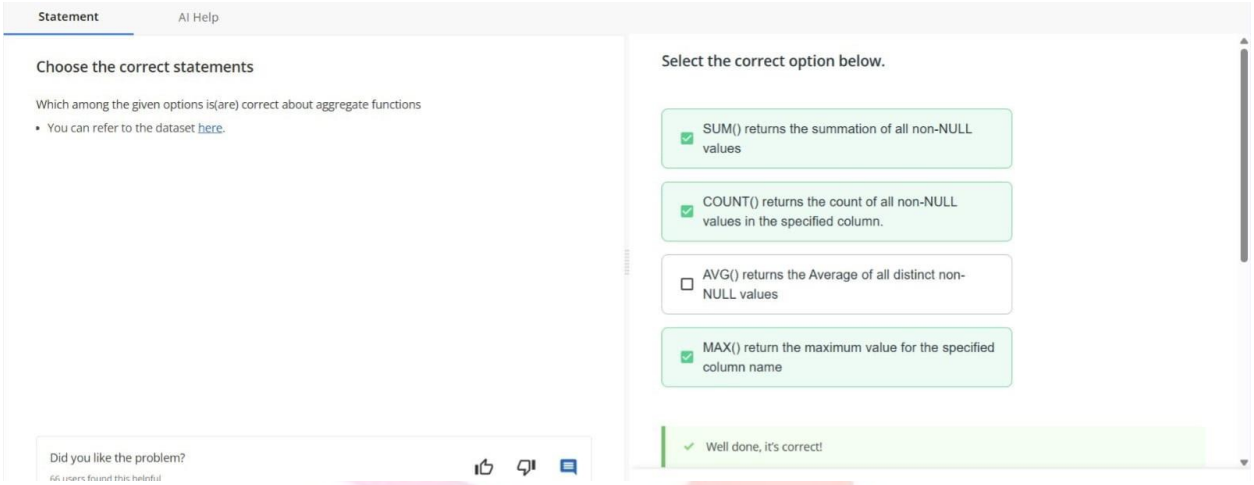
5.NESTED IN AGGREGATE FUNCTIONS



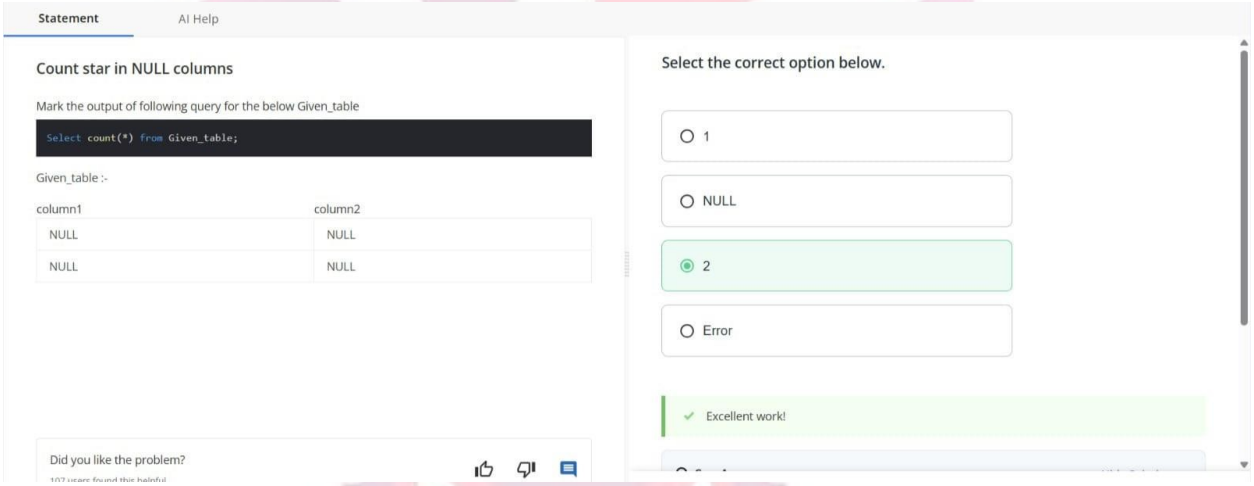
6. OUTPUT OF A QUERY



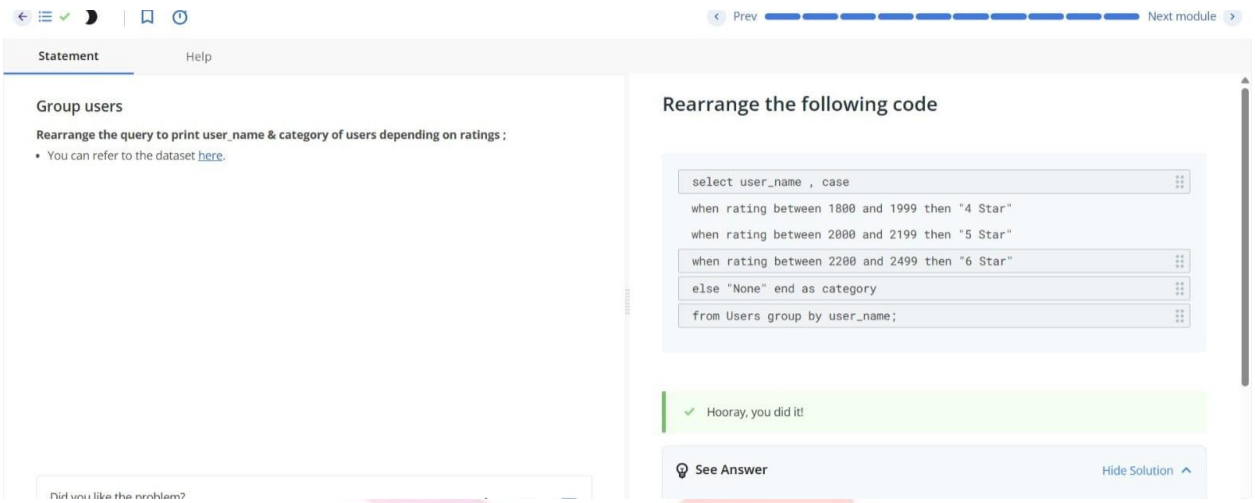
7. CHOOSE THE CORRECT STATEMENTS



8. COUNT STAR IN NULL COLUMNS



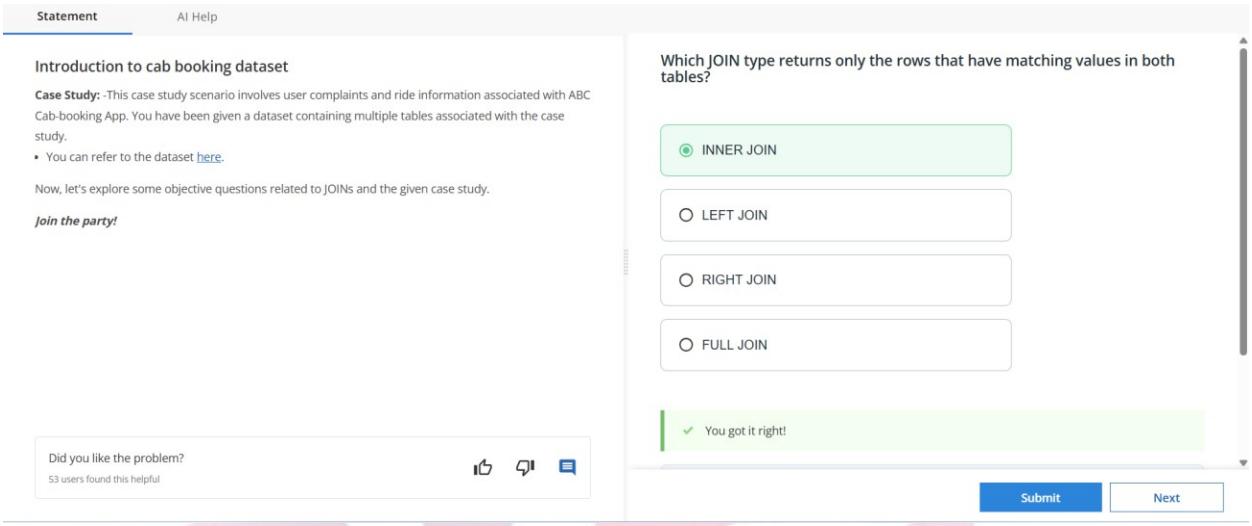
9.GROUP USERS



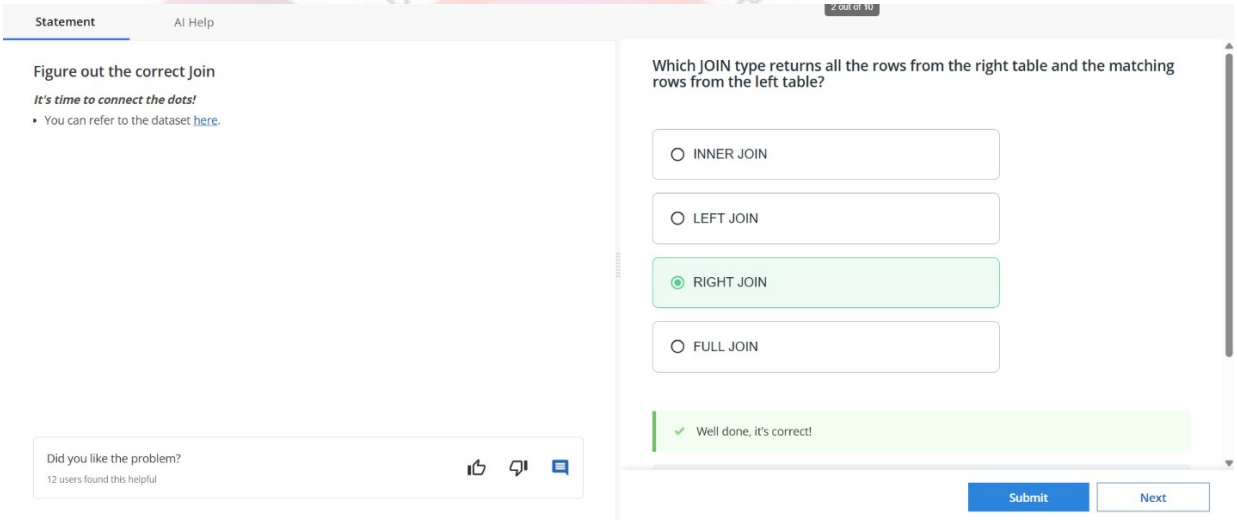
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CASE STUDY – ABC CAB BOOKIN APP

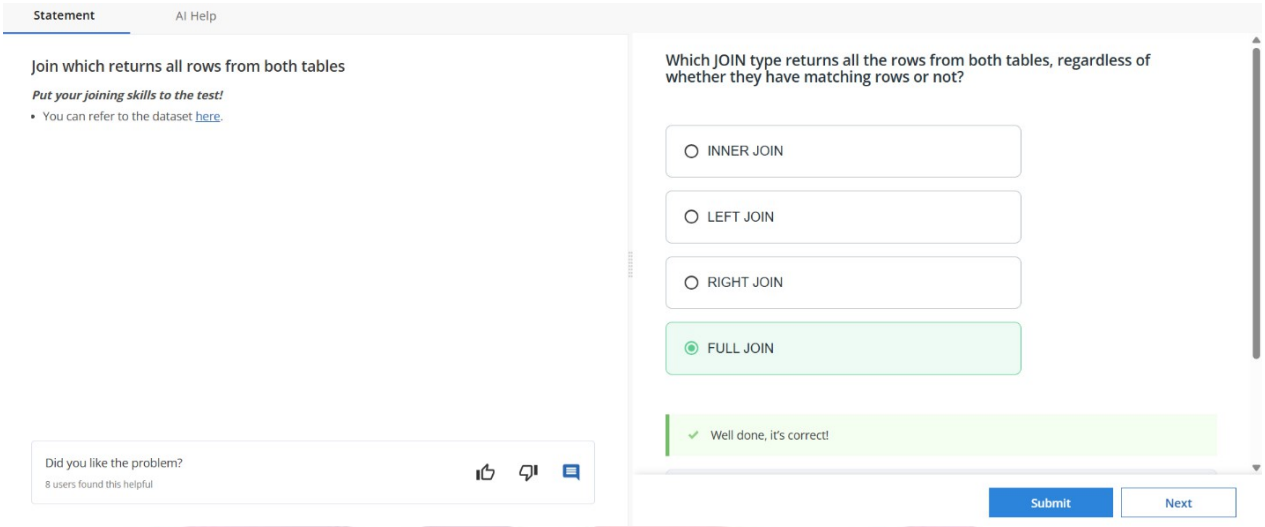
1.INTRODUCTION TO CAB BOOKING DATASET



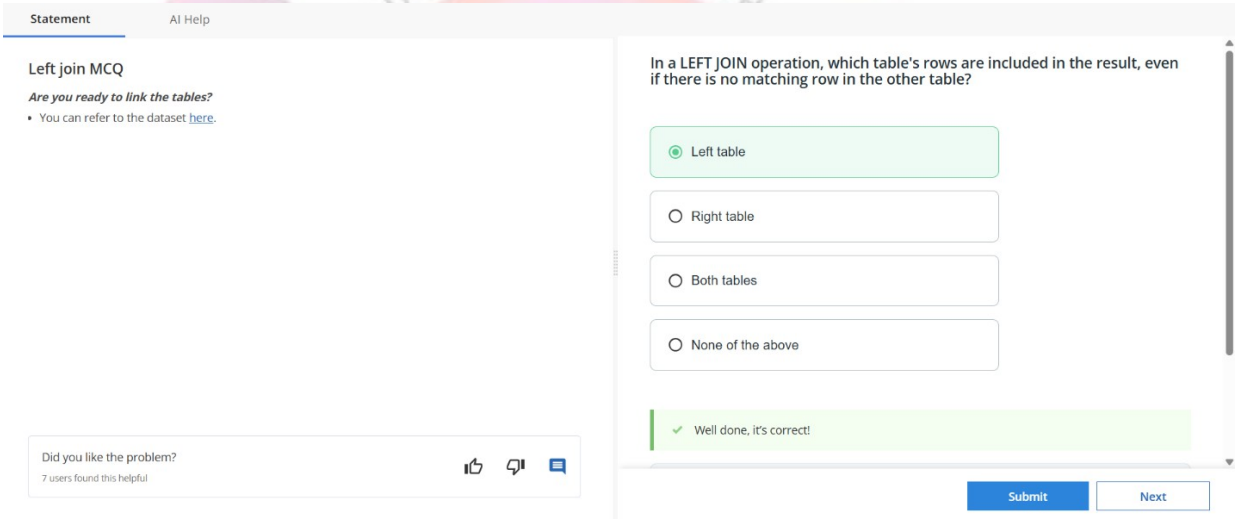
2. FIGURE OUT THE CORRECT JOIN



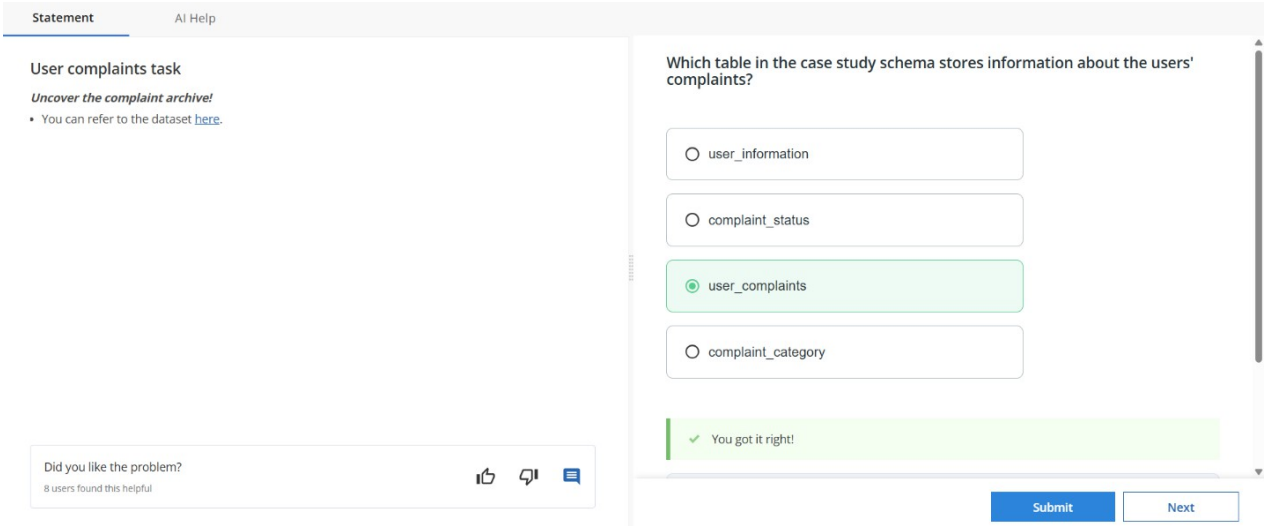
3. JOIN WHICH RETURN ALL ROWS FROM BOTH TABLES



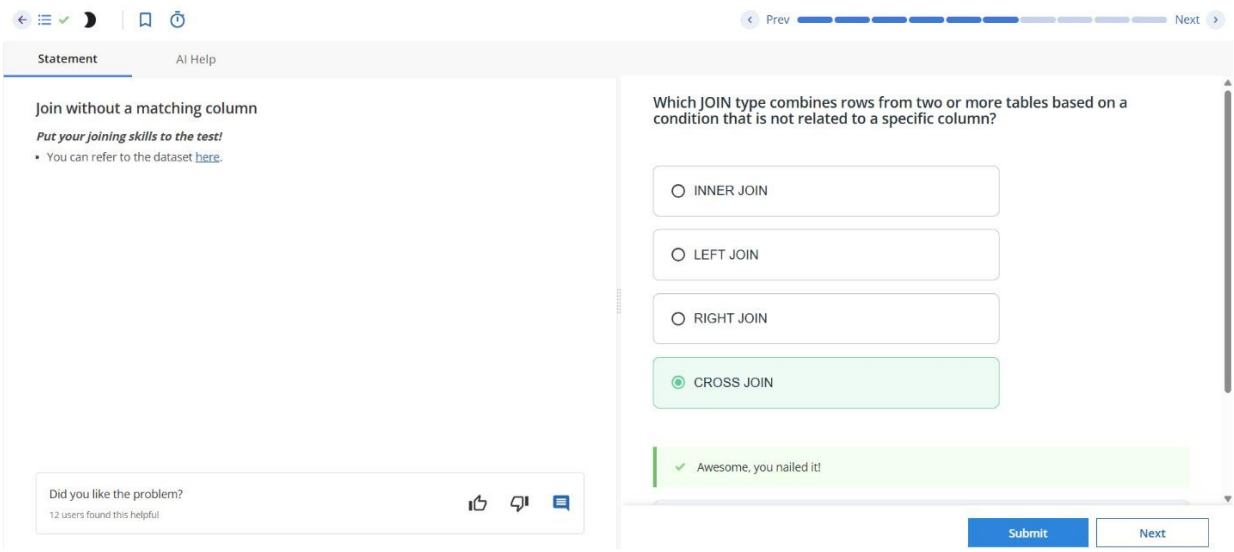
4. LEFT JOIN MCQ



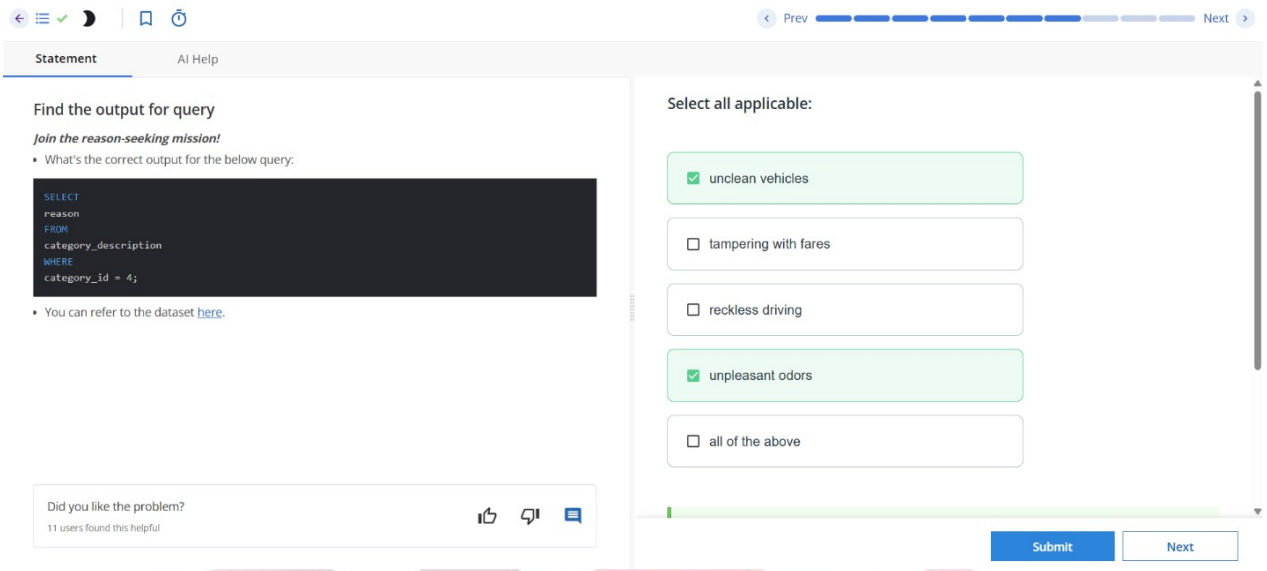
5. USER COMPLAINTS TASK



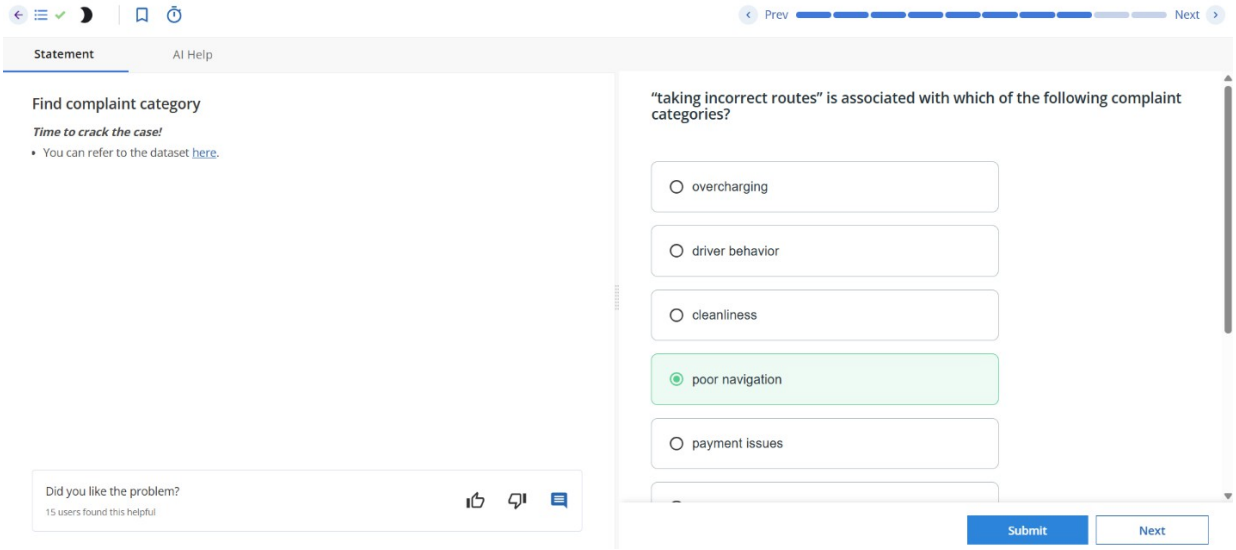
6. JOIN WITHOUT A MATCHING COLUMN



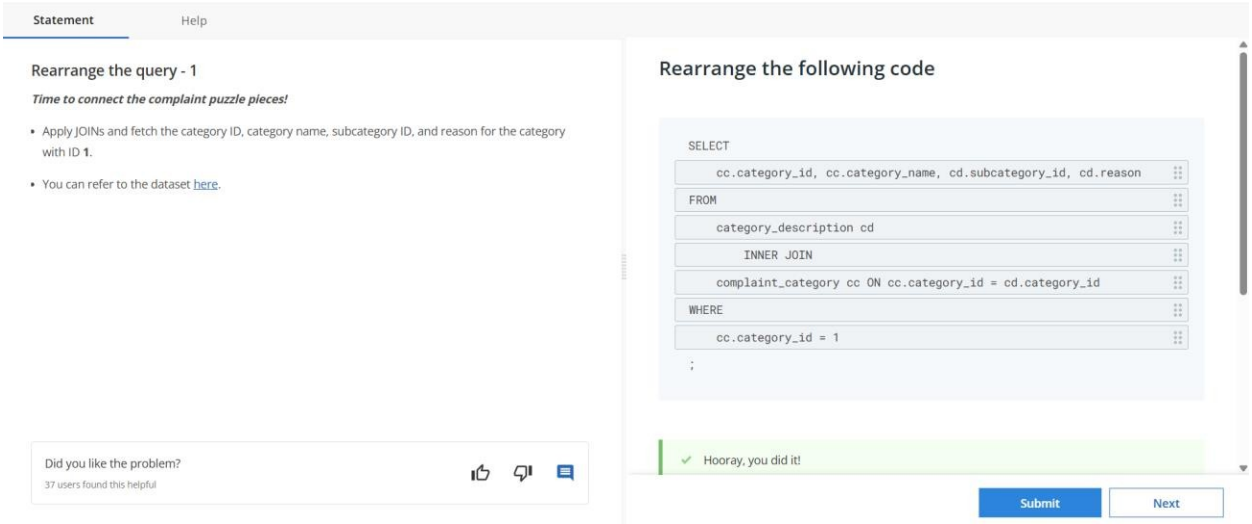
7. FIND THE OUTPUT FOR QUERY



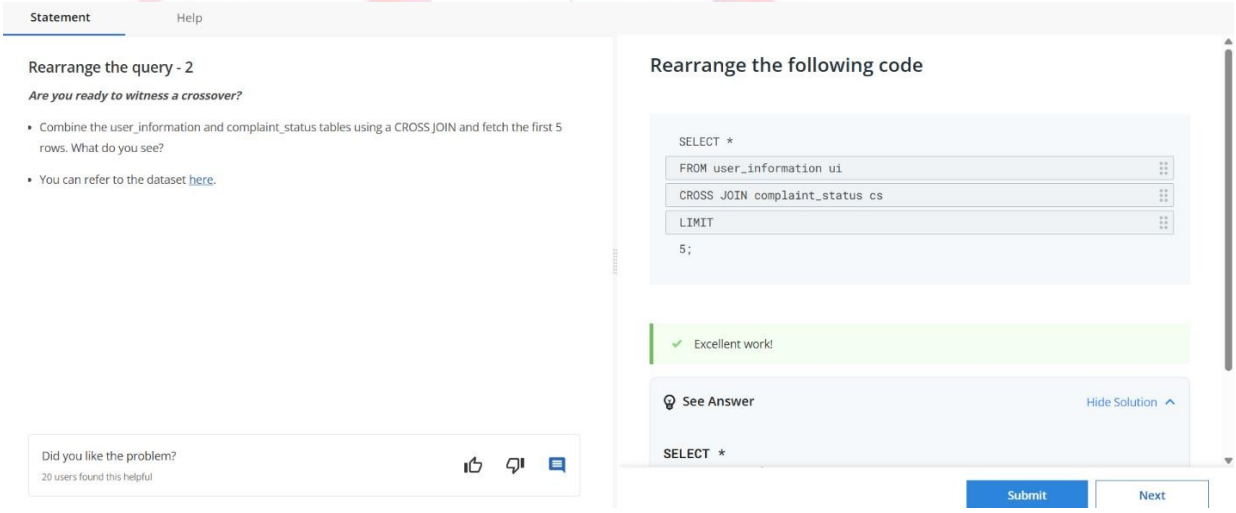
8. FIND COMPLAINT CATEGORY



9. REARRANGE THE QUERY - 1

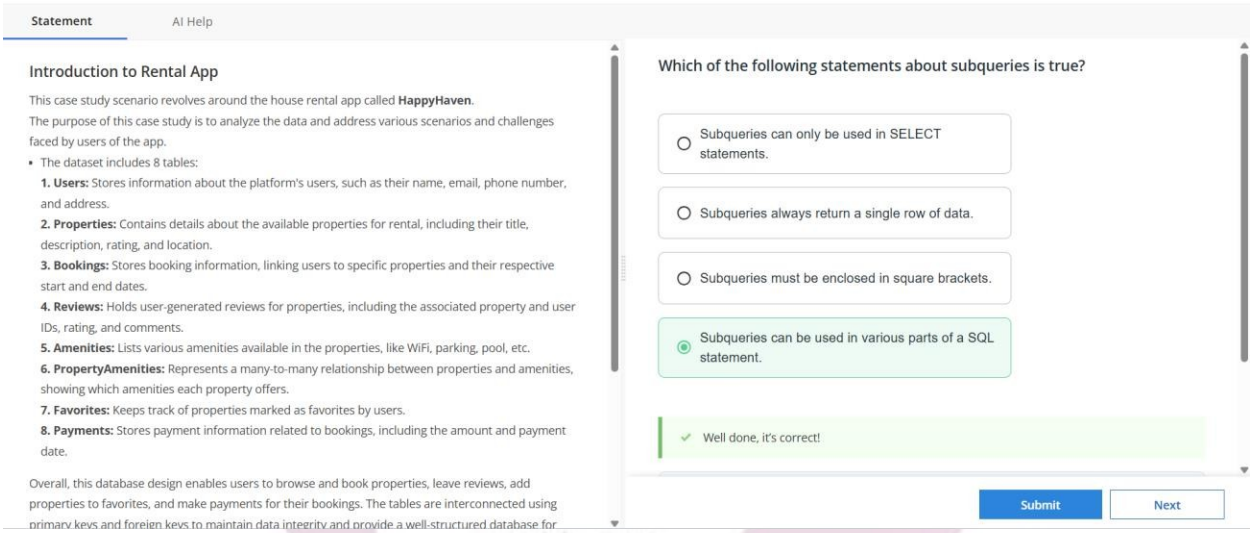


10 . REARRANGE THE QUERY - 2

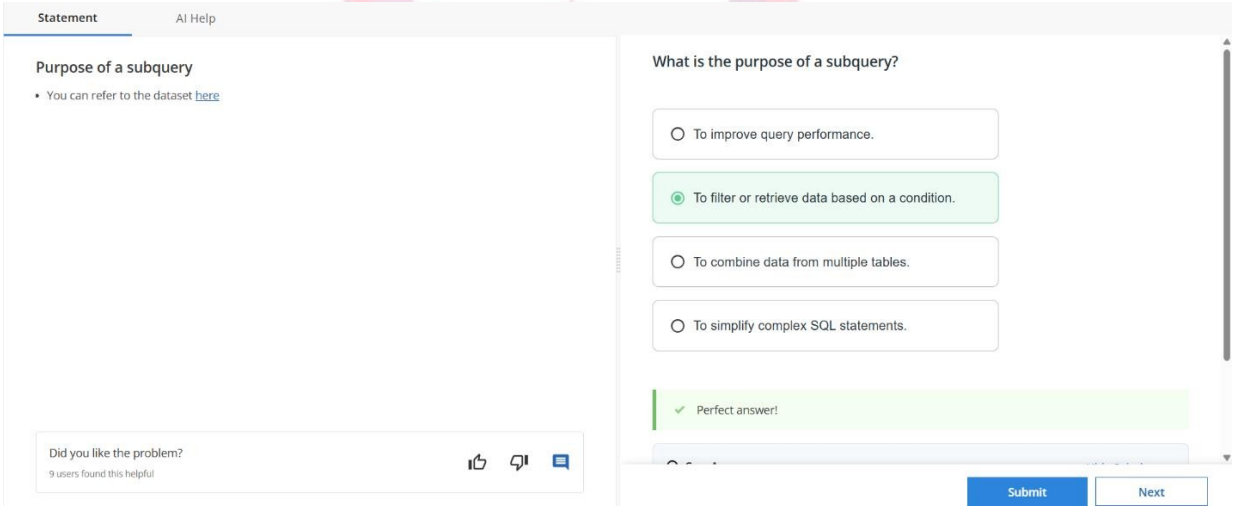


CASE STUDY – HAPPY HAVEN HOUSE RENTAL APP

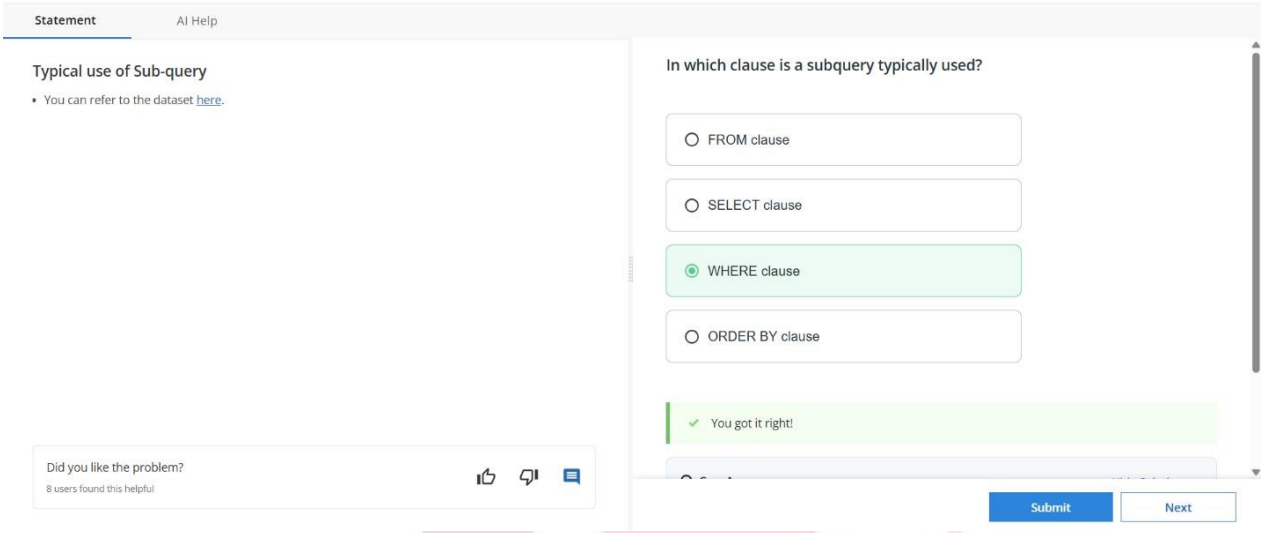
1.INTRODUCTION TO RENTAL APP



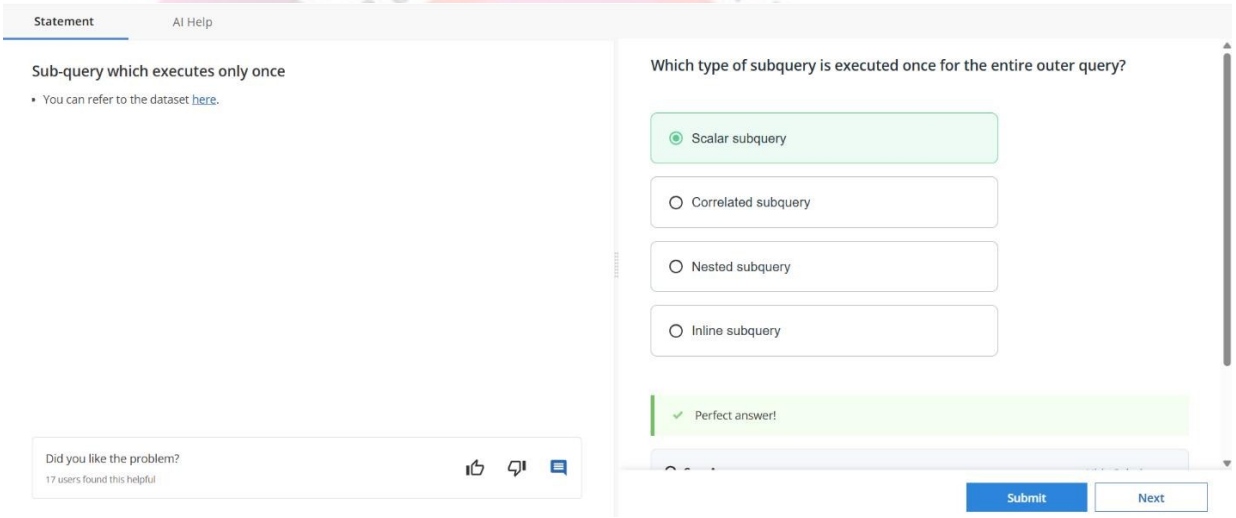
2. PURPOSE OF A SUBQUERY



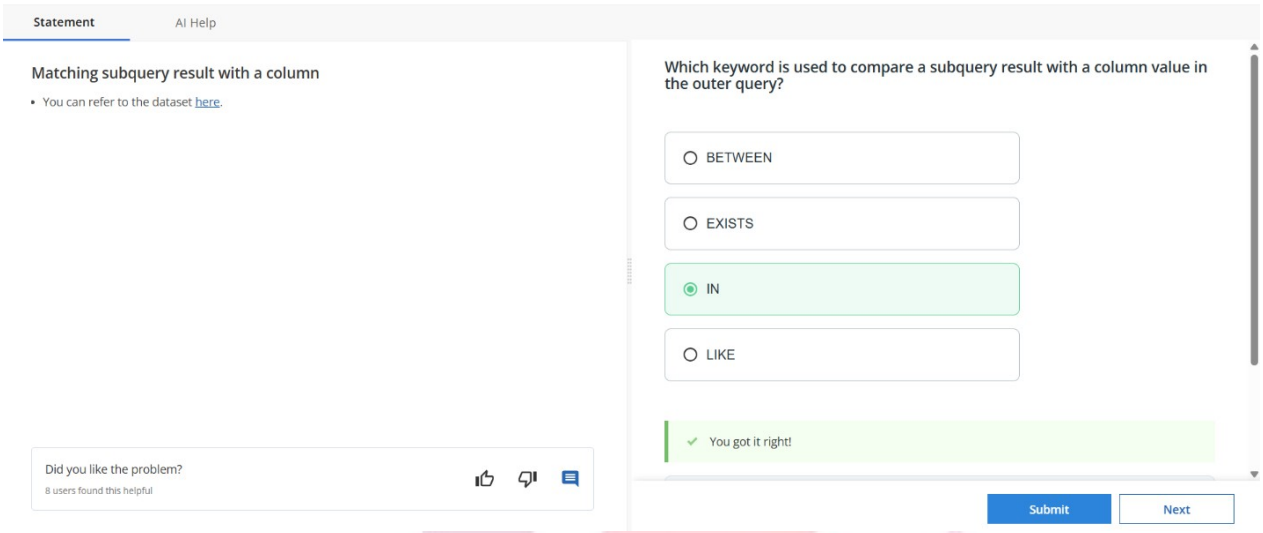
3. TYPICAL USE OF SUB-QUERY



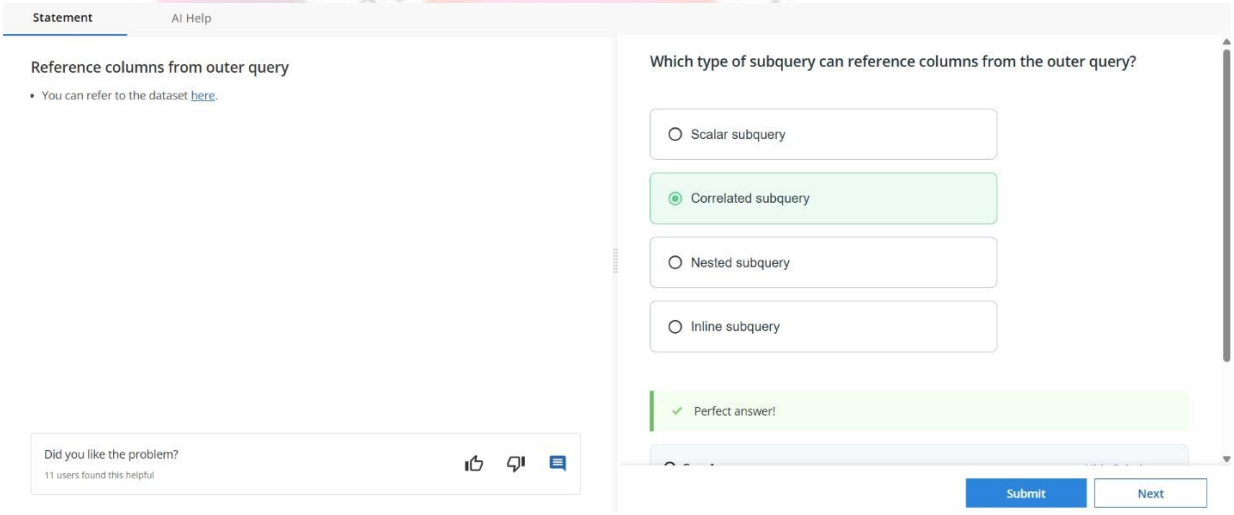
4. SUB-QUERY WHICH EXECUTES ONLY ONCE



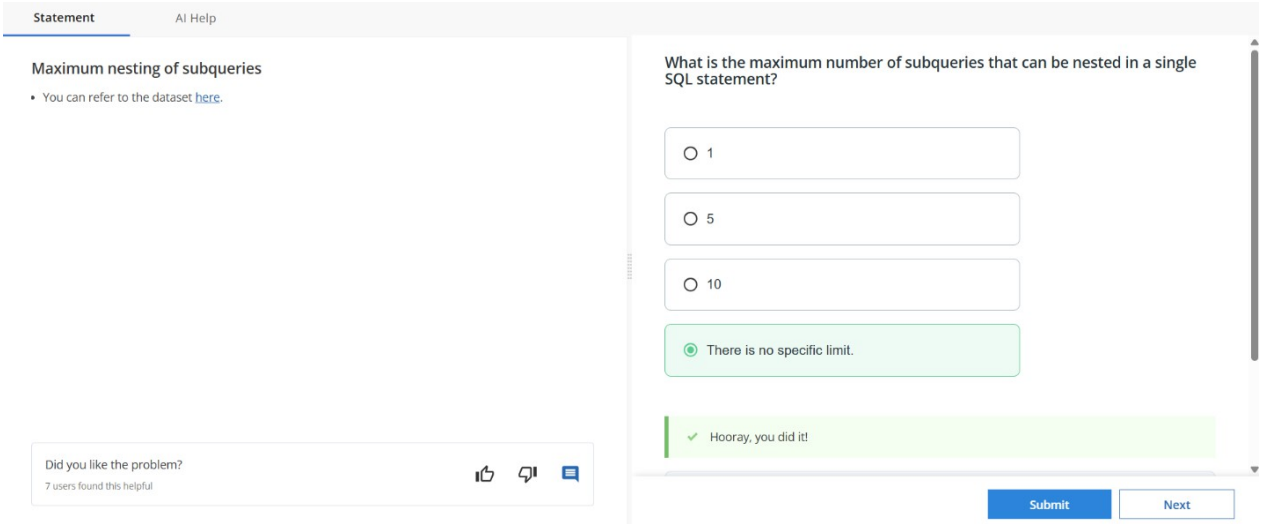
5. MATCHING SUBQUERY RESULT WITH A COLUMN



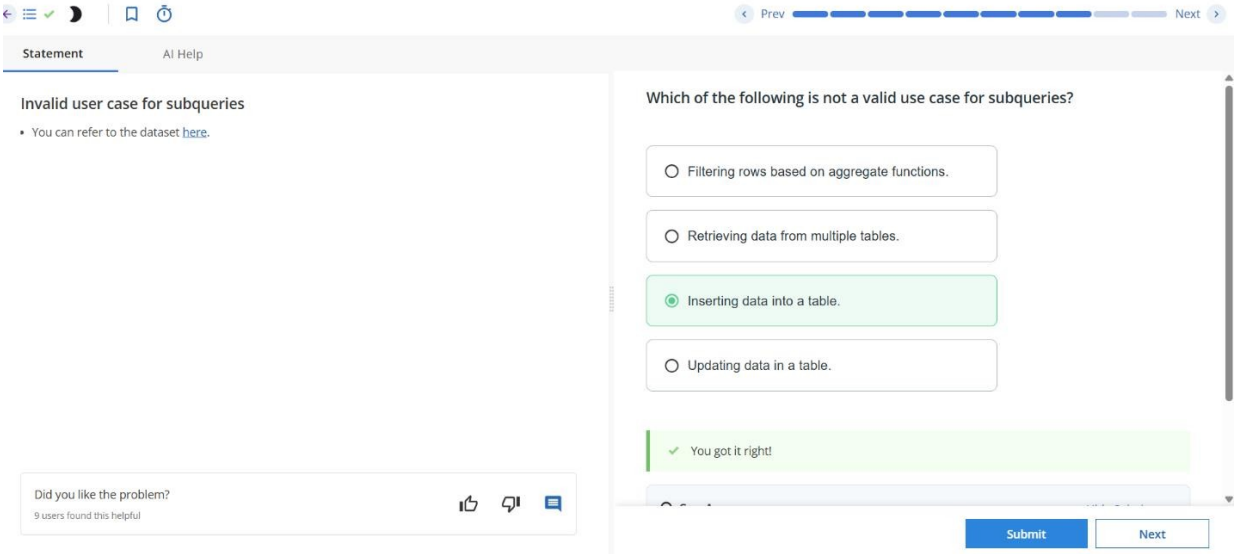
6. REFERENCE COLUMNS FROM OUTER QUERY



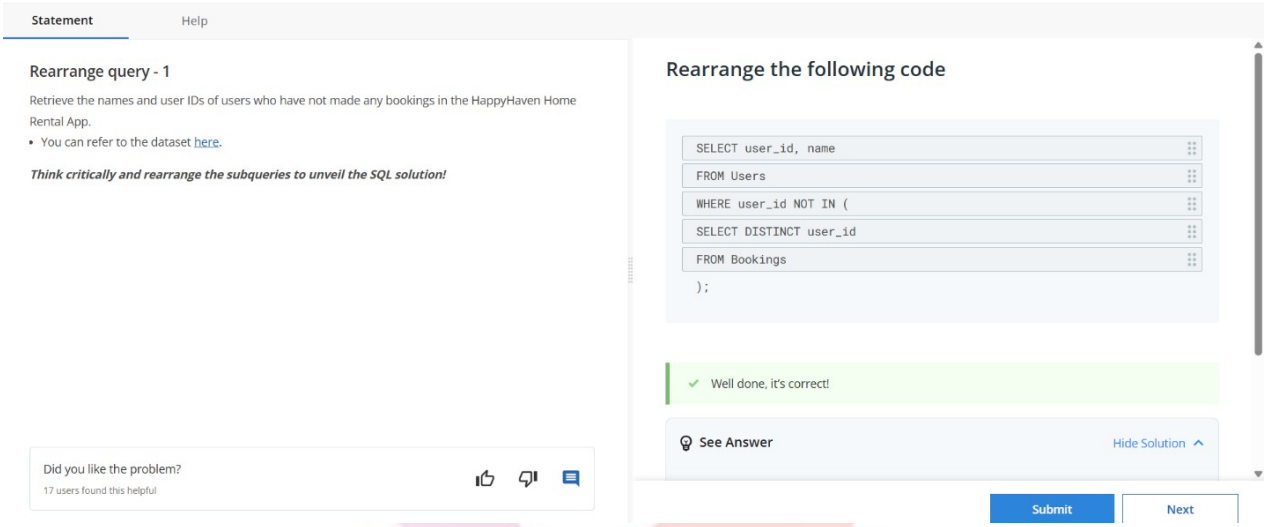
7. MAXIMUM NESTING OF SUBQUERIES



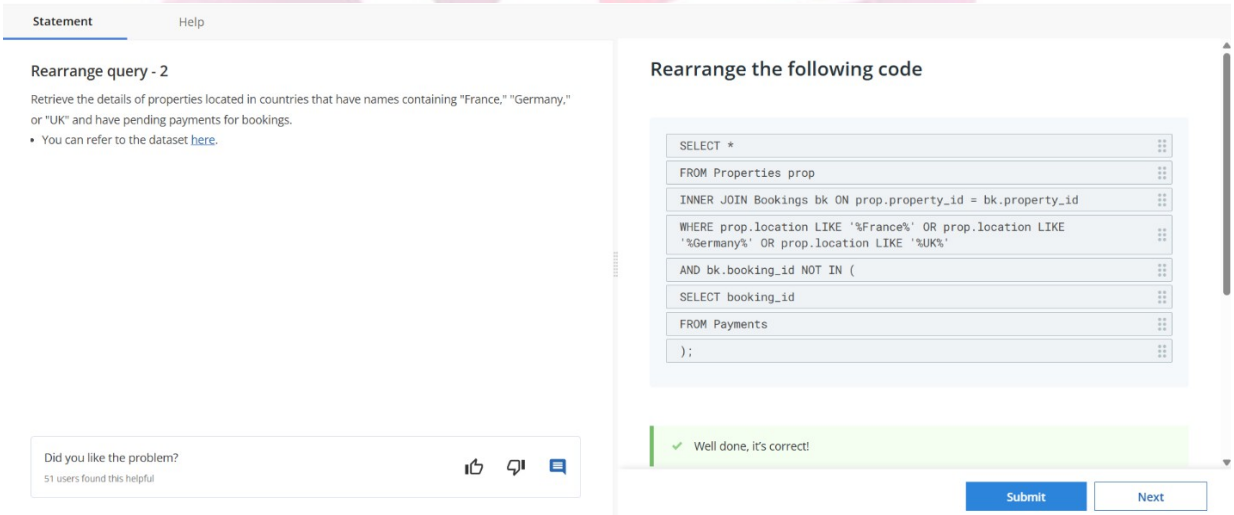
8. INVALID USER CASE FOR SUBQUERIES



9. REARRANGE QUERY-1

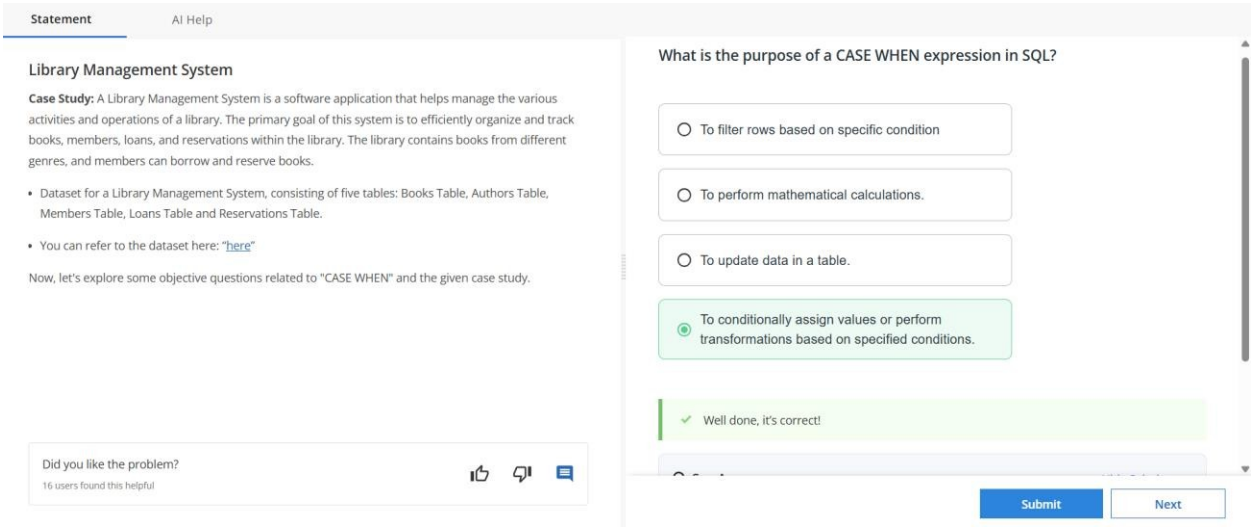


10. REARRANGE QUERY - 2

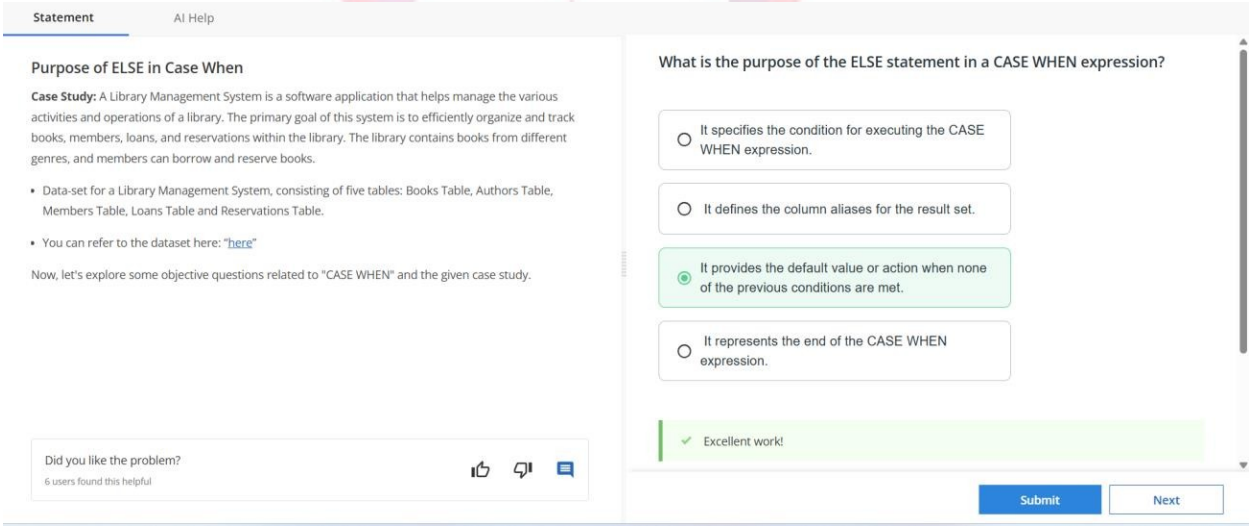


CASE STUDY – LIBRARY MANAGEMENT SYSTEM

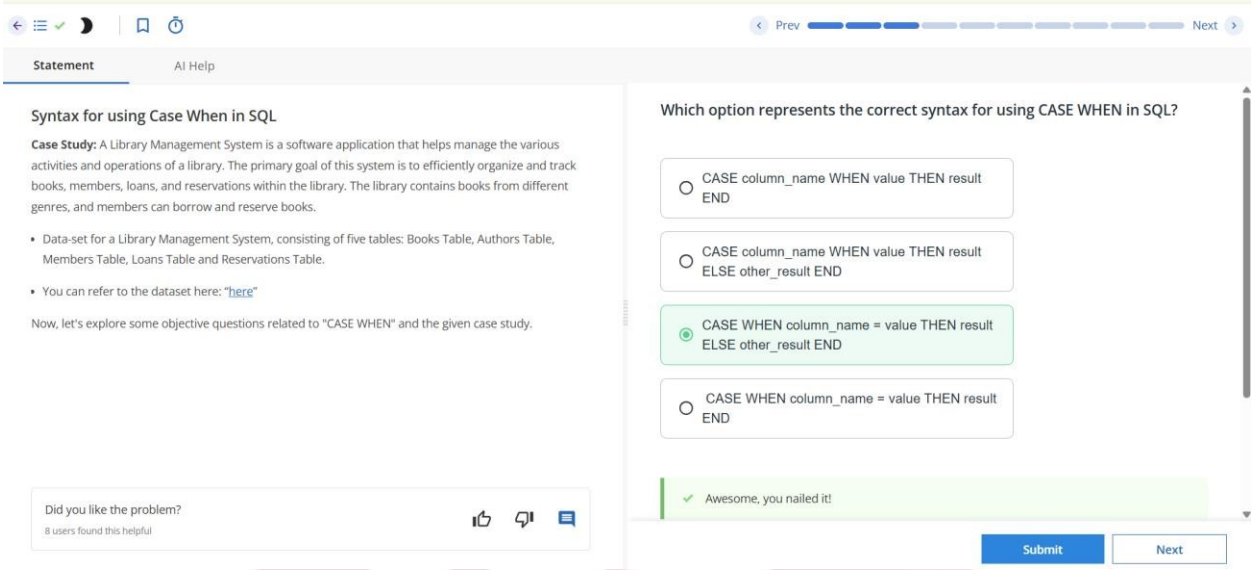
1. LIBRARY MANAGEMENT SYSTEM



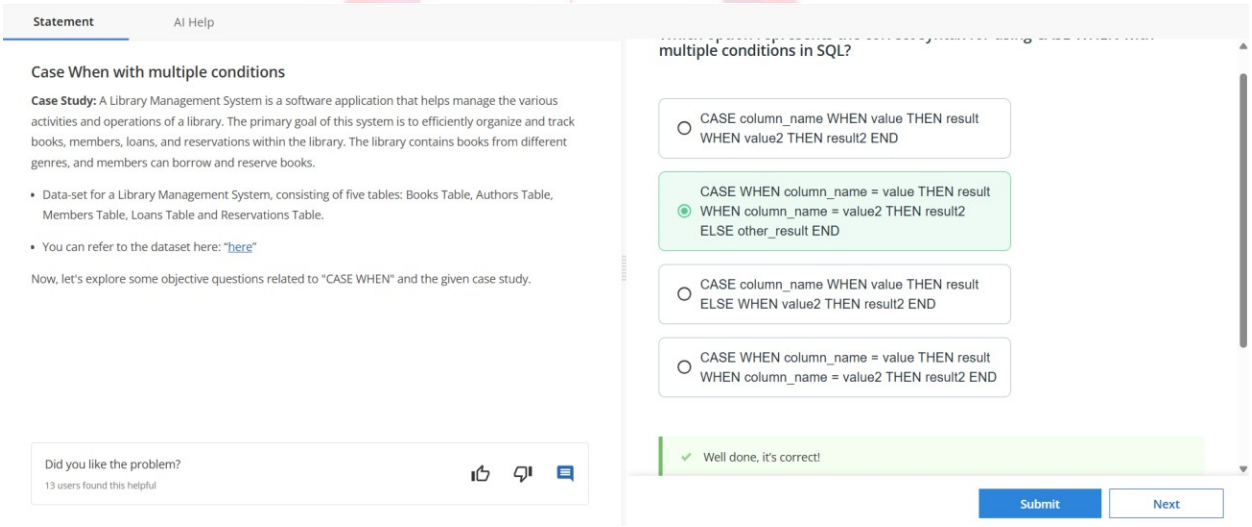
2. PURPOSE OF ELSE IN CASE WHEN



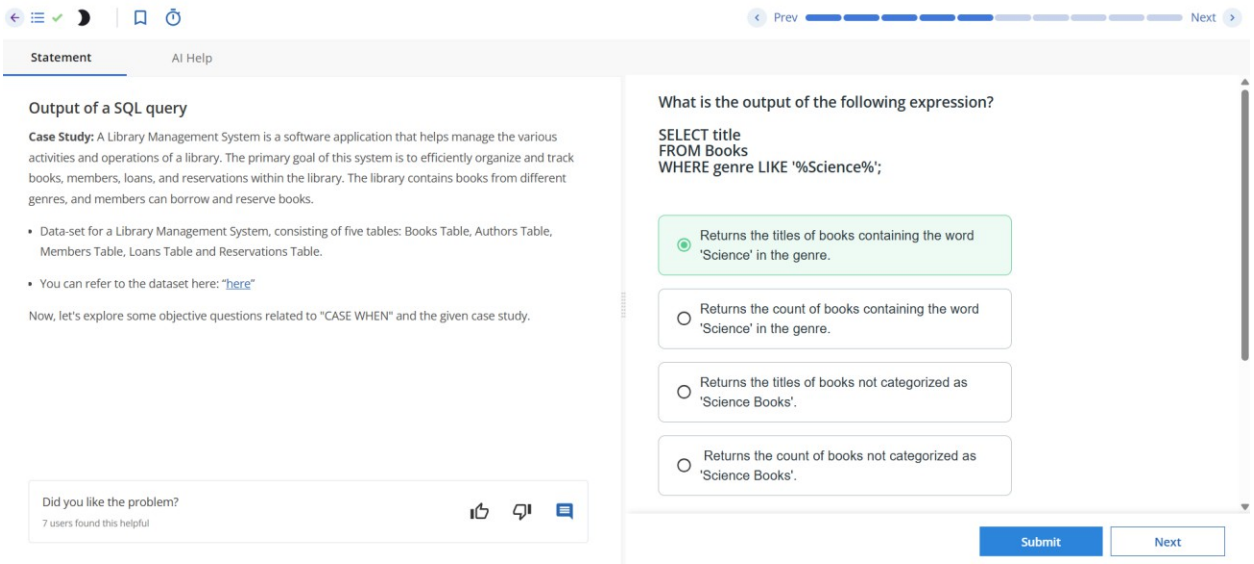
3.SYNTAX FOR USING CASE WHEN IN SQL



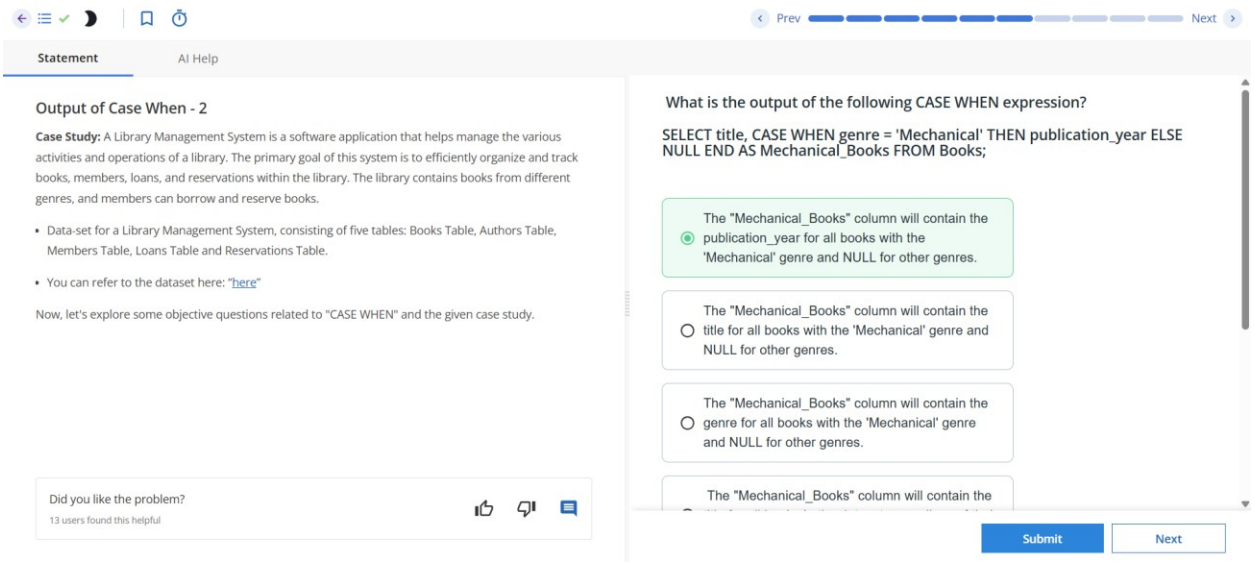
4. CASE WHEN WITH MULTIPLE CONDITIONS



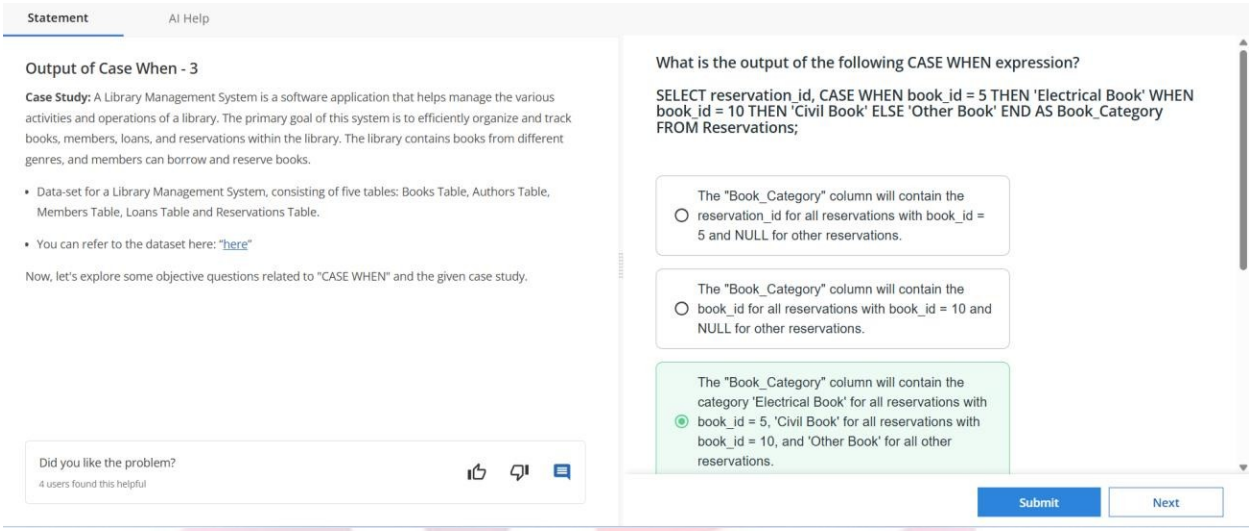
5. OUTPUT OF A SQL QUERY



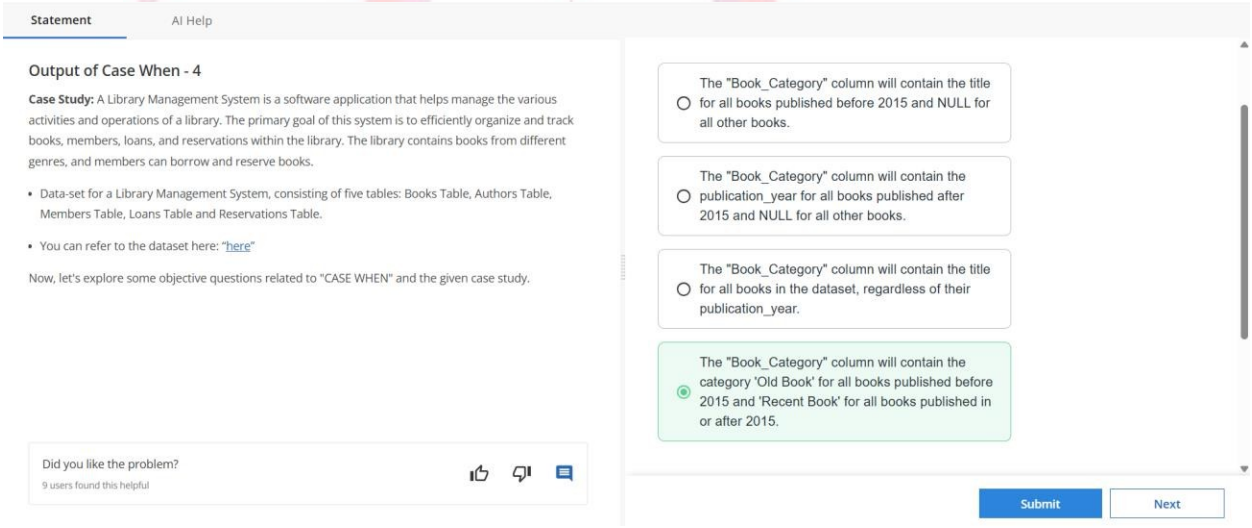
6. OUTPUT OF CASE WHEN - 2



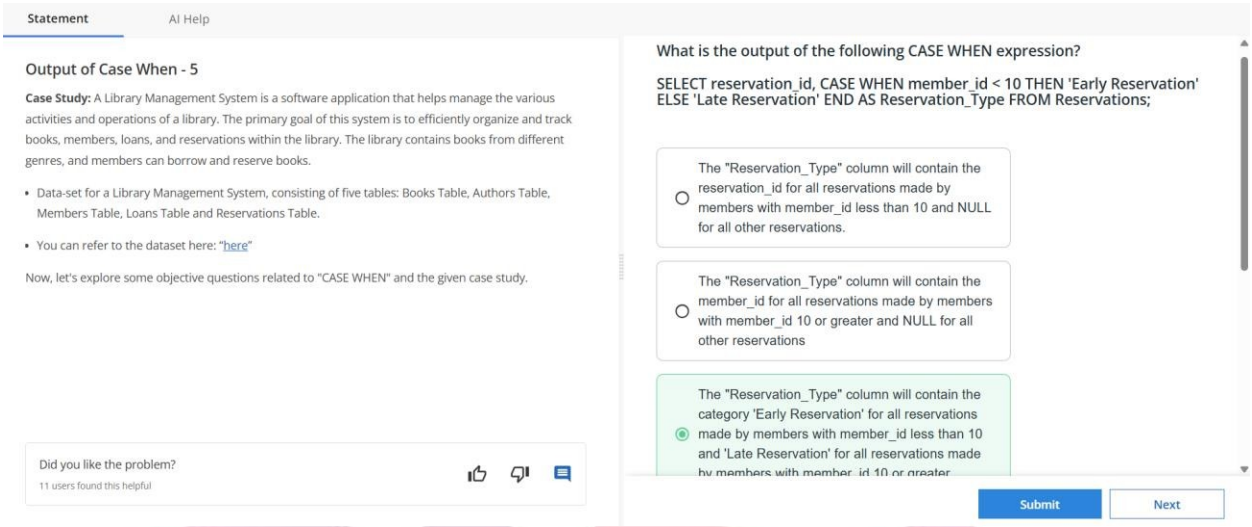
7. OUTPUT OF CASE WHEN - 3



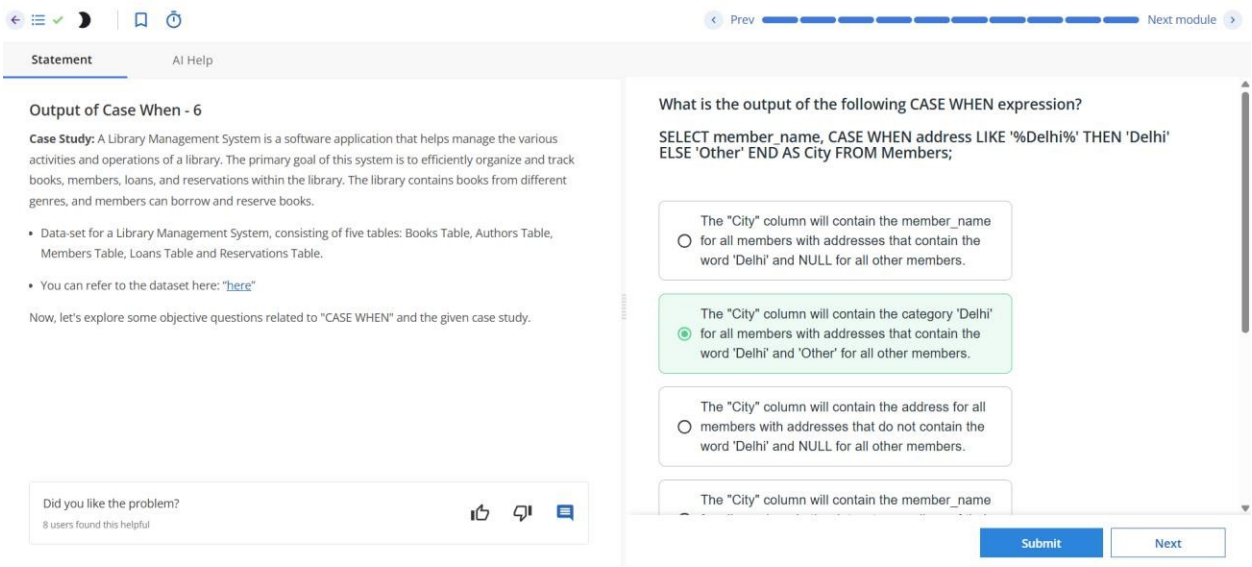
8. OUTPUT OF CASE WHEN - 4



9. OUTPUT OF CASE WHEN -5



10. OUTPUT OF CASE WHEN - 6



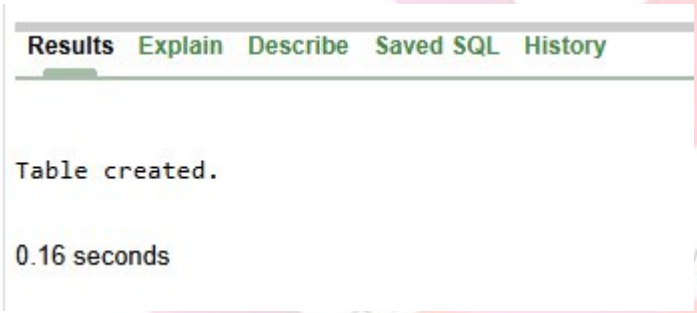
PRACTICE CODING

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

Create CABHUB Table

CREATE TABLE CABHUB (Vcode INT, VehicleName VARCHAR(20), Make VARCHAR(20), Color VARCHAR(10), Capacity INT, Charges INT);



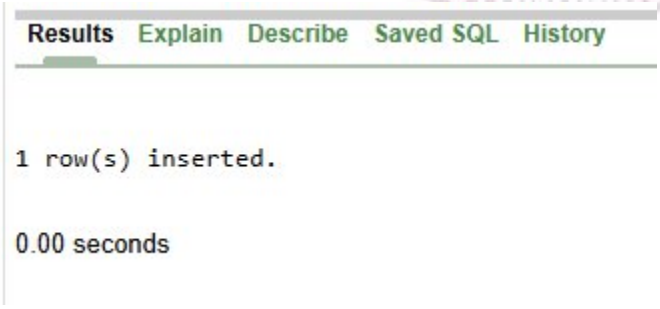
INSERT INTO CABHUB VALUES (100, 'Innova', 'Toyota', 'WHITE', 7, 15),

INSERT INTO CABHUB VALUES (102, 'SX4', 'Suzuki', 'BLUE', 4, 14),

INSERT INTO CABHUB VALUES (104, 'C-Class', 'Mercedes', 'RED', 4, 35),

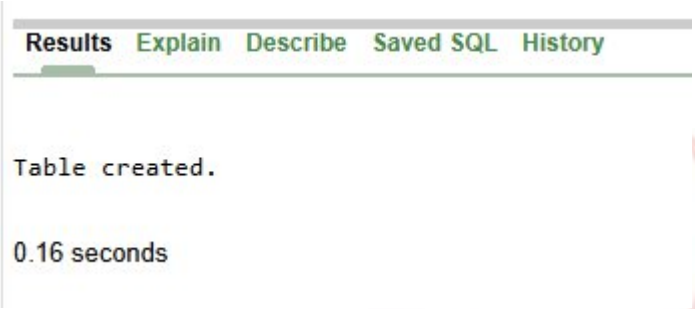
INSERT INTO CABHUB VALUES (105, 'A-Star', 'Suzuki', 'WHITE', 3, 14),

INSERT INTO CABHUB VALUES (108, 'Indigo', 'Tata', 'SILVER', 3, 12);



Create CUSTOMER Table

CREATE TABLE CUSTOMER (Code INT, CName VARCHAR(30), VCode INT);

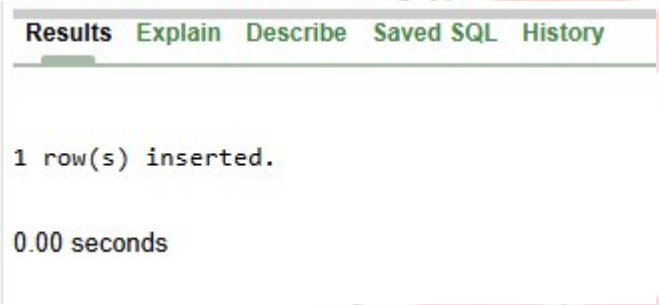


INSERT INTO CUSTOMER VALUES (1, 'Hemant Sahu', 101),

INSERT INTO CUSTOMER VALUES (2, 'Raj Lal', 108),

INSERT INTO CUSTOMER VALUES (3, 'Feroza Shah', 105),

INSERT INTO CUSTOMER VALUES (4, 'Ketan Dhal', 104);



(i) To display the names of all the white colored vehicles.

SELECT VehicleName FROM CABHUB WHERE Color = 'WHITE';



(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

SELECT VehicleName, Make, Capacity FROM CABHUB ORDER BY Capacity ASC;

Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
Indigo	Tata	3
A-Star	Suzuki	3
SX4	Suzuki	4
C-Class	Mercedes	4
Innova	Toyota	7

5 rows returned in 0.00 seconds CSV Export

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

SELECT MAX(Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

(iv) To display the customer and the corresponding name of the vehicle hired by them.

SELECT CUSTOMER.CName, CABHUB.VehicleName FROM CUSTOMER INNER JOIN CABHUB ON CUSTOMER.VCode = CABHUB.VCode;

Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
Ketan Dhal	C-Class
Feroza Shah	A-Star
Raj Lal	Indigo

3 rows returned in 0.00 seconds CSV Export

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT(*), Make FROM CABHUB;

Results Explain Describe Saved SQL History

MAKE	COUNT(*)
Suzuki	2
Mercedes	1
Toyota	1
Tata	1

4 rows returned in 0.02 seconds CSV Export

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

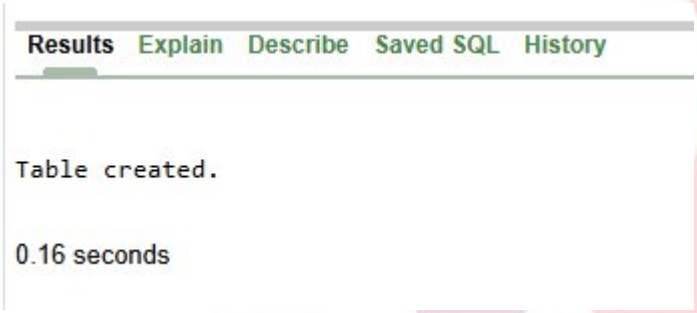
Results Explain Describe Saved SQL History

VEHICLENAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

CREATE TABLE CO2_EMISSION (VehicleID INT, Company VARCHAR(20), Model VARCHAR(20), FuelType VARCHAR(10), EngineSize DECIMAL(2,1), CO2Emission INT);



INSERT INTO CO2_EMISSION VALUES (101, 'Toyota', 'Corolla', 'Petrol', 1.8, 165);

INSERT INTO CO2_EMISSION VALUES (102, 'Hyundai', 'Creta', 'Diesel', 1.5, 145);

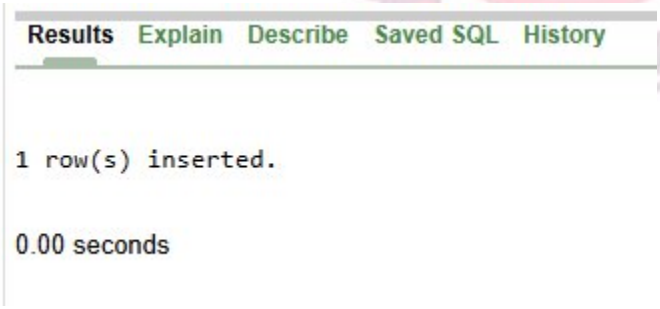
INSERT INTO CO2_EMISSION VALUES (103, 'Honda', 'City', 'Petrol', 1.5, 152);

INSERT INTO CO2_EMISSION VALUES (104, 'Tata', 'Nexon', 'Diesel', 1.5, 138);

INSERT INTO CO2_EMISSION VALUES (105, 'Maruti', 'Swift', 'Petrol', 1.2, 125);

INSERT INTO CO2_EMISSION VALUES (106, 'Kia', 'Seltos', 'Diesel', 1.5, 150);

INSERT INTO CO2_EMISSION VALUES (107, 'Mahindra', 'XUV700', 'Diesel', 2.2, 185);



(a) Display all the details of vehicles that use Petrol as the fuel type.

SELECT * FROM CO2_EMISSION WHERE FuelType = 'Petrol';

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125

3 rows returned in 0.02 seconds CSV Export

(b) Display the Company and Model

of vehicles whose EngineSize is greater than 1.5 litres.

SELECT Company, Model FROM CO2_EMISSION WHERE EngineSize > 1.5;

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.01 seconds CSV Export

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

SELECT Company, Model, CO2Emission FROM CO2_EMISSION ORDER BY CO2Emission ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
Toyota	Corolla	165
Mahindra	XUV700	185

7 rows returned in 0.00 seconds CSV Export

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

SELECT * FROM CO2_EMISSION WHERE CO2Emission > 150;

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185

3 rows returned in 0.00 seconds CSV Export

(e) Count the number of vehicles that use Diesel as the fuel type.

SELECT COUNT(*) FROM CO2_EMISSION WHERE FuelType = 'Diesel';

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record into the CO2_EMISSION table.

108, 'MG', 'Hector', 'Petrol', 1.5, 160

INSERT INTO CO2_EMISSION VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);

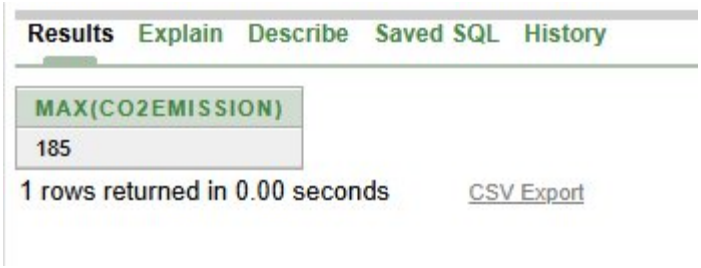
Results Explain Describe Saved SQL History

COUNT(*)
4

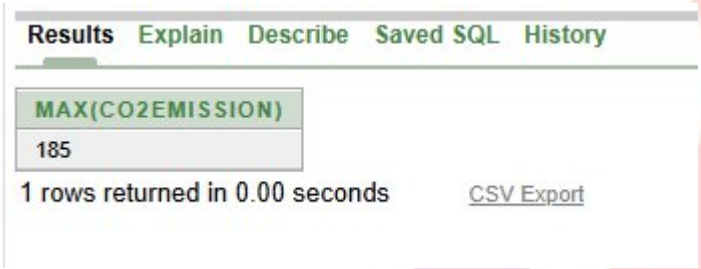
1 rows returned in 0.00 seconds CSV Export

(g) Write the output of the following SQL commands.

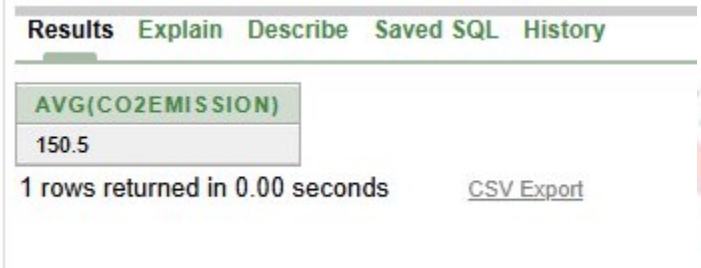
(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;



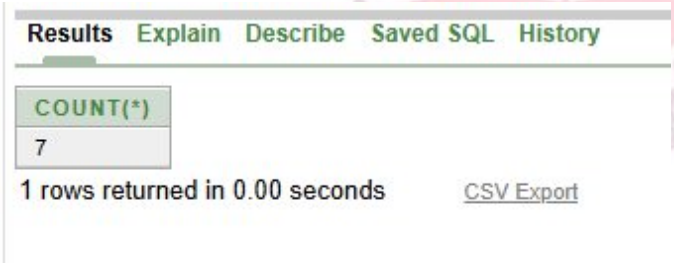
(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;



(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';



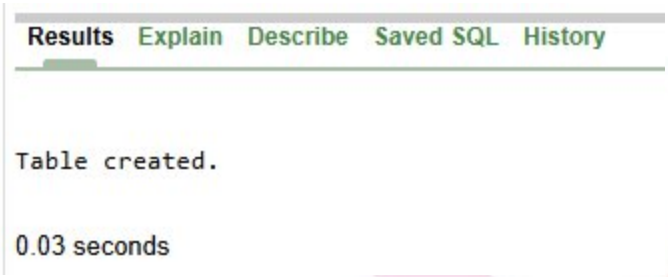
(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;



3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

Create APPLICANTS Table

CREATE TABLE APPLICANTS (NO NUMBER(4),NAME VARCHAR2(20), FEE NUMBER(10),GENDER CHAR(1),C_ID VARCHAR2(3), JOINYEAR NUMBER(4));



INSERT INTO APPLICANTS VALUES (1012, 'Amandeep', 30000, 'M', 'A01', 2012);

INSERT INTO APPLICANTS VALUES (1102, 'Avisha', 25000, 'F', 'A02', 2009);

INSERT INTO APPLICANTS VALUES (1103, 'Ekant', 30000, 'M', 'A02', 2011);

INSERT INTO APPLICANTS VALUES (1049, 'Arun', 30000, 'M', 'A03', 2009);

INSERT INTO APPLICANTS VALUES (1025, 'Amber', 40000, 'M', 'A02', 2011);

INSERT INTO APPLICANTS VALUES (1106, 'Ela', 40000, 'F', 'A05', 2010);

INSERT INTO APPLICANTS VALUES (1017, 'Nikita', 35000, 'F', 'A03', 2012);

INSERT INTO APPLICANTS VALUES (1108, 'Arleena', 30000, 'F', 'A03', 2012);

INSERT INTO APPLICANTS VALUES (2109, 'Shakti', 35000, 'M', 'A04', 2011);

INSERT INTO APPLICANTS VALUES (1101, 'Kirat', 25000, 'M', 'A01', 2012);

COMMIT;



Create COURSES Table

CREATE TABLE COURSES (C_ID VARCHAR2(3), COURSE VARCHAR2(30)) ;



INSERT INTO COURSES VALUES ('A01', 'Fashion Design');

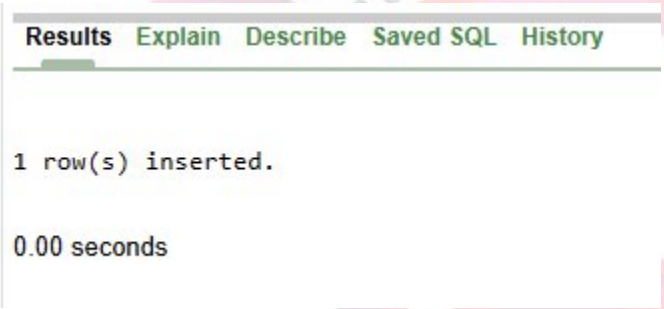
INSERT INTO COURSES VALUES ('A02', 'Networking');

INSERT INTO COURSES VALUES ('A03', 'Hotel Management');

INSERT INTO COURSES VALUES ('A04', 'Event Management');

INSERT INTO COURSES VALUES ('A05', 'Office Management');

COMMIT;



(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANTS WHERE JOINYEAR < 2010;

Results Explain Describe Saved SQL History

NAME	FEE	GENDER	JOINYEAR
Avisha	25000	F	2009
Arun	30000	M	2009

2 rows returned in 0.01 seconds CSV Export

(b) To display the names of applicants, who are paying fee more than 30000.

SELECT NAME FROM APPLICANTS WHERE FEE > 30000;

Results Explain Describe Saved SQL History

NAME
Amber
Ela
Nikita
Shakti

4 rows returned in 0.00 seconds CSV Export

(c) To display name of all applicants in ascending order of their joinyear.

SELECT NAME, JOINYEAR FROM APPLICANTS ORDER BY JOINYEAR ASC;

Results Explain Describe Saved SQL History

NAME	JOINYEAR
Arun	2009
Avisha	2009
Ela	2010
Ekant	2011
Amber	2011
Shakti	2011
Nikita	2012
Arleena	2012
Amandeep	2012
Kirat	2012

10 rows returned in 0.01 seconds CSV Export

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

SELECT JOINYEAR, COUNT(*) AS TOTAL_APPLICANTS FROM APPLICANTS GROUP BY JOINYEAR ORDER BY JOINYEAR;

Results Explain Describe Saved SQL History

JOINYEAR	TOTAL_APPLICANTS
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.02 seconds CSV Export

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

SELECT C_ID, COUNT(*) AS NO_OF_APPLICANTS FROM APPLICANTS GROUP BY C_ID ORDER BY C_ID;

Results Explain Describe Saved SQL History

C_ID	NO_OF_APPLICANTS
A01	2
A02	3
A03	3
A04	1
A05	1

5 rows returned in 0.00 seconds CSV Export

(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

SELECT A.NAME, C.COURSE FROM APPLICANTS A, COURSES C WHERE A.C_ID = C.C_ID;

Results Explain Describe Saved SQL History

NAME	COURSE
Amandeep	Fashion Design
Avisha	Networking
Ekant	Networking
Arun	Hotel Management
Amber	Networking
Ela	Office Management
Nikita	Hotel Management
Arleena	Hotel Management
Shakti	Event Management
Kirat	Fashion Design

10 rows returned in 0.00 seconds CSV Export

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOINYEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= 'A02';

Results Explain Describe Saved SQL History

NAME	JOINYEAR
Avisha	2009

1 rows returned in 0.00 seconds CSV Export

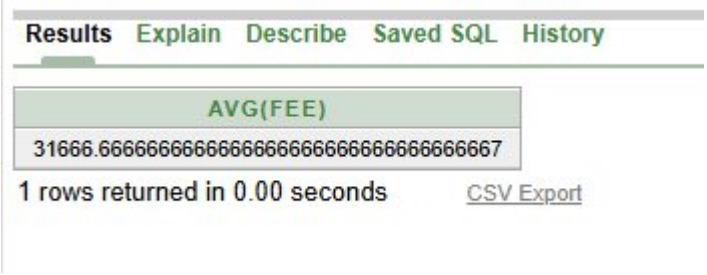
(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';

Results Explain Describe Saved SQL History

MIN(JOINYEAR)
2009

1 rows returned in 0.00 seconds CSV Export

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= ‘A01’ OR C_ID= ‘A05’;



Results Explain Describe Saved SQL History

AVG(FEE)
31666.666666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

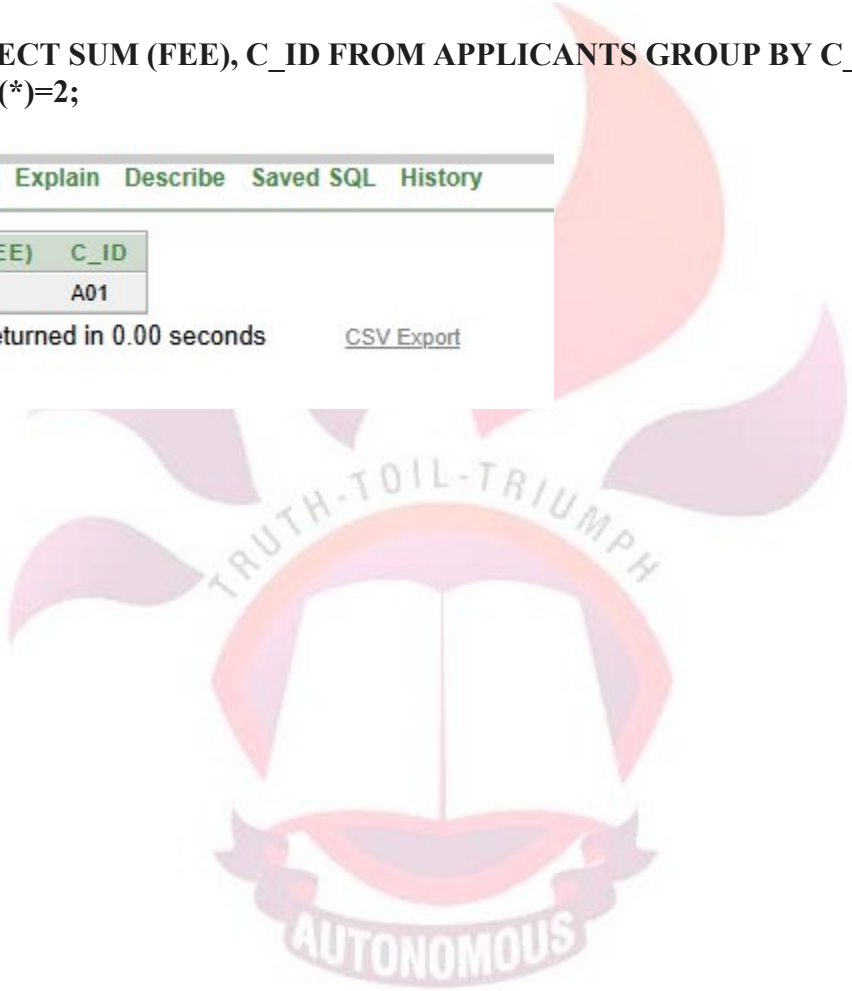
(g4) SELECT SUM (FEE), C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT (*)=2;



Results Explain Describe Saved SQL History

SUM(FEE)	C_ID
55000	A01

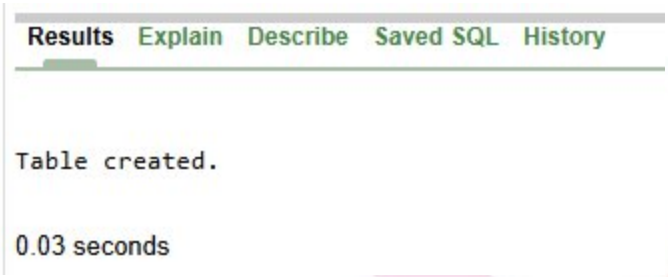
1 rows returned in 0.00 seconds CSV Export



4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables PRODUCTS AND SUPPLIERS

Create SUPPLIERS Table

CREATE TABLE SUPPLIERS (SUPCODE VARCHAR2(3),SNAME VARCHAR2(30),CITY VARCHAR2(20));

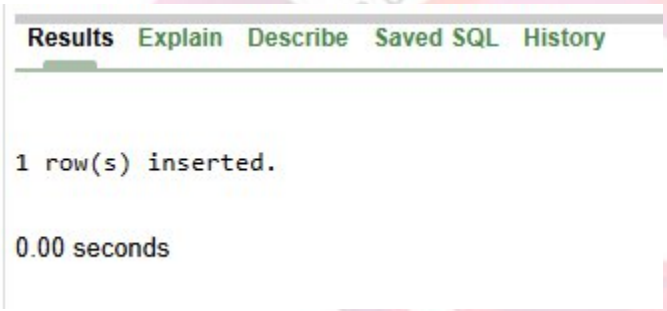


INSERT INTO SUPPLIERS VALUES ('S01', 'GET ALL INC', 'KOLKATA');

INSERT INTO SUPPLIERS VALUES ('S03', 'EASY MARKET CORP', 'DELHI');

INSERT INTO SUPPLIERS VALUES ('S02', 'DIGI BUSY GROUP', 'CHENNAI');

COMMIT;



Create PRODUCTS Table

CREATE TABLE PRODUCTS (PID NUMBER(3),SNAME VARCHAR2(30), QTY NUMBER(5),PRICE NUMBER(10),COMPANY VARCHAR2(30),SUPCODE VARCHAR2(3));



```
INSERT INTO PRODUCTS VALUES (101, 'DIGITAL CAMERA14X', 120, 12000, 'RENIX',
'S01');

INSERT INTO PRODUCTS VALUES (102, 'DIGITAL PAD 11i', 100, 22000, 'DIGI POP',
'S02');

INSERT INTO PRODUCTS VALUES (104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING',
'S01');

INSERT INTO PRODUCTS VALUES (106, 'LED SCREEN', 70, 28000, 'DISEXPERTS',
'S02');

INSERT INTO PRODUCTS VALUES (105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON',
'S03');

COMMIT;
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

```
SELECT * FROM PRODUCTS ORDER BY SNAME ASC;
```

Results Explain Describe Saved SQL History

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 11i	100	22000	DIGI POP	S02
106	LED SCREEN	70	28000	DISEXPERTS	S02
104	PENDRIVE 16GB	500	1100	STOREKING	S01

5 rows returned in 0.00 seconds CSV Export

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

SELECT SNAME, PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;



Results Explain Describe Saved SQL History

SNAME	PRICE
DIGITAL CAMERA14X	12000
CAR GPS SYSTEM	12000

2 rows returned in 0.02 seconds CSV Export

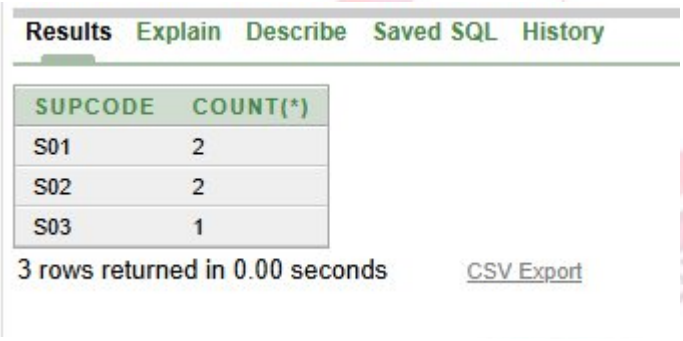
(c) To display the number of products which are supplied by each supplier i.e. the expected output should be

S01 2

S02 2

S03 1

SELECT SUPCODE, COUNT(*) FROM PRODUCTS GROUP BY SUPCODE ORDER BY SUPCODE;



Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.00 seconds CSV Export

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

SELECT PRICE, SNAME, QTY FROM PRODUCTS WHERE QTY > 100;

Results Explain Describe Saved SQL History

PRICE	SNAME	QTY
12000	DIGITAL CAMERA14X	120
1100	PENDRIVE 16GB	500

2 rows returned in 0.00 seconds CSV Export

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

SELECT SNAME FROM SUPPLIERS WHERE CITY = 'DELHI' OR CITY = 'CHENNAI';

Results Explain Describe Saved SQL History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.02 seconds CSV Export

(f) To display the name of the companies and the name of the products in descending order of company names.

SELECT COMPANY, SNAME FROM PRODUCTS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	SNAME
STOREKING	PENDRIVE 16GB
RENIX	DIGITAL CAMERA14X
MOVEON	CAR GPS SYSTEM
DISEXPERTS	LED SCREEN
DIGI POP	DIGITAL PAD 11i

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.01 seconds CSV Export

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

Results Explain Describe Saved SQL History

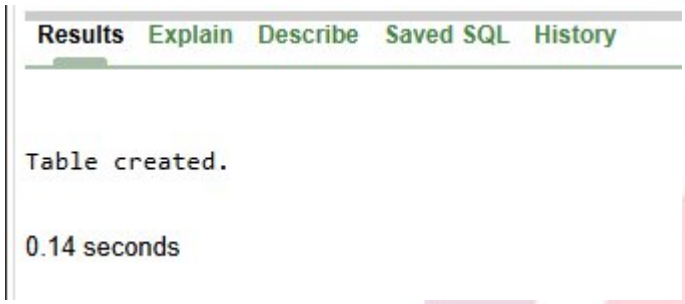
AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

Create SHOPPE table

CREATE TABLE SHOPPE (ID VARCHAR2(5), SNAME VARCHAR2(30), AREA VARCHAR2(20));



INSERT INTO SHOPPE VALUES ('S0001', 'ABC Computeronics', 'CP');

INSERT INTO SHOPPE VALUES ('S0002', 'All Infotech Media', 'GK II');

INSERT INTO SHOPPE VALUES ('S0003', 'Tech Shoppe', 'CP');

INSERT INTO SHOPPE VALUES ('S0004', 'Greeks Techno Soft', 'Nehru Place');

INSERT INTO SHOPPE VALUES ('S0005', 'Hitech Tech Store', 'Nehru Place');

COMMIT;



Create ACCESSORIES table

CREATE TABLE ACCESSORIES (NO VARCHAR2(5), NAME VARCHAR2(30), PRICE NUMBER(10), ID VARCHAR2(5));




```
INSERT INTO ACCESSORIES VALUES ('A01', 'Mother Board', 12000, 'S01');
INSERT INTO ACCESSORIES VALUES ('A02', 'Hard Disk', 5000, 'S01');
INSERT INTO ACCESSORIES VALUES ('A03', 'Keyboard', 500, 'S02');
INSERT INTO ACCESSORIES VALUES ('A04', 'Mouse', 300, 'S01');
INSERT INTO ACCESSORIES VALUES ('A05', 'Mother Board', 13000, 'S02');
INSERT INTO ACCESSORIES VALUES ('A06', 'Keyboard', 400, 'S03');
INSERT INTO ACCESSORIES VALUES ('A07', 'LCD', 6000, 'S04');
INSERT INTO ACCESSORIES VALUES ('T08', 'LCD', 5500, 'S05');
INSERT INTO ACCESSORIES VALUES ('T09', 'Mouse', 350, 'S05');
INSERT INTO ACCESSORIES VALUES ('T10', 'Hard Disk', 4500, 'S03');
COMMIT;
```



(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

SELECT NAME, PRICE FROM ACCESSORIES ORDER BY PRICE ASC;

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

(ii) To display Id and SName of all Shoppe in Nehru Place.

SELECT ID, SNAME FROM SHOPPE WHERE AREA = 'Nehru Place';

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.02 seconds CSV Export

(iii) To display Minimum and Maximum Price of each Name of accessories.

SELECT NAME, MIN(PRICE) AS MIN_PRICE, MAX(PRICE) AS MAX_PRICE FROM ACCESSORIES GROUP BY NAME;

Results Explain Describe Saved SQL History

NAME	MIN_PRICE	MAX_PRICE
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

(iv) To display Name, Price of all accessories and their respective SName where they are available.

SELECT A.NAME, A.PRICE, S.SNAME FROM ACCESSORIES A, SHOPPE S WHERE A.ID = S.ID;

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.00 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM SHOPPE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;



Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT Name, Price*0.05 AS DISCOUNT FROM ACCESSORIES WHERE ID IN ('S02', 'S03');



Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

6. Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL

Create HOSPITAL Table

CREATE TABLE HOSPITAL (No INT PRIMARY KEY,Name VARCHAR(50), Age INT, Department VARCHAR(50), Date_of_Admission DATE, Charges INT, Gender CHAR(1));



INSERT INTO HOSPITAL VALUES (1, 'Arpit', 62, 'Surgery', TO_DATE('1998-01-21', 'YYYY-MM-DD'), 300, 'M');

INSERT INTO HOSPITAL VALUES (2, 'Zarina', 22, 'ENT', TO_DATE('1997-12-12', 'YYYY-MM-DD'), 250, 'F');

INSERT INTO HOSPITAL VALUES (3, 'Kareem', 32, 'Orthopedic', TO_DATE('1998-02-19', 'YYYY-MM-DD'), 200, 'M');

INSERT INTO HOSPITAL VALUES (4, 'Arun', 12, 'Surgery', TO_DATE('1998-01-11', 'YYYY-MM-DD'), 300, 'M');

INSERT INTO HOSPITAL VALUES (5, 'Zubin', 30, 'ENT', TO_DATE('1998-02-24', 'YYYY-MM-DD'), 250, 'M');

INSERT INTO HOSPITAL VALUES (6, 'Ketaki', 16, 'ENT', TO_DATE('1998-01-12', 'YYYY-MM-DD'), 250, 'M');

INSERT INTO HOSPITAL VALUES (7, 'Ankita', 29, 'Cardiology', TO_DATE('1998-02-20', 'YYYY-MM-DD'), 800, 'F');

INSERT INTO HOSPITAL VALUES (8, 'Zareen', 45, 'Gynecology', TO_DATE('1998-02-22', 'YYYY-MM-DD'), 300, 'F');

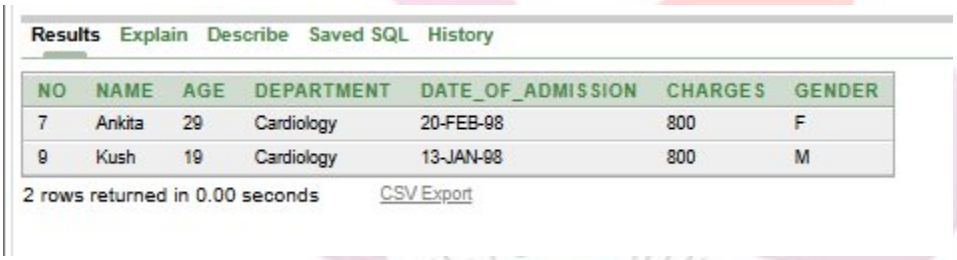
INSERT INTO HOSPITAL VALUES (9, 'Kush', 19, 'Cardiology', TO_DATE('1998-01-13', 'YYYY-MM-DD'), 800, 'M');

INSERT INTO HOSPITAL VALUES (10, 'Shilpa', 23, 'Nuclear Medicine', TO_DATE('1998-02-21', 'YYYY-MM-DD'), 400, 'F');



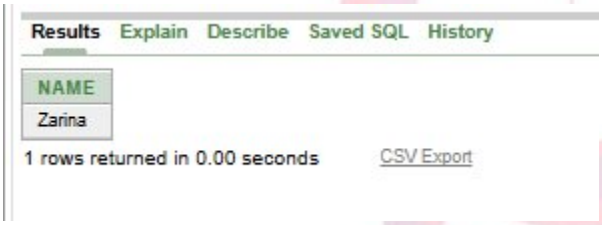
(a) To select all the information of patients of cardiology department.

SELECT * FROM HOSPITAL WHERE Department = 'Cardiology';



(b) To list the names of female patients who are in ENT department.

SELECT Name FROM HOSPITAL WHERE Gender = 'F' AND Department = 'ENT';



(c) To list name of all patients with their date of admission in ascending order.

SELECT Name, Date_of_Admission FROM HOSPITAL ORDER BY Date_of_Admission ASC;

Results Explain Describe Saved SQL History

NAME	DATE_OF_ADMISSION
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankita	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98

10 rows returned in 0.00 seconds CSV Export

(d) To display Patient's Name, Charges, Age for only female patients.

SELECT Name, Charges, Age FROM HOSPITAL WHERE Gender = 'F';

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankita	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.00 seconds CSV Export

(e) To count the number of patients with Age<30.

SELECT COUNT(*) FROM HOSPITAL WHERE Age < 30;

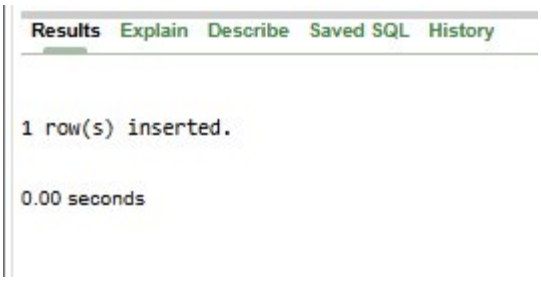
Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

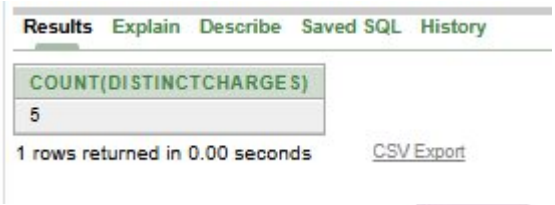
(f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M"

INSERT INTO HOSPITAL VALUES (11, 'Aftab', 24, 'Surgery', TO_DATE('1998-02-25', 'YYYY-MM-DD'),300, 'M');

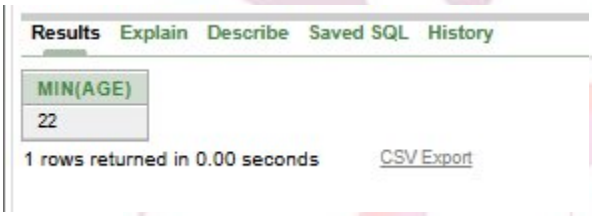


(g) Give the output of following SQL statements:

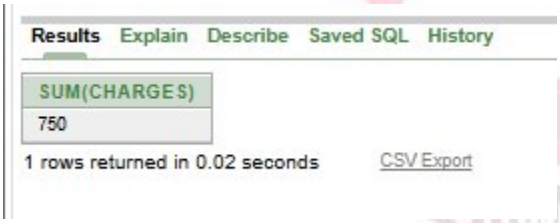
(i) Select COUNT (DISTINCT charges) from HOSPITAL;



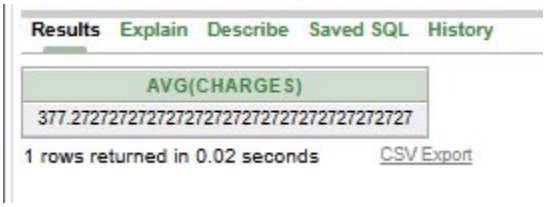
(ii) Select MIN (Age) from HOSPITAL where Gender = "F"



(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"



(iv) SELECT AVG(Charges) FROM HOSPITAL WHERE Date_of_Admission < TO_DATE('1998-08-12', 'YYYY-MM-DD');



7. Answer the questions (a) and (b) on the basis of the following tables STORE and ITEM

Create STORE Table

CREATE TABLE STORE (SNo VARCHAR(5) PRIMARY KEY, SName VARCHAR(50), Area VARCHAR(50)

);



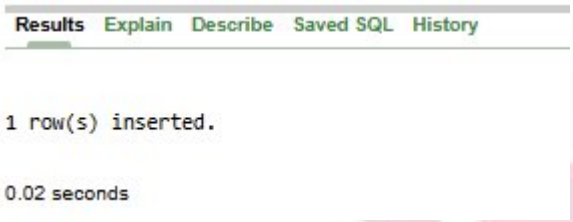
INSERT INTO STORE VALUES ('S01', 'ABC Computronics', 'GK II');

INSERT INTO STORE VALUES ('S02', 'All InfoTech Media', 'CP');

INSERT INTO STORE VALUES ('S03', 'Tech Shoppe', 'Nehru Place');

INSERT INTO STORE VALUES ('S04', 'Geeks Techno Soft', 'Nehru Place');

INSERT INTO STORE VALUES ('S05', 'Hitech Tech Store', 'CP');



Create ITEM Table

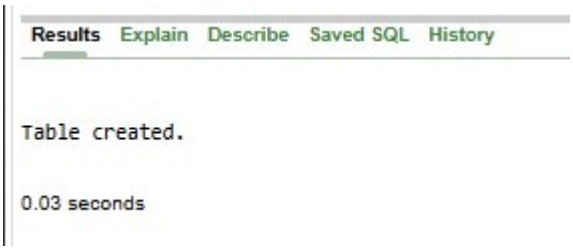
CREATE TABLE ITEM (

INo VARCHAR(5) PRIMARY KEY,

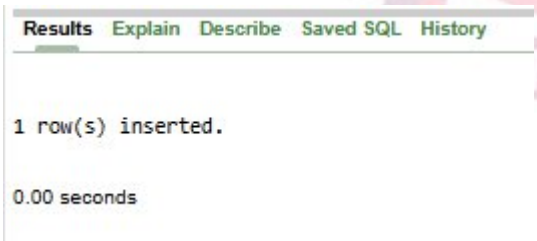
IName VARCHAR(50),

Price INT,

```
    SNo VARCHAR(5),
    CONSTRAINT fk_store FOREIGN KEY (SNo) REFERENCES STORE(SNo)
);
```




```
INSERT INTO ITEM VALUES ('T01', 'Mother Board', 12000, 'S01');
INSERT INTO ITEM VALUES ('T02', 'Hard Disk', 5000, 'S01');
INSERT INTO ITEM VALUES ('T03', 'Keyboard', 500, 'S02');
INSERT INTO ITEM VALUES ('T04', 'Mouse', 300, 'S01');
INSERT INTO ITEM VALUES ('T05', 'Mother Board', 13000, 'S02');
INSERT INTO ITEM VALUES ('T06', 'Keyboard', 400, 'S03');
INSERT INTO ITEM VALUES ('T07', 'LCD', 6000, 'S04');
INSERT INTO ITEM VALUES ('T08', 'LCD', 5500, 'S05');
INSERT INTO ITEM VALUES ('T09', 'Mouse', 350, 'S05');
INSERT INTO ITEM VALUES ('T10', 'Hard Disk', 4500, 'S03');
```



(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

SELECT IName, Price FROM ITEM ORDER BY Price ASC;

Results Explain Describe Saved SQL History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

(ii) To display SNo and SName of all store location in CP.

SELECT SNo, SName FROM STORE WHERE Area = 'CP';

Results Explain Describe Saved SQL History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

SELECT IName, MIN(Price) AS Min_Price, MAX(Price) AS Max_Price FROM ITEM GROUP BY IName;

Results Explain Describe Saved SQL History

INAME	MIN_PRICE	MAX_PRICE
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

(iv) To display IName, Price of all items and their respective SName where they are available.

SELECT I.IName, I.Price, S.SName FROM ITEM I JOIN STORE S ON I.SNo = S.SNo;

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computronics
Hard Disk	5000	ABC Computronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Geeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.00 seconds CSV Export

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.00 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.01 seconds CSV Export

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

Results Explain Describe Saved SQL History

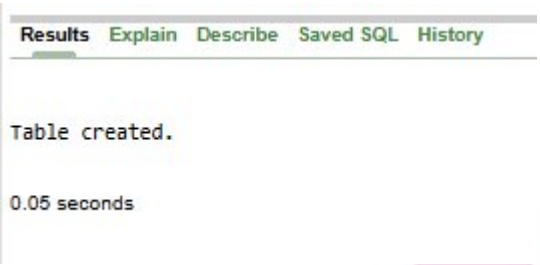
INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

Create TEACHER Table

CREATE TABLE TEACHER (No NUMBER(2), Name VARCHAR2(30), Age NUMBER(3), Department VARCHAR2(20), Date_of_Join DATE, Salary NUMBER(10), Gender CHAR(1));



INSERT INTO TEACHER VALUES (1, 'Jugal', 34, 'Computer', TO_DATE('10/01/97','DD/MM/YY'), 12000, 'M');

INSERT INTO TEACHER VALUES (2, 'Sharmila', 31, 'History', TO_DATE('24/03/98','DD/MM/YY'), 20000, 'F');

INSERT INTO TEACHER VALUES (3, 'Sandeep', 32, 'Maths', TO_DATE('12/12/96','DD/MM/YY'), 30000, 'M');

INSERT INTO TEACHER VALUES (4, 'Sangeeta', 35, 'History', TO_DATE('01/07/99','DD/MM/YY'), 40000, 'F');

INSERT INTO TEACHER VALUES (5, 'Rakesh', 42, 'Maths', TO_DATE('05/09/97','DD/MM/YY'), 25000, 'M');

INSERT INTO TEACHER VALUES (6, 'Shyam', 50, 'History', TO_DATE('27/06/98','DD/MM/YY'), 30000, 'M');

INSERT INTO TEACHER VALUES (7, 'Shiv Om', 44, 'Computer', TO_DATE('25/02/97','DD/MM/YY'), 21000, 'M');

INSERT INTO TEACHER VALUES (8, 'Shalakha', 33, 'Maths', TO_DATE('31/07/97','DD/MM/YY'), 20000, 'F');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To show all information about the teacher of history department

SELECT * FROM TEACHER WHERE Department = 'History';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATE_OF_JOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-99	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.00 seconds CSV Export

(b) To list the names of female teacher who are in Hindi department

SELECT Name FROM TEACHER WHERE Gender = 'F' AND Department = 'Hindi';

Results Explain Describe Saved SQL History

no data found

(c) To list names of all teachers with their date of joining in ascending order.

SELECT Name, Date_of_Join FROM TEACHER ORDER BY Date_of_Join ASC;

Results Explain Describe Saved SQL History

NAME	DATE_OF_JOIN
Sandeep	12-DEC-96
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98
Sangeeta	01-JUL-99

8 rows returned in 0.00 seconds CSV Export

(d) To display teacher's Name, Salary, Age for male teacher only.

SELECT Name, Salary, Age FROM TEACHER WHERE Gender = 'M';

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44

5 rows returned in 0.00 seconds CSV Export

(e) To count the number of teachers with Age>23.

SELECT COUNT(*) FROM TEACHER WHERE Age > 23;

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Export

(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

INSERT INTO TEACHER VALUES (9, 'Raja', 26, 'Computer',
TO_DATE('13/05/95','DD/MM/YY'), 2300, 'M');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.03 seconds
```

(g) Give the output of following SQL statements:

(i) Select COUNT (distinct department) from TEACHER;

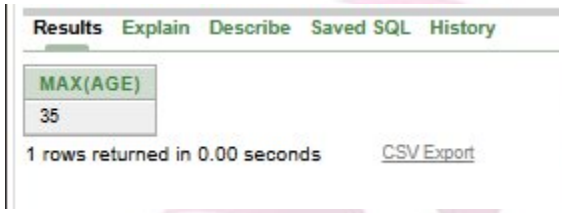


Results Explain Describe Saved SQL History

COUNT(DISTINCTDEPARTMENT)
3

1 rows returned in 0.00 seconds CSV Export

(II) Select MAX (Age) from TEACHER where Gender= "F"



Results Explain Describe Saved SQL History

MAX(AGE)
35

1 rows returned in 0.00 seconds CSV Export

(iii) Select AVG (Salary) from TEACHER where Date_of_join < To_Date (12/07/96);



Results Explain Describe Saved SQL History

AVG(SALARY)
-

1 rows returned in 0.00 seconds CSV Export

(iv) Select SUM (Salary) from TEACHER where Date of join < To_Date (12/07/96);



Results Explain Describe Saved SQL History

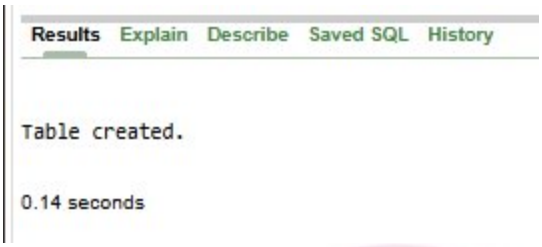
SUM(SALARY)
2300

1 rows returned in 0.02 seconds CSV Export

9. Given the following tables for a database LIBRARY:

Create BOOKS Table

CREATE TABLE BOOKS (Book_Id VARCHAR2(5) PRIMARY KEY, Book_Name VARCHAR2(50), Author_Name VARCHAR2(50),Publishers VARCHAR2(30), Price NUMBER(8), Type VARCHAR2(20), Qty NUMBER(5));



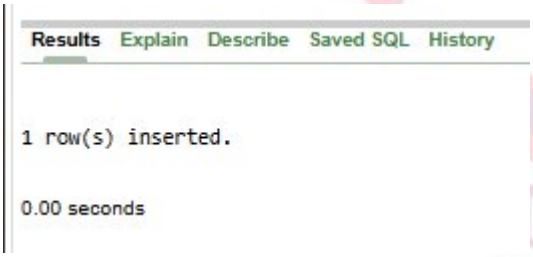
INSERT INTO BOOKS VALUES ('F0001', 'The Tears', 'William Hopkins', 'First Publ.', 750, 'Fiction', 10)

INSERT INTO BOOKS VALUES ('F0002', 'Thunderbolts', 'Anna Roberts', 'First Publ.', 700, 'Fiction', 5)

INSERT INTO BOOKS VALUES ('T0001', 'My First C++', 'Brijan & Brooke', 'EPB', 250, 'Text', 10)

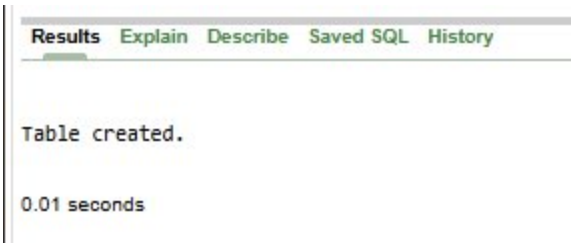
INSERT INTO BOOKS VALUES ('T0002', 'C++ Brainworks', 'A. W. Rossaine', 'TDH', 325, 'Text', 5)

INSERT INTO BOOKS VALUES ('C0001', 'Fast Cook', 'Lata Kapoor', 'EPB', 350, 'Cookery', 8)



Create ISSUED Table

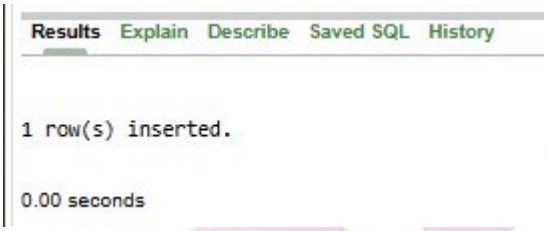
CREATE TABLE ISSUED (Book_Id VARCHAR2(5), Quantity_Issued NUMBER(5), FOREIGN KEY (Book_Id) REFERENCES BOOKS(Book_Id))



INSERT INTO ISSUED VALUES ('F0001', 3)

INSERT INTO ISSUED VALUES ('T0001', 1)

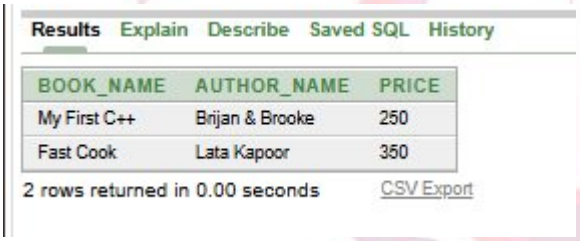
INSERT INTO ISSUED VALUES ('C0001', 5)



Write SQL queries for (a) to (f):

(a) To show Book name, Author name and Price of books of EPB publishers.

SELECT Book_Name, Author_Name, Price FROM BOOKS WHERE Publishers = 'EPB'



(b) To list the names of the books of Fiction Type.

SELECT Book_Name FROM BOOKS WHERE Type = 'Fiction'



(c) To display the names and price of the books in descending order of their price.

SELECT Book_Name, Price FROM BOOKS ORDER BY Price DESC

Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250

5 rows returned in 0.00 seconds CSV Export

(d) To increase the price of all books of first publisher by 50.

UPDATE BOOKS SET Price = Price + 50 WHERE Publishers = 'First Publ.'

Results Explain Describe Saved SQL History

```
2 row(s) updated.

0.00 seconds
```

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

SELECT B.Book_Id, B.Book_Name, I.Quantity_Issued FROM BOOKS B JOIN ISSUED I ON B.Book_Id = I.Book_Id

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
F0001	The Tears	3
T0001	My First C++	1
C0001	Fast Cook	5

3 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the table Issued following the data: “F0002”,4

INSERT INTO ISSUED VALUES ('F0002', 4)

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.00 seconds

(g) Give the output of the following queries based on the above tables:

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

Results Explain Describe Saved SQL History

SUM(PRICE)
1400

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME
My First C++	Brijan & Brooke
C++ Brainworks	A. W. Rossaine
Fast Cook	Lata Kapoor

3 rows returned in 0.00 seconds CSV Export

(iv) SELECT COUNT (*) FROM Books;

Results Explain Describe Saved SQL History

COUNT(*)
5

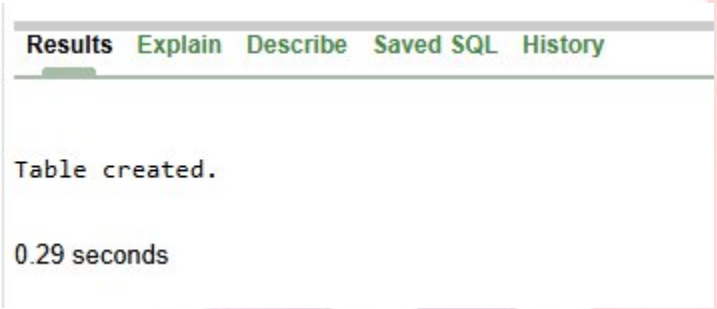
1 rows returned in 0.00 seconds CSV Export

10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

Create CARDEN Table

CREATE TABLE CARDEN (Ccode INT PRIMARY KEY, CarName VARCHAR(20), Make VARCHAR(20), Color VARCHAR(15), Capacity INT, Charges INT);



INSERT INTO CARDEN VALUES (501, 'A-star', 'Suzuki', 'RED', 3, 14);

INSERT INTO CARDEN VALUES (503, 'Indigo', 'Tata', 'SILVER', 3, 12);

INSERT INTO CARDEN VALUES (502, 'Innova', 'Toyota', 'WHITE', 7, 15);

INSERT INTO CARDEN VALUES (509, 'SX4', 'Suzuki', 'SILVER', 4, 14);

INSERT INTO CARDEN VALUES (510, 'C-Class', 'Mercedes', 'RED', 4, 35);



Create CUSTOMER Table

CREATE TABLE CUSTOMER (CustCode INT PRIMARY KEY, Cname VARCHAR(30), Ccode INT, FOREIGN KEY (Ccode) REFERENCES CARDEN(Ccode));

Results Explain Describe Saved SQL History

```
Table created.

0.05 seconds
```

INSERT INTO CUSTOMER VALUES (1001, 'HamantSahu', 501);

INSERT INTO CUSTOMER VALUES (1002, 'Raj Lal', 509);

INSERT INTO CUSTOMER VALUES (1003, 'Feroja Shah', 503);

INSERT INTO CUSTOMER VALUES (1004, 'Ketan Dhal', 502);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.02 seconds
```

(i) To display the name of all the SILVER colored cars.

SELECT CarName FROM CARDEN WHERE Color = 'SILVER';

Results Explain Describe Saved SQL History

CARNAME
Indigo
SX4

2 rows returned in 0.03 seconds CSV Export

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.02 seconds CSV Export

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

SELECT MAX(Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.02 seconds CSV Export

(iv) To display the customer name and the corresponding name of the cards hired by them.

SELECT Cname, CarName FROM CUSTOMER, CARDEN WHERE CUSTOMER.Ccode = CARDEN.Ccode;

Results Explain Describe Saved SQL History

CNAME	CARNAME
HamantSahu	A-star
Feroja Shah	Indigo
Ketan Dhal	Innova
Raj Lal	SX4

4 rows returned in 0.00 seconds CSV Export

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.02 seconds CSV Export

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT(*), Make FROM CARDEN GROUP BY Make;

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.02 seconds CSV Export

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

Results Explain Describe Saved SQL History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

Create SALGRADE Table

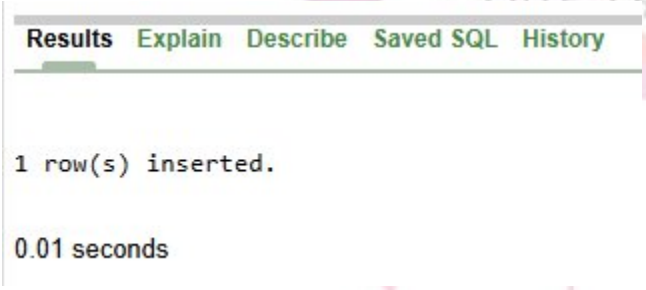
CREATE TABLE SALGRADE (SGRADE VARCHAR2(5) PRIMARY KEY, SALARY NUMBER(8), HRA NUMBER(8));



INSERT INTO SALGRADE VALUES ('S01', 56000, 18000);

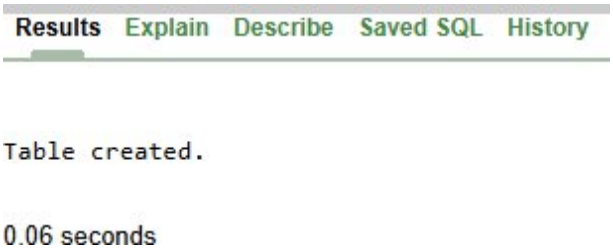
INSERT INTO SALGRADE VALUES ('S02', 32000, 12000);

INSERT INTO SALGRADE VALUES ('S03', 24000, 8000);



Create EMPLOYEE Table

CREATE TABLE EMPLOYEE (ECODE NUMBER(5) PRIMARY KEY, NAME VARCHAR2(30), DESIG VARCHAR2(20), SGRADE VARCHAR2(5), DOJ DATE, DOB DATE, FOREIGN KEY (SGRADE) REFERENCES SALGRADE(SGRADE));



```
INSERT INTO EMPLOYEE VALUES (101, 'Abdul Ahmad',  'EXECUTIVE',    'S03',
TO_DATE('23-MAR-2003', 'DD-MON-YYYY'), TO_DATE('13-JAN-1980', 'DD-MON-
YYYY'));

INSERT INTO EMPLOYEE VALUES (102, 'Ravi Chander', 'HEAD-IT',      'S02',
TO_DATE('12-FEB-2010', 'DD-MON-YYYY'), TO_DATE('22-JUL-1987', 'DD-MON-
YYYY'));

INSERT INTO EMPLOYEE VALUES (103, 'John Ken',     'Receptionist', 'S03', TO_DATE('24-
JUN-2009', 'DD-MON-YYYY'), TO_DATE('24-FEB-1983', 'DD-MON-YYYY'));

INSERT INTO EMPLOYEE VALUES (105, 'NazarAmeen',   'GM',           'S02', TO_DATE('11-
AUG-2006', 'DD-MON-YYYY'), TO_DATE('03-MAR-1984', 'DD-MON-YYYY'));

INSERT INTO EMPLOYEE VALUES (108, 'PriyamSen',    'CEO',          'S01', TO_DATE('29-
DEC-2004', 'DD-MON-YYYY'), TO_DATE('19-JAN-1982', 'DD-MON-YYYY'));
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.03 seconds
```

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

```
SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;
```

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazarAmeen	GM	S02	11-AUG-06	03-MAR-84
108	PriyamSen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.02 seconds CSV Export

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

```
SELECT NAME, DESIG FROM EMPLOYEE WHERE SGRADE IN ('S02', 'S03');
```

Results Explain Describe Saved SQL History

NAME	DESIG
Abdul Ahmad	EXECUTIVE
Ravi Chander	HEAD-IT
John Ken	Receptionist
NazarAmeen	GM

4 rows returned in 0.00 seconds CSV Export

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB-2006' and '08-AUG-2009'.

SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN '09-FEB-2006' AND '08-AUG-2009';

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazarAmeen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.01 seconds CSV Export

(iv) TO add a new row in the EMPLOYEE table with the following data:

109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

INSERT INTO EMPLOYEE VALUES (109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.03 seconds

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S03
3	S02
1	S01

3 rows returned in 0.01 seconds CSV Export

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S

WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

Results Explain Describe Saved SQL History

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.02 seconds CSV Export

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

Results Explain Describe Saved SQL History

SGRADE	SALARY+HRA
S02	44000

1 rows returned in 0.00 seconds CSV Export

12. Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

Create PAYLEVEL Table

CREATE TABLE PAYLEVEL (PLEVEL VARCHAR2(5) PRIMARY KEY, PAY NUMBER(8), ALLOWANCE NUMBER(8));




```
Table created.
```

0.08 seconds

INSERT INTO PAYLEVEL VALUES ('P001', 26000, 12000);

INSERT INTO PAYLEVEL VALUES ('P002', 22000, 10000);

INSERT INTO PAYLEVEL VALUES ('P003', 12000, 6000);




```
1 row(s) inserted.
```

0.03 seconds

Create WORKER Table

CREATE TABLE WORKER (ECODE NUMBER(5) PRIMARY KEY, NAME VARCHAR2(30), DESIGN VARCHAR2(20), PLEVEL VARCHAR2(5),DOJ DATE, DOB DATE, FOREIGN KEY (PLEVEL) REFERENCES PAYLEVEL(PLEVEL));




```
Table created.
```

0.06 seconds

INSERT INTO WORKER VALUES (11, 'RadheShyam', 'Supervisor', 'P001', TO_DATE('13-SEP-2004', 'DD-MON-YYYY'), TO_DATE('23-AUG-1981', 'DD-MON-YYYY'));

INSERT INTO WORKER VALUES (12, 'ChanderNath', 'Operator', 'P003', TO_DATE('22-FEB-2010', 'DD-MON-YYYY'), TO_DATE('12-JUL-1987', 'DD-MON-YYYY'));

INSERT INTO WORKER VALUES (13, 'Fizza', 'Operator', 'P003', TO_DATE('14-JUN-2009', 'DD-MON-YYYY'), TO_DATE('14-OCT-1983', 'DD-MON-YYYY'));

INSERT INTO WORKER VALUES (15, 'Ahmeen Ahmad', 'Mechanic', 'P002', TO_DATE('21-AUG-2006', 'DD-MON-YYYY'), TO_DATE('13-MAR-1984', 'DD-MON-YYYY'));

INSERT INTO WORKER VALUES (18, 'Sanya', 'Clerk', 'P002', TO_DATE('19-DEC-2005', 'DD-MON-YYYY'), TO_DATE('09-JUN-1983', 'DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.01 seconds

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

SELECT * FROM WORKER ORDER BY DOB DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.01 seconds CSV Export

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

SELECT NAME, DESIGN FROM WORKER WHERE PLEVEL IN ('P001', 'P002');

Results Explain Describe Saved SQL History

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.00 seconds CSV Export

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

SELECT * FROM WORKER WHERE DOB BETWEEN TO_DATE('19-JAN-1984', 'DD-MON-YYYY') AND TO_DATE('18-JAN-1987', 'DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

(iv) To add a new row with the following:

19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

INSERT INTO WORKER VALUES (19, 'Daya Kishore', 'Operator', 'P003', TO_DATE('19-JUN-2008', 'DD-MON-YYYY'), TO_DATE('11-JUL-1984', 'DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.03 seconds
```

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

Results Explain Describe Saved SQL History

COUNT(PLEVEL)	PLEVEL
1	P001
3	P003
2	P002

3 rows returned in 0.02 seconds CSV Export

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

Results Explain Describe Saved SQL History

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13;

Results Explain Describe Saved SQL History

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.01 seconds CSV Export

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PAYLEVEL WHERE PLEVEL = 'P003';

Results Explain Describe Saved SQL History

PLEVEL	PAY+ALLOWANCE
P003	18000

1 rows returned in 0.00 seconds CSV Export

13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

Create SUPPLIERS Table

CREATE TABLE SUPPLIERS (Scode NUMBER(3) PRIMARY KEY, Sname VARCHAR2(30));




```
Table created.

0.07 seconds
```

INSERT INTO SUPPLIERS VALUES (21, 'Premium Stationers');

INSERT INTO SUPPLIERS VALUES (23, 'Soft Plastics');

INSERT INTO SUPPLIERS VALUES (22, 'Tetra Supply');




```
1 row(s) inserted.

0.00 seconds
```

Create STORE Table

CREATE TABLE STORE (ItemNo NUMBER(5) PRIMARY KEY, Item VARCHAR2(30), Scode NUMBER(3), Qty NUMBER(5), Rate NUMBER(5), LastBuy DATE, FOREIGN KEY (Scode) REFERENCES SUPPLIERS(Scode));




```
Table created.

0.01 seconds
```

```
INSERT INTO STORE VALUES (2005, 'Sharpener Classic', 23, 60,  8,  TO_DATE('30-JUN-2009', 'DD-MON-YYYY'));

INSERT INTO STORE VALUES (2003, 'Ball Pen 0.25',     22, 50,  25, TO_DATE('01-FEB-2009', 'DD-MON-YYYY'));

INSERT INTO STORE VALUES (2002, 'Gel Pen Premium',   21, 150, 12, TO_DATE('24-FEB-2010', 'DD-MON-YYYY'));

INSERT INTO STORE VALUES (2006, 'Gel Pen Classic',   21, 250, 20, TO_DATE('11-MAY-2009', 'DD-MON-YYYY'));

INSERT INTO STORE VALUES (2001, 'Eraser Small',      22, 220, 6,  TO_DATE('19-JAN-2009', 'DD-MON-YYYY'));

INSERT INTO STORE VALUES (2004, 'Eraser Big',        22, 110, 8,  TO_DATE('02-DEC-2009', 'DD-MON-YYYY'));

INSERT INTO STORE VALUES (2009, 'Ball Pen 0.5',      21, 180, 18, TO_DATE('03-NOV-2009', 'DD-MON-YYYY'));
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

```
SELECT * FROM STORE ORDER BY LastBuy ASC;
```

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2006	Gel Pen Classic	21	250	20	11-MAY-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09
2002	Gel Pen Premium	21	150	12	24-FEB-10

7 rows returned in 0.00 seconds CSV Export

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

SELECT ItemNo, Item FROM STORE WHERE Rate > 15;

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.00 seconds CSV Export

(iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

SELECT * FROM STORE WHERE Scode = 22 OR Qty > 110;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.00 seconds CSV Export

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

SELECT Scode, MIN(Rate) FROM STORE GROUP BY Scode;

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.00 seconds CSV Export

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
3

1 rows returned in 0.00 seconds CSV Export

(ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

Results Explain Describe Saved SQL History

RATE*QTY
880

1 rows returned in 0.00 seconds CSV Export

(iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

Results Explain Describe Saved SQL History

ITEM	SNAME
Gel Pen Classic	Premium Stationers

1 rows returned in 0.01 seconds CSV Export

(iv) SELECT MAX(LastBuy) FROM STORE;

Results Explain Describe Saved SQL History

MAX(LASTBUY)
24-FEB-10

1 rows returned in 0.00 seconds CSV Export

14. Consider the following table GARMENT and FABRIC,

Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

Create FABRIC Table

CREATE TABLE FABRIC (FCODE VARCHAR2(5) PRIMARY KEY, TYPE VARCHAR2(20));




```
Table created.

0.01 seconds
```

INSERT INTO FABRIC VALUES ('F04', 'Polyester');

INSERT INTO FABRIC VALUES ('F02', 'Cotton');

INSERT INTO FABRIC VALUES ('F03', 'Silk');

INSERT INTO FABRIC VALUES ('F01', 'Terelene');




```
1 row(s) inserted.

0.01 seconds
```

Create GARMENT Table

CREATE TABLE GARMENT (GCODE NUMBER(5) PRIMARY KEY, DESCRIPTION VARCHAR2(30), PRICE NUMBER(6), FCODE VARCHAR2(5),READYDATE DATE,FOREIGN KEY (FCODE) REFERENCES FABRIC(FCODE));

Results Explain Describe Saved SQL History

```
Table created.

0.03 seconds
```

```
INSERT INTO GARMENT VALUES (10023, 'Pencil Skirt',   1150, 'F03', TO_DATE('19-DEC-2008', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10001, 'Formal Shirt',   1250, 'F01', TO_DATE('12-JAN-2008', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10012, 'Informal Shirt', 1550, 'F02', TO_DATE('06-JUN-2008', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10024, 'Baby Top',       750,  'F03', TO_DATE('07-APR-2007', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10090, 'Tulip Skirt',    850,  'F02', TO_DATE('31-MAR-2007', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10019, 'Evening Gown',   850,  'F03', TO_DATE('06-JUN-2008', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10009, 'Informal Pant',  1500, 'F02', TO_DATE('20-OCT-2008', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10007, 'Formal Pant',    1350, 'F01', TO_DATE('09-MAR-2008', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10020, 'Frock',          850,  'F04', TO_DATE('09-SEP-2007', 'DD-MON-YYYY'));

INSERT INTO GARMENT VALUES (10089, 'Slacks',         750,  'F03', TO_DATE('20-OCT-2008', 'DD-MON-YYYY'));
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

SELECT GCODE, DESCRIPTION FROM GARMENT ORDER BY GCODE DESC;

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.00 seconds CSV Export

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

SELECT * FROM GARMENT WHERE READYDATE BETWEEN TO_DATE('08-DEC-2007', 'DD-MON-YYYY') AND TO_DATE('16-JUN-2008', 'DD-MON-YYYY');

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.00 seconds CSV Export

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

SELECT AVG(PRICE) FROM GARMENT WHERE FCODE = 'F03';

Results Explain Describe Saved SQL History

AVG(PRICE)
875

1 rows returned in 0.00 seconds CSV Export

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

SELECT FCODE, MAX(PRICE), MIN(PRICE) FROM GARMENT GROUP BY FCODE;

Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.02 seconds CSV Export

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

Results Explain Describe Saved SQL History

SUM(PRICE)
2600

1 rows returned in 0.00 seconds CSV Export

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT.FCODE = FABRIC.FCODE AND GARMENT.PRICE >= 1260;

Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Informal Pant	Cotton
Informal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.00 seconds CSV Export

(vii) SELECT MAX (FCODE) FROM FABRIC;

Results Explain Describe Saved SQL History

MAX(FCODE)
F04

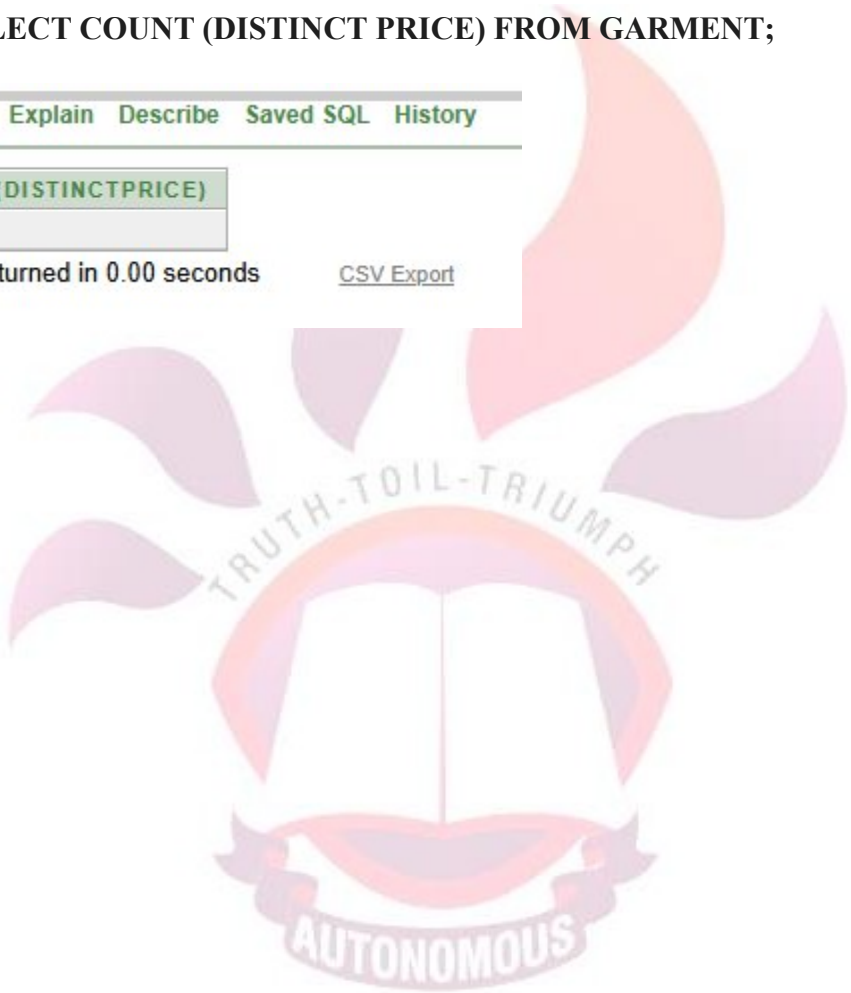
1 rows returned in 0.00 seconds CSV Export

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
7

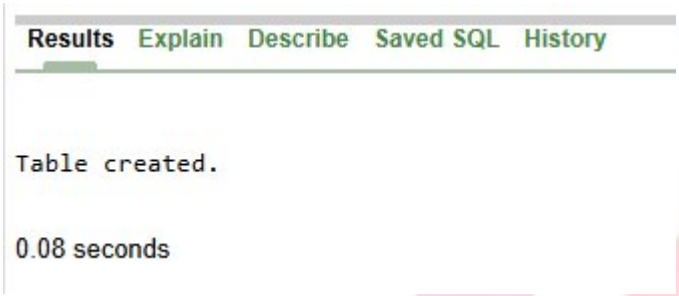
1 rows returned in 0.00 seconds CSV Export



15. Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables FURNITURE AND ARRIVALS.

Create FURNITURE Table

```
CREATE TABLE FURNITURE (ITEMNO NUMBER(3) PRIMARY KEY, ITEMNAME
VARCHAR2(30), TYPE VARCHAR2(20), DATEOFSTOCK DATE, PRICE NUMBER(7),
DISCOUNT NUMBER(3));
```




```
INSERT INTO FURNITURE VALUES (1, 'White lotus', 'Double Bed',
TO_DATE('23/02/2002', 'DD/MM/YYYY'), 30000, 25);

INSERT INTO FURNITURE VALUES (2, 'Pink feather', 'Baby cot',
TO_DATE('20/01/2002', 'DD/MM/YYYY'), 7000, 20);

INSERT INTO FURNITURE VALUES (3, 'Dolphin', 'Baby cot',
TO_DATE('19/02/2002', 'DD/MM/YYYY'), 9500, 20);

INSERT INTO FURNITURE VALUES (4, 'Decent', 'Office Table',
TO_DATE('01/01/2002', 'DD/MM/YYYY'), 25000, 30);

INSERT INTO FURNITURE VALUES (5, 'Comfort zone', 'Double Bed',
TO_DATE('12/01/2002', 'DD/MM/YYYY'), 25000, 25);

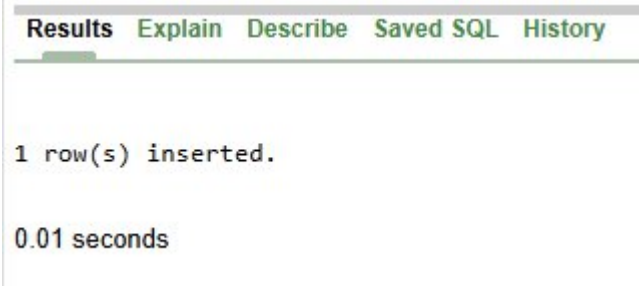
INSERT INTO FURNITURE VALUES (6, 'Donald', 'Baby cot',
TO_DATE('24/02/2002', 'DD/MM/YYYY'), 6500, 15);

INSERT INTO FURNITURE VALUES (7, 'Royal Finish', 'Office Table',
TO_DATE('20/02/2002', 'DD/MM/YYYY'), 18000, 30);

INSERT INTO FURNITURE VALUES (8, 'Royal tiger', 'Sofa',
TO_DATE('22/02/2002', 'DD/MM/YYYY'), 31000, 30);

INSERT INTO FURNITURE VALUES (9, 'Econo sitting', 'Sofa',
TO_DATE('13/12/2001', 'DD/MM/YYYY'), 9500, 25);
```

INSERT INTO FURNITURE VALUES (10, 'Eating paradise', 'Dining Table', TO_DATE('19/02/2002', 'DD/MM/YYYY'), 11500, 25);



Create ARRIVALS Table

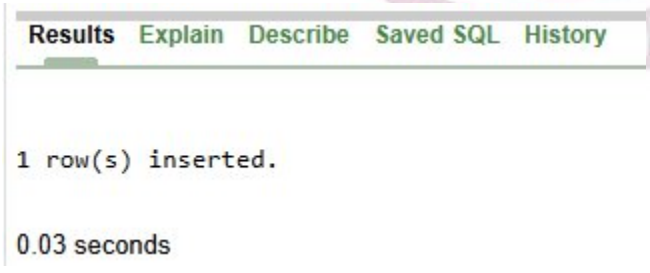
CREATE TABLE ARRIVALS (ITEMNO NUMBER(3) PRIMARY KEY, ITEMNAME VARCHAR2(30), TYPE VARCHAR2(20), DATEOFSTOCK DATE, PRICE NUMBER(7), DISCOUNT NUMBER(3));



INSERT INTO ARRIVALS VALUES (11, 'Wood Comfort', 'Double Bed', TO_DATE('23/03/2003', 'DD/MM/YYYY'), 25000, 25);

INSERT INTO ARRIVALS VALUES (12, 'Old Fox', 'Sofa', TO_DATE('20/02/2003', 'DD/MM/YYYY'), 17000, 20);

INSERT INTO ARRIVALS VALUES (13, 'Micky', 'Baby cot', TO_DATE('21/02/2002', 'DD/MM/YYYY'), 7500, 15);



(a) To show all information about the baby cots from the FURNITURE table.

SELECT * FROM FURNITURE WHERE TYPE = 'Baby cot';

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.02 seconds CSV Export

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

SELECT ITEMNAME FROM FURNITURE WHERE PRICE > 15000;

Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

SELECT ITEMNAME, TYPE FROM FURNITURE WHERE DATEOFSTOCK < TO_DATE('22/01/2002', 'DD/MM/YYYY') ORDER BY ITEMNAME DESC;

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Pink feather	Baby cot
Econo sitting	Sofa
Decent	Office Table
Comfort zone	Double Bed

4 rows returned in 0.02 seconds CSV Export

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

SELECT ITEMNAME, DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT > 25;

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.00 seconds CSV Export

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.

SELECT COUNT(*) FROM FURNITURE WHERE TYPE = 'Sofa';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.01 seconds CSV Export

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30

INSERT INTO ARRIVALS VALUES (14, 'Velvet touch', 'Double Bed',
TO_DATE('25/03/2003', 'DD/MM/YYYY'), 25000, 30);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.03 seconds

(g) Give the output of following SQL statement:

(i) Select COUNT(distinct TYPE) from FURNITURE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTTYPE)
5

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX(FURNITURE.DISCOUNT) FROM FURNITURE, ARRIVALS;

Results Explain Describe Saved SQL History

MAX(FURNITURE.DISCOUNT)
30

1 rows returned in 0.02 seconds CSV Export

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = “Baby cot”;

Results Explain Describe Saved SQL History

AVG(DISCOUNT)
18.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT SUM(PRICE) FROM FURNITURE WHERE DATEOFSTOCK < TO_DATE('12/02/2002', 'DD/MM/YYYY');

Results Explain Describe Saved SQL History

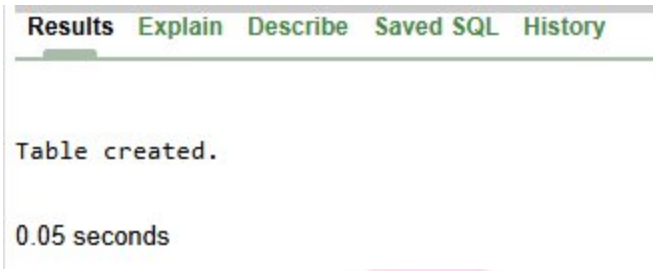
SUM(PRICE)
66500

1 rows returned in 0.00 seconds CSV Export

16. Given the following COVID19 relation, write SQL commands for questions (a) to (f).

Create COVID19 Table

CREATE TABLE COVID19 (PatientID NUMBER(5) PRIMARY KEY, PatientName VARCHAR2(30), State VARCHAR2(30), Age NUMBER(3), Status VARCHAR2(15),Vaccinated VARCHAR2(3));



INSERT INTO COVID19 VALUES (101, 'Arun', 'Tamil Nadu', 35, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES (102, 'Priya', 'Kerala', 42, 'Active', 'Yes');

INSERT INTO COVID19 VALUES (103, 'Karthik','Karnataka', 28, 'Recovered', 'No');

INSERT INTO COVID19 VALUES (104, 'Divya', 'Tamil Nadu', 31, 'Active', 'Yes');

INSERT INTO COVID19 VALUES (105, 'Rahul', 'Andhra Pradesh', 55, 'Deceased', 'No');

INSERT INTO COVID19 VALUES (106, 'Meena', 'Kerala', 24, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES (107, 'Suresh', 'Tamil Nadu', 48, 'Active', 'No');



a) Display all details of patients from Tamil Nadu.

SELECT * FROM COVID19 WHERE State = 'Tamil Nadu';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.01 seconds CSV Export

b) Display the names of patients whose Status is Recovered.

SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.02 seconds CSV Export

c) Display PatientName and Age in ascending order of Age.

SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.00 seconds CSV Export

d) Display the names of patients who are Vaccinated = 'Yes'.

SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0.01 seconds CSV Export

e) Count the number of patients whose status is Active.

SELECT COUNT(*) FROM COVID19 WHERE Status = 'Active';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

f) Insert the following record into the COVID19 table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

INSERT INTO COVID19 VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.03 seconds

(g) Write the output of the following SQL commands.

a. SELECT COUNT(*) FROM COVID19;

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.02 seconds CSV Export

b. SELECT MAX(Age) FROM COVID19;

Results Explain Describe Saved SQL History

MAX(AGE)
55

1 rows returned in 0.00 seconds CSV Export

c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';

Results Explain Describe Saved SQL History

COUNT(*)
5

1 rows returned in 0.00 seconds CSV Export

d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

Results Explain Describe Saved SQL History

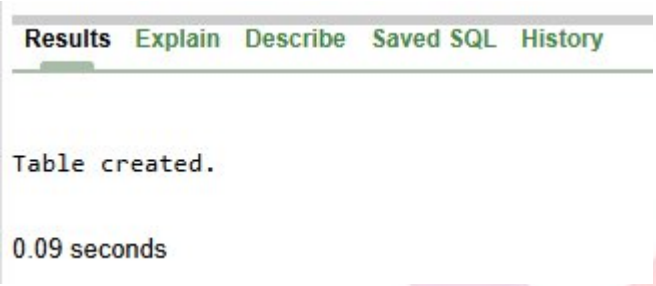
MIN(AGE)
24

1 rows returned in 0.00 seconds CSV Export

17. Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).

Create HOUSEPRICE Table

CREATE TABLE HOUSEPRICE (HouseID NUMBER(5) PRIMARY KEY, Location VARCHAR2(20), Bedrooms NUMBER(2), AreaSqft NUMBER(5), Price NUMBER(10), HouseType VARCHAR2(15));



INSERT INTO HOUSEPRICE VALUES (201, 'Chennai', 2, 1100, 5200000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (202, 'Coimbatore', 3, 1450, 6800000, 'Villa');

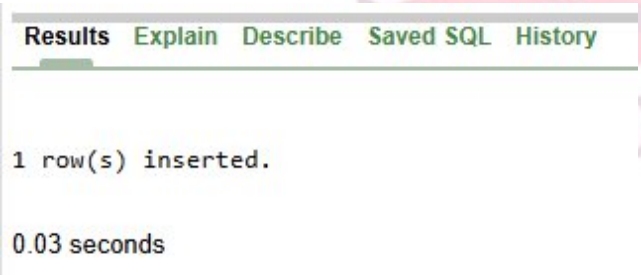
INSERT INTO HOUSEPRICE VALUES (203, 'Madurai', 2, 980, 4100000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (204, 'Chennai', 4, 2200, 9800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES (205, 'Trichy', 3, 1650, 7200000, 'Independent');

INSERT INTO HOUSEPRICE VALUES (206, 'Salem', 2, 1200, 5000000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (207, 'Erode', 3, 1800, 7600000, 'Independent');



a) Display all houses located in Chennai.

SELECT * FROM HOUSEPRICE WHERE Location = 'Chennai';

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

2 rows returned in 0.01 seconds CSV Export

b) Display the Location and Price of houses having 3 Bedrooms.

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;

Results Explain Describe Saved SQL History

LOCATION	PRICE
Coimbatore	6800000
Trichy	7200000
Erode	7600000

3 rows returned in 0.00 seconds CSV Export

c) Display Location, AreaSqft, and Price in descending order of Price.

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Trichy	1650	7200000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

7 rows returned in 0.00 seconds CSV Export

d) Display the details of houses costing more than ₹70,00,000.

SELECT * FROM HOUSEPRICE WHERE Price > 7000000;

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
207	Erode	3	1800	7600000	Independent

3 rows returned in 0.00 seconds CSV Export

e) Count the number of Apartments.

SELECT COUNT(*) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.01 seconds CSV Export

f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

INSERT INTO HOUSEPRICE VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.02 seconds

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

Results Explain Describe Saved SQL History

MAX(PRICE)
9800000

1 rows returned in 0.02 seconds CSV Export

(ii) SELECT MIN(AreaSqft)FROM HOUSEPRICE;



Results Explain Describe Saved SQL History

MIN(AREASQFT)
980

1 rows returned in 0.02 seconds CSV Export

(iii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';



Results Explain Describe Saved SQL History

AVG(PRICE)
4766666.666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;



Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.02 seconds CSV Export

18. Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).

Create CANCERDATA Table

CREATE TABLE CANCERDATA(PatientID INT,PatientName VARCHAR(30), CancerType VARCHAR(20), Age INT, Stage VARCHAR(5) SurvivalYears INT);

Results Explain Describe Saved SQL History

```
Table created.
```

0.03 seconds

INSERT INTO CANCERDATA VALUES(301,'Kavya','Breast',45,'II',8);

INSERT INTO CANCERDATA VALUES(302,'Arun','Lung',60,'III',4);

INSERT INTO CANCERDATA VALUES(303,'Divya','Breast',38,'I',10);

INSERT INTO CANCERDATA VALUES(304,'Hari','Colon',52,'II',6);

INSERT INTO CANCERDATA VALUES(305,'Meena','Lung',57,'IV',2);

INSERT INTO CANCERDATA VALUES(306,'Ravi','Prostate',64,'III',5);

INSERT INTO CANCERDATA VALUES(307,'Priya','Breast',41,'II',9);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

a) Display all patients diagnosed with Breast Cancer.

SELECT * FROM CANCERDATA WHERE CancerType='Breast';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Kavya	Breast	45	II	8
303	Divya	Breast	38	I	10
307	Priya	Breast	41	II	9

3 rows returned in 0.00 seconds CSV Export

b) Display the names of patients whose Stage is III.

SELECT PatientName FROM CANCERDATA WHERE Stage='III';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Ravi

2 rows returned in 0.00 seconds CSV Export

c) Display PatientName and Age in ascending order of Age.

SELECT PatientName, Age FROM CANCERDATA ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	64

7 rows returned in 0.00 seconds CSV Export

d) Display the names of patients having SurvivalYears greater than 5.

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears > 5;

Results Explain Describe Saved SQL History

PATIENTNAME
Kavya
Divya
Hari
Priya

4 rows returned in 0.00 seconds CSV Export

e) Count the number of patients diagnosed with Lung Cancer.

SELECT COUNT(*) FROM CANCERDATA WHERE CancerType='Lung';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

f) Insert the following record:

308, 'Anand', 'Leukemia', 50, 'II', 7

INSERT INTO CANCERDATA VALUES (308,'Anand','Leukemia',50,'II',7);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCANCERTYPE)
5

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

Results Explain Describe Saved SQL History

MAX(SURVIVALYEARS)
10

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

Results Explain Describe Saved SQL History

AVG(AGE)
41.333333333333333333333333333333333333

1 rows returned in 0.01 seconds CSV Export

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

Results Explain Describe Saved SQL History

MIN(AGE)
41

1 rows returned in 0.00 seconds CSV Export

19. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS

Create ITEMS Table

CREATE TABLE ITEMS(CODE INT, INAME VARCHAR(30), QTY INT, PRICE INT, COMPANY VARCHAR(30), TCODE VARCHAR(5));




```
Table created.

0.00 seconds
```

INSERT INTO ITEMS VALUES(1001,'DIGITAL PAD12i',120,11000,'XENITA','T01');

INSERT INTO ITEMS VALUES(1006,'LED SCREEN 40',70,38000,'SANTORA','T02');

INSERT INTO ITEMS VALUES(1004,'CAR GPS SYSTEM',50,21500,'GEOKNOW','T01');

INSERT INTO ITEMS VALUES(1003,'DIGITAL CAMERA 12X',160,8000,'DIGICLICK','T02');

INSERT INTO ITEMS VALUES(1005,'PEN DRIVE 32 GB',600,1200,'STOREHOME','T03');




```
1 row(s) inserted.

0.00 seconds
```

Create TRADERS Table

CREATE TABLE TRADERS(TCODEVARCHAR(5),TNAMEVARCHAR(30),CITY VARCHAR(20));

Results Explain Describe Saved SQL History

```
Table created.

0.00 seconds
```

INSERT INTO TRADERS VALUES('T01','ELECTRONIC SALES','MUMBAI');

INSERT INTO TRADERS VALUES('T03','BUSY STORE CORP','DELHI');

INSERT INTO TRADERS VALUES('T02','DISP HOUSE INC','CHENNAI');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

SELECT * FROM ITEMS ORDER BY INAME ASC;

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

5 rows returned in 0.00 seconds CSV Export

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.01 seconds CSV Export

(c) To display the number of items, which are traded by each trader. The expected output of this query should be

T01 2

T02 2

T03 1

SELECT TCODE, COUNT(*) FROM ITEMS GROUP BY TCODE;

Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.01 seconds CSV Export

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;

Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

2 rows returned in 0.00 seconds CSV Export

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

SELECT TNAME FROM TRADERS WHERE CITY IN ('DELHI','MUMBAI');

Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.00 seconds CSV Export

(f) To display the name of the companies and the bane of the items in descending order of company names.

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD12i
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV Export

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

Results Explain Describe Saved SQL History

AMOUNT
1075000

1 rows returned in 0.00 seconds CSV Export

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

Results Explain Describe Saved SQL History

TCODE
T01
T03
T02

3 rows returned in 0.00 seconds CSV Export

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

Results Explain Describe Saved SQL History

INAME	TNAME
CAR GPS SYSTEM	ELECTRONIC SALES
LED SCREEN 40	DISP HOUSE INC

2 rows returned in 0.00 seconds CSV Export